



VRIJE
UNIVERSITEIT
BRUSSEL



Master thesis submitted in partial fulfilment of the requirements for the degree of
Master of Science in Applied Sciences and Engineering: Computer Science

A JAX FRAMEWORK FOR MULTI-AGENT RESEARCH IN DECENTRALISED ENERGY MARKETS

Kenneth Emerson

2023-2024

Promotor: Prof. Dr. Ann Nowé

Advisor: Andries Rosseau

Science and Bio-Engineering Sciences



VRIJE
UNIVERSITEIT
BRUSSEL



Proefschrift ingediend met het oog op het behalen van de graad van Master of
Science in de Ingenieurswetenschappen: Computerwetenschappen

A JAX FRAMEWORK FOR MULTI-AGENT RESEARCH IN DECENTRALISED ENERGY MARKETS

Kenneth Emerson

2023-2024

Promotor: Prof. Dr. Ann Nowé

Advisor: Andries Rosseau

wetenschappen en bio-ingenieurswetenschappen

Declaration of originality

I hereby declare that this thesis was entirely my own work and that any additional sources of information have been duly cited. I certify that, to the best of my knowledge, my thesis does not infringe upon anyone's copyright nor violate any proprietary rights and that any ideas, techniques, quotations, or any other material from the work of other people included in my thesis, published or otherwise, are fully acknowledged in accordance with the standard referencing practices. Furthermore, to the extent that I have included copyrighted material, I certify that I have obtained a written permission from the copyright owner(s) to include such material(s) in my thesis and have included copies of such copyright clearances to my appendix. I declare that this thesis has not been submitted for a higher degree to any other University or Institution.

Abstract

The emergence of alternative and decentralised energy sources has introduced significant complexity in managing contemporary energy networks, challenging the traditional distinction between large-scale energy producers and local consumers. The expanding integration of technologies such as solar panels, residual heat installations, home batteries, and electric vehicles has increased demand from local residential actors to adopt a more active role within the energy market.

To address these challenges, the introduction of local decentralised energy markets has been proposed, enabling members to actively engage in market transactions to procure and sell energy in response to their changing energy needs. In this context, artificial intelligence, particularly reinforcement learning, has emerged as a promising approach to facilitate automated market transactions and act as autonomous agents on behalf of the local market participants they represent.

This master's thesis introduces a novel simulation framework written in JAX and designed to investigate and develop new agent algorithms for decentralised energy markets. We disclose the design principles and implementation details fundamental to the framework's development and present the results of several representative experiments that demonstrate the framework's capabilities and offer insights for further exploration.

Acknowledgements

I would like to express my sincere appreciation to Professor Nowé for granting me the opportunity to undertake this master's thesis. Your expertise and support have been invaluable, and I am truly thankful for the trust you placed in me and the chance to explore my research interests under your guidance.

A special thank you goes to Andries Rosseau, for your intensive support, guidance, and insights throughout the year. Your mentorship has been crucial in helping me navigate the challenges, and I am thankful for the time and effort you dedicated to shaping this thesis.

I am grateful to all the professors in the Computer Science department of the Vrije Universiteit Brussel for their willingness to guide me through the learning process. Your expertise, experience, and flexibility toward me as a working student have made this journey an interesting and enriching experience.

To my friends and family, thank you for your support during this demanding time. I am truly grateful for your patience and understanding whenever I was unable to attend celebrations, trips, or even just a regular get-together. Your encouragement has meant the world to me.

And finally, my deepest gratitude to you, An, who encouraged me to just jump in the deep end, and to my most wonderful daughter, Miep. Your love and support have been my greatest motivation, and without you both, I would never have been able to accomplish this milestone. Thank you for being my pillars of strength and for believing in me every step of the way.

Contents

1	Introduction	1
1.1	Context	1
1.2	Objectives	2
1.3	Structure of the Thesis	2
2	Background	3
2.1	Energy Markets	3
2.1.1	Introduction	3
2.1.2	Energy Trading	4
2.2	Single-Agent Reinforcement Learning	6
2.2.1	Introduction	6
2.2.2	Proximal Policy Optimisation	7
2.3	Multi-Agent Reinforcement Learning	8
2.3.1	Environments	8
2.3.2	Training and Execution Modes	8
2.3.3	Challenges	8
2.4	Auctions	9
2.4.1	General	9
2.4.2	Double Auction	10
2.5	Related Work	10
3	Implementation of the Framework	13
3.1	Introduction	13
3.2	Implementation of the Environment	13
3.2.1	General	13
3.2.2	Implementation	15
3.3	Instantiating the Prosumers and the Global Market	17
3.3.1	Generated Data by means of a Mathematical Function	17
3.3.2	Generated Actor Energy Production and Consumption Data	18
3.3.3	Historical Market Day-Ahead Price Data	23
3.4	Agent-Environment Interaction and Transformers	23
3.5	Implementation of the Agents and the Training Loop	28
3.6	Experiment Configurations	29
3.7	The Main Function	30
4	Experimental Results	31
4.1	Scope of the Experiments	31
4.2	Demonstrating Learning	33
4.2.1	A Constant Global Market with Constant Prosumers	33
4.2.2	A Sine-Wave Global Market with Constant Prosumers	36
4.2.3	A Constant Market with Sine-Wave Prosumers	37
4.2.4	Using Synthetic Data for Both Prosumers and the Global Market	39
4.3	The Impact of Batteries	40
4.4	The impact of the Day-Ahead Window and Data-Sharing between Agents	41
5	Conclusion	45

A	Referential Information	51
A.1	VREG Yearly Distribution Tariffs for 2024.	51
B	Experimental results	53
B.1	Overview of the Experiments	53
B.2	With no Batteries, Day-Ahead Window or Data Sharing	54
B.2.1	Cte. Prosumers and Cte. Global Market	54
B.2.2	Cte. Prosumers and a Yearly Oscillating Global Market	60
B.2.3	Cte. Prosumers and a Yearly and Daily oscillating Global Market	67
B.2.4	Yearly Oscillating Prosumers and a Cte. Global Market	74
B.2.5	Using the Synthetic Datasets	80
B.3	With Batteries and no Day-Ahead Window or Data Sharing	86
B.3.1	Cte. Prosumers and a Yearly Oscillating Global Market	86
B.3.2	Cte. Prosumers and a Yearly and Daily oscillating Global Market	90
B.3.3	Yearly Oscillating Prosumers and a Cte. Global Market	94
B.3.4	Using the Synthetic Datasets	98
B.4	With Batteries and Day-Ahead Window and no Data Sharing	102
B.4.1	Cte. Prosumers and a Yearly Oscillating Global Market	102
B.4.2	Cte. Prosumers and a Yearly and Daily oscillating Global Market	106
B.4.3	Using the Synthetic Datasets	110
B.5	With Batteries, Day-Ahead Window and Data Sharing	114
B.5.1	Yearly Oscillating Prosumers and a Cte. Global Market	114
B.5.2	Using the Synthetic Datasets	118

List of Figures

2.1	Schematic overview of a combined energy network	4
2.2	Illustration of the double auction clearing principle	5
2.3	The basic interacting principle between an agent and its environment	6
2.4	Illustration of a basic Markov Decision Process	7
3.1	Action diagram describing the environment's step function	15
3.2	Graphical representation of the internal cleared market and network balancing	17
3.3	Data generated by the double sine wave function	19
3.4	Energy consumption data as used in the experiments	21
3.5	Energy Consumption data of the individual appliances, as used by actor 03	22
3.6	Energy production data as used in the experiments	24
3.7	Historic day-ahead price data as used in the experiments	25
3.8	Action price clipping and hyperbolic tangent price normalisation	26
3.9	Effect of action amount clipping to the remain within the battery limits	27
3.10	Action diagram describing the training loop	28
4.1	Fully cleared market where buyers get the highest bidding price	33
4.2	Offers and internal market clearing price for experiment B.2.1 with rule-based agents . .	34
4.3	Offers and internal market clearing price for experiment B.2.1 with PPO agents	35
4.4	Offers and internal market clearing price for experiment B.2.1 with self-play PPO agents	35
4.5	Offer prices for each agent in experiment B.2.1 with PPO agents	36
4.6	Offers and internal market clearing price for experiment B.2.2 with rule-based agents . .	37
4.7	Offers and internal market clearing price for experiment B.2.2 with PPO agents	37
4.8	Offers and internal market clearing price for experiment B.2.2 with self-play PPO agents	38
4.9	Example of a market amount imbalance due to oscillating energy demands	38
4.10	Double auction with amount imbalance and cte. ask and bid prices 1	39
4.11	Double auction with amount imbalance and cte. ask and bid prices 2	39
4.12	Offers and internal market clearing price for experiment B.2.4 with PPO agents	40
4.13	Offers and internal market clearing price for experiment B.2.4 with self-play PPO agents	40
4.14	Battery usage of individual and self-play PPO agents	42
4.15	Battery usage of individual and self-play PPO agents	43
A.1	VREG Yearly distribution Tariffs for 2024	51
B.1	Mean rewards per epoch for all agents for experiment B.2.1 with rule-based agents . . .	54
B.2	Reward distribution for experiment B.2.1 with rule-based agents	54
B.3	Key metrics overview of experiment B.2.1 with rule-based agents	55
B.4	Mean rewards per epoch for all agents for experiment B.2.1 with PPO agents	56
B.5	Reward distribution for experiment B.2.1 with PPO agents	56
B.6	Key metrics overview of experiment B.2.1 with PPO agents	57
B.7	Mean rewards per epoch for all agents for experiment B.2.1 with self-play PPO agents .	58
B.8	Reward distribution for experiment B.2.1 with self-play PPO agents	58
B.9	Key metrics overview of experiment B.2.1 with self-play PPO agents	59
B.10	Mean rewards per epoch for all agents for experiment B.2.2 with rule-based agents . . .	60
B.11	Reward distribution for experiment B.2.2 with rule-based agents	60
B.12	Key metrics overview of experiment B.2.2 with rule-based agents	61

B.13	Mean rewards per epoch for all agents for experiment B.2.2 with PPO agents	62
B.14	Reward distribution for experiment B.2.2 with PPO agents	62
B.15	Key metrics overview of experiment B.2.2 with PPO agents	63
B.16	Offer prices for each agent in experiment B.2.2 with PPO agents	64
B.17	Mean rewards per epoch for all agents for experiment B.2.2 with self-play PPO agents .	65
B.18	Reward distribution for experiment B.2.2 with self-play PPO agents	65
B.19	Key metrics overview of experiment B.2.2 with self-play PPO agents.	66
B.20	Mean rewards per epoch for all agents for experiment B.2.3 with rule-based agents . . .	67
B.21	Reward distribution for experiment B.2.3 with rule-based agents	67
B.22	Key metrics overview of experiment B.2.3 with rule-based agents	68
B.23	Mean rewards per epoch for all agents for experiment B.2.3 with PPO agents	69
B.24	Reward distribution for experiment B.2.3 with PPO agents	69
B.25	Key metrics overview of experiment B.2.3 with PPO agents	70
B.26	Offer prices for each agent in experiment B.2.3 with PPO agents	71
B.27	Mean rewards per epoch for all agents for experiment B.2.3 with self-play PPO agents .	72
B.28	Reward distribution for experiment B.2.3 with self-play PPO agents	72
B.29	Key metrics overview of experiment B.2.3 with self-play PPO agents	73
B.30	Mean rewards per epoch for all agents for experiment B.2.4 with rule-based agents . . .	74
B.31	Reward distribution for experiment B.2.4 with rule-based agents	74
B.32	Key metrics overview of experiment B.2.4 with rule-based agents	75
B.33	Mean rewards per epoch for all agents for experiment B.2.4 with PPO agents	76
B.34	Reward distribution for experiment B.2.4 with PPO agents	76
B.35	Key metrics overview of experiment B.2.4 with PPO agents	77
B.36	Mean rewards per epoch for all agents for experiment B.2.4 with self-play PPO agents .	78
B.37	Reward distribution for experiment B.2.4 with self-play PPO agents	78
B.38	Key metrics overview of experiment B.2.4 with self-play PPO agents	79
B.39	Mean rewards per epoch for all agents for experiment B.2.5 with rule-based agents . . .	80
B.40	Reward distribution for experiment B.2.5 with rule-based agents	80
B.41	Key metrics overview of experiment B.2.5 with rule-based agents	81
B.42	Mean rewards per epoch for all agents for experiment B.2.5 with PPO agents	82
B.43	Reward distribution for experiment B.2.5 with PPO agents	82
B.44	Key metrics overview of experiment B.2.5 with PPO agents	83
B.45	Mean rewards per epoch for all agents for experiment B.2.5 with self-play PPO agents .	84
B.46	Reward distribution for experiment B.2.5 with self-play PPO agents	84
B.47	Key metrics overview of experiment B.2.5 with self-play PPO agents	85
B.48	Mean rewards per epoch for all agents for experiment B.3.1 with PPO agents	86
B.49	Reward distribution for experiment B.3.1 with PPO agents	86
B.50	Key metrics overview of experiment B.3.1 with PPO agents	87
B.51	Mean rewards per epoch for all agents for experiment B.3.1 with self-play PPO agents .	88
B.52	Reward distribution for experiment B.3.1 with self-play PPO agents	88
B.53	Key metrics overview of experiment B.3.1 with self-play PPO agents.	89
B.54	Mean rewards per epoch for all agents for experiment B.3.2 with PPO agents	90
B.55	Reward distribution for experiment B.3.2 with PPO agents	90
B.56	Key metrics overview of experiment B.3.2 with PPO agents	91
B.57	Mean rewards per epoch for all agents for experiment B.3.2 with self-play PPO agents .	92
B.58	Reward distribution for experiment B.3.2 with self-play PPO agents	92
B.59	Key metrics overview of experiment B.3.2 with self-play PPO agents	93
B.60	Mean rewards per epoch for all agents for experiment B.3.3 with PPO agents	94
B.61	Reward distribution for experiment B.3.3 with PPO agents	94
B.62	Key metrics overview of experiment B.3.3 with PPO agents	95
B.63	Mean rewards per epoch for all agents for experiment B.3.3 with self-play PPO agents .	96
B.64	Reward distribution for experiment B.3.3 with self-play PPO agents	96
B.65	Key metrics overview of experiment B.3.3 with self-play PPO agents	97
B.66	Mean rewards per epoch for all agents for experiment B.3.4 with PPO agents	98
B.67	Reward distribution for experiment B.3.4 with PPO agents	98
B.68	Key metrics overview of experiment B.3.4 with PPO agents	99
B.69	Mean rewards per epoch for all agents for experiment B.3.4 with self-play PPO agents .	100

B.70	Reward distribution for experiment B.3.4 with self-play PPO agents	100
B.71	Key metrics overview of experiment B.3.4 with self-play PPO agents	101
B.72	Mean rewards per epoch for all agents for experiment B.4.1 with PPO agents	102
B.73	Reward distribution for experiment B.4.1 with PPO agents	102
B.74	Key metrics overview of experiment B.4.1 with PPO agents	103
B.75	Mean rewards per epoch for all agents for experiment B.4.1 with self-play PPO agents .	104
B.76	Reward distribution for experiment B.4.1 with self-play PPO agents	104
B.77	Key metrics overview of experiment B.4.1 with self-play PPO agents.	105
B.78	Mean rewards per epoch for all agents for experiment B.4.2 with PPO agents	106
B.79	Reward distribution for experiment B.4.2 with PPO agents	106
B.80	Key metrics overview of experiment B.4.2 with PPO agents	107
B.81	Mean rewards per epoch for all agents for experiment B.4.2 with self-play PPO agents .	108
B.82	Reward distribution for experiment B.4.2 with self-play PPO agents	108
B.83	Key metrics overview of experiment B.4.2 with self-play PPO agents	109
B.84	Mean rewards per epoch for all agents for experiment B.4.3 with PPO agents	110
B.85	Reward distribution for experiment B.4.3 with PPO agents	110
B.86	Key metrics overview of experiment B.4.3 with PPO agents	111
B.87	Mean rewards per epoch for all agents for experiment B.4.3 with self-play PPO agents .	112
B.88	Reward distribution for experiment B.4.3 with self-play PPO agents	112
B.89	Key metrics overview of experiment B.4.3 with self-play PPO agents	113
B.90	Mean rewards per epoch for all agents for experiment B.5.1 with PPO agents	114
B.91	Reward distribution for experiment B.5.1 with PPO agents	114
B.92	Key metrics overview of experiment B.5.1 with PPO agents	115
B.93	Mean rewards per epoch for all agents for experiment B.5.1 with self-play PPO agents .	116
B.94	Reward distribution for experiment B.5.1 with self-play PPO agents	116
B.95	Key metrics overview of experiment B.5.1 with self-play PPO agents	117
B.96	Mean rewards per epoch for all agents for experiment B.5.2 with PPO agents	118
B.97	Reward distribution for experiment B.5.2 with PPO agents	118
B.98	Key metrics overview of experiment B.5.2 with PPO agents	119
B.99	Mean rewards per epoch for all agents for experiment B.5.2 with self-play PPO agents .	120
B.100	Reward distribution for experiment B.5.2 with self-play PPO agents	120
B.101	Key metrics overview of experiment B.5.2 with self-play PPO agents	121

List of Tables

2.1	Overview of auction classifications	10
3.1	Input values used in the market clearing example	16
3.2	Parameter values used to create the synthetic energy production dataset	23
4.1	Overview of the PPO parameter settings used in the experiments	32
4.2	Overview of the general parameter settings used in the experiments	32
B.1	Experiments and their respective codes as used in the code repository	53
B.2	Overview of settings for experiment B.2.1	54
B.3	Overview of settings for experiment B.2.2	60
B.4	Overview of settings for experiment B.2.3	67
B.5	Overview of settings for experiment B.2.4	74
B.6	Overview of settings for experiment B.2.5	80
B.7	Overview of settings for experiment B.3.1	86
B.8	Overview of settings for experiment B.3.2	90
B.9	Overview of settings for experiment B.3.3	94
B.10	Overview of settings for experiment B.3.4	98
B.11	Overview of settings for experiment B.4.1	102
B.12	Overview of settings for experiment B.4.2	106
B.13	Overview of settings for experiment B.4.3	110
B.14	Overview of settings for experiment B.5.1	114
B.15	Overview of settings for experiment B.5.2	118

Chapter 1

Introduction

1.1 Context

Recent decades have been marked by a significant paradigm shift in the management of energy networks. Driven by growing concerns regarding the reliance on fossil fuels and its impact on climate change (Calvin et al., 2023), there has been a growing interest in the energy transition and the imperative transformation of our energy distribution systems, prompting the European Union to include both in its strategy for a carbon-neutral economy (European Commission, 2020).

Traditionally, both electrical and heat network topologies feature a clear distinction between large-scale energy producers connected to a backbone network that distributes energy to smaller local networks, thus interlinking consumers to the energy production units in the backbone network (Pérez-Arriaga, 2013). However, the rise of alternative and primarily local and decentralised energy systems such as photovoltaic (PV) or waste heat installations, heat pumps, home battery systems and electric vehicles (EV) allow local energy market participants to play a more active role within the energy market, dramatically obscuring the clear classical distinction between producers and consumers.

The rise of these so-called "prosumers" (Ancona et al., 2021; Parra-Domínguez et al., 2023) within the energy landscape does, however, pose significant challenges to the current management of the energy markets. In contrast to conventional commodity markets, energy markets are subject to their affiliated energy networks' electrical or thermodynamical limitations. Electrical networks, for instance, must remain in a state of equilibrium at all times such that the total amount of energy consumed is accounted for by an equal amount of energy produced. The integration of highly variable small-scale energy production sites can thus lead to temporal imbalances between energy demand and production, which other market actors must promptly mitigate to guarantee the operational integrity of the electrical network. Furthermore, transformations to the traditional unidirectional network topologies are imperative to attain the growing flexibility and capacity required (Oduro & Taylor, 2023).

One of the solutions proposed to alleviate these challenges is the introduction of combined electricity heat networks (Oduro & Taylor, 2023), the expansion of energy storage facilities such as home batteries (Nadeem et al., 2023) and the establishment of local energy markets or microgrids (Parag & Sovacool, 2016) in which neighbouring producers, prosumers and consumers interact directly within a regional electricity or heat market. While these propositions alleviate the burden on the main energy grid by facilitating the storage and consumption of energy in proximity to its production, they necessitate the implementation of advanced control systems designed to automate the energy exchange between prosumers within a P2P trading platform (Parag & Sovacool, 2016). The development of such control systems has, therefore, emerged as an increasingly significant area of research.

Multi-agent Reinforcement Learning (MARL) has presented a promising approach to address these challenges of developing advanced control systems for local P2P energy markets (Keren et al., 2024; Nadeem et al., 2023). In MARL, multiple agents interact concurrently within a shared environment and learn to execute an optimal policy through trial and error (Albrecht et al., 2024; Gronauer & Diepold, 2022). In the context of P2P energy markets, MARL can be employed to develop and research control systems that enable prosumers to automate their interactions with the P2P energy platform.

In this master's thesis, we propose a novel framework engineered to assess various computational models that engage with a simulated decentralised energy market on behalf of individual prosumers. The framework functions as an intermediary, empowering agents to submit offers specifying their energy

surpluses or shortages at a price of their choice. By implementing a market-clearing mechanism, the framework processes these offers to ascertain a uniform price and settle the market. The framework accommodates multiple configurations for the market environment and participating prosumers to foster a diverse range of experimental arrangements. This includes integrating individual home batteries, formulating mathematical energy demand and market pricing profiles, or integrating existing historical or synthesised datasets within the experimental setup. Designed with extensibility in mind, the framework further permits developing novel market-clearing mechanisms and various agent decision-making algorithms to be evaluated within the framework. To ensure efficient computation and scalability, the framework is developed using Python and JAX, a high-performance numerical computation library. (Bradbury et al., 2018). In order to delimit the scope and mitigate the implementation complexities associated with the mathematical and physical modelling of heat networks, the proposed framework will be limited to the simulation of electrical decentralised energy markets.

1.2 Objectives

The primary objective of this thesis is to design and implement a comprehensive framework that facilitates the simulation of a decentralised energy market environment. This framework will incorporate a mechanism for processing offers submitted by agents, employing a double auction market-clearing approach to determine the clearing price and to determine which offers will be successfully executed. Furthermore, the simulated environment will feature a global market that acts as an interconnection between the microgrid under investigation and the broader energy network. This global market will function as a market participant of last resort, effectively mitigating any energy discrepancies that may arise within the microgrid. In addition to facilitating market transactions, the framework must manage the simulated prosumer entities represented by the agents within the market. This includes determining their current energy demand or surplus and monitoring their battery levels (if applicable). In support of these core functionalities, the framework will provide the necessary infrastructure to log and evaluate agent behaviour, thereby enabling the testing and assessment of various multi-agent reinforcement learning (MARL) implementations within the simulated environment.

To demonstrate the framework’s efficacy, a series of experiments featuring three distinct agent algorithms will be conducted. These experiments will evaluate the ability of each algorithm to learn and converge on a suitable policy that is advantageous to the prosumer they represent. Furthermore, the impact of home battery installations on agent behaviour and performance will be investigated, as the presence of such batteries equips agents with the capability to make more informed long-term decisions by utilising battery capacity to store or deplete energy in response to market price fluctuations.

1.3 Structure of the Thesis

The structure of this master’s thesis is organised as follows. Chapter 2 provides an introductory overview of energy markets, multi-agent reinforcement learning, and the foundational concepts of auctions. In chapter 3, we present a detailed overview of the framework, including its implementation specifics and the design choices made throughout the development process. Chapter 4 provides an extended overview of the empirical results obtained from experiments conducted using the framework. These results illustrate the framework’s workings and potential applications. Finally, chapter 5 concludes the thesis. Here, the findings from the experiments presented in Chapter 4 are summarised, and suggestions for potential future work are made.

Chapter 2

Background

2.1 Energy Markets

2.1.1 Introduction

Classical electrical networks are typically characterised by the presence of one or more generating companies that produce the energy required by the consumers in the network. The energy generated by these companies is distributed throughout the network by transmission and distribution companies, which often maintain a geographical monopoly position to facilitate this service. Retailing companies, as the third party within the network, purchase electrical energy on the market and, subsequently resell it to their affiliated energy consumers connected to the distribution network (Kirschen & Strbac, 2004).

In addition to electrical networks, many districts worldwide utilise heat networks where residual heat, originating from large power plants, waste-to-energy plants, supermarkets, data centres, etc., is transported throughout a pipeline network to local users. The first- and second-generation heat networks used steam and high-temperature water to facilitate energy transport in a closed- or open-loop network. With advancements in piping technology and the incorporation of substations, these systems could evolve into third and fourth-generation heat networks, where the operational temperature could be gradually reduced (Werner, 2017). In contrast to previous generations, fifth-generation district heating networks use a dual piping network in which water circulates at temperatures ranging from 8°C to 20°C and to which decentralised Water-Source Heat Pumps (HP) are connected (Bünning et al., 2018). These modifications enhance the system’s overall efficiency while integrating heat pumps enables users to leverage the network for heating and cooling purposes (Buffa et al., 2019).

Although centralised energy networks have been the prevailing model for many decades, the rise of decentralised energy generation installations, such as solar panels and fifth-generation district heating and cooling systems, has instigated an increasing interest in what is referred to as Distributed Energy Systems (DES) (Idries et al., 2022). Distributed Energy Networks, which may consist of an electrical grid, a thermal grid or a combination of both, enable local energy producers, such as local industries, wind turbines and residences equipped with photovoltaic cells, to make their excess energy available to nearby consumers or use the network to monetise their residual heat (Nadeem et al., 2023). Owners of clean energy installations, whose production is often dependent upon weather conditions or the time of day, can use the network to offer surplus energy to potential local energy consumers or can become consumers themselves when their energy production falls short of meeting their own needs. Participants in Distributed Energy Networks, commonly referred to as prosumers (Qiu et al., 2021), can thus, at any given time, act as energy producers or consumers depending on their individual energy needs. While these DES can, in principle, operate autonomously, they are typically integrated into the primary electrical or thermal network. This integration ensures that the primary network can absorb any energy deficit or surplus within the DES as needed (Bari et al., 2014).

Both heat and electrical DES possess distinct characteristics. In electrical networks, the total amount of electrical energy produced must always be accounted for by an equal amount of energy consumption. Any imbalance within the network may induce the activation of safety systems or require the shutdown of producers or consumers to protect the network and its components (Kirschen & Strbac, 2004). Additionally, standalone local battery installations or bidirectional charging units for electric vehicles may play a crucial role within the network (Xu, 2019). They can serve as a buffer capacity, enabling

the storage of temporary energy surpluses and providing additional energy during peak loads. In the absence of battery installations or when their capacity is insufficient, the connection to the main grid can provide the required buffer capacity to alleviate any energy discrepancies. Due to its extensive scale, The main grid possesses the capacity to manage temporary local energy shortages or surpluses, effectively serving as an inexhaustible resource within the context of the local energy grid¹.

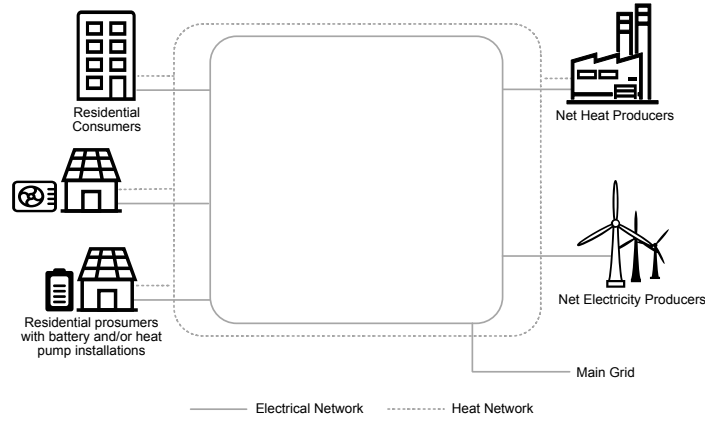


Figure 2.1: Schematic overview of a combined energy network where heat and electricity producers, consumers and prosumers are connected through a local Combined Decentralised Energy Network.

In contrast, heat networks offer greater flexibility in balancing the energy produced and consumed. When heat energy consumption temporarily exceeds the contributions from heat producers, the total energy and, consequently, the temperature within the network will decrease. Conversely, a temporary surplus in heat production will result in an increase in both the total energy and temperature of the network. While it is essential to maintain temperature fluctuations within established operational thresholds to ensure the integrity of the network, a heat network with a sufficiently large capacity relative to the present producers and consumers typically does not encounter the same specific balancing challenges as electrical energy networks. Thus, unlike electrical networks, which require battery installations or reliance on the national grid to buffer imbalances, the heat network functions as a buffer, effectively absorbing fluctuations in heat production and consumption. To address any seasonal oscillations or fluctuations that the network cannot absorb, heat storage units or geothermal buffers can be implemented within the system (Boesten et al., 2019; Buffa et al., 2019).

2.1.2 Energy Trading

As previously described, four main parties operate in classical centralised electricity networks: consumers, generating, transmission/distribution and retailing companies. Retailing companies acquire electricity from generating companies and subsequently resell it to their individual customers through long-term contracts that stipulate a fixed or variable price. Transmission and distribution companies are responsible for the physical infrastructure and the distribution of electricity from power plants to consumers. They typically operate under regulations imposed by a governmental agency which acts as the network regulator. This regulatory body often determines the tariffs that transmission or distribution companies are permitted to charge for utilising their network. Conversely, electrical energy trading between generating and retailing companies or large consumers is frequently conducted within a free energy market. To facilitate this trading process, two additional parties are required. The first is an independent market operator tasked with ensuring the accurate execution of transactions within the market. The second party is the Transmission System Operator (TSO), which must ensure the network remains balanced and acts as a broker of last resort (Kirschen & Strbac, 2004). How these markets are organised can differ significantly from country to country, both in the extent to which the markets

¹It should be noted that this assumes a sufficient and quickly deployable production capacity available within the main network to accommodate any consumption peaks at the local level. Furthermore, the main network will only be able to absorb and redistribute a finite amount of excess energy produced from the local networks. We will, however, always assume that these capabilities are guaranteed for the remainder of this thesis.

are liberalised and in how supply and demand are matched throughout the individual markets (Maurer et al., 2011). Although it is beyond the scope of this thesis to provide a comprehensive analysis of how electricity markets can be organised, it is important to provide a general overview of the mechanisms involved in facilitating transactions and maintaining network balance. First and foremost, buyers and sellers in the electricity market can engage in long- or short-term bilateral contracts, which establish a fixed price for the supply of a specified amount of energy at a designated time. These contracts allow the participants to mitigate the risks associated with excessive market fluctuations and are essential to managing possible price risks (Cramton, 2017). In addition to these over-the-counter transactions, many regions feature regulated futures markets where market participants and speculators can trade standardised energy futures and options (Meeus, 2020).

Although future markets and bilateral agreements allow participants to hedge any potential pricing risks, they do not account for unforeseen circumstances, such as changing weather conditions or the unexpected failure of an electrical production unit. Consequently, an additional spot market, which consists of the day-ahead and intraday markets, is created. The market operator manages the day-ahead market, offering both generating and retailing companies the opportunity to make an offer for each time slot of the next day through a mechanism commonly referred to as a double auction (Hubbard & Paarsch, 2015). The market operator subsequently aggregates the offers, which include both the amount of energy and the corresponding asking or bidding price, from generating and retailing companies into two distinct pools. The pool of offers from the generating companies will be sorted in ascending order, while those from the retailing companies will be sorted in descending order. These offers can be graphically represented as shown in figure 2.2, with the former forming the supply curve and the latter the demand curve. The intersection of the two curves subsequently determines the market equilibrium. This market equilibrium will dictate which offers are accepted by the market in such that for every unit of energy demanded on the left side of the equilibrium, there exists a corresponding energy supply for which the asking price is lower than what the retailing company offered and vice versa. Furthermore, the equilibrium establishes the uniform market price for executing these accepted offers. By utilising a uniform market price for all accepted offers, retailing companies are assured they will never pay a price higher than their original offer while generating companies will receive a price that is equal or greater than their initial bidding price (Pérez-Arriaga, 2013).

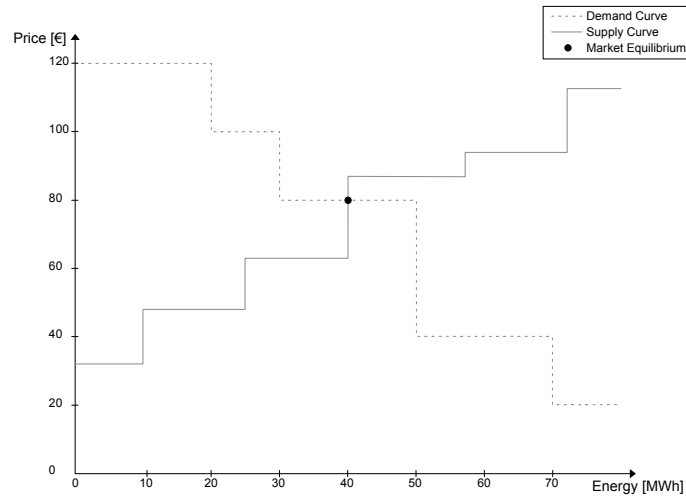


Figure 2.2: Illustration of the double auction clearing principle.

The application of a uniform market price may initially appear counterintuitive, as generating companies receive a price that exceeds their initial offer. However, by using this method of price determination, an incentive is created for generating companies to set their offers at the marginal cost of electricity production. By doing so, they enhance their likelihood of securing a buyer in the market and can capture the difference between the marginal cost and the market price as profit (Kirschen & Strbac, 2004).

The day-ahead market enables both generating and retailing companies to reconcile any discrepancies between the previously established long-term contracts and the short-term forecasts for the next day. Once the market closes, the market operator collects all offers and establishes the market equilibrium for each time slot. Subsequently, the System Operator will collect all long-term positions, the day-ahead

offers, and the associated market equilibria to assess the overall balance of the network. To address any imbalances, the System operator uses the intraday market. In this market, generating companies are permitted to offer deviations from their original positions at a price they determine, which can be activated on short notice at the request of the system operator. Similarly, large consumers can propose adjustments to their consumption levels at a price of their choosing. The system operator can then use this offered capacity at its own discretion to guarantee the network balance at all times (Kirschen & Strbac, 2004).

The preceding description clearly illustrates that consumers, although represented by the retailing companies, do not actively participate in the traditional electricity market. However, the rise of photovoltaic solar panel installations has significantly altered this dynamic. Private owners of these installations have transitioned from being mere consumers to participating, albeit on a small scale, as energy producers. In response, many countries have introduced the concept of Feed-In-Tariffs (FIT), which determine the sales price of energy injected into the electricity network by prosumers (Bukar et al., 2023). Further innovations, such as household battery installations, heat pumps and even demand-side management systems capable of controlling household appliances based on current energy prices have underscored the need for a more extensive participation of the prosumer in the market (Bari et al., 2014; Nadeem et al., 2023). This shift has, in turn, generated increased interest in decentralised energy trading (Bukar et al., 2023).

2.2 Single-Agent Reinforcement Learning

2.2.1 Introduction

To facilitate the participation of actors within a decentralised energy market, it is crucial to develop mechanisms that enable the automatic engagement of such actors with the market. We thus need a control system capable of making advantageous decisions in real-time, thereby serving the interests of both the actor or even the broader network. Reinforcement learning, a promising domain within machine learning, has gained significant momentum in recent years due to its efficacy in making successful decisions within complex environments. Notable achievements in this field include the development of algorithms that have mastered Atari games (Mnih et al., 2015), defeated the world champion in the game of Go (Silver et al., 2016), and controlled robotic hands to solve a Rubik's Cube (OpenAI, Akkaya, et al., 2019). In each of these instances, the algorithms demonstrated the ability to learn and select the best possible actions based on the current state of their respective environments.

Within reinforcement learning, an agent interacts with a stochastic environment and pursues the maximisation of its expected return G_t by optimising its internal policy $\pi(a|s)$, which elects the action to take given the current state of the environment (Sutton & Barto, 2018). This agent-environment interaction can be visualised as shown in Figure 2.3.

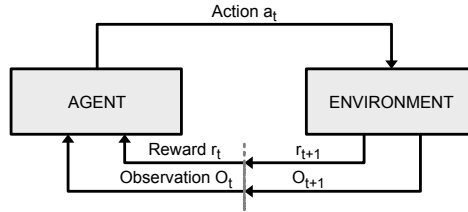


Figure 2.3: The basic interacting principle between an agent and its environment

In each time step t , the agent selects an action $A_t \in A(s)$ that, once executed, changes the environment's state S_t to S_{t+1} . In conjunction with the state change, the action leads to a numerical reward signal R_{t+1} , which is presented to the agent together with an observation O_{t+1} derived from the environment's new state S_{t+1} . Depending on the environment, this mutual feedback loop can continue indefinitely or end if the environment enters a terminal state.

$$p(s'|s, a) = \Pr\{S_{t+1} = s' | S_t = s, A_t = a\} \quad (2.1)$$

$$r(s, a) = \Pr\{R_{t+1} | S_t = s, A_t = a\} \quad (2.2)$$

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.3)$$

The environment can be formulated as a Markov Decision Process (Sutton & Barto, 2018) defined by a tuple (S, A, p, r, γ) for which S and A are the environment's state and action space and p and r are the state-transition and reward function, respectively. As illustrated in Figure 2.4 the environment will uphold a distinct state s in each timestep t . Whenever an action $a \in A(s)$ is invoked upon the environment, the state-transition function p (2.1) and the reward function (2.2) will determine the next state s' and the accompanying reward r respectively, given action a and the current state s . It should be noted that the result of the function p is independent of past state trajectories, an attribute often referred to as the Markov Property (Sutton & Barto, 2018).

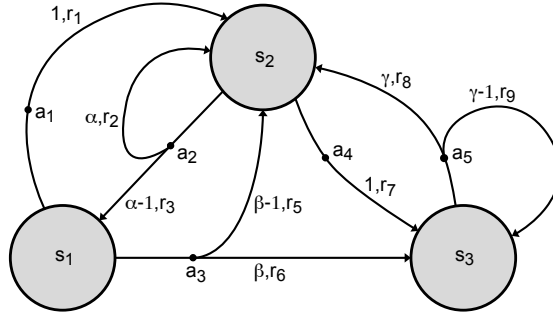


Figure 2.4: Illustration of a basic Markov Decision Process.

Finally, the return as defined in (2.3) includes a parameter $\gamma \in [0, 1]$, often referred to as the discount rate. When set to a value close to zero, the agent's policy will prioritise immediate rewards while disregarding rewards in a more distant future (Sutton & Barto, 2018).

2.2.2 Proximal Policy Optimisation

As stated in paragraph 2.2.1 a crucial element within the agent-environment interaction loop is the agent's capacity to learn to improve its policy π . Although many reinforcement learning algorithms have been developed, we specifically introduce Proximal Policy Optimization (PPO) (Schulman et al., 2017) as it is used within the experiments described in chapters 3 and 4.

As a policy gradient method, PPO optimises its policy through stochastic gradient ascent in which policy weights are adjusted so that they move the policy in the direction with the highest expected cumulative reward. While classical policy gradient methods use (2.4) as their objective function, they carry the inherent risk of overstepping the policy weight update and thus inflicting significant changes to the underlying policy (Albrecht et al., 2024). In order to mitigate this concern, PPO incorporates the concept of trust regions in which a surrogate objective function (2.6) with ratio (2.5) is used (Schulman et al., 2017).

$$L^{PG}(\theta) = \mathbb{E}_t \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right] \quad (2.4)$$

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \quad (2.5)$$

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2.6)$$

The ratio, in which $\pi_{\theta_{old}}$ denotes the old policy and π_{θ} represents the updated policy, serve as a measure of the deviation between these two policies. Using the objective function (2.6), ratios who exceed the interval $[1 - \epsilon, 1 + \epsilon]$ will be clipped to remain within these set boundaries and thus effectively mitigates the risk of inducing excessive policy updates (Schulman et al., 2017).

2.3 Multi-Agent Reinforcement Learning

2.3.1 Environments

With the incorporation of multiple agents, environments can be classified along two distinct axes. A primary axis differentiates environments based on the mutual alignment of the agent’s objectives (Albrecht et al., 2024; K. Zhang et al., 2021). In a cooperative or common-reward setting, the agents’ objectives mutually align by sharing a common reward function. In contrast, fully competitive settings are often modelled as zero-sum games in which agents pursue opposing objectives. Finally, general-sum or mixed settings are distinguished by the absence of any inherent restriction regarding the agents’ objectives.

The secondary axis hierarchically discerns environments, with complexity serving as its primary ordering criterion (Albrecht et al., 2024). The absence of any state transitions characterises normal-form games which are solely defined by (I, A, R) , in which $I = \{1, 2, \dots, n\}$ is the finite set of agents, $A = \{A_1, A_2, \dots, A_n\}$ the finite set of actions for each agent $i \in I$ and $R_i : A \rightarrow \mathbb{R}$ is the reward function.

Stochastic or Markov games introduce the aspect of state transitions such that the environment’s state can evolve depending on the agents’ actions. Formally, stochastic games can be defined by (I, S, A, R, T) with $I = \{1, 2, \dots, n\}$ as the finite set of agents, S the finite set of states, $A = \{A_1, A_2, \dots, A_n\}$ the finite set of actions for each agent $i \in I$, a reward function $R_i : (S \times A \times S) \rightarrow \mathbb{R}$ and the state-transition function $T : S \times A \times S \rightarrow [0, 1]$.

Finally, Partially Observable Stochastic Games (POSG) differ from Stochastic games in that the latter fully discloses the environment state to the agents, while the former introduces the concept of partial observability by providing agents with individual observations that do not fully represent the actual state of the environment. A formal definition can be formulated by extending the Markov game definition by a finite set of observations O_i and an observation function $O_i : A \times S \times O_i \rightarrow [0, 1]$.

2.3.2 Training and Execution Modes

Introducing multiple agents gives rise to three variations of training methods applicable within a MARL setting (Albrecht et al., 2024). Within a centralised training and execution method, the different agents share their observations collectively with a centralised controller, resulting in one global policy guiding all agent’s actions and thus effectively reducing the multi-agent problem to a single-agent variant. A significant drawback of this type of training mode resides in the fact that the centralised controller has to obtain the most optimal policy from within the joint action space, which grows exponentially with the number of agents (Wong et al., 2023). Moreover, real-life applications of such training and execution methods are limited as they require some form of communication mechanism allowing the agents to dispatch their observations to the centralised controller in return for their selected action (Albrecht et al., 2024).

Secondly, in a fully decentralised training and execution mode, no information is shared between agents. This implies that agents must select their actions and optimise their policies based on the limited information encapsulated within their set of observations. Due to these criteria, decentralised training and execution is a well-suited training strategy when confronted with isolated agents unable to share information (Albrecht et al., 2024). The individual PPO agents will use this training method during the experiments as described in chapter 4.

As a final training and execution strategy, centralised training with decentralised execution combines a centralised learning strategy while maintaining a decentralised action selection process. This is the training strategy used by the self-play PPO agents during the experiments. More specifically, information, such as observations, is shared between agents during training, while agents can select their actions solely based on their own individual observations.

2.3.3 Challenges

With respect to single-agent reinforcement learning, multi-agent environments introduce numerous new challenges which must be addressed to facilitate the agents’ effective learning (Wong et al., 2023). The first challenge that impedes effective learning is that all agents learn and interact with the environment simultaneously. This results in the environment’s reward and transition functions becoming non-stationary from the agents’ perspective (Gronauer & Diepold, 2022), and thus, the environment

state is no longer sufficient in providing all the required information to allow the agents to engage in optimal decision-making (Wong et al., 2023).

A second challenge resides in the partial observability of the environment’s state. When agents are subject to partial observations of the state of their environment, agents become inadequate in attributing the state transition to their actions (Wong et al., 2023).

Thirdly, to improve their policy, agents must be able to assign the impact of their actions to the reward received by the environment. This, however, becomes non-trivial when multiple agents interact with the environment simultaneously and thus leads to what is referred to as the credit assignment problem (Gronauer & Diepold, 2022).

A final, albeit more practical challenge resides in the computational complexity associated with the concurrent training of multiple agents. Reinforcement learning algorithms generally suffer from low sample efficiency, necessitating a high amount of interactions with the environment to collect a sufficient amount of data samples, a shortcoming which only exacerbates when introduced in a multi-agent environment (Wong et al., 2023).

2.4 Auctions

2.4.1 General

Throughout history auctions have played a crucial role in various economic and financial markets. They provide a highly structured and regulated process through which goods or services can be allocated (Hubbard & Paarsch, 2015; Peters, 2015) and transparently offer a means to what is often referred to as "price discovery" (Hubbard & Paarsch, 2015).

Auctions can be conceptualised as games characterised by specific features. At the core of each auction lies the product or service being offered, alongside a set of market participants denoted as $N=1,2,3,\dots,n$. Each participant, whether acting as a buyer or a seller, possesses a valuation or valuation function v_i , which quantifies the intrinsic value of the product or service to that individual. While participants have complete knowledge of their valuation, they remain uninformed about the valuation functions of other participants within the auction. Next to their valuation, each participant will require a strategy s_i which maps its valuation to an actual price p_i as shown in (2.7). Finally, we define a utility function $u_i \Rightarrow \mathbb{R}$, which maps an auction outcome to a value which serves as a means to validate the preference for specific outcomes (Mashler et al., 2013).

$$s_i(v_i) = p_i \quad (2.7)$$

As auctions can be formulated as games, game theoretical concepts such as dominant strategies and Nash-equilibria can be applied to auctions. A dominant strategy as formulated in (2.8) in which s_i is the strategy played by the player i and for which s_{i-1} represent the strategies used by all other players. A strategy vector S is considered a dominant strategy solution if requirement (2.8) holds for each player i and each alternative vector $s - 1$.

$$u_i(s_i, s'_{-i}) \geq u_i(s'_i, s'_{-i}) \quad (2.8)$$

Similarly, we can include the formal definition for a pure strategy Nash Equilibrium in which no player can improve its strategy while all other players’ strategies remain as defined in (2.9) (Mashler et al., 2013).

$$u_i(s_i, s_{-i}) \geq u_i(s'_i, s_{-i}) \quad (2.9)$$

Auctions can be classified based on two distinct metrics (Hubbard & Paarsch, 2015; Mashler et al., 2013). A first class is delineated by the auction format, which can be open or sealed-bid. In auctions following the open format, all offers are disclosed to the auction participants and are usually conducted orally. Conversely, in sealed-bid auctions, auction offers are only disclosed to the broker to establish the winner and the prevailing auction price. Auction participants, therefore, are unaware of their opponents’ strategies. A second classification method emphasises the pricing rule used within the auction for which a first price results in the auction’s winner paying what was bid (Hubbard & Paarsch, 2015). With a second-price rule, the winner is only obliged to pay the one-but-highest bid. This classification leads to an overview of four well-known auction archetypes, as shown in Table 2.1.

	First-price	Second-price
Open	Dutch	English
Sealed-bid	Pay-your-bid	Vickrey

Table 2.1: Overview of auction classifications

To illustrate the influence of auction configurations on the behaviour of agents and the resulting outcome, we will compare the Vickrey or sealed-bid second-price auction to its sealed-bid first-price counterpart. In a sealed-bid first-price auction, there is no dominant strategy for the bidders, as they can only speculate on their opponents' strategies. This uncertainty can lead to suboptimal outcomes, as bidders may hesitate to submit their true valuation for fear of overpaying. In contrast, the Vickrey auction mitigates this issue by awarding the highest bidder with the product at the second-highest bid price. This design thus encourages players to submit their true valuation and consequently defines a dominant strategy (Mashler et al., 2013).

2.4.2 Double Auction

As discussed in paragraph 2.1.2, double auctions facilitate buyers and sellers to place their offers within the market. To illustrate this behaviour, consider a double auction scenario involving one seller and one buyer, with respective valuations denoted as v_s and v_b . Both valuations are considered to be independently drawn out of the uniform interval $[0, 1]$. The market price clearing mechanism operates under the principle that when the buyer's valuation exceeds the seller's, the average of the two valuations is established as the market price. This scenario will result in an expected utility payoff for the buyer represented in (2.10) and the seller in (2.11) (Peters, 2015).

$$\left[v_b - \frac{p_b + E[p_s(v_s)|p_b \geq p_s(v_s)]}{2} \right] \text{Prob}[p_b \geq p_s(v_s)] \quad (2.10)$$

$$\left[\frac{p_s + E[p_b(v_b)|p_s \leq p_b(v_b)]}{2} - v_s \right] \text{Prob}[p_s \geq p_b(v_b)] \quad (2.11)$$

2.5 Related Work

The growing interest in both reinforcement learning and decentralised energy networks has resulted in a growing body of work, leading to the proposal of various inspiring solutions, such as: Bünning et al., 2018 successfully introduced a fundamental agent control system within low-energy bidirectional heat systems, employing a cascading control framework that integrates classical control theory with agent-based systems to regulate set point temperatures within the network. Harrold et al., 2022 developed a multi-agent actor-critic control unit to manage different network battery systems, utilising a centralised learning and decentralised execution training methodology. He et al., 2021 proposed a peer-to-peer energy trading framework governed by a central energy pawn, which can manage a centralised battery installation within the sharing network to address potential energy fluctuations. Khorasany et al., 2017 developed a transactive energy management system that facilitates direct connections between prosumers and consumers for energy trading, incorporating an auction-based approach to clear the market with the additional introduction of a subscription charge to incentive prosumers to reduce their reliance on the energy network. Liang and Su, 2018 initiated their analysis with a game-theoretical approach to examine and develop the market characteristics necessary for effectively managing prosumer participation. Mumbere et al., 2023 proposed a synthetic prosumer model, incorporating an integrated energy management system simulator facilitating the coordination of prosumer components and enabling flexible switching mechanisms. Calvano et al., 2019 demonstrated that Q-learning pricing algorithms exhibited a propensity for systematic collusion, which was generally maintained through temporary punitive actions for deviations, facilitating a gradual return to prior pricing levels. These algorithms acquired the ability to execute such strategies via trial and error, operating without prior knowledge of their environment, and did not communicate or coordinate with one another. Sun et al., 2023 used a data-driven hybrid learning approach to facilitate real-time hierarchical energy sharing among building prosumers using both XGBoost to characterise the price-based demand response behaviours of prosumers, facilitating the estimation of online peer-to-peer energy sharing outcomes among them.

Tushar et al., 2020 investigated a peer-to-peer energy trading scheme that leverages social cooperation among various prosumers within the network by creating a coalition formation game that enables each participating prosumer to opportunistically determine whether to engage its battery in the peer-to-peer market for energy trading. Wang et al., 2023 introduced a rolling horizon optimisation-based real-time operational strategy that comprehensively addresses the implications of peer-to-peer energy trading and the inherent uncertainties associated with renewable energy sources. Vázquez-Canteli et al., 2020 introduced CityLearn, an OpenAI Gym environment, along with a simulated framework designed to implement reinforcement learning in demand response and urban energy management.

Chapter 3

Implementation of the Framework

3.1 Introduction

In order to accommodate a wide range of experiments in a multi-agent decentralised energy market environment, the proposed framework must meet several key requirements. These include accommodating various configurations for the production and consumption patterns of market participants, incorporating global market price fluctuations, and supporting the use of battery systems by the participating market actors. We have developed the necessary codebase to facilitate the execution of these experiments, which involved implementing the market environment, market actors, and agents representing these actors. Subsequent sections of this chapter will thus provide a detailed account of how these components were implemented¹.

The code repository was developed using Python 3.11 and the Google JAX numerical computing framework (Bradbury et al., 2018). JAX enables both automatic vectorisation and the application of a specific just-in-time (JIT) compiler decorator to designated functions, ensuring that all code within these functions is compiled to an XLA-compatible format (Bradbury et al., 2018). Through the XLA compiler, a high-performance execution of the compiled functions can be achieved on various hardware platforms such as GPUs (OpenXLA, n.d.). The application of the JAX JIT compiler does, however, impose certain limitations to the implementation of the concerning functions (JAX, 2024). Firstly, any function that is subject to the JIT compiler must be functionally pure. Consequently, JAX arrays, which represent the fundamental data type within the JAX framework, are immutable, and thus, any function designed to modify these arrays should operate out of place. Moreover, it is imperative to exercise caution when accessing elements within a JAX array, as the framework automatically clamps indices to remain within the array’s boundaries without raising any out-of-bound exceptions. Lastly, it is essential to recognise that the use of standard Python control flow syntax is significantly restricted when employing the JIT compiler. Therefore, it is advisable to utilise the specific control flow primitives provided by JAX to manage execution flow during runtime effectively (JAX, 2024).

3.2 Implementation of the Environment

3.2.1 General

A concrete market environment was implemented to allow the various agents to interact with the market and enable them to place the required offers. This market environment is tasked with iteratively processing the agents’ offers, clearing the internal market, and thus determining the prices and quantities of energy traded between the agents during each iteration. Offers that cannot be fully or partially executed within the internal market must subsequently be transacted on the global market to ensure the network’s equilibrium. This must happen at the applicable Time-of-Use price (i.e., the price to buy from the global market) and Feed-In price (i.e., the price at which one can sell energy to the global market) of the global market. Furthermore, the market environment must account for any discrepancies between the energy traded by the agents and their required or produced energy quantities. After all,

¹The source code of the framework can be consulted at:
<https://github.com/KennethEmerson/JAX-Framework-for-Multi-Agent-Research-in-Decentralised-Energy-Markets>

the agents' offers may not fully align with their actual energy needs. Additionally, some agents may have access to a battery installation that they can use as a buffer to meet their energy needs or store any surpluses. The market environment must consider these factors during the final processing step. Therefore, any current surpluses or shortages will first be addressed by the agent's battery installation. If the battery's minimum or maximum capacity is exceeded or no battery is available to the agent, the remaining surplus or deficit will be traded on the global market to compensate for this discrepancy and maintain the balance in the energy network.

After processing all necessary information, the market environment can calculate the reward obtained by each agent during the iteration. The reward assigned to each agent is defined as the total revenue or cost for the energy traded on both the internal and global markets. Specifically, a positive reward represents a yield, while a negative value represents the cost of the purchased energy. It is important to note that any energy discrepancies that must invariably be addressed on the global market will most likely result in a penalty for the agent. After all, it is fair to assume that the global market consistently offers energy at a higher price than what agents could have secured in the internal market and will acquire any unallocated surpluses at a price lower than what agents could have achieved by selling the energy to other agents.

In addition to calculating the rewards, the environment must compute the new battery levels for each agent based on their current battery level, the associated maximum storage and peak capacity of the battery, their actual energy demands and the energy quantities traded on both the local and global market, including any energy discrepancies.

This process ultimately enables the environment to ascertain its new state $\mathcal{S}$, including the adjusted battery level values (BL), the agents' energy requirements (ED), and the Time-of-Use (toup) and Feed-In (fip) prices established by the global market for the subsequent iteration. Additionally, the state includes two counters that indicate the current epoch (e) and timestep iteration (t) of the environment, a day-ahead window (DAW) containing the future day-ahead prices of the global market within a preconfigured horizon and the maximum potential energy production (MEP), energy storage capacity (MBC) and maximum (dis)charge capacity (MBPC) for each agent.

$$\mathcal{S} = \langle e, t, BL, toup, fip, DAW, ED, MEP, BL, MBC, MBPC \rangle \quad (3.1)$$

As a final step, the environment uses the new state to create the necessary observations $\mathcal{O}_x$ for each agent. By differentiating between the actual new state and the observations accessible to each agent, we can ensure that agents receive only the information relevant to their circumstances. Consequently, this results in a state that is only partially observable to individual agents. The observations will encompass several parameters, including the agent's energy demand (ED_x), current battery level (BL_x), maximum energy production capacity (MEP_x), maximum battery capacity (MBC_x), and the corresponding maximum peak power capacity ($MBPC_x$). Although the latter three parameters remain constant, they may facilitate the agent in making more informed and optimal decisions and thus are included in the observations. Additionally, the observations incorporate a day-ahead window (DAW) provided by the global market. This configurable array of day-ahead prices offers the agent an insight into the anticipated evolution of the global market in subsequent iterations. Both the size of the day-ahead window and the hour of the day at which the global market will refresh this day-ahead window can be set as a global market configuration parameter when the environment is created. Lastly, the observations provide the agent with the week's number² (w), ranging from 0 to 52, the hour of the day (hod), and the day of the week (dow), which spans from 0 to 6. These elements might offer the agent additional context and help detect potential temporal patterns within the evolution of its energy demand and/or the global market's prices. It is, however, important to note that any agent participating within the environment will only have their individual observations at their disposal to select the actions for the following timestep; in other words, from any agent's perspective, the environment is only partially observable.

$$\mathcal{O}_x = \langle w, hod, dow, toup, fip, BL_x, ED_x, MEP_x, BL_x, MBC_x, MBPC_x, DAW \rangle \quad (3.2)$$

At the end of each iteration, the environment determines whether the epoch has ended based on the state's counter. This counter will be compared against a preconfigured parameter, allowing for flexibility in the duration of the epochs. However, it is important to note that there is a limitation. For both agents and the global market, the number of steps is constrained by the availability of data regarding

²The implementation uses the standard ISO calendar format for which a year can contain 51 or 52 weeks and in which the first week including a Thursday is considered the first week of the year (Python Software Foundation, 2024).

energy requirements or Time-of-Use and Feed-In pricing. Once either of these thresholds is reached, the environment will signal the conclusion of the epoch, necessitating the initiation of a new one.

3.2.2 Implementation

The experimental environment utilised within the framework is encapsulated within the file *energy-MarketEnv.py* and features a class named *DoubleAuctionEnv*, which is a child class of the standard *MultiAgentEnv* class provided by the JaxMarl library (Rutherford et al., 2023). Through this inheritance, we ensure that our environment adheres to a standardised application programming interface (API) that is consistent with several existing multi-agent libraries (Lange, 2022; Rutherford et al., 2023; Terry et al., 2021). Our *DoubleAuctionEnv* class thus contains all required methods for managing the environment and facilitating the agent interactions.

Upon instantiation, the class requires several parameters to be provided, such as the maximum steps in one epoch, a *GlobalMarket* named tuple containing all relevant global market data and a list of *Prosumer* tuples which in their turn contain all the actors' specific information regarding their energy demand data, battery characteristics, maximum energy production, etc. Once the object is created, two main methods are available to interact with the environment. The first method, referred to as *reset*, (re)initialises the environment to its original starting state by generating a new *EnvironmentState* object and the corresponding observations. The second method, *step*, requires the current *EnvironmentState* object and a dictionary, mapping the agents to their respective actions, as input. It subsequently uses both inputs to execute all necessary calculations, as described in the previous section 3.2.1 and as visualised in Figure 3.1, which results in the generation of the subsequent state and observations. To enable intermediate tracking of the agents' performance, the method produces an *info* dictionary, which contains logging information for the post-evaluation of the experiment and which is never presented to the agents. Ultimately, the state, observations, rewards, and info dictionary are returned by the *step* function for further processing upstream.

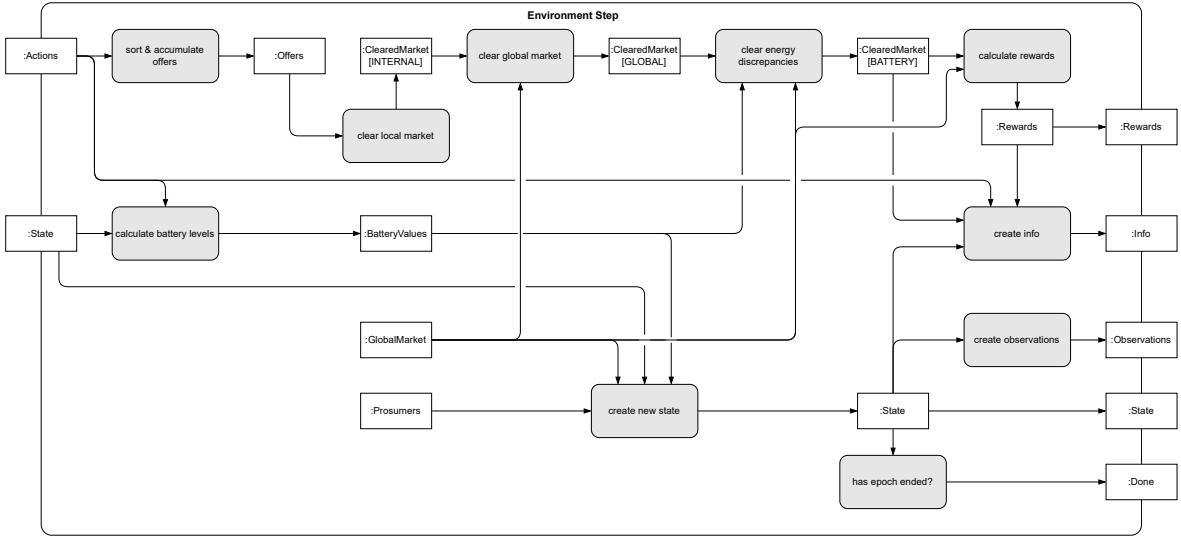


Figure 3.1: Action diagram describing the environment's step function

As described above, the *step* procedure uses several auxiliary functions to facilitate its execution, with a first set of these functions located within the module *ledger.py*. This module distinguishes offers between asks and bids based on the sign of their energy amounts, wherein bid amounts are represented as negative and ask amounts as positive values. As such, it converts actions received from the agents into a named tuple, referred to as *Offers*. This named tuple contains two two-dimensional arrays, called *bids* and *asks*, in which all agent sales and purchase offers are stored, respectively. Each array holds three columns, which include the agent's index, the price, and the amount of energy offered. To ensure that the array sizes are constant and equal to the number of actors, the bid of a specific agent is set to zero whenever the agent submits an ask, and vice versa.

The *clear_market* function will start by ordering and subsequently accumulating all asks and bids stored in the *Offers* tuple. It will create an internally used tuple *IntermittentClearingValues*, which will

contain all temporary data required throughout the clearing process of the local market, the ordered and accumulated offers included. Once this tuple is created, an iterative loop will be triggered. This loop will try to link an ask to a bid in each step, starting with the highest and lowest, respectively, and taking any potential differences in the requested and supplied energy quantities into consideration. When a bid is insufficient to completely satisfy the demand specified in an ask, the bid is fully cleared, while the ask retains the remaining energy amount. Conversely, if an ask cannot fulfil the entirety of a bid, the ask is fully cleared, and the bid remains, holding the outstanding energy amount. A counter for both the asks and the bids is contained within the *IntermittentClearingValues* tuple and will only be incremented when the respective ask or bid has been fully processed. This way, the loop will continue to process all offers until one of the arrays is exhausted or when the price of the next bid exceeds the price of the next ask. Once all potential offers are matched, the function generates a *ClearedMarket* tuple, which includes the clearing price and volume and six arrays, of which four contain the cleared and the remaining uncleared bids and asks. The two final arrays, although completely filled with zeros, represent the battery shortages and overflows and will be used in a subsequent stage to mitigate any energy discrepancies. After the internal market is cleared, the environment calculates any actors' battery shortages or excesses upon which the module *globalMarket* is invoked to clear the remaining outstanding offers and energy discrepancies on the global market.

A final aspect that requires further examination is how the environment determines the new battery levels and any energy shortages or surpluses that are not accounted for by the energy stored or transacted by the agent. During the initialisation, the environment is provided with a list of *Prosumer* tuples, each corresponding to an individual actor. These tuples contain all relevant parameters and data, including the maximum energy production, the maximum battery capacity, its maximum peak power capacity, and the actual energy demand or surplus generated by the actor. During each iteration, the environment calculates the delta between the offered amounts (notably, this value is negative when energy is sold) and the actual energy demand (which is also negative when there is a surplus of energy produced) for each agent. This delta thus corresponds to the discrepancy between the amount of energy exchanged on the market and the actual amount of energy required, which will need to be mitigated by the battery. However, the amount of energy the battery can account for is limited by both its capacity and maximum peak power capacity. When the battery level falls below zero, or the required amount of energy to extract from the battery exceeds the maximum (dis)charge rate, procuring additional energy from the global market becomes necessary to compensate for the energy not provided by the battery. Conversely, if the new battery level exceeds the maximum capacity or the required amount of energy to inject into the battery exceeds the maximum (dis)charge rate, surplus energy must be sold on the global market.

To illustrate the market clearing procedure, a fictitious market scenario involving four agents is presented in Table 3.1 and Figure 3.2. Each agent operates with a battery that is 50% charged, possessing a total capacity of 200 watt-hours (Wh) and a maximum (dis)charge rate of 50 Wh.

Agent identifier	energy demand [Wh]	battery level [Wh]	Offer price [€]	Offer amount [Wh]
agent 00	-100	100	0.0001	-100
agent 01	150	100	0.0004	200
agent 02	-200	100	0.0003	-150
agent 03	200	100	0.0005	-200

Table 3.1: Input values used in the market clearing example where negative energy demand values represent an energy excess and negative offer amounts express an agent's bid to sell energy. All agents have a 50 % charged battery with a capacity of 200 Wh and a maximum (dis)charge rate of 50 Wh at their disposal.

At the start of the iteration, the environment clears the internal market, resulting in Agent 01 purchasing all the energy it requested through bids submitted by Agent 00 and Agent 02. While Agent 00's bid is fully cleared, Agent 01's offer is only partially fulfilled, culminating in a totally cleared amount of 200 Wh at a price of €(0.0030) per Wh. Following the market clearing results, the environment must balance the energy amounts, as depicted in the right-hand plot of Figure 3.2. For Agent 00, the overview indicates that the agent placed a bid that precisely matched its actual energy surplus of 100 Wh, which was fully cleared on the internal market, thereby maintaining its battery level at a 50% charge. Conversely, Agent 01's offer to buy 200 Wh was completely cleared on the internal market, surpassing its energy demand of 150 Wh by 50 Wh. This surplus does not exceed the battery's maximum charging rate or additional capacity, allowing the excess energy to be stored in the battery. Agent 02's bid of 150 Wh was only partially cleared on the internal market for 100 Wh, necessitating that the remaining 50 Wh be cleared through the global market. Notably, Agent 02's bid did not account for the total 200

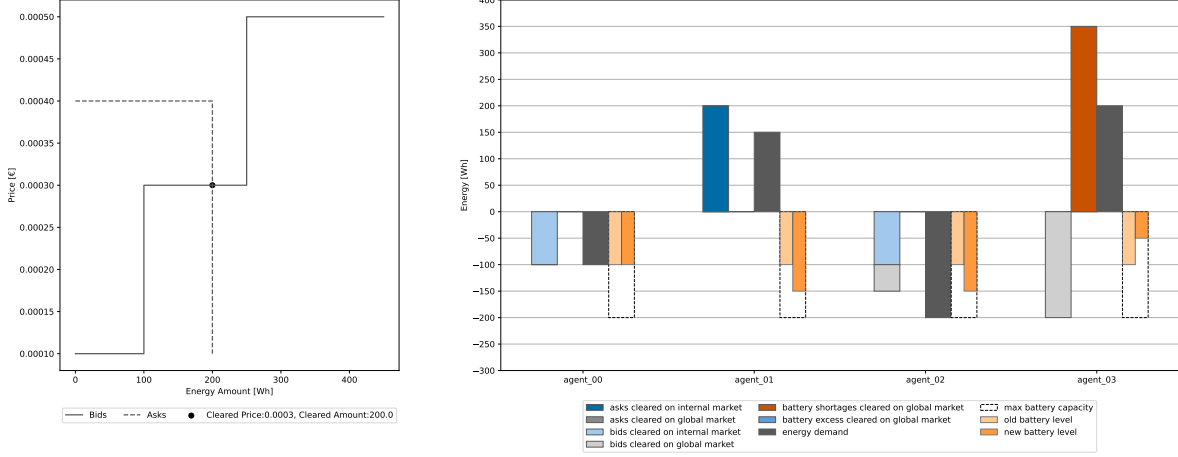


Figure 3.2: Graphical representation of the internal cleared market and network balancing for the example in Table 3.1

Wh of excess energy produced. However, since this surplus does not exceed the maximum charging rate or the additional capacity of the battery, the remaining 50 Wh will also be stored in the battery. In contrast, Agent 03's bid to sell 200 Wh was not cleared on the internal market, resulting in the entire amount being sold to the global market. Despite this, Agent 03 had an actual energy demand of 200 Wh and, due to selling an additional 200 Wh, is now short by a total of 400 Wh. The first 50 Wh of this deficit will be covered by the battery, reaching its maximum peak capacity. The remaining 350 Wh must be procured from the global market, leading to a significant penalty for the agent. Specifically, Agent 03 sold 200 Wh to the global market at the feed-in price but must repurchase 350 Wh at the higher Time-of-Use price, thereby incurring additional costs.

3.3 Instantiating the Prosumers and the Global Market

As stated in paragraph 3.2, the environment requires access to a *GlobalMarket* and a list of *Prosumer* tuples to perform the necessary computations for clearing the market and determining the state transition. The framework includes two factory methods to facilitate the creation of these required components, namely *create_global_market* and *create_prosumer*, located in their respective module folders. Each factory method accepts all required configuration parameters through a designated configuration data class and, based on this configuration, produces the corresponding named tuple.

To support the use of both mathematically generated and prerecorded data, as well as to enable the extension of the framework with new prosumer and/or global market configurations, the configuration classes inherit from the abstract classes *ProsumerConfig* and *GlobalMarketConfig*, respectively. Based on the child class of the configuration data class object, the factory methods are capable of selecting the appropriate code to execute for generating the data necessary to populate the respective *GlobalMarket* and *Prosumer* tuples.

Two distinct variations of the configuration data classes are available for both the global market and the prosumers. The *DummyDataGlobalMarketConfig* and *DummyProsumerConfig* classes result in the factory method generating a global market or prosumer, respectively, where the day-ahead prices or energy demand are created through a mathematical function. As such, they provide a means for generating synthetic datasets with customisable parameters, enabling the exploration of various scenarios and agent behaviour. Alternatively, the concrete configuration data classes *SyntheticDataGlobalMarketConfig* and *SyntheticProsumerConfig* can be employed. When using these classes, the factory methods create the respective tuples in which the data is loaded from precomputed synthetic data files, allowing for the integration of more realistic datasets.

3.3.1 Generated Data by means of a Mathematical Function

In order to research the behaviour of the agents in a controlled environment, both The *DummyDataGlobalMarketConfig* and *DummyProsumerConfig* data classes were created. By using these dummy configurations and their corresponding implementation in the factory methods, a global market or one

or more prosumers can be created for which a mathematical function can generate day-ahead price or energy demand patterns.

$$f(t) = M + \cos\left(\frac{t + \text{HPO}}{\frac{365 \times 24}{2\pi}}\right) \cdot \text{YA} + \sin\left(\frac{t + \text{HPO} - 6}{\frac{24}{2\pi}}\right) \cdot \text{DA} + \mathcal{U}(-\text{NA}, \text{NA}) \quad (3.3)$$

For both patterns, the current implementation uses the mathematical function (3.3) to create the required outputs for every sample hour t using a double sine wave to simulate both yearly and daily oscillations, where M is the average value around which the outputs oscillate and YA and DA are the amplitudes of the yearly and daily output oscillations. Parameter NA represents the maximum range of uniform random noise added to the output, while HPO is the hourly phase offset applied to the time variable t . The function is constructed as such that yearly peaks occur during midwinter and -summer while daily peaks arise at 0:00h and 12:00h when the offset parameter HPO is set to zero. An example of a dataset generated using the mathematical function (3.3) is shown in figure 3.3 in which these yearly and daily patterns are clearly distinguishable. By adjusting these parameters, the function provides a means to explore the agents' behaviour and performance in relation to constant, yearly and/or daily oscillating and noisy input observations.

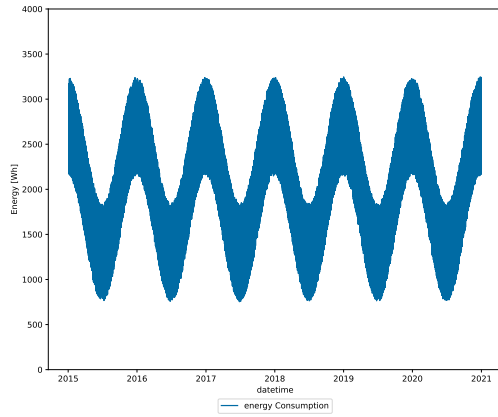
3.3.2 Generated Actor Energy Production and Consumption Data

Using mathematical functions to generate data presents a logical approach to investigating the agents' behaviour in detail. However, to allow more realistic research scenarios, the framework must accommodate the incorporation of historical data or more intricately generated synthetic datasets. To this end, the current implementation of the framework introduces the *SyntheticProsumerConfig* data class alongside the required procedures within the *create_prosumer* factory method. This factory method facilitates the retrieval of synthetically generated energy consumption and production data located within the *data_synthetic/consuming_actors_hours* folder. The files contained within this folder contain data spanning a single epoch. Each entry within these files includes a timestep alongside the corresponding energy consumption value, measured in watt-hours. When a *SyntheticProsumerConfig* data class is used as a parameter, the factory method will load the appropriate file based on the agent identifier, which is a unique reference used throughout the framework to distinguish each agent and its associated energy market actor. The loaded data is subsequently injected into the *Prosumer* tuple. While this process is straightforward and inherently generic, it allows for the potential extension of the factory method to accommodate data extraction from alternative datasets, thereby allowing the creation of custom *Prosumer* tuples.

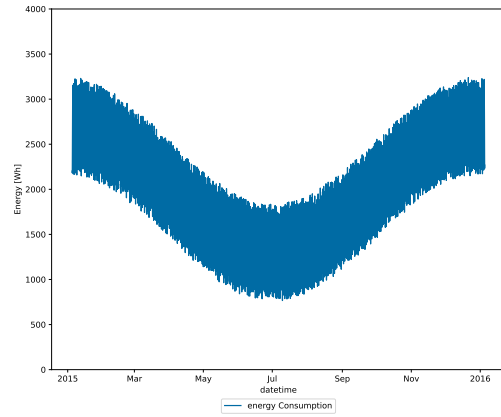
Although several household energy consumption datasets are publicly available (Hebrail & Berard, 2012; Pullinger et al., 2021; Shin et al., 2019), it was decided to use the ANTgen Python library to generate a synthetic electricity consumption dataset. This library is specifically designed to create synthetic household electricity consumption patterns by integrating household, user, appliance, and activity profiles (Reinhardt & Klemenjak, 2020). The ANTgen Python library contains a set of realistic appliance load signatures representing the energy consumption characteristics of various household devices, such as toasters, dishwashers, and microwaves. These appliance load signatures are systematically scheduled within defined activity profiles, such as ironing, preparing breakfast, or operating the dishwasher. Each activity profile, which also specifies whether or not the user's presence is required, can subsequently be incorporated into a user profile, determining when the user is present and when specific activities can be performed and how often. Ultimately, these user profiles can be aggregated into one or more household profiles. A household profile, therefore, contains multiple users, each characterised by a user profile that defines distinct activity routines. These routines, in turn, dictate the utilisation of specific appliances, each associated with a defined electricity consumption rate (Reinhardt & Klemenjak, 2020).

When generating the actual dataset, the ANTgen Python library will consider whether a user is present, whether or not the profile allows the user to perform a particular activity during the time under consideration, and whether the appliances required to perform the activity are available or already in use by another inhabitant. The algorithm will then randomly schedule the activities based on the preconditions and restrictions defined by the various profiles (Reinhardt & Klemenjak, 2020).

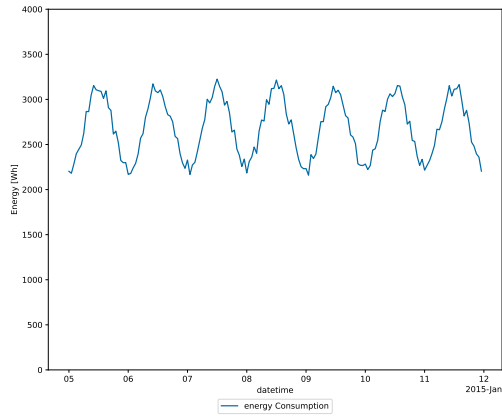
Although not directly used by the framework, two additional Python scripts are included in the codebase to facilitate the creation of a synthetic dataset using the above-described ANTgen library, which can be found in the *data_synthetic/preprocess* folder. The first script implemented in *consumption-create_profiles.py* can be used to randomly generate household profiles based on a preset collection



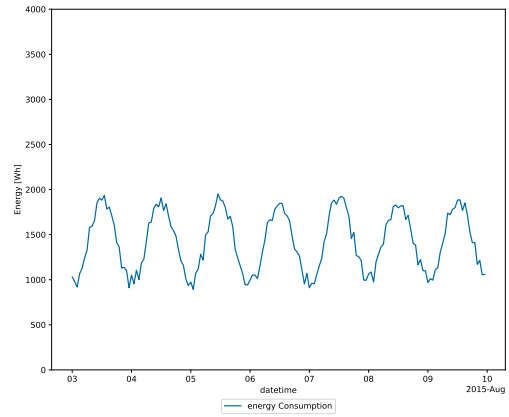
(a) Data created for one epoch



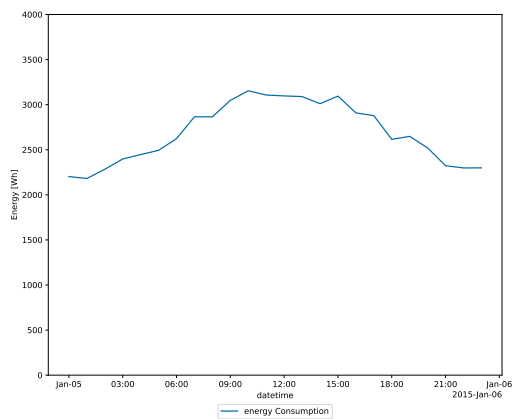
(b) Data created for one year



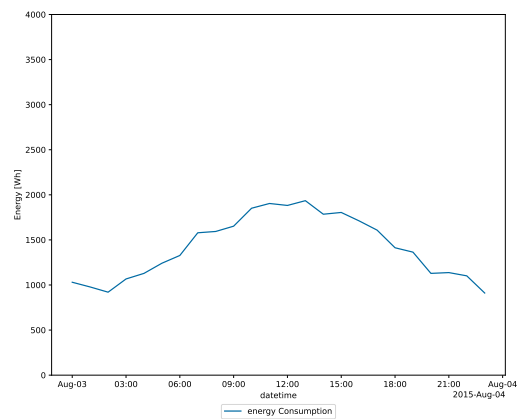
(c) Data created for one week in January



(d) Data created for one week in August



(e) Data created for one day in January



(f) Data created for one day in August

Figure 3.3: Illustration of data generated by the double sine wave function. All images are generated from one dataset and only differ in the time window scale used in the X-axis. (The following parameters were used to create the dataset: $HPO = 120$, $M = 2000$, $YA = 700$, $DA = 450$, $NA = 100$, $seed = 5$ for a value of t ranging from 0 to 52512)

of user, activity and appliance profiles already available within the ANTgen Python library. The script will, based on the provided command line parameters and additional settings stored within the *config.ini* file, create one or more household configurations containing a list of user profiles and one base load profile, for which the latter is included to represent the energy consumption of the house independent of its occupation (Reinhardt & Klemenjak, 2020).

The second script, implemented in *consumption_create_data.py*, generates consumption data, using the ANTgen Python library, for each household profile based on the provided terminal line parameters and additional configurations specified within the accompanying *config.ini* file. It is important to note that the ANTgen library was originally developed for the training and testing of load disaggregation algorithms within the context of Non-Intrusive Load Monitoring of residential electricity consumers (Reinhardt & Klemenjak, 2020). Consequently, the dataset produced by the ANTgen Python library includes synthetic data for the household as a whole, as well as for individual appliances and users, created at a per-second sampling rate. To align the output with the requirements of our framework, the script subsequently discards the appliance- and user-specific data, resampling the household dataset to a one-hour sampling rate. This process generates a single file for each household or actor, wherein each record comprises a timestamp and the corresponding fictitious energy consumption value expressed in watt-hours.

Using both scripts, a synthetic dataset was generated for eight actors, covering a period from January 5th, 2015, to December 31st, 2021, with the corresponding plots presented in Figure 3.4. An initial analysis of these plots reveals that the energy consumption patterns during a typical week in January do not significantly differ from those observed in a week in August. This observation highlights a limitation inherent in using the ANTgen Python library for synthetic dataset generation, as it operates solely on user profiles structured around a weekly schedule, thereby neglecting seasonal variations in energy consumption. A further examination of the daily energy consumption plots for January 5th and August 3rd indicates that nighttime energy consumption appears disproportionately high. A closer inspection of the user profiles clarifies the underlying reasons for this phenomenon; many profiles allow for the scheduling of high-energy-consuming activities, such as washing and drying clothes, at any time of day or night, thereby increasing the likelihood of these activities occurring during nighttime hours. This is confirmed by the graphs in Figure 3.5, which illustrate the energy consumption of individual appliances for Actor 03 during the relevant weeks and days depicted in Figure 3.4. Notably, the peaks in energy consumption for Actor 03 on January 5th at approximately 6:00 AM and August 3rd at around 3:00 AM correspond to using the dryer and a combination of the dishwasher and washing machine, respectively. While one might argue against the feasibility of such profiles that involve heavy energy-consuming appliances being utilised during nighttime, it is crucial to consider that market retailers often incentivise consumers to use electricity during off-peak hours when energy is presumed to be more abundant and cost-effective. This market dynamic counters the argument regarding the unrealistic nature of nighttime energy consumption. Moreover, utilising these profiles introduces a significant discrepancy between energy production from photovoltaic (PV) installations during daylight hours and energy consumption peaks at night. This discrepancy will force agents in the experiments to address both periods of excess energy production and subsequent energy shortages, thereby enhancing the complexity and realism of the energy market simulations. Consequently, it was determined that the dataset would be retained in its current form.

In order to enable our prosumer actors to function not only as consumers within the energy market but also as producers of their own energy, it is essential to incorporate relevant data regarding the energy production capabilities of these agents into the framework. While some datasets are available based on actual PV installation measurements (Boyd, 2016; Yao et al., 2021), such datasets are relatively scarce and limited within a specific geographical area or to a small number of installations (Vogt et al., 2022; Yao et al., 2021). As an alternative to utilising real-world data, tools have been developed to generate synthetic PV installation production data. One such tool is the *pvlb* Python library, which enables the mapping of a time series of weather data to solar power generation data, given a predefined PV system configuration and location (Holmgren et al., 2018). Two additional scripts *production_create_profiles.py* and *production_create_data.py* are therefore included alongside the framework. The former script generates random PV installation profiles within the limits and configurations specified in the *config.ini* file and as summarised in Table 3.2. Each actor’s PV installation configuration profile will include a single default inverter, which is essential for converting the direct current (DC) produced by the solar panels into alternating current (AC) suitable for residential consumption and one or two circuits, each

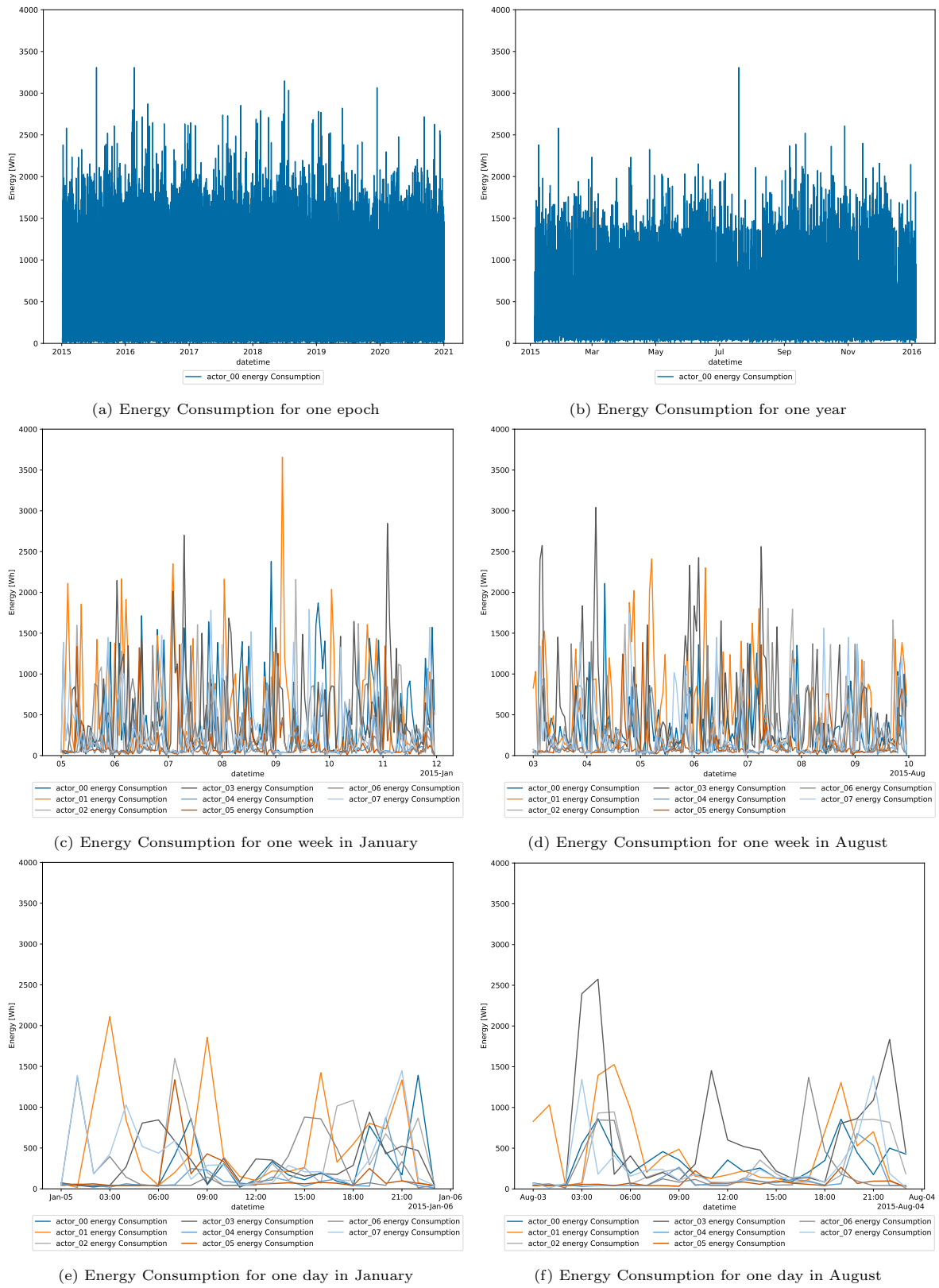
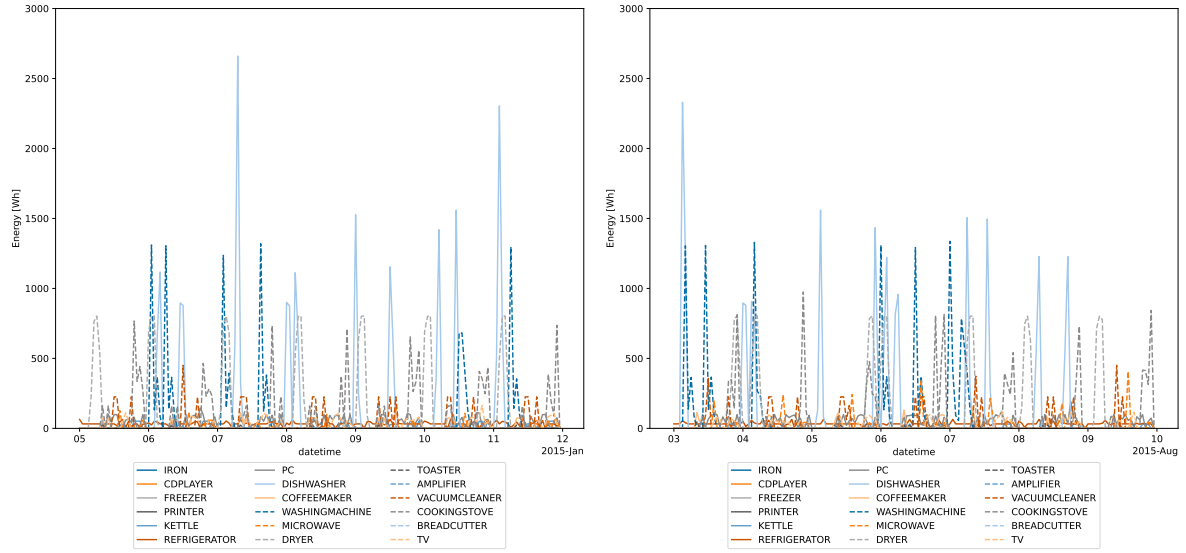
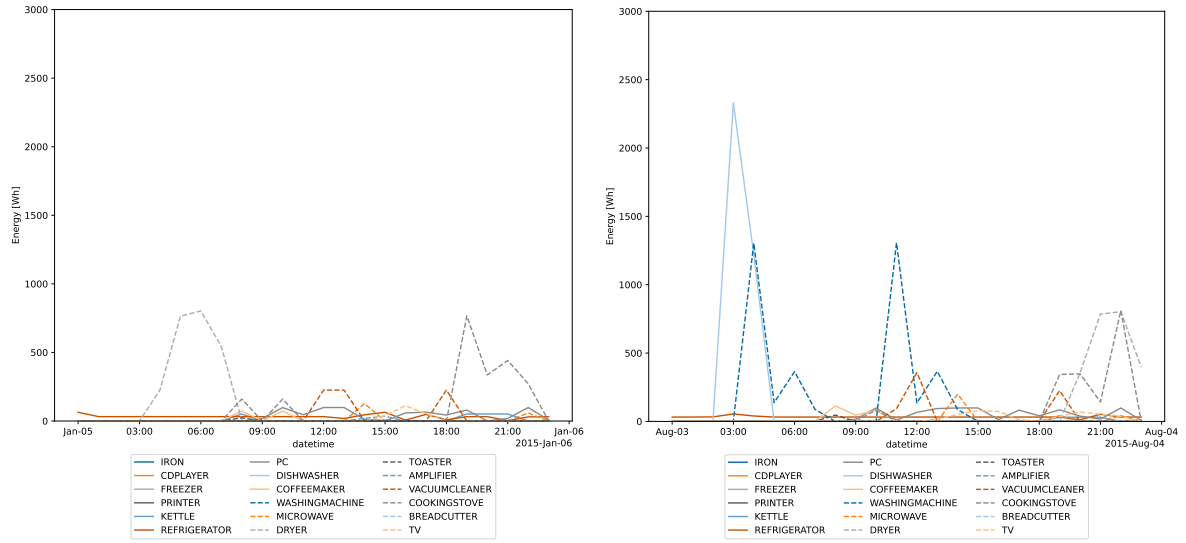


Figure 3.4: Graphical representation of the energy Consumption data as used in the experiments, generated using Randomly generated Household Profiles. All images are generated from one dataset and only differ in the actor data presented and the time window scale used in the X-axis.



(a) Appliance consumption of actor 03 for one week in January (b) Appliance consumption of actor 03 for one week in August



(c) Appliance consumption of actor 03 for one day in January (d) Appliance consumption of actor 03 for one day in August

Figure 3.5: Graphical representation of the energy consumption data of the individual appliances, as used to generate the total energy consumption of actor 03 in the experiments.

comprising eight to twelve default solar panels. The vertical tilt of the solar panels can range from 20 to 50 degrees, while the azimuth angle can vary between a minimum of 100 degrees and a maximum of 260 degrees with respect to true north. Subsequently, the *production_create_data.py* script utilises these profiles, in conjunction with the pvlib Python library and the historical hourly solar irradiation data obtained through the Photovoltaic Geographical Information System (PV-GIS) API (Šúri et al., 2005), to calculate the anticipated energy production for each PV installation. As The PV-GIS API does not provide any data beyond December 31st, 2021, it does, however, impose a rigid end date to our dataset.

Parameter	Value	Parameter	Value
min. tilt	20	max. tilt	50
min. azimuth	100	max. azimuth	260
min. modules per string	8	max. modules per string	12
min. strings per inverter	1	max. strings per inverter	2

Table 3.2: Parameter values used to create the synthetic energy production dataset

To facilitate the experiments, eight fictitious production actor profiles were created in the vicinity of the Vrije Universiteit Brussels campus³ and their estimated energy production was generated. The resulting dataset, as illustrated in Figure 3.6, demonstrates a clear yearly and daily pattern in energy production, consistent with the solar irradiance data retrieved from the expected PV-GIS historical weather data. Moreover, the weekly and daily energy production plots reveal that the individual energy production patterns of the different actors exhibit variations in both timing and amplitude, attributable to differences in azimuth, tilt, and PV installation size.

3.3.3 Historical Market Day-Ahead Price Data

In addition to accommodating the injection of actual datasets for energy production and consumption of individual prosumers, the framework also enables the utilisation of day-ahead price datasets within the global market. To facilitate this, a final script, *globalmarket_create_data.py*, is provided. This script uses the entsoe-py client, a Python interface for the European Network of Transmission System Operators for Electricity (ENTSO-E) API (<https://transparency.entsoe.eu>), to fetch Belgian day-ahead price data. The retrieved data is subsequently transformed to Watt-hours to ensure consistency with the previously generated datasets and stored in a separate file, where each record comprises a timestamp and the corresponding day-ahead price. In the context of the experiments conducted, a dataset was created comprising historic day-ahead price data spanning from January 5th 2015, the earliest date for which ENTSO-E API data is available, to December 31st, 2020, as illustrated in Figure 3.7. The plots reveal pronounced fluctuations in market prices, characterised by dramatic spikes throughout the five-year period and instances of negative prices. Notably, the relatively small magnitude of the prices is attributed to the fact that day-ahead prices are generally denominated in euros per MegaWatt-hour instead of Watt-hour.

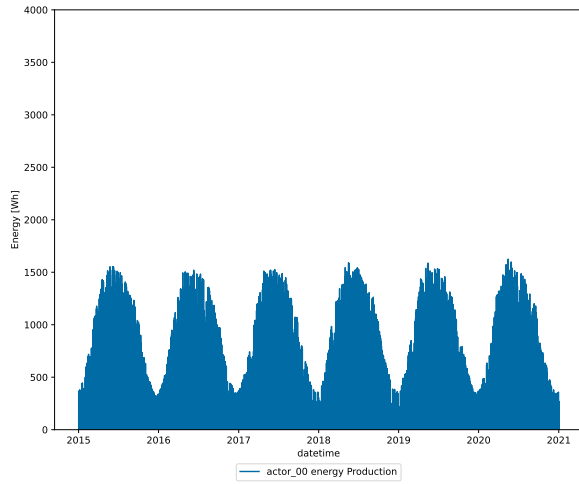
3.4 Agent-Environment Interaction and Transformers

As outlined in paragraph 3.2, at each timestep, the environment will provide a set of observations for each individual agent, which serve as the input for the agent’s decision-making process. Within the context of the framework, these observations are represented as a dictionary, where each key is mapped to a specific value for that timestep. This implies that the agent must be capable of processing these dictionary inputs. Similarly, the environment expects a dictionary of action key-value pairs as input, which includes the agent’s offering price p and the amount a of energy to be bought or sold. Consequently, it is imperative that the agent provides its actions in the aforementioned data type and structure for the environment to effectively process them. To address this potential mismatch, a separate *Transformer* class was implemented, serving a dual purpose.

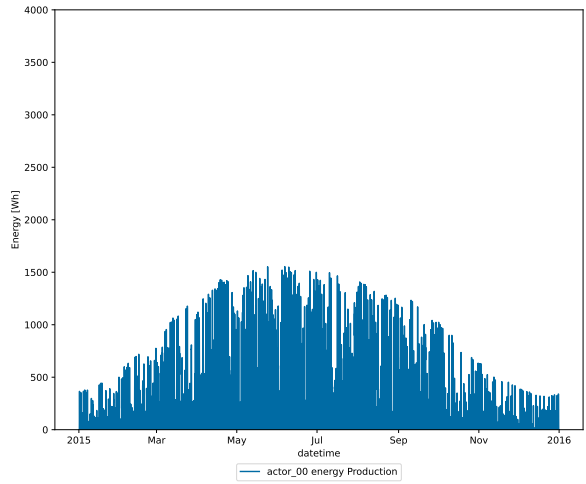
$$\mathcal{A}_x = \langle p_x, a_x \rangle \quad (3.4)$$

First and foremost, the *Transformer* object, for which one is instantiated for every accompanying agent, acts as an interface between the environment and the agent, transforming the environment’s

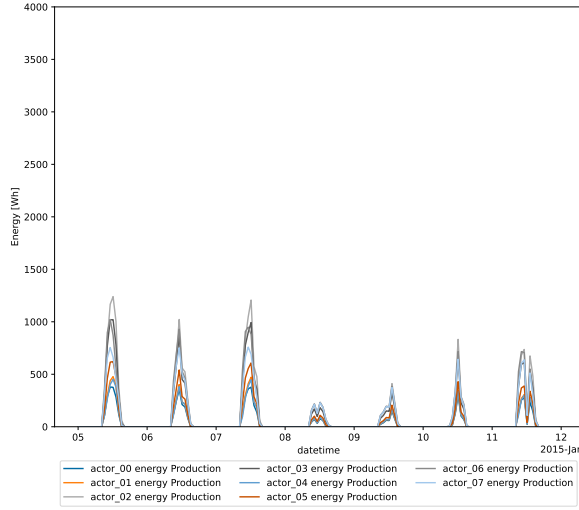
³A random point at the Boulevard Général Jacques in front of the campus was chosen for which the actual latitude (50.82451), longitude (4.39335) and altitude (81m) were retrieved using <https://earth.google.com/>



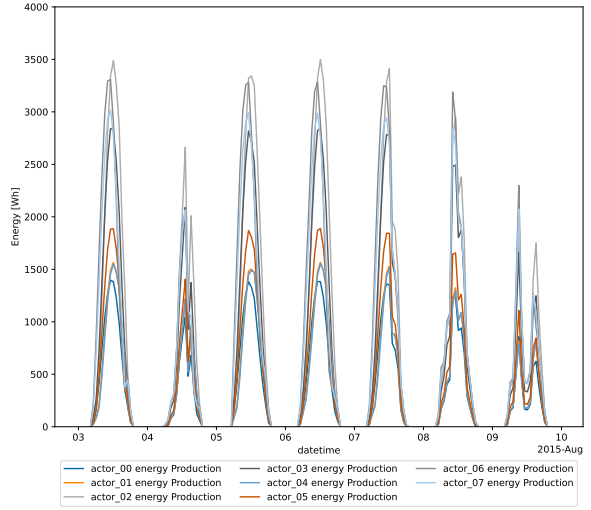
(a) Energy Production for one epoch



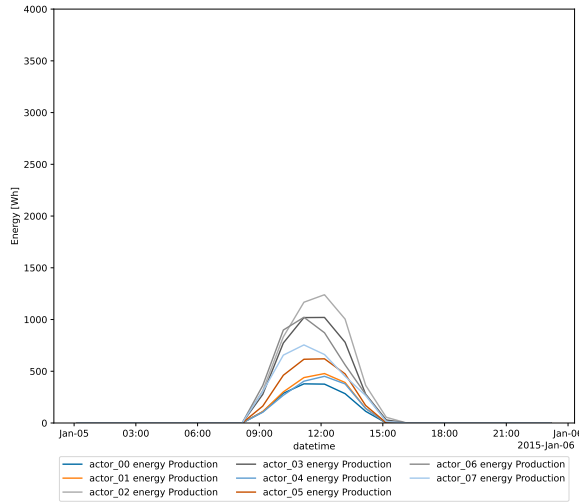
(b) Energy Production for one year



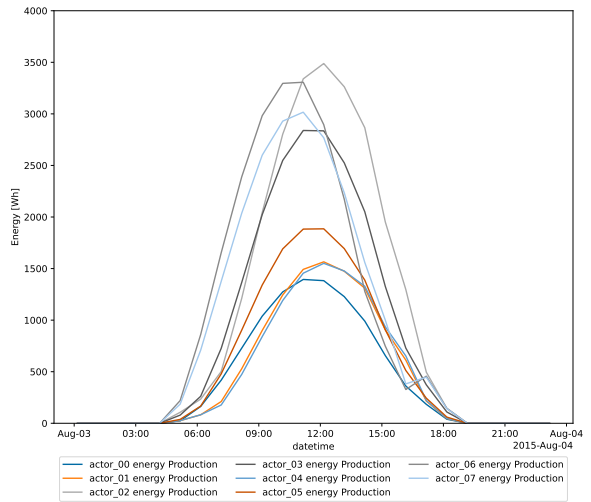
(c) Energy Production for one week in January



(d) Energy Production for one week in August

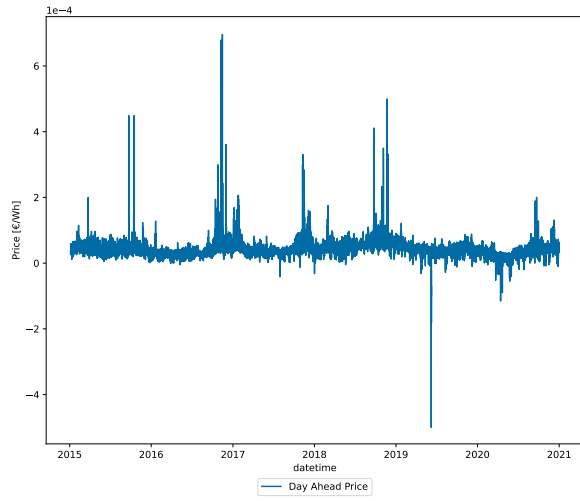


(e) Energy Production for one day in January

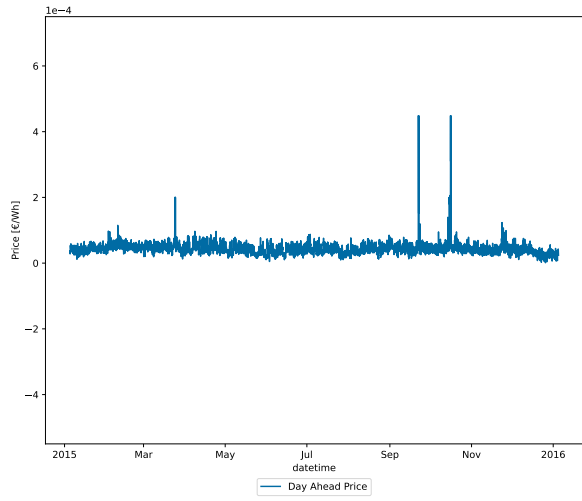


(f) Energy Production for one day in August

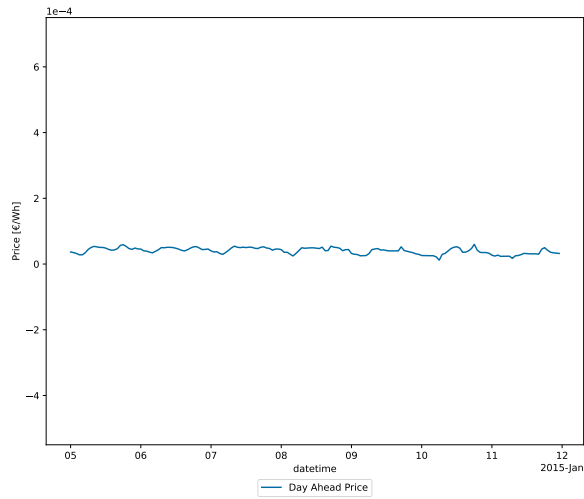
Figure 3.6: Graphical representation of the energy Production data as used in the experiments, generated using randomly generated Photo Voltaic installation profiles and historical weather data. All images are generated from one dataset and only differ in the actor data presented and the time window scale used in the X-axis.



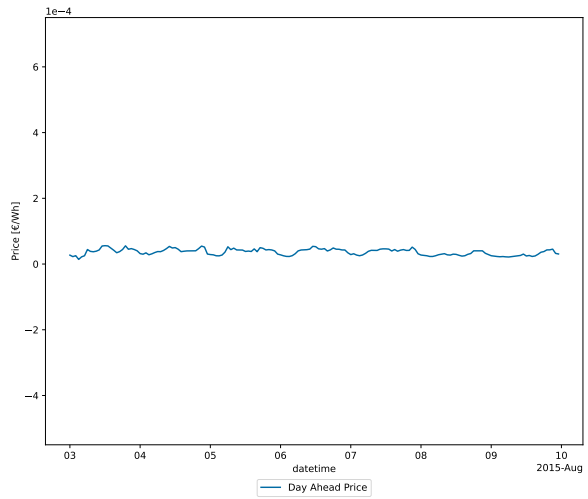
(a) Day-ahead prices for one epoch



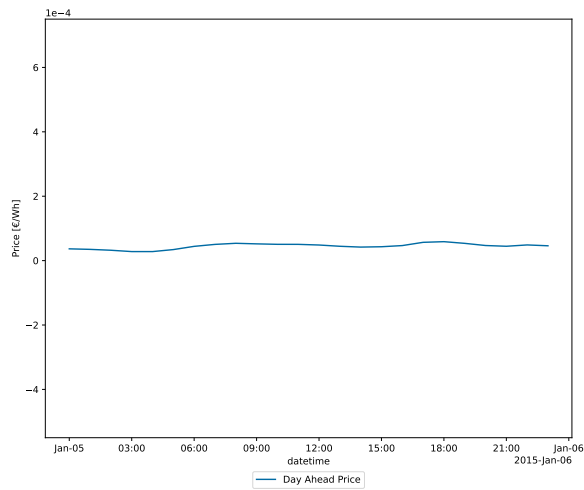
(b) Day-ahead prices for one year



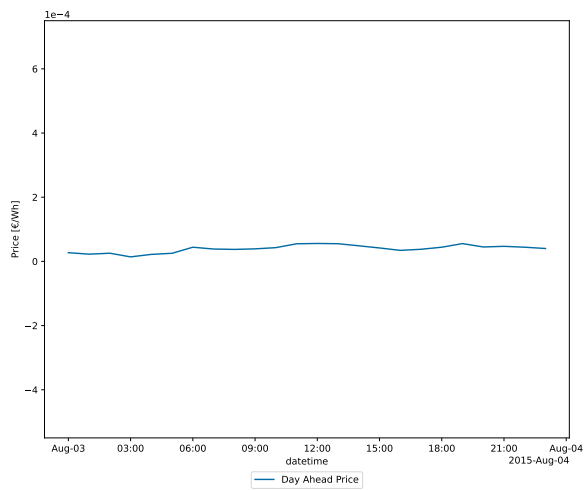
(c) Day-ahead prices for one week in January



(d) Day-ahead prices for one week in August



(e) Day-ahead prices for one day in January



(f) Day-ahead prices for one day in August

Figure 3.7: Graphical representation of the historic Day-ahead price data as used in the experiments. All images are generated from one dataset and only differ in the actor data presented and the time window scale used in the X-axis.

observations into a data type that the agent is designed to accept. Conversely, the *Transformer* object transforms the actions received from the agent into the expected dictionary structure, enabling the environment to process them accordingly.

In addition to these datatype transformations, the *Transformer* objects also provide the option for normalising incoming observations to a scale that is more suitable for the agent’s algorithm and implementation, as well as denormalising any action values to ensure they conform to the units and scales anticipated by the environment. Furthermore, the *Transformer* objects can, if required by the experimental design, function as intermediaries to filter or extend the set of observations or to impose constraints on the actions taken by the agent, such as clipping unrealistic price or quantity values for the offer made by the agent.

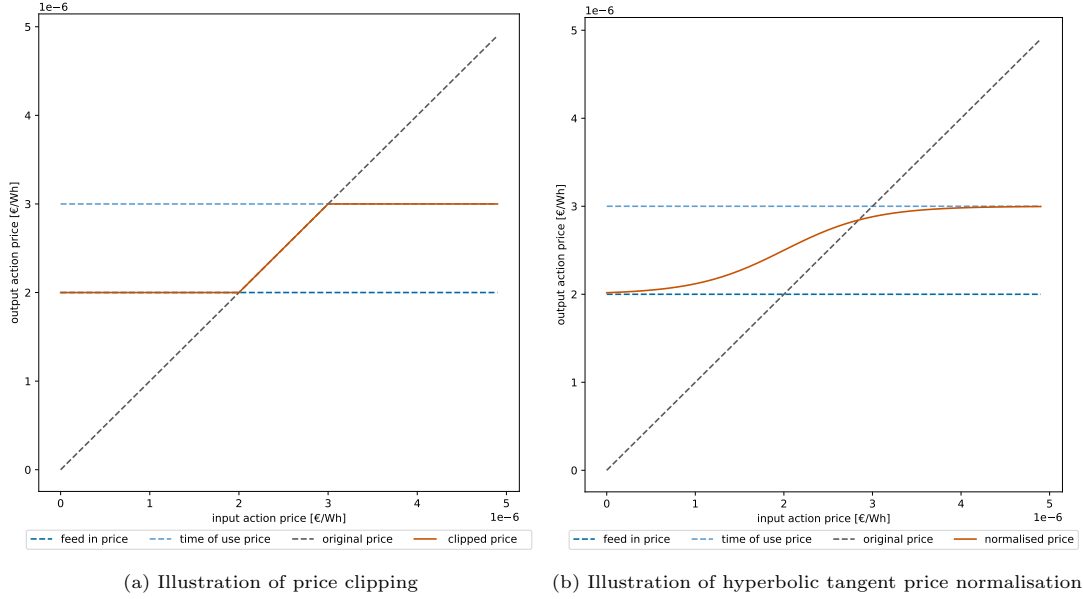


Figure 3.8: Illustration of the action price clipping and hyperbolic tangent price normalisation functions as implemented in the *Transformer* Class

As of the time of writing, the *Transformer* class includes several implementations of the functionalities previously described. Firstly, it can address the normalisation of the observation values by using either a min-max or a robust scaler. The min-max scaler rescales the observation values for both energy demands and global market prices to a range between minus one and one. In contrast, the robust scaler employs the median and interquartile range of the dataset to scale the observation values, rendering it less susceptible to the influence of outliers. Depending on the chosen scaler, an inverse scaler function will subsequently rescale the action amounts to the original scaling used for the energy demands, and the action price will be rescaled using the inverse of the scalar applied to the original global market prices. A notable disadvantage of these solutions is, however, that the statistical properties, including expected minimum, maximum, and quantile information, must either be known a priori or estimated based on previously recorded historical data. This requirement poses a potential limitation as the underlying data distributions of the actors’ energy demands and/or the market’s day-ahead pricing may be non-stationary or subject to significant changes over time.

A second optional functionality implemented in the current *Transformer* is the ability to limit the agent’s action price. Four distinct options are available: the first two either clip negative prices to zero or take the absolute value of the price set by the agent, thereby preventing the agent from submitting offers with negative prices to the environment. The latter two implemented price limiters will constrain the agent’s offering price within the boundaries of the Time-of-Use and Feed-in prices of the global market, either by simply clipping the agent’s offering price, as shown in Figure 3.8a or by using a hyperbolic tangent function to normalise the action prices within the set global market prices as illustrated in Figure 3.8b.

A third value modifier implemented within the *Transformer* class is the ability to clip the action amounts as included in the agent’s offers. As stated in paragraph 3.2, any energy discrepancies between the amount of energy used or produced by the agent versus the energy traded on the energy market

will need to be accounted for, resulting in the environment acquiring or selling this energy discrepancy on the global market on behalf of the agent. This action may be assumed to be detrimental to the agent’s reward and is, therefore, difficult to justify to the actor represented by the agent. To mitigate this response by the environment, the agent’s *Transformer* can be configured to only allow agents to set an offer for which the amount can only deviate from its actual energy demand (which, as stated before, is a negative value when more energy is produced than consumed) by the maximum amount of energy the battery can account for. This intervention can be motivated by the previous example illustrated in Figure 3.2, where Agent 03 offered to sell 200 Wh of energy while simultaneously requiring 200 Wh to meet its own energy demand. This scenario resulted in the procurement of 350 Wh from the global market after exhausting its battery capacity to address the discrepancy. If the offered amount were clipped according to the aforementioned principle, the energy balancing would yield a result as illustrated in Figure 3.9. Notably, Agent 03’s bid of 200 Wh would be adjusted to an ask to buy 150 Wh, significantly impacting the internal market clearing by turning agent 03 from a seller into an energy buyer.

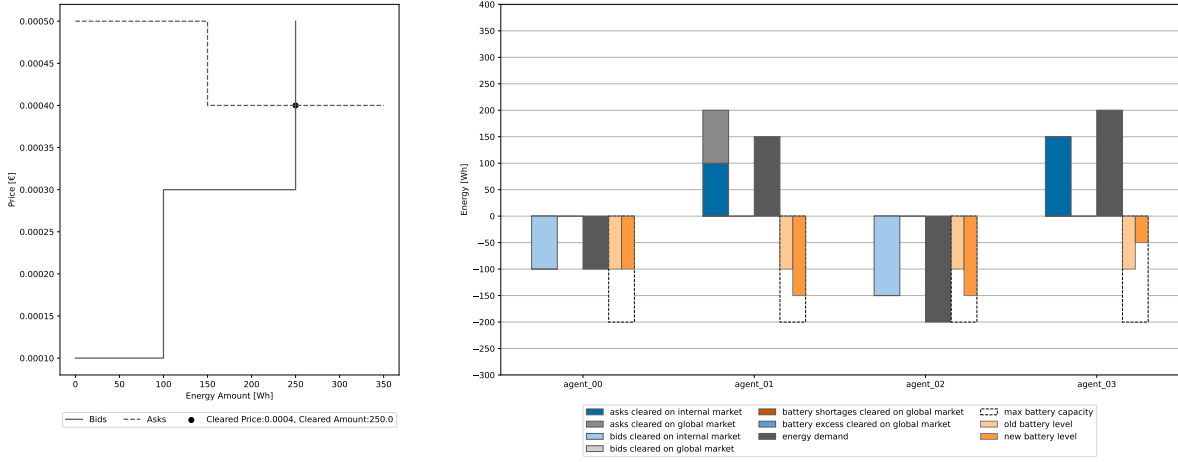


Figure 3.9: Illustration of how the action amount can be clipped by the Transformer to remain within the limits of the agent’s battery

The implementation of the *Transformer* class includes two additional options for filtering or augmenting the observations provided to the agents. The first option enables the exclusion of the day-ahead window from the observations supplied by the environment. This feature would facilitate experiments aimed at investigating the impact of having access to future market trend data, thereby creating a potential information advantage for agents equipped with this additional insight. The second option enables the sharing of individual energy demand ED_Y and battery level BL_Y data between one or more agents, leading to an observation space as formulated in (3.5). This functionality supports potential research into the dynamics of agent collaboration, as it allows for the examination of how agents with knowledge of each other’s demand and battery statuses might be tempted to cooperate or exploit this advantage against other agents that do not share their data, potentially leading to a form of market collusion. In the current implementation, the number of agents that share their energy status data for the next time step can be configured such that all agents in the agent list stored within the environment, with an index below this number, will share their data. This leads to an additional array of energy demand and battery level data in the observations of these agents. In contrast, others will receive an array of zero values of the same size within their observations.

$$\mathcal{O}_x = \langle w, \text{hod}, \text{dow}, \text{toup}, \text{fip}, BL_x, ED_x, MEP_x, BL_x, MBC_x, MBPC_x, \text{DAW}, ED_Y, BL_Y \rangle \quad (3.5)$$

As with the *Prosumer* and *GlobalMarket* classes, a *TransformerConfig* data class is provided, which encapsulates all required configuration data to instantiate a *Transformer* object for a given *Agent*. This configuration data is used by the training implementation, as described in paragraph 3.5, to create one *Transformer* per *Agent* before starting the actual training run. It is important to note that each *Transformer* is uniquely assigned to one specific agent, thus effectively allowing experiments where different agents within one experiment use a different *Transformer* configuration.

3.5 Implementation of the Agents and the Training Loop

To execute the experiments, a specific function named *train* was implemented within the *train.py* file and was heavily inspired by the original examples provided in the JaxMARL library (Rutherford et al., 2023). The execution process begins by creating the relevant *Agent* and *Transformer* objects based on their respective configuration data classes. Subsequently, the function utilises two helper functions, *_env_step* and *_update_step*, to initiate the iterative environment-agent interaction and the training of the agents. The structure of the training process is organised into an inner and outer loop, as illustrated in Figure 3.10. The inner loop will execute the environment step function a preset number of times, during which the agents interact with the environment. Throughout this iterative process, a *_RunnerState* tuple is passed between iterations, encapsulating the current environment state, the resulting observations obtained from the agents’ interactions, and the *TrainState* of each agent. The *TrainState* data class contains all agent’s training-related data and follows a common design pattern utilised by the Flax library to facilitate neural network training (Heek et al., 2023). At each environment step, the observations included in the *_RunnerState* will be transformed by the agent’s designated Transformer, to be used by the agent to select the appropriate action to take. Once transformed again by the agent’s Transformer, the actions are submitted to the environment alongside its previous state. All relevant data is compiled into a new *_RunnerState* tuple, which serves as input for the subsequent iteration. Alongside the *_RunnerState* tuple, a *Transitions* tuple is created, which contains the observations of and actions taken by the agents, their resulting rewards, and other data that needs to be logged and acts as an experience buffer (OpenAI, Berner, et al., 2019) for use during the training of the agents or the final evaluation of the experiment.

After the inner loop’s completion, the outer loop will invoke the update step, which takes the latest *_RunnerState* and the trajectory data logged through the *Transitions* as input to update the agents’ policy parameters, e.g. the weights of the neural networks used by the agents. The total execution of the training results in a final *_RunnerState* tuple, along with the metrics logged in the *Transitions* and subsequently *Trajectory_batch*. These metrics can finally be used to explore and evaluate the agents’ behaviour during the entire training execution.

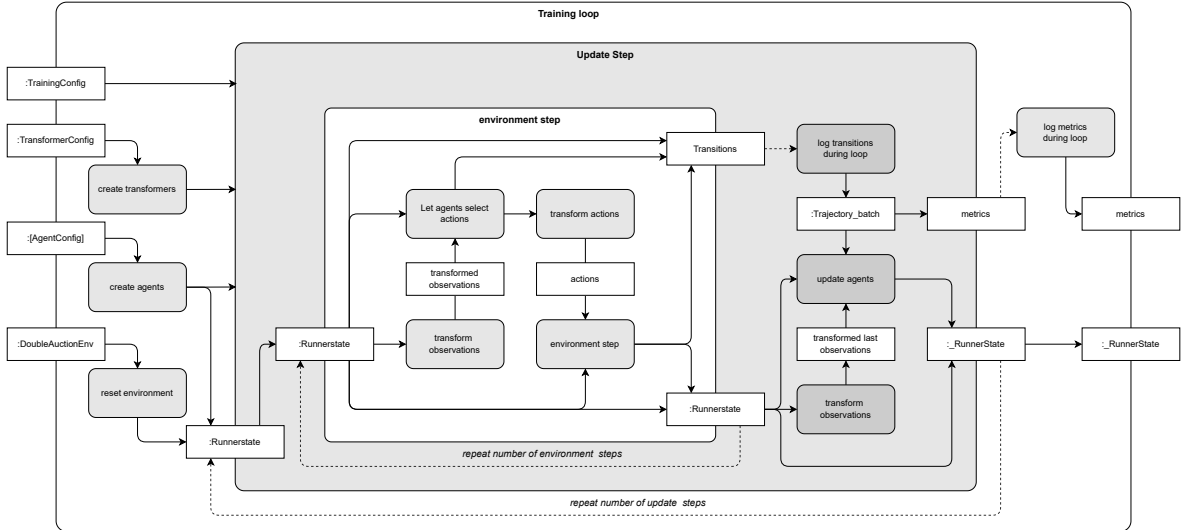


Figure 3.10: Action Diagram describing the training loop. Dotted arrows represent activities which are executed repetitively during the environment and update step iterations.

As with the actual design of the training procedure, the implementation of agents is based on the baseline examples provided in the JaxMARL library (Rutherford et al., 2023). The framework introduces an abstract *Agent* class defining methods, such as *select_action* and *update_networks_for_one_epoch*, which must be implemented by all derived child agent classes to comply with the training function’s operational requirements.

Currently, three concrete *Agent* classes are included within the framework. The *RuleBasedAgent* class comprises a naive agent which, at any given time step, determines whether energy should be sold or bought on the energy market based on the sign of its observed energy demand. Based on this

observation, it will place an offer for which the amount is identical to this energy demand and where the global market Feed-In price is asked or the Time-of-Use price is offered, depending on its need to buy or sell energy, respectively. Notably, the agent’s behaviour remains unaffected by any access to a battery; it consistently sells any surplus energy while fully procuring energy to address any shortages from the market.

In contrast, the other two agent implementations leverage the (Independent) Proximal Policy Optimization (PPO/IPPO) algorithm (de Witt et al., 2020; Schulman et al., 2017). The first implementation, located in *ppo_agent.py*, creates an individual PPO *Agent* which is fully isolated from all other agents, such that it uses and trains its own private neural network on its own prior experience, in order to achieve an optimal policy (Schulman et al., 2017). In the second implementation, implemented in *ppo_self-play-agent*, the IPPO agent uses a form of self-play (OpenAI, Berner, et al., 2019; R. Zhang et al., 2024) in which all agents share the same Neural Network weights and prior experiences (de Witt et al., 2020; OpenAI, Berner, et al., 2019). At each time step, the agent will provide the network with its own observations and subsequently let the network select the action to take. During this action selection process, the network thus only has a partial observable view of the state of the environment provided by the agent’s specific observations. However, during the training phase, the logged trajectory data from all agents, as described above, is aggregated to train the singular neural network (Horgan et al., 2018). Consequently, while the former implementation allows for substantial divergence in policies among agents due to varying configurations between agents and their respective *Transformers* or *Prosumers*, the latter implementation will converge towards a unified and generic policy applicable to all agents.

3.6 Experiment Configurations

As previously described, the framework uses various data classes to encapsulate the configuration of its components. The use of such Python data classes was a deliberate design choice driven by several key considerations. Firstly, it provides future users of the framework with essential building blocks that constitute a complete configuration for conducting experiments, thereby offering a descriptive interface that inherently supports the use of type hints supported by the data class datatype. Moreover, Python’s data class datatype allows for the application of object-oriented principles, specifically inheritance, which is not fully supported by other data structures such as named tuples, typed dictionaries, or regular dictionaries within Python. This capability enhances the framework’s extensibility, allowing for the creation of new types of *Prosumer* or *GlobalMarket* that can utilise different datasets for their energy demands or day-ahead prices.

To implement this, developers need only to define a new child class derived from the abstract *AgentConfig*, *GlobalMarketConfig*, or *ProsumerConfig* classes, incorporating the necessary parameters. Subsequently, the factory method for the respective component can be extended by verifying the class of the configuration object and executing the specific code required to instantiate the *Agent*, *GlobalMarket*, or *Prosumer* accordingly. Furthermore, data classes can be rendered immutable, and by grouping all relevant configuration parameters within a single data structure, they facilitate the easy passing of parameters throughout the codebase as a unified container.

In addition to the aforementioned configuration data classes for the individual components of the framework, an additional *ExperimentConfig* data class has been implemented in *experiments/experiment-config.py*. Although the use of this data class is not strictly mandatory for operating the framework, it is used as input by the *training_init_and_execute* function in *main.py* to establish the complete framework and execute the training experiment. The *ExperimentConfig* data class enables users to consolidate all necessary parameters for the complete experimental setup, allowing for the organisation and storage of these parameters in a separate file or their export to their export as a JSON file, as demonstrated in the *training_init_and_execute* function.

Moreover, users have the flexibility to create their own scripts that combine predefined configurations for different Prosumer, GlobalMarket, and/or Agent types within one or more *ExperimentConfig* data classes, thereby allowing for the generation of multiple variations of an experiment in which a single component type or parameter is modified and thus facilitating a systematic exploration of different configurations.

3.7 The Main Function

To conclude this chapter, we must take a look at the main *training-init-and-execute* function implemented in the *main.py* file, which serves as the entry point for executing the experiments. This function accepts an *ExperimentConfig* data class and a logging folder as inputs. Initially, it instantiates the *GlobalMarket*, *Environment*, and *Prosumers*, followed by a data integrity check to ensure that the number of records, as well as the start and end dates of the datasets associated with both the *Prosumers* and *GlobalMarket* tuples, are consistent and aligned. Upon successful creation of these components, the function proceeds to invoke the *train* function, thereby initiating the actual training process. As detailed in Paragraph 3.5, this function returns the metrics logged throughout the training loop upon completion of the training process. If desired, these metrics, along with the experiment configuration, can finally be saved locally for future reference. Alternatively, the Weights and Biases API (<https://wandb.ai/>) can be employed to facilitate remote storage of both metrics and configuration data. For debugging purposes, the JIT compilation, as discussed in Paragraph 3.1, can be turned off by setting the optional input parameter *disable_jit* to True.

Chapter 4

Experimental Results

4.1 Scope of the Experiments

The fully implemented framework facilitates a diverse array of experimental setups. It is, therefore, imperative to acknowledge that the extensive range of potential variations permitted within the framework precludes the execution of all conceivable tests. Furthermore, optimising the agents’ performance was not the primary focus of this study; all agents were configured with standard hyperparameter values, which remained constant throughout the conducted experiments. As a result, the findings presented in this chapter are intended to illustrate the framework’s potential rather than to provide a comprehensive examination of the agents used within the context of the framework.

The experimental setup involved configuring the framework in specific ways for each test, enabling the evaluation of the three distinct implemented agent types: the rule-based agent, the Proximal Policy Optimization (PPO) agents, and the self-play variant of the PPO agents. Each experimental condition was replicated five times, with variations introduced solely through the application of random seeds (0, 10, 20, 30 and 40). Metrics were logged at a sample rate of 100 timesteps, and for each experiment, a set of general overview plots is provided in appendix B.

In adherence to the limitations imposed by the synthetic dataset outlined in paragraphs 3.3.2 and 3.3.3, each experimental epoch begins on January 5, 2015, at 00:00 hours and concludes on December 31, 2021, at 23:00 hours. This temporal window includes both leap years 2016 and 2020, resulting in 52,512 timesteps, each representing one hour. It is essential to highlight that throughout all experiments, the *Transformers* associated with the rule-based agents did not incorporate any normalisation of the observations or actions. In contrast, the observations and actions of the PPO agents and their self-play counterparts underwent normalisation and denormalisation by means of the implemented robust scaler, as elaborated in paragraph 3.4. Additionally, no clipping of action prices or action amounts was applied during any of the experiments, although these options were implemented within the framework. Finally, it needs to be reiterated that the parameters for both the individual PPO agents and self-play PPO agents remained constant across all conducted experiments, with the specific values summarised in Table 4.1.

In order to execute the required experiments, a Docker container was created to run the framework based on the python:3.11.0-slim-bullseye image fetched from DockerHub (<https://hub.docker.com>). The container was used in conjunction with the DevContainer extension, as provided by Visual Studio Code (VisualStudioCode, 2024). All experiments were conducted on a 2021 MacBook Pro equipped with an M1 Pro chipset, featuring 10 CPU cores and 16 GB of RAM, running macOS Sonoma 14.3 as the operating system. To facilitate the execution of the Docker containers, Docker Desktop version 4.33.0 (<https://www.docker.com>) was installed, with 10 CPU cores and 7.9 GB of RAM allocated to its resource pool. To provide full transparency, a set of graphs depicting the overall experimental results is included for each conducted experiment in Appendix B.

The initial set of experiments encompasses five distinct scenarios in which eight agents act on behalf of their respective market actors¹. The first scenario maintains a constant global market and prosumer

¹While a clear distinction exists between the actor interacting with the energy market and the corresponding agent responsible for selecting appropriate actions on the actor’s behalf, this distinction will not be consistently upheld throughout the remainder of this chapter to enhance readability. Consequently, it is assumed that references to the agent’s energy demand, battery level, etc., will pertain to the corresponding characteristics of the actor represented by that agent.

Parameter	setting	Parameter	setting
Actor network layer size	128,128,128	Critic network layer size	128,128,128
Activation function	relu	Maximum Gradient Norm	0.5
Gamma	0.000021	GAE Lambda	0.95
Clip Epsilon	0.2	Value Function Coefficient	0.5
Entropy Coefficient	0.01	Learning rate (with annealing)	28e-4

Table 4.1: Overview of the PPO parameter settings used for both the individual PPO as the self-play agents.

conditions. Although this scenario may be perceived as trivial, its simplicity allows for the verification of learning outcomes for both the individual PPO agents and their self-play counterparts, with the rule-based agent serving as a primary benchmark. The second scenario introduces variability in the global market price by applying yearly sine-wave fluctuations to the market’s day-ahead prices, as delineated in paragraph 3.3.1, thereby evaluating the agents’ adaptability to dynamic market conditions. The third scenario further complicates the environment by incorporating both yearly and daily fluctuations in the global market. In the fourth scenario, while the global market remains static, the prosumers’ energy demand exhibits yearly sinusoidal fluctuations. Finally, the fifth scenario utilises the synthetic data described in paragraphs 3.3.2 and 3.3.3 to present the agents with a more realistic operational context.

Parameter	setting
global market Time-of-use cost	0.000045 €/Wh
global market mean day-ahead value (M)	0.000042 €/Wh
global market yearly day-ahead amplitude (YA)	0.000021 €/Wh
global market daily day-ahead amplitude (DA)	0.000005 €/Wh
prosumer mean energy demand value (M)	+/-320 Wh
prosumer yearly energy demand amplitude (YA)	350 Wh

Table 4.2: Overview of the general parameter settings used in the first four scenarios where the day-ahead and energy demands are calculated using formula (3.3). The mean values are always applicable in those scenarios, while the amplitudes will only be applied when the scenario requires this parameter.

To generate the global market day-ahead and prosumer energy demand data for the first four scenarios, the mathematical function (3.3) was used. The mean value of the global market (M) was set to 0.000042 euro/Wh, reflecting the rounded mean day-ahead price obtained from the ENTSO-E API data (<https://transparency.entsoe.eu>), as referenced in paragraph 3.3.3. The yearly (YA) and daily amplitudes (DA) were selected arbitrarily to ensure sufficient variability in the day-ahead prices. The time-of-use cost was determined based on the tariff list for 2024 (appendix A.1), as provided by the Vlaamse Regulator van de Elektriciteits- en Gasmarkt (VREG, 2018) indicating a distribution cost of approximately 45 euro/MWh.

Similarly, the prosumer energy production and demand values of -320 Wh and 320 Wh, respectively, correspond to the rounded mean values of energy produced and consumed in the synthetic dataset created as described in paragraph 3.3.2. In each of the first four scenarios, four dedicated net producers with a mean energy demand of -320 Wh are introduced to the market together with four net consumers, each having a mean energy demand of 320 Wh. In all initial five scenarios, the battery capacity and its corresponding peak capacity were set to zero for all agents, simulating the absence of a battery installation. Lastly, the *Transformers* for all agents were configured to exclude the day-ahead window from observations, ensuring no data sharing occurred among the agents.

In the subsequent phases of the experiments, relevant initial scenarios are subjected to three distinct variations. The first variation introduces batteries to all agents, with each battery configured to a capacity of 13500 Wh and a peak output of 5000 Wh, aligning with the specifications of the Powerwall 2 home battery system produced by Tesla (Tesla, 2024). These scenarios aim to test the agents’ ability to effectively manage the new option of storing or retrieving energy from the battery to optimise their long-term returns. The second variation builds upon the first by incorporating both batteries and a day-ahead pricing window, as delineated in paragraph 3.2. This enhancement allows for assessing whether the agents can leverage foresight into future price fluctuations to make more informed long-term decisions. Finally, the third variation further expands the agents’ observation space by introducing the concept of data sharing by allowing four agents to share their current battery status and energy demand. In those scenarios, which utilise the sine-wave function (3.3) to generate the global market prices and/or the energy demand of individual prosumers, two net producers and two consumers will engage in data sharing, while the observations of the remaining four other agents will include a zero-filled

array as described in paragraph 3.4.

It is essential to state that throughout these three variations, rule-based agents will not be tested, as their existing framework does not incorporate the functionalities of batteries, the day-ahead pricing window, or data sharing. However, future, more complex implementations of the rule-based agents could potentially incorporate these features, making the experiments more comprehensive.

4.2 Demonstrating Learning

4.2.1 A Constant Global Market with Constant Prosumers

To demonstrate the operational functionality of the proposed framework, we initiate our analysis with a simplified scenario in which both the global market's day-ahead price and the energy demand of prosumers remain constant. In this scenario, the first four prosumers function as net producers, each generating 320 Wh during one time step, while the remaining four consumers consume an equivalent amount of energy within the same time frame. The global market price is maintained at a constant value of 0.000042 €/Wh, with a time-of-use cost of 0.000045 €/Wh. None of the consumers have access to a battery installation, nor is the day-ahead window included in their observations. Furthermore, the framework is configured such that no agents can share any energy demand or battery level data.

Upon analysing the behaviour of the rule-based agents within this controlled environment, it becomes evident that, despite the simplicity of the scenario, the system operates as anticipated. As expected, the overall mean reward for all agents combined results in zero. This outcome arises from the rule-based agents' consistent strategy of purchasing their required energy at the time-of-use price established by the global market. Conversely, when acting on behalf of a net-producing agent, these agents will invariably attempt to sell all their surplus energy at the lower feed-in tariff, which, in this scenario, corresponds to the day-ahead price of the global market. Consequently, the internal market is fully cleared, and no energy transactions are required with respect to the global market. This is demonstrated in Figure B.2 where the first four agents, representing the producers, receive a mean reward during the last epoch of 0.01344 €/Wh, which corresponds to 320 Wh multiplied by the feed-in price of 0.000042 €/Wh. On the other hand, their consuming counterparts are subject to a negative mean reward of the same size.

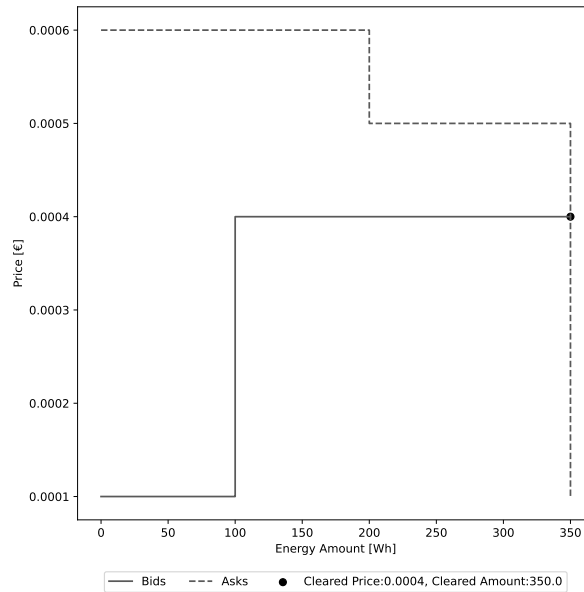


Figure 4.1: Illustration of an internal market scenario where two sellers place an asking offer of 200 Wh at 0.0006 €/Wh and 150 Wh at 0.0005 €/Wh while two sellers place a bid of 100 Wh at 0.0001 €/Wh and 250 Wh at 0.0004 €/Wh, fully clearing the market at the highest bidding price.

This finding elucidates a critical implementation detail concerning the clearing mechanism of the internal market. In scenarios where the internal market is fully cleared, all bids are allocated to corresponding asks. However, this situation presents a challenge in defining the actual internal market

price, as there may be no distinct cross-point at which supply and demand intersect. In such cases, the current implementation will adopt the highest bidding price as the prevailing market price. This approach ensures that market sellers receive at least the price specified in their bids, safeguarding their interests, while simultaneously, buyers can benefit from the price discrepancy. This dynamic is illustrated in Figure 4.1 where two asking offers of 200 Wh at 0.0006 €/Wh and 150 Wh at 0.0005 €/Wh are cleared against two bids of 100 Wh at 0.0001 €/Wh and 250 Wh at 0.0004 €/Wh. Although there is no clear intersection point between the asks and bids, the double auction mechanism is implemented so that the highest bidding price will be chosen as the final market price. This is confirmed by the plots in Figure 4.2 depicting the actual ask and bid prices of the agents versus the internal market’s clearing price in the experiment.

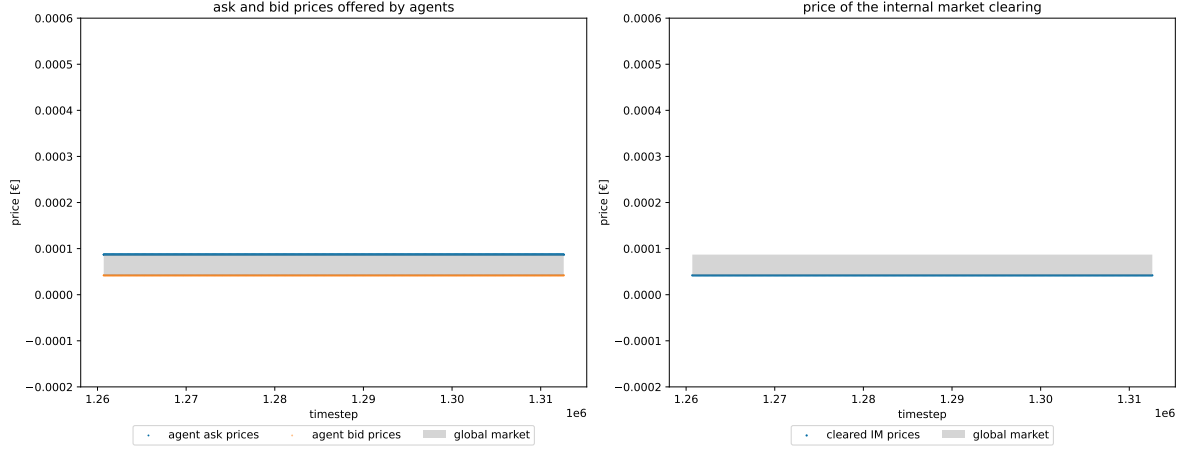


Figure 4.2: Scatter plot containing all asks, bids and internal market clearing price for experiment B.2.1 with rule-based agents during the last epoch.

Upon examining the same scenario using the implemented PPO agents, it becomes evident that the overall mean reward for all agents in Figure B.4 exhibits a consistent improvement over time across all five distinct runs. The mean rewards of the individual agents in the boxplot in figure B.5 demonstrate that the producing agents 00 up to 03 achieved a better result than their rule-based counterparts in the previous scenario. This does, however, come at the expense of the results for the consuming agents 04 to 07. All agents combined thus perform slightly worse than the rule-based benchmark due to the presence of global market transactions.

A closer inspection of the key metrics, as presented in Figure B.6, reveals that by the conclusion of the training run, all agents, with the exception of Agent 05, have successfully learned to approximate the target energy amount of 320 Wh in their ask or bid offers. This advancement results in a notable reduction in battery discrepancies, which is expected to be detrimental to the agents’ rewards as described in paragraph 3.4.

Furthermore, the plots clearly demonstrate that by the end of the training period, nearly all agents, apart from Agent 05, are capable of successfully buying or selling their required energy demands or surplus within the internal market, leading to a decline in global market transactions.

In comparing the performance of individual PPO agents (Figure B.5) to their self-play variants (Figure B.8), it is fair to conclude that on average, the former agents outperform their self-play counterparts with a mean reward of -0.00009 versus -0.00359. Notably, the learning trajectory of the self-play agents exhibits a decline over time across all five consecutive runs, following an initial steep learning curve, as illustrated in Figure B.7. In contrast, the individual PPO agents demonstrate a more stable learning progression.

Furthermore, a comparative analysis of Figures B.5 and B.8 reveals that while the producing self-play agents marginally outperform their individual PPO counterparts, this advantage is offset by the underperformance of the consuming self-play agents. A similar pattern is observed when examining the key metrics in Figure B.9. While the behaviour is much more consistent than observed with the individual PPO agent, both the selling and buying self-play agents’ offering amounts fall significantly below the target of 320 Wh, resulting in increased energy discrepancies that necessitate resolution in

the global market, as well as a reduction in the amounts cleared within the internal market.

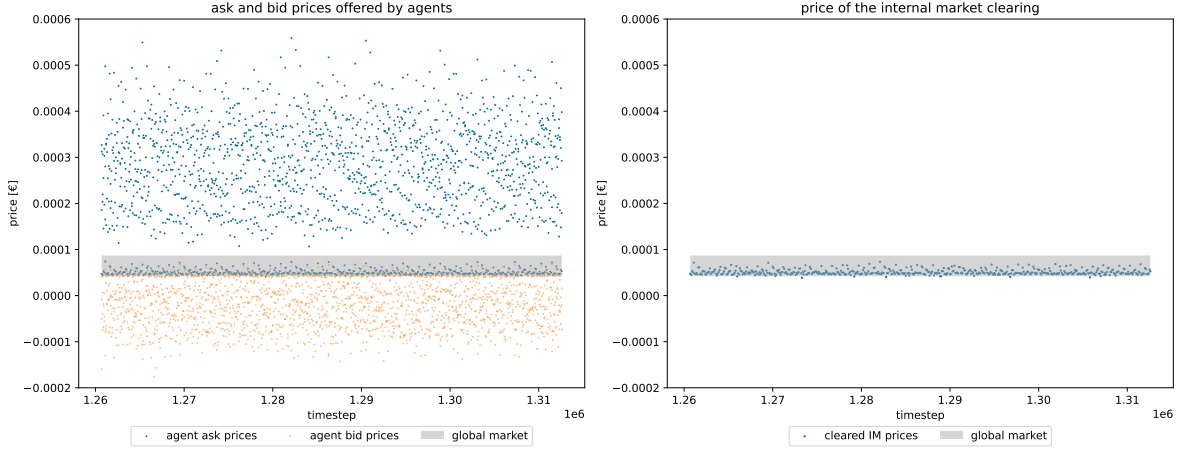


Figure 4.3: Scatter plot containing all asks, bids, and internal market clearing price for experiment B.2.1 with PPO agents during the last epoch.

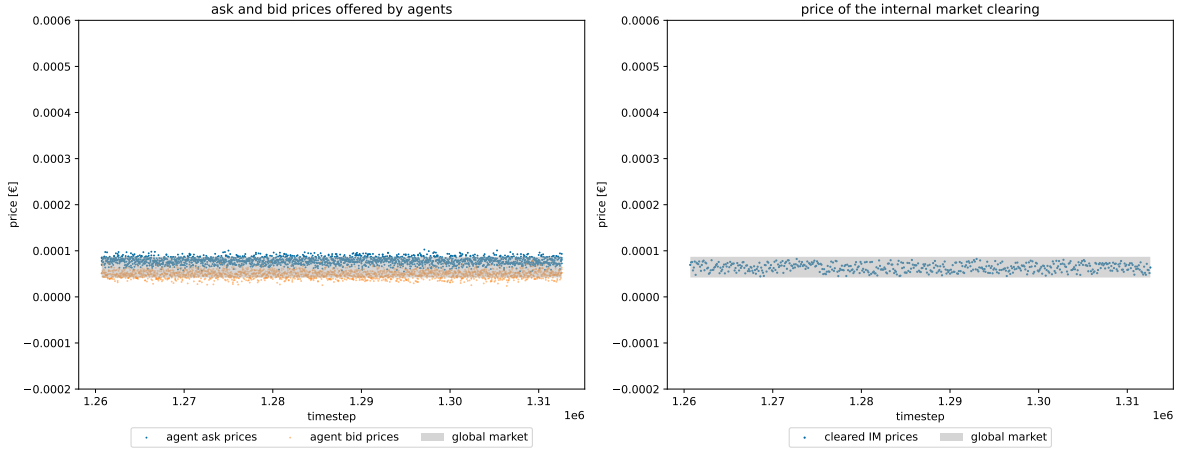


Figure 4.4: Scatter plot containing all asks, bids and internal market clearing price for experiment B.2.1 with self-play PPO agents during the last epoch.

A closer examination of the offering prices between the individual and self-play PPO agents reveals an intriguing peculiarity. The self-play agents exhibit behaviour akin to that of the rule-based agents, as depicted in Figure 4.4. Specifically, the selling agents tend to set their offering prices near the feed-in price of the global market. Simultaneously, the buying agents' offering prices approximate the time-of-use price to maximise the chance of clearing the offers on the internal market, resulting in internal market clearing prices being determined well within the global market boundaries. In contrast, the individual PPO agents display a more significant divergence in their pricing strategies, as visualised in Figure 4.3. A more detailed analysis of the individual agents' prices, presented in Figure 4.5, reveals an interesting pattern: Agents 00 and 05 appear to have learned to position their offering prices in close proximity to one another. On the other hand, the other selling agents, 01, 02, and 03, have adopted a strategy of undercutting their fellow producer, Agent 00, by systematically placing bids at lower prices to enhance their chances of having their bids cleared first. A similar dynamic is observed among the buying agents, where Agents 06, 07, and 08 consistently place their buying offers above those of Agent 05, thereby increasing their likelihood of having their offers cleared while assuming Agent 05's lower offering price as the probable final market clearing price. This behaviour is confirmed by the mean amounts cleared in the internal market, as illustrated in Figure B.6 and the lower rewards, especially for

agent 05, as shown in Figure B.5. Consequently, one could assume that Agents 00 and 05 are cornered by their respective producing and consuming counterparts, limiting their ability to optimise their market positions effectively.

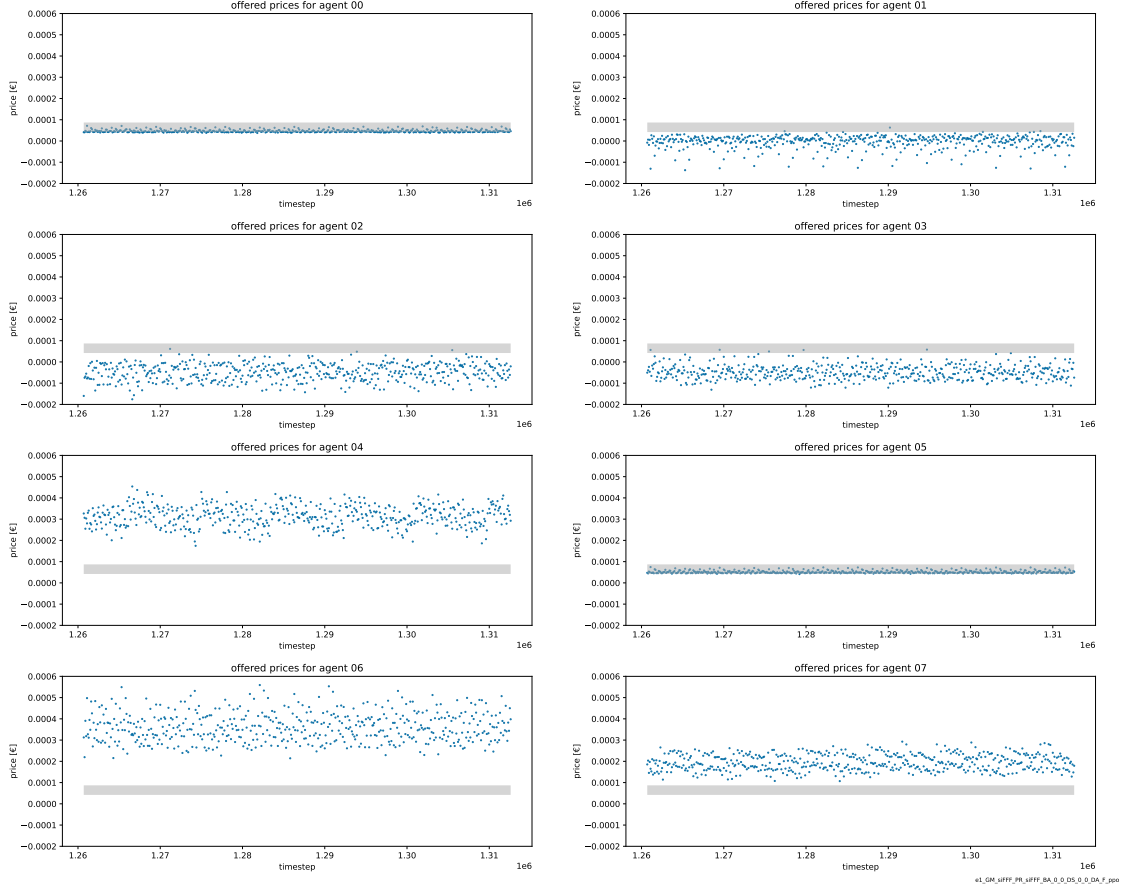


Figure 4.5: Scatter plot containing all individual PPO agent offer prices during the last epoch for experiment B.2.1.

4.2.2 A Sine-Wave Global Market with Constant Prosumers

When confronting the agents with slightly more challenging environments, the introduction of oscillating global market day-ahead prices on both a yearly and a combined yearly and daily basis reveals similar behavioural patterns as those noted under a constant global market price regime. The rule-based agents adhere strictly to the prevailing global market prices, as illustrated in Figure 4.6, which consequently leads to the same overall mean reward, depicted in Figure B.11, as with a constant global market. The increased spread in rewards hereby is solely caused by the changing global market prices, which affect the agents' rewards as fluctuating global market prices let both buying and selling agents follow these price alterations. Again, when using both the individual PPO and the self-play PPO agents in these settings, the same behavioural patterns are observed as with the constant global market. The self-play PPO agents' policy remains relatively consistent, where the net-producing agents place their bids near the time-of-use price while the consumers' offer prices approximate the global market's feed-in price (Figure 4.8). This leads to a similar overall performance as illustrated in Figures B.18 and B.28.

In the case of individual PPO agents, the previously noted specific behaviours are again evident, as illustrated in Figure 4.7. In the yearly fluctuating global market scenario, Agents 03 and 06 adjust their offer pricing strategies to converge more closely, enabling other agents to capitalise on this alignment as shown in Figure B.16. This pattern persists in the context of a market with yearly and daily price fluctuations as depicted in Figure B.26, where Agents 00 and 07 assume the role of stabilising the clearing price within the confines of the global market. The fact that the impacted agents differ across the three observed scenarios allows us to assume that this is not an implementation issue but the result

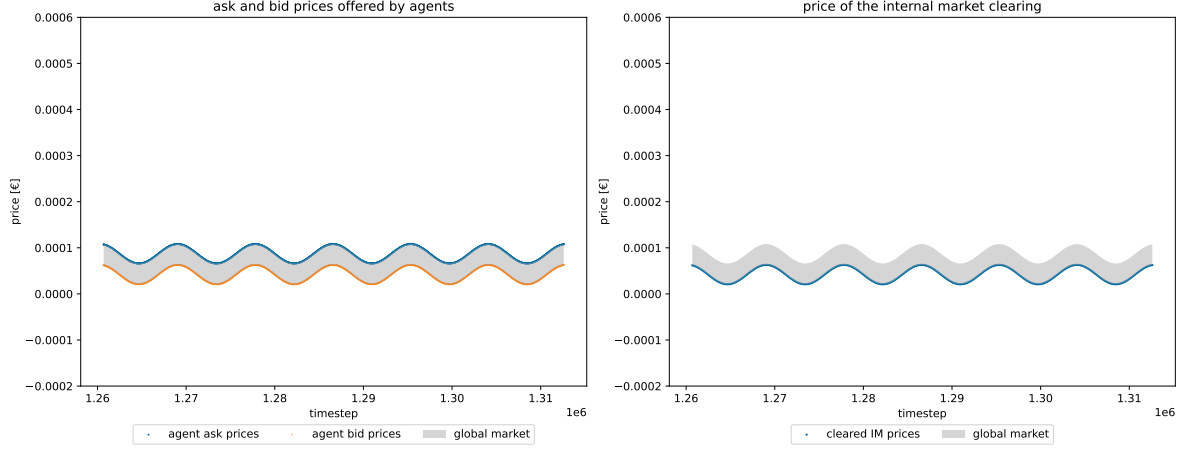


Figure 4.6: Scatter plot containing all asks, bids and internal market clearing price for experiment B.2.2 with rule-based agents a yearly oscillating global market during the last epoch.

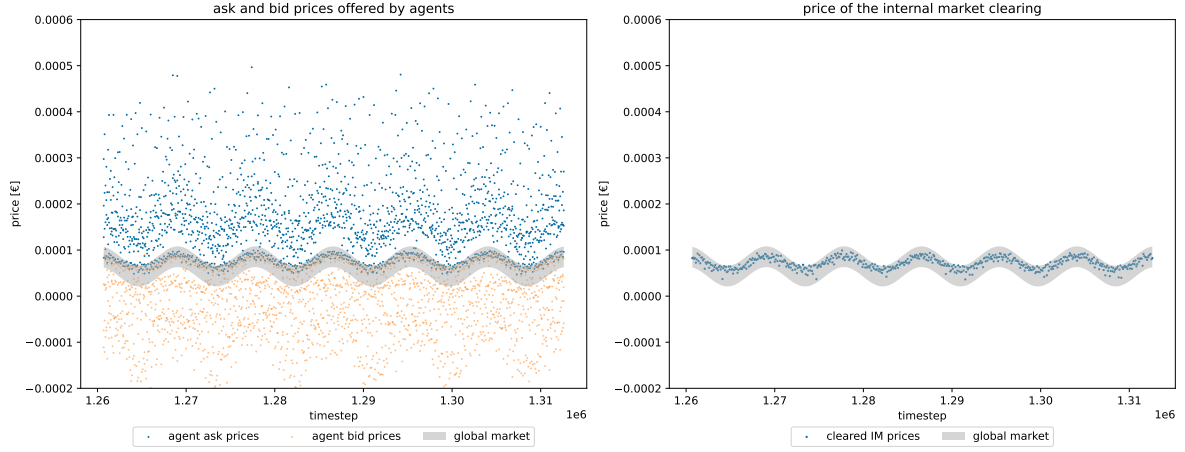


Figure 4.7: Scatter plot containing all asks, bids and internal market clearing price for experiment B.2.2 with PPO agents and a yearly oscillating global market, during the last epoch.

of the intrinsic interplay between agents.

4.2.3 A Constant Market with Sine-Wave Prosumers

The fourth scenario comprises an environment where the individual energy demands of the agents, rather than the global market, fluctuate with an amplitude of 380 Wh around a mean of -320 Wh and 320 Wh for the first four agents and the remaining agents, respectively. By choosing these specific parameters, two elements are introduced within the scenario. First, agents are no longer strict consumers or producers, as the amplitude will shift the sign of the energy demand during specific periods. Furthermore, since the sine wave energy demands of all agents are in phase with one another, there is no longer a strict energy amount equilibrium within the local market, as exemplified by a representative example in Figure 4.9. When the rule-based agents are exposed to this new environment, we observe an effect most likely caused by a specific implementation detail. As the rule-based agents are designed to always place their complete energy demand or surplus in a buying or selling offer at the current time-of-use or feed-in price, respectively, the double auction will receive a set of four asks and bids with the same respective offer prices, disabling the initial sorting of the ask and bid offers based on their offer prices to clear the internal market. To illustrate this point, consider the example presented in Figure 4.10, where four agents participate in the market. Agents 00 and 01 submit their total energy surplus of 100 Wh at a

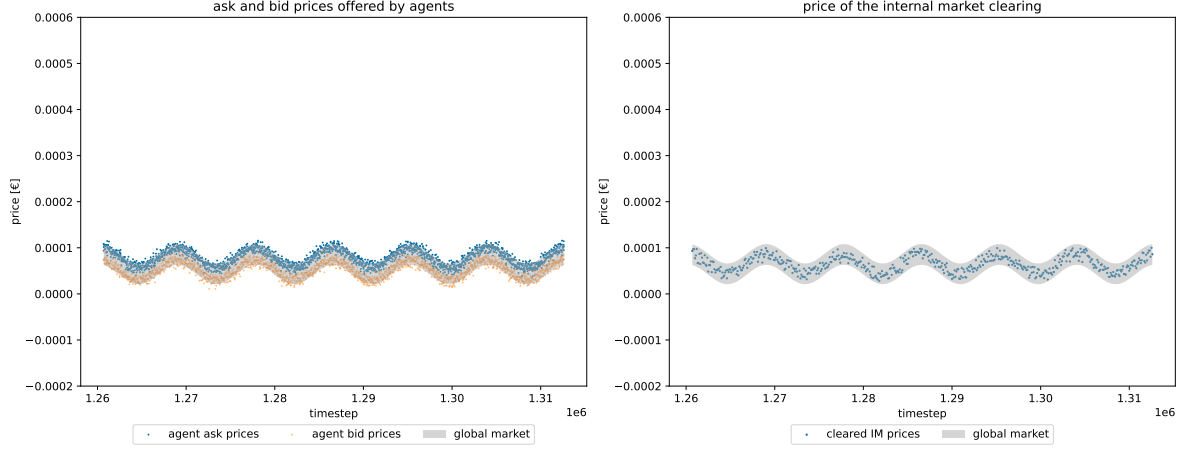


Figure 4.8: Scatter plot containing all asks, bids and internal market clearing price for experiment B.2.2 with self-play PPO agents and a yearly oscillating global market during the last epoch.

price of 0.0001 €/Wh, while agents 02 and 03 place an ask for their energy demand of 50 Wh at a price of 0.0003 €/Wh.

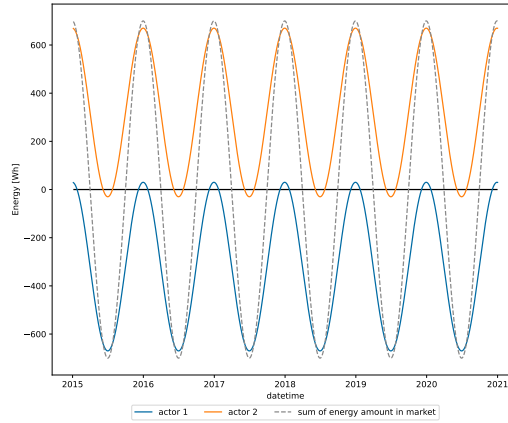


Figure 4.9: Example of a market amount imbalance due to oscillating energy demands.

When using the double auction implementation as used in the framework, only Agent 00's bid gets cleared on the internal market, while Agent 02 must fulfil its offer on the global market. Figure 4.11 shows the opposite scenario, in which now Agent 03's buying offer gets cleared on the internal market, while Agent 02 must buy its required energy demand on the global market. This implementation detail leads to a (dis)advantage between agents solely based on their index numbers, as visible in Figure B.32, where the amounts cleared on the internal markets put Agents 00 and 07 in an advantageous position to the following agents in order of their appearing within the ask and bid list. The behaviour in such specific circumstances can, and with reason, be considered a bug within the framework. Given, however, that it is highly improbable for PPO agents to submit offers at identical prices, it is anticipated that the implementation issue identified above will not adversely affect the performance of the PPO agent scenarios.

Upon examining the results of the same scenario concerning both individual and self-play PPO agents, we observe the impact of the changing availability of energy within the internal markets. Both individual PPO agents (Figure 4.12) and their self-play counterparts (Figure 4.13) demonstrate adaptive pricing behaviours in response to the prevailing energy surplus or shortage within the market. This behaviour corresponds to what one would expect in a real-life scenario, where prices typically increase

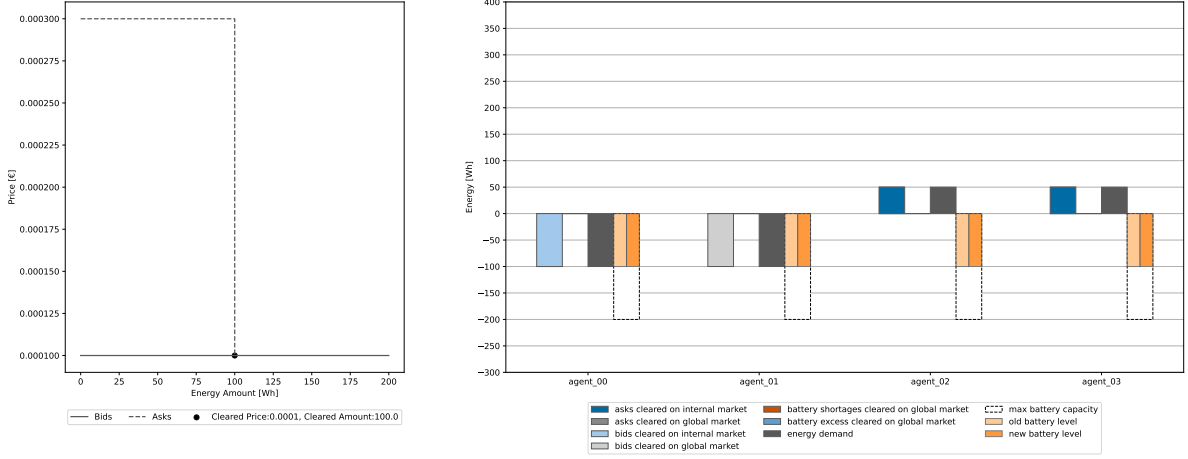


Figure 4.10: Example of a double auction clearing where four agents participate in the market. Agents 00 and 01 submit their total energy surplus of 100 Wh at a price of 0.0001 €/Wh, while agents 02 and 03 place an ask for their energy demand of 50 Wh at a price of 0.0003 €/Wh.

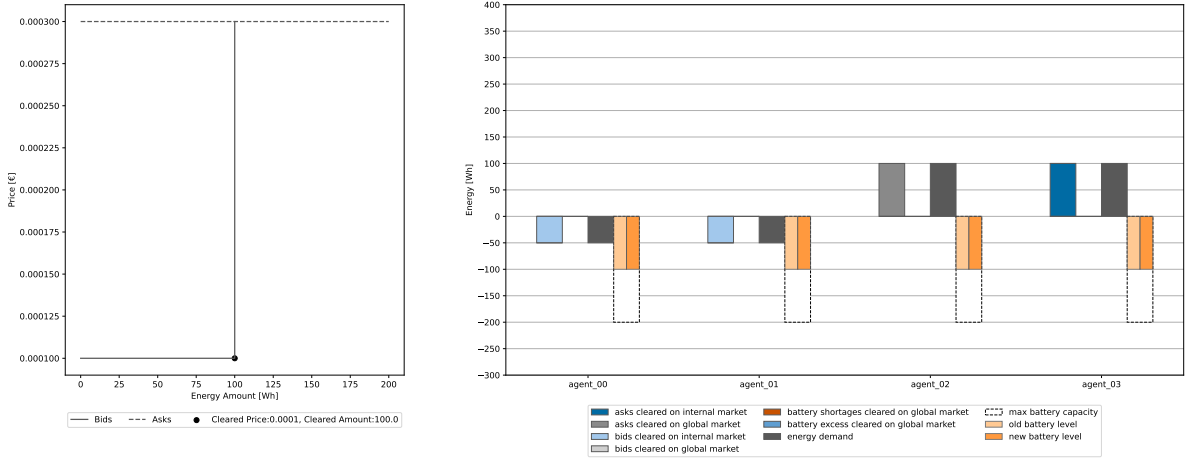


Figure 4.11: Example of a double auction clearing where four agents participate in the market. Agents 00 and 01 submit their total energy surplus of 50 Wh at a price of 0.0001 €/Wh, while agents 02 and 03 place an ask for their energy demand of 100 Wh at a price of 0.0003 €/Wh.

in response to high demand for a commodity and decrease when supply exceeds demand. However, the presence of the global market as a backup source, providing or purchasing energy at fixed time-of-use and feed-in prices, constrains these price fluctuations. The price evolution observed when applying self-play agents reveals another rather distinct pattern. While this phenomenon cannot be definitively verified, it is reasonable to hypothesise that it arises from the agents sharing one common policy. In instances of energy scarcity, all agents experiencing a deficit, therefore, tend to exhibit similar behaviours, mirroring the actions of agents with an energy surplus. This leads to internal market clearing prices approximating the time-of-use price of the global market when there is an overall energy shortage within the market while levitating towards the feed-in price when the internal market has an overall excess amount of energy. Overall, the box plots in Figures B.34 and B.37 show that, measured by the mean rewards, both PPO agents and self-play PPO agents are well capable of coping with these fluctuations in energy demand, leading to mean rewards that rank among the highest thus far.

4.2.4 Using Synthetic Data for Both Prosumers and the Global Market

In the final scenario, all three agent implementations are confronted with the synthetically generated dataset. While it is impossible to perform a detailed examination of the agents' behaviour due to the

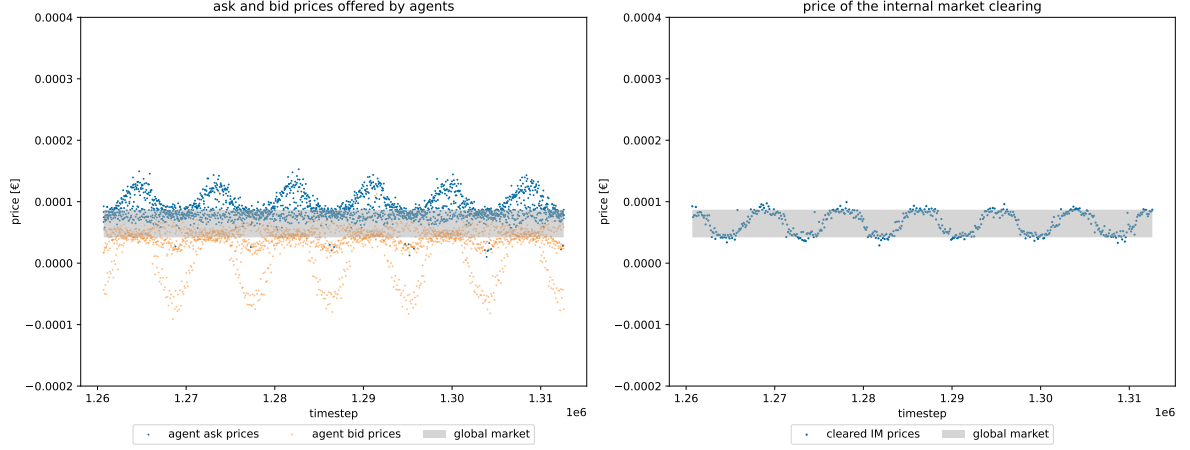


Figure 4.12: Scatter plot containing all asks, bids and internal market clearing price for experiment B.2.4 with PPO agents and a yearly oscillating global market, during the last epoch.

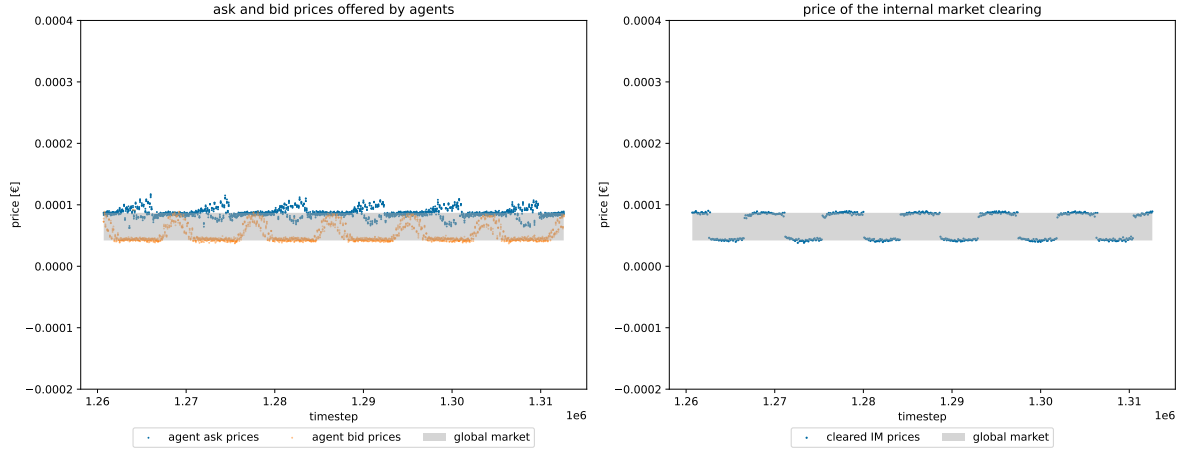


Figure 4.13: Scatter plot containing all asks, bids and internal market clearing price for experiment B.2.4 with self-play PPO agents and a yearly oscillating global market during the last epoch.

complexity and unpredictability of the underlying data, one particularly surprising observation emerges from our findings. A comparative analysis of the overall mean rewards across five executive runs, as illustrated in Figures B.39, B.42, and B.45, alongside the box plots depicting the reward distribution for run 0 in Figures B.40, B.43, and B.46, reveals that neither the individual PPO agents nor the self-play PPO agents were able to surpass the performance of the relatively simplistic rule-based agent.

The rule-based agent's straightforward strategy of consistently aligning the offer amount with the prevailing energy shortage or surplus and matching the asking or bidding price with the time-of-use or feed-in price demonstrates a surprising level of efficiency in the absence of battery storage, which would otherwise, most likely, facilitate a more nuanced, long-term strategy.

4.3 The Impact of Batteries

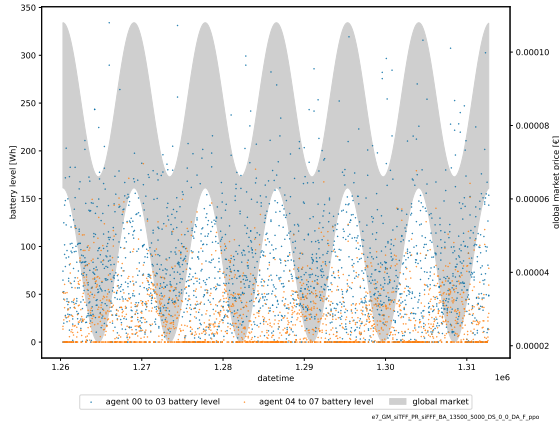
In this next phase, we introduce the availability of batteries to our agents and assess their impact on the agents' behaviour. It is essential to recognise that incorporating batteries significantly increases the complexity of the environment, as agents are now equipped to engage in long-term planning, in contrast to their previous capacity to respond solely to immediate market conditions at any given timestep. It should be noted that the first scenario, in which both the global market and the agents' energy requirement patterns remain constant, was not considered for the remainder of the experiments due to

the absence of any temporal variation in this scenario. Overall, several key observations can be made. First, based on the box plots of all scenarios under consideration, we can conclude that the presence of batteries positively impacts the overall mean rewards of most scenarios. In the scenario with a yearly fluctuating global market (Figures B.49 and B.52) and when using the synthetic data (Figures B.67 and B.70), both the individual and the self-play PPO agents perform better than without a battery. In the scenarios where the global market fluctuates both yearly and daily (Figure B.58), or when the prosumers' energy demand fluctuates (Figure B.64), the self-play agents perform better, with the latter case even resulting in an overall mean reward.

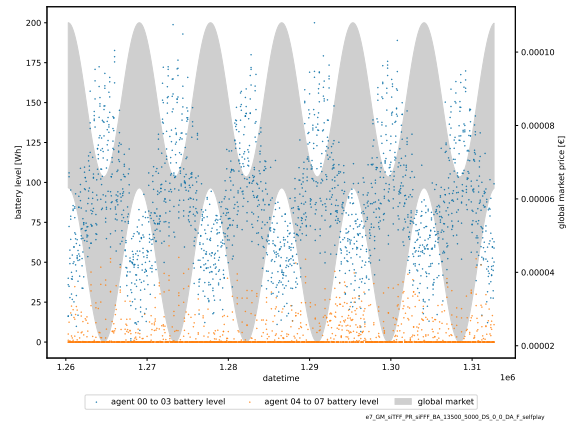
A detailed examination of the battery levels during the last epoch, as depicted in Figures 4.14 and 4.15, reveals that, except one scenario involving individual agents interacting with a yearly fluctuating global market, agents utilise their batteries to anticipate yearly fluctuations in global market prices and to respond to changing energy surpluses or shortages. Even when operating with the synthetic dataset, agents effectively charge their batteries during summer periods when energy is more abundantly available. A final observation is that in the first three scenarios where no synthetic data was used, the agents only use a small range of the battery's capacity. This most likely indicates that in relation to the rather small amount of energy demand or surplus of ± 320 Wh, the chosen battery with a capacity of 13500 Wh is drastically over-dimensioned. When confronted with more realistic values, as one can presume in the synthetic dataset, agents do tend to use the complete range of the battery.

4.4 The impact of the Day-Ahead Window and Data-Sharing between Agents

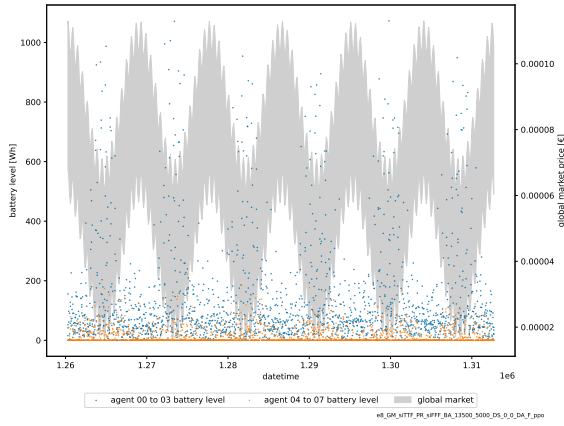
Before evaluating the impact of the day-ahead window and the data-sharing functionality, it is essential to address several preliminary considerations. First, the day-ahead window is only relevant in the context of a fluctuating global market; therefore, scenarios in which only the agents' energy demands fluctuate will be excluded from these experiments. Additionally, it is important to note that there is no direct way to visualize the impact of these functionalities. Therefore, we can only evaluate their impact based on the box plots depicting the mean reward during the last epoch in the run with seed 0. Regarding the impact of the day-ahead window, the results observed are fairly disconcerting. In all scenarios examined, the inclusion of the day-ahead window in the agents' observations did not yield an improvement in mean rewards compared to identical scenarios without this feature. At the time of writing, the reasons for this lack of improvement remain unclear, particularly given the expectation that the availability of this additional information would enable agents to better predict short-term fluctuations in global market prices. Ideally, agents could thus anticipate by using the battery buffer capacity to stockpile additional energy before the market prices go up or deplete the battery when knowing the market prices will decline in the coming 24 hours. Ideally, agents would utilize their battery buffer capacity to stockpile energy prior to anticipated price increases or to deplete their batteries when foreseeing a decline in market prices over the subsequent 24 hours. A similar conclusion arises when assessing the data-sharing option. A comparison of the individual mean rewards in Figures B.91 B.94 B.97 and B.100 of the first four agents engaged in mutual sharing of energy demand and battery level data with their counterparts that do not share data reveals no significant distinction in mean rewards within the same scenario nor can any significant difference be observed in comparison to the same scenario without data-sharing in figure B.85 versus B.88 and B.97 versus B.100. Both observations thus present issues that require further investigation.



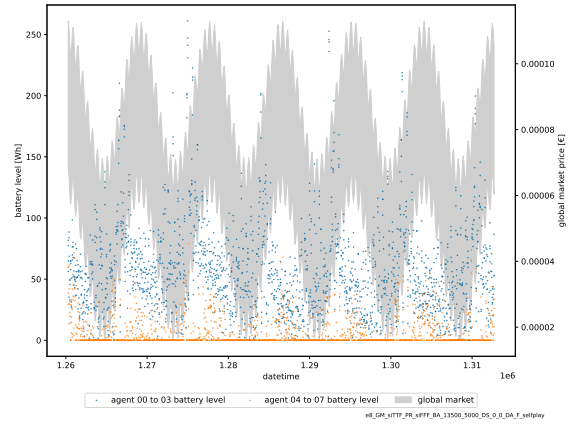
(a) Individual PPO agents



(b) Self-play PPO agents



(c) Individual PPO agents



(d) Self-play PPO agents

Figure 4.14: Plots representing the battery levels of individual and self-play PPO agents with a fluctuating global market as conducted in experiments B.3.1 (top) and B.3.2 (bottom). Note that the daily fluctuations within the bottom plots are not correctly represented due to the logging sample rate of 100 timesteps.

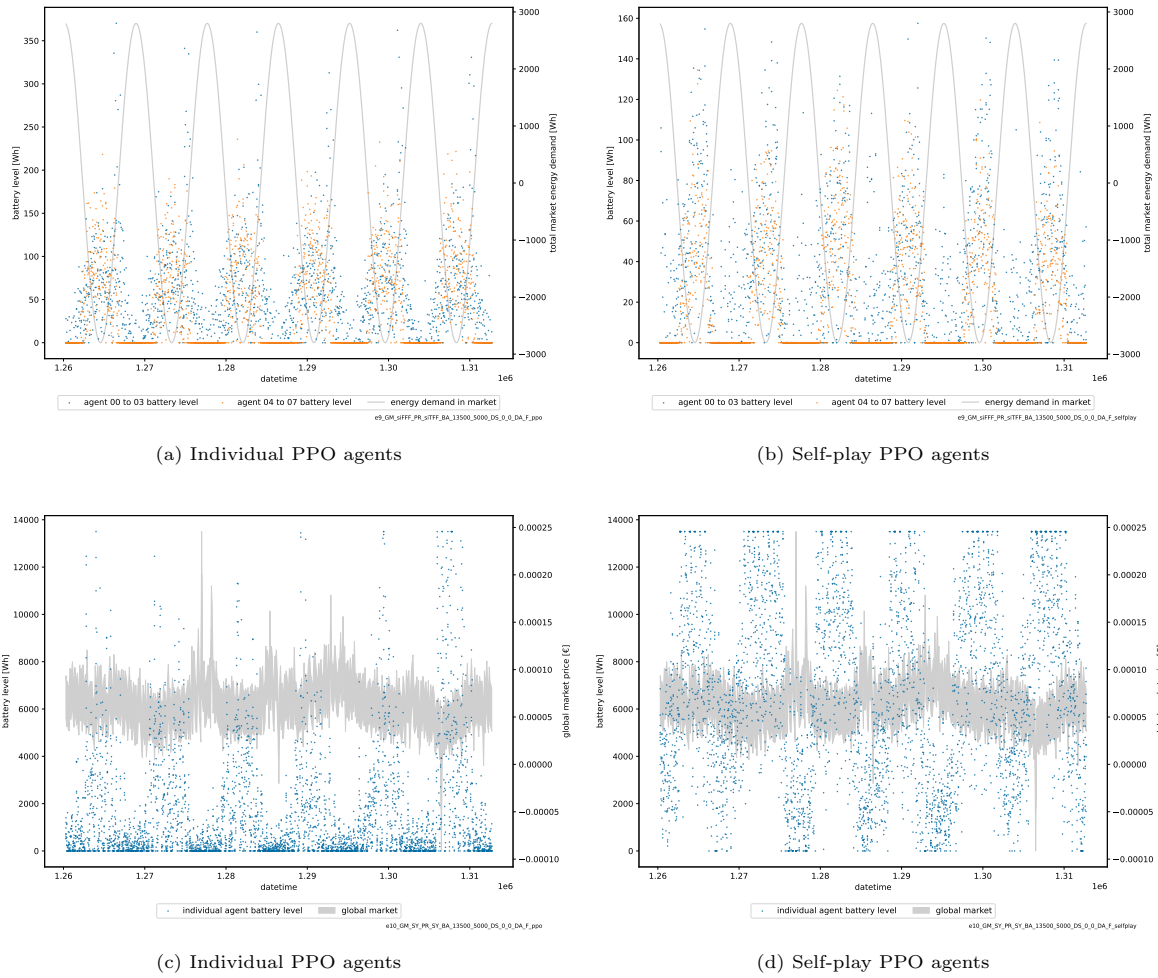


Figure 4.15: Plots representing the battery levels of individual and self-play PPO agents with a fluctuating energy demand as conducted in experiment B.3.3 (top) and the synthetic data as conducted in experiment B.3.4 (bottom).

Chapter 5

Conclusion

In this thesis, we presented a novel JAX framework for simulating a decentralised energy market. This framework is designed to facilitate future research and the evaluation of both existing and innovative agent algorithms in the context of these market scenarios. We detailed the overall architecture of the framework, the design choices and limitations encountered during the framework’s development, and the implementation specifics concerning the market environment, the global market, and the prosumers participating in this decentralised system.

The framework underwent a series of testing scenarios that validated its functionality and showcased its capacity to allow agents employing different algorithmic variations to learn and adopt effective trading policies. A significant finding was the behavioural distinction between individual and self-play Proximal Policy Optimization agents; the latter, by sharing a joint neural network, exhibited a more consistent policy across all agents. In contrast, individual PPO agents displayed markedly different behaviours, where two agents were compelled to place offers within strict limits to secure favourable clearing prices, while others exploited this situation to maximise their chances of having their offers cleared. Notably, the experiments highlighted the advantageous role of home battery systems, enabling agents to anticipate seasonal fluctuations in energy demand and market conditions.

However, introducing a day-ahead window, which ostensibly provided agents with foresight into future market fluctuations, did not yield the expected improvement in policy performance. Furthermore, the investigation into sharing energy demand and battery level data among a subset of agents did not reveal any evidence of collaboration or market manipulation that could enhance the agents’ overall reward.

Given the extensive range of configuration options, not all conceivable experiments could be conducted, presenting a clear path for future research. Potential experimental setups could include the implementation of a centralised learning agent acting on behalf of all agents or mixing various agent implementation types, such as rule-based and PPO agents, to assess whether the latter can leverage the more straightforward strategies of the former. Additionally, future investigations could explore the impact of different battery installation configurations to determine optimal sizes under varying market conditions or examine the influence of larger pools of agents, which may uncover more complex behaviours or even the formation of coalitions.

While the framework has been shown to be effective in its current form, there remains ample opportunity for further optimization and enhancement of the framework’s functionality and realism. Future work could focus on accommodating more complex and realistic testing scenarios, such as developing more sophisticated reward functions that better align with the intricacies of real-world energy trading. Additionally, incorporating limitations on the maximum amount of energy exchanged between the internal market and the global market would reflect the technical constraints inherent in actual energy networks, thereby increasing the realism and applicability of the simulations. Moreover, while significant implementation efforts would be required to introduce the necessary thermodynamical models and ensure a realistic experimental setup, the framework’s potential for application in other domains, such as heat networks, is an area worthy of exploration.

Bibliography

- Albrecht, S. V., Christianos, F., & Schäfer, L. (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press. <https://www.marl-book.com>
- Ancona, M. A., Bianchi, M., Branchini, L., De Pascale, A., Melino, F., Peretto, A., & Rosati, J. (2021). Influence of the Prosumer Allocation and Heat Production on a District Heating Network. *Frontiers in Mechanical Engineering*, 7. <https://doi.org/10.3389/fmech.2021.623932>
- Bari, A., Jiang, J., Saad, W., & Jaekel, A. (2014). Challenges in the Smart Grid Applications: An Overview. *International Journal of Distributed Sensor Networks*, 10(2), 974682. <https://doi.org/10.1155/2014/974682>
- Boesten, S., Ivens, W., Dekker, S. C., & Eijdens, H. (2019). 5th generation district heating and cooling systems as a solution for renewable urban thermal energy supply. *Advances in Geosciences*, 49, 129–136. <https://doi.org/10.5194/adgeo-49-129-2019>
- Boyd, M. T. (2016, March). *NIST Weather Station for Photovoltaic and Building System Research* (tech. rep. No. NIST TN 1913). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.TN.1913>
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). JAX: Composable transformations of Python+NumPy programs. <http://github.com/google/jax>
- Buffa, S., Cozzini, M., D’Antoni, M., Baratieri, M., & Fedrizzi, R. (2019). 5th generation district heating and cooling systems: A review of existing cases in Europe. *Renewable and Sustainable Energy Reviews*, 104, 504–522. <https://doi.org/10.1016/j.rser.2018.12.059>
- Bukar, A. L., Hamza, M. F., Ayub, S., Abobaker, A. K., Modu, B., Mohseni, S., Brent, A. C., Ogbonnaya, C., Mustapha, K., & Idakwo, H. O. (2023). Peer-to-peer electricity trading: A systematic review on current developments and perspectives. *Renewable Energy Focus*, 44, 317–333. <https://doi.org/10.1016/j.ref.2023.01.008>
- Bünning, F., Wetter, M., Fuchs, M., & Müller, D. (2018). Bidirectional low temperature district energy systems with agent-based control: Performance comparison and operation optimization. *Applied Energy*, 209, 502–515. <https://doi.org/10.1016/j.apenergy.2017.10.072>
- Calvano, E., Calzolari, G., Denicolò, V., & Pastorello, S. (2019, April). Artificial Intelligence, Algorithmic Pricing and Collusion. <https://doi.org/10.2139/ssrn.3304991>
- Calvin, K., Dasgupta, D., Krinner, G., Mukherji, A., Thorne, P. W., Trisos, C., Romero, J., Aldunce, P., Barrett, K., Blanco, G., Cheung, W. W., Connors, S., Denton, F., Diongue-Niang, A., Dodman, D., Garschagen, M., Geden, O., Hayward, B., Jones, C., ... Péan, C. (2023, July). *IPCC, 2023: Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change [Core Writing Team, H. Lee and J. Romero (eds.)]*. IPCC, Geneva, Switzerland. (tech. rep.). Intergovernmental Panel on Climate Change (IPCC). <https://doi.org/10.59327/IPCC/AR6-9789291691647>
- Cramton, P. (2017). Electricity market design. *Oxford Review of Economic Policy*, 33(4), 589–612. <https://doi.org/10.1093/oxrep/grx041>
- de Witt, C. S., Gupta, T., Makoviychuk, D., Makoviychuk, V., Torr, P. H. S., Sun, M., & Whiteson, S. (2020, November). Is Independent Learning All You Need in the StarCraft Multi-Agent Challenge? <https://doi.org/10.48550/arXiv.2011.09533>
- European Commission. (2020). Communication from the Commission to the European Parliament, the Council, the European Economic and Social Committee and the Committee of the Regions: Powering a climate-neutral economy: An EU Strategy for Energy System Integration. Retrieved

- December 25, 2023, from <https://eur-lex.europa.eu/legal-content/EN/ALL/?uri=COM:2020:299:FIN>
- Gronauer, S., & Diepold, K. (2022). Multi-agent deep reinforcement learning: A survey. *Artificial Intelligence Review*, 55(2), 895–943. <https://doi.org/10.1007/s10462-021-09996-w>
- Harrold, D. J. B., Cao, J., & Fan, Z. (2022). Renewable energy integration and microgrid energy trading using multi-agent deep reinforcement learning. *Applied Energy*, 318, 119151. <https://doi.org/10.1016/j.apenergy.2022.119151>
- He, L., Liu, Y., & Zhang, J. (2021). Peer-to-peer energy sharing with battery storage: Energy pawn in the smart grid. *Applied Energy*, 297, 117129. <https://doi.org/10.1016/j.apenergy.2021.117129>
- Hebrail, G., & Berard, A. (2012). Individual Household Electric Power Consumption. <https://doi.org/https://doi.org/10.24432/C58K54>
- Heek, J., Levskaya, A., Oliver, A., Ritter, M., Rondepierre, B., Steiner, A., & van Zee, M. (2023). Flax: A neural network library and ecosystem for JAX. <http://github.com/google/flax>
- Holmgren, W., Hansen, C., & Mikofski, M. (2018). Pvlib python: A python package for modeling solar energy systems. *Journal of Open Source Software*, 3, 884. <https://doi.org/10.21105/joss.00884>
- Horgan, D., Quan, J., Budden, D., Barth-Maron, G., Hessel, M., van Hasselt, H., & Silver, D. (2018, March). Distributed Prioritized Experience Replay. <https://doi.org/10.48550/arXiv.1803.00933>
- Hubbard, T. P., & Paarsch, H. J. (2015). *Auctions*. MIT Press.
- Idries, A., Krogstie, J., & Rajasekharan, J. (2022). Challenges in platforming and digitizing decentralized energy services. *Energy Informatics*, 5(1), 8. <https://doi.org/10.1186/s42162-022-00193-9>
- JAX. (2024). JAX-The Sharp Bits—JAX documentation. Retrieved August 11, 2024, from https://jax.readthedocs.io/en/latest/notebooks/Common_Gotchas_in_JAX.html
- Keren, S., Essayeh, C., Albrecht, S. V., & Morstyn, T. (2024, May). Multi-Agent Reinforcement Learning for Energy Networks: Computational Challenges, Progress and Open Problems. <https://doi.org/10.48550/arXiv.2404.15583>
- Khorasany, M., Mishra, Y., & Ledwich, G. (2017). Auction based energy trading in transactive energy market with active participation of prosumers and consumers. *2017 Australasian Universities Power Engineering Conference (AUPEC)*, 1–6. <https://doi.org/10.1109/AUPEC.2017.8282470>
- Kirschen, D. S., & Strbac, G. (2004). *Fundamentals of power system economics*. John Wiley & Sons.
- Lange, R. T. (2022). Gymnax: A {JAX}-based Reinforcement Learning Environment Library. <http://github.com/RobertTLange/gymnax>
- Liang, Z., & Su, W. (2018). Game theory based bidding strategy for prosumers in a distribution system with a retail electricity market. *IET Smart Grid*, 1(3), 104–111. <https://doi.org/10.1049/iet-stg.2018.0048>
- Mashler, M., Solan, E., & Zamir, S. (2013). *Game theory* (M. Borns, Ed.; Z. Hellman, Trans.). Cambridge University Press.
- Maurer, L. T. A., Barroso, L. A., & Chang, J. M. (2011). *Electricity auctions: An overview of efficient practices*. World Bank. <https://doi.org/10.1596/978-0-8213-8822-8>
- Meeus, L. (2020, November). *The Evolution of Electricity Markets in Europe*. Edward Elgar Publishing. <https://doi.org/10.4337/9781789905472>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>
- Mumbere, K. S., Sasaki, Y., Yorino, N., Zoka, Y., Tanioka, Y., & Bedawy, A. (2023). A Resilient Prosumer Model for Microgrid Communities with High PV Penetration. *Energies*, 16(2), 621. <https://doi.org/10.3390/en16020621>
- Nadeem, T. B., Siddiqui, M., Khalid, M., & Asif, M. (2023). Distributed energy systems: A review of classification, technologies, applications, and policies. *Energy Strategy Reviews*, 48, 101096. <https://doi.org/10.1016/j.esr.2023.101096>
- Oduro, R. A., & Taylor, P. G. (2023). Future pathways for energy networks: A review of international experiences in high income countries. *Renewable and Sustainable Energy Reviews*, 171, 113002. <https://doi.org/10.1016/j.rser.2022.113002>
- OpenAI, Akkaya, I., Andrychowicz, M., Chociej, M., Litwin, M., McGrew, B., Petron, A., Paino, A., Plappert, M., Powell, G., Ribas, R., Schneider, J., Tezak, N., Tworek, J., Welinder, P., Weng, L., Yuan, Q., Zaremba, W., & Zhang, L. (2019, October). Solving Rubik’s Cube with a Robot Hand. <https://doi.org/10.48550/arXiv.1910.07113>

- OpenAI, Berner, C., Brockman, G., Chan, B., Cheung, V., Debiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Jozefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., Pinto, H. P. d. O., Raiman, J., Salimans, T., . . . Zhang, S. (2019, December). Dota 2 with Large Scale Deep Reinforcement Learning. <https://doi.org/10.48550/arXiv.1912.06680>
- OpenXLA. (n.d.). OpenXLA Project. Retrieved August 11, 2024, from <https://openxla.org/>
- Parag, Y., & Sovacool, B. K. (2016). Electricity market design for the prosumer era. *Nature Energy*, 1(4), 1–6. <https://doi.org/10.1038/nenergy.2016.32>
- Parra-Domínguez, J., Sánchez, E., & Ordóñez, Á. (2023). The Prosumer: A Systematic Review of the New Paradigm in Energy and Sustainable Development. *Sustainability*, 15(13), 10552. <https://doi.org/10.3390/su151310552>
- Pérez-Arriaga, I. J. (Ed.). (2013). *Regulation of the Power Sector* (1st ed.). Springer London. <https://doi.org/10.1007/978-1-4471-5034-3>
- Peters, H. J. M. (2015). *Game theory: A multi-leveled approach* (2. ed). Springer.
- Pullinger, M., Kilgour, J., Goddard, N., Berliner, N., Webb, L., Dzikovska, M., Lovell, H., Mann, J., Sutton, C., Webb, J., & Zhong, M. (2021). The IDEAL household energy dataset, electricity, gas, contextual sensor data and survey data for 255 UK homes. *Scientific Data*, 8(1), 146. <https://doi.org/10.1038/s41597-021-00921-y>
- Python Software Foundation. (2024, August). Datetime — Basic date and time types. Retrieved August 24, 2024, from <https://docs.python.org/3/library/datetime.html>
- Qiu, D., Wang, J., Wang, J., & Strbac, G. (2021). Multi-Agent Reinforcement Learning for Automated Peer-to-Peer Energy Trading in Double-Side Auction Market. *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, 2913–2920. <https://doi.org/10.24963/ijcai.2021/401>
- Reinhardt, A., & Klemenjak, C. (2020). How does Load Disaggregation Performance Depend on Data Characteristics?: Insights from a Benchmarking Study. *Proceedings of the Eleventh ACM International Conference on Future Energy Systems*, 167–177. <https://doi.org/10.1145/3396851.3397691>
- Rutherford, A., Ellis, B., Gallici, M., Cook, J., Lupu, A., Ingvarsson, G., Willi, T., Khan, A., de Witt, C. S., Souly, A., Bandyopadhyay, S., Samvelyan, M., Jiang, M., Lange, R. T., Whiteson, S., Lacerda, B., Hawes, N., Rocktaschel, T., Lu, C., & Foerster, J. N. (2023, December). JaxMARL: Multi-Agent RL Environments in JAX. <https://doi.org/10.48550/arXiv.2311.10090>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017, August). Proximal Policy Optimization Algorithms. <https://doi.org/10.48550/arXiv.1707.06347>
- Shin, C., Lee, E., Han, J., Yim, J., Rhee, W., & Lee, H. (2019). The ENERTALK dataset, 15 Hz electricity consumption data from 22 houses in Korea. *Scientific Data*, 6(1), 193. <https://doi.org/10.1038/s41597-019-0212-5>
- Silver, D., Huang, A., Maddison, C., Guez, A., Sifre, L., Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529, 484–489. <https://doi.org/10.1038/nature16961>
- Sun, H., Liu, N., Tan, L., & Han, J. (2023). Data-driven energy sharing for multi-microgrids with building prosumers: A hybrid learning approach. *IET Renewable Power Generation*, n/a(n/a). <https://doi.org/10.1049/rpg2.12821>
- Šúri, M., Huld, T. A., & Dunlop, E. D. (2005). PV-GIS: A web-based solar radiation database for the calculation of PV potential in Europe. *International Journal of Sustainable Energy*, 24(2), 55–67. <https://doi.org/10.1080/14786450512331329556>
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (Second edition). The MIT Press.
- Terry, J., Black, B., Grammel, N., Jayakumar, M., Hari, A., Sullivan, R., Santos, L. S., Dieffendahl, C., Horsch, C., Perez-Vicente, R., Williams, N., Lokesh, Y., & Ravi, P. (2021). PettingZoo: Gym for multi-agent reinforcement learning. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, & J. W. Vaughan (Eds.), *Advances in neural information processing systems* (pp. 15032–15043, Vol. 34). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2021/file/7ed2d3454c5eea71148b11d0c25104ff-Paper.pdf
- Tesla. (2024). Powerwall. Retrieved August 21, 2024, from <https://www.tesla.com/powerwall>

- Tushar, W., Saha, T. K., Yuen, C., Azim, M. I., Morstyn, T., Poor, H. V., Niyato, D., & Bean, R. (2020). A coalition formation game framework for peer-to-peer energy trading. *Applied Energy*, 261, 114436. <https://doi.org/10.1016/j.apenergy.2019.114436>
- Vázquez-Canteli, J. R., Dey, S., Henze, G., & Nagy, Z. (2020). CityLearn: Standardizing Research in Multi-Agent Reinforcement Learning for Demand Response and Urban Energy Management. *ArXiv*.
- VisualStudioCode. (2024, August). Developing inside a Container using Visual Studio Code Remote Development. Retrieved August 21, 2024, from <https://code.visualstudio.com/docs/devcontainers/containers>
- Vogt, S., Schreiber, J., & Sick, B. (2022, April). Synthetic Photovoltaic and Wind Power Forecasting Data. <https://doi.org/10.48550/arXiv.2204.00411>
- VREG. (2018, December). Nettareven. Retrieved August 20, 2024, from <https://www.vreg.be/nl/nettarieven>
- Wang, X., Wang, Z., Mu, Y., Deng, Y., & Jia, H. (2023). Rolling horizon optimization for real-time operation of prosumers with Peer-to-Peer energy trading. *Energy Reports*, 9, 321–328. <https://doi.org/10.1016/j.egyr.2022.10.438>
- Werner, S. (2017). International review of district heating and cooling. *Energy*, 137, 617–631. <https://doi.org/10.1016/j.energy.2017.04.045>
- Wong, A., Bäck, T., Kononova, A. V., & Plaat, A. (2023). Deep multiagent reinforcement learning: Challenges and directions. *Artificial Intelligence Review*, 56(6), 5023–5056. <https://doi.org/10.1007/s10462-022-10299-x>
- Xu, Z. (2019, July). *The electricity market design for decentralized flexibility sources*: (tech. rep.). Oxford Institute for Energy Studies. <https://doi.org/10.26889/9781784671433>
- Yao, T., Wang, J., Wu, H., Zhang, P., Li, S., Wang, Y., Chi, X., & Shi, M. (2021). A photovoltaic power output dataset: Multi-source photovoltaic power output dataset with Python toolkit. *Solar Energy*, 230, 122–130. <https://doi.org/10.1016/j.solener.2021.09.050>
- Zhang, K., Yang, Z., & Başar, T. (2021, April). Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms. <https://doi.org/10.48550/arXiv.1911.10635>
- Zhang, R., Xu, Z., Ma, C., Yu, C., Tu, W.-W., Huang, S., Ye, D., Ding, W., Yang, Y., & Wang, Y. (2024, August). A Survey on Self-play Methods in Reinforcement Learning. <https://doi.org/10.48550/arXiv.2408.01072>

Referential Information

A.1 VREG Yearly Distribution Tariffs for 2024.

[illegible]

Figure A.1: Vlaamse Regulator van de Elektriciteits- en Gasmarkt (VREG) Yearly distribution Tariffs for 2024
(Source: Vlaamse Regulator van de Elektriciteits- en Gasmarkt at <https://www.vreg.be/nl/nettarieven>)

Appendix B

Experimental results

B.1 Overview of the Experiments

The following pages contain the general results for all experiments conducted in the context of this Master’s Thesis, for which an overview can be found in table B.1. During the experiments’ execution, each experimental setup and its respective configuration file within the code repository was systematically assigned a unique identifier, adhering to a structured nomenclature designed to encapsulate critical configuration parameters. The identifier follows the format:

$$e\langle A \rangle_{GM} \langle Sgm \rangle [\langle Ygm \rangle \langle Dgm \rangle \langle Ngm \rangle]_{PR} \langle Spr \rangle [\langle Ypr \rangle \langle Dpr \rangle \langle Npr \rangle]_{BA} \langle BC \rangle \langle BPC \rangle_{DS} \langle DSy \rangle \langle DSn \rangle_{DA} \langle DA \rangle$$

In this schema, $\langle A \rangle$ serves as an index to identify each experiment distinctly. In contrast, $\langle Sgm \rangle$ denotes the methodology employed to generate the global market day-ahead prices, with ‘si’ indicating the utilisation of the mathematical function (3.3) and ‘SY’ signifying the use of the synthetic dataset described in paragraphs 3.3.2 and 3.3.3. In instances where the mathematical function is applied, the suffix ‘si’ is further qualified by three letters T (true) or F (false), which indicate whether yearly fluctuations $\langle Ygm \rangle$, daily fluctuations $\langle Dgm \rangle$, or noise $\langle Ngm \rangle$ were added to the price signal, respectively, using the values summarised in Table 4.2. A similar structure was employed for the prosumer configurations, using the ‘PR’ prefix and ‘pr’ post-fix. To denote the inclusion of battery systems within the experimental framework, the prefix ‘BA’ is succeeded by two numerical values: $\langle BC \rangle$ and $\langle BPC \rangle$, which correspond to the battery’s capacity and peak capacity, both measured in Watt-hours. Furthermore, the prefix ‘DS’ indicates the extent of data sharing among agents, with $\langle DSy \rangle$ and $\langle DSn \rangle$ denoting the number of agents (starting from index 00) that shared their data and the number of non-data-sharing agents, respectively. Lastly, the inclusion of the day-ahead window in the agents’ observations is denoted by the $\langle DA \rangle$ parameter, which is set to ‘T’ (true) if the day-ahead window is included or ‘F’ (false) if it is obfuscated from the agents’ view.

Experiment	Experiment code
Experiment B.2.1	e1_GM_siFFF_PR_siFFF_BA_0.0_DS_0.0_DA_F
Experiment B.2.2	e2_GM_siTFF_PR_siFFF_BA_0.0_DS_0.0_DA_F
Experiment B.2.3	e3_GM_siTTF_PR_siFFF_BA_0.0_DS_0.0_DA_F
Experiment B.2.4	e4_GM_siFFF_PR_siTFF_BA_0.0_DS_0.0_DA_F
Experiment B.2.5	e5_GM_SY_PR_SY_BA_0.0_DS_0.0_DA_F
Experiment B.3.1	e7_GM_siTFF_PR_siFFF_BA_13500_5000_DS_0.0_DA_F
Experiment B.3.2	e8_GM_siTTF_PR_siFFF_BA_13500_5000_DS_0.0_DA_F
Experiment B.3.3	e9_GM_siFFF_PR_siTFF_BA_13500_5000_DS_0.0_DA_F
Experiment B.3.4	e10_GM_SY_PR_SY_BA_13500_5000_DS_0.0_DA_F
Experiment B.4.1	e12_GM_siTFF_PR_siFFF_BA_13500_5000_DS_0.0_DA_T
Experiment B.4.2	e13_GM_siTTF_PR_siFFF_BA_13500_5000_DS_0.0_DA_T
Experiment B.4.3	e15_GM_SY_PR_SY_BA_13500_5000_DS_0.0_DA_T
Experiment B.5.1	e19_GM_siFFF_PR_siTFF_BA_13500_5000_DS_0.0_DA_T
Experiment B.5.2	e20_GM_SY_PR_SY_BA_13500_5000_DS_0.0_DA_T

Table B.1: Experiments and their respective codes as used in the code repository

B.2 With no Batteries, Day-Ahead Window or Data Sharing

B.2.1 Cte. Prosumers and Cte. Global Market

Parameter	Setting	Parameter	Setting
global market data generation	formula	global market Time-of-use cost	0.000045 €/Wh
prosumers data generation	formula	global market mean day-ahead value	0.000042 €/Wh
Day-ahead window	deactivated	global market yearly day-ahead amplitude	0 €/Wh
Data Sharing	deactivated	global market daily day-ahead amplitude	0 €/Wh
Producer agents	00,01,02,03	prosumer mean energy demand value	+/-320 Wh
Consumer Agents	04,05,06,07	prosumer yearly energy demand amplitude	0 Wh

Table B.2: Overview of settings for experiment B.2.1

Rule-Based Agents

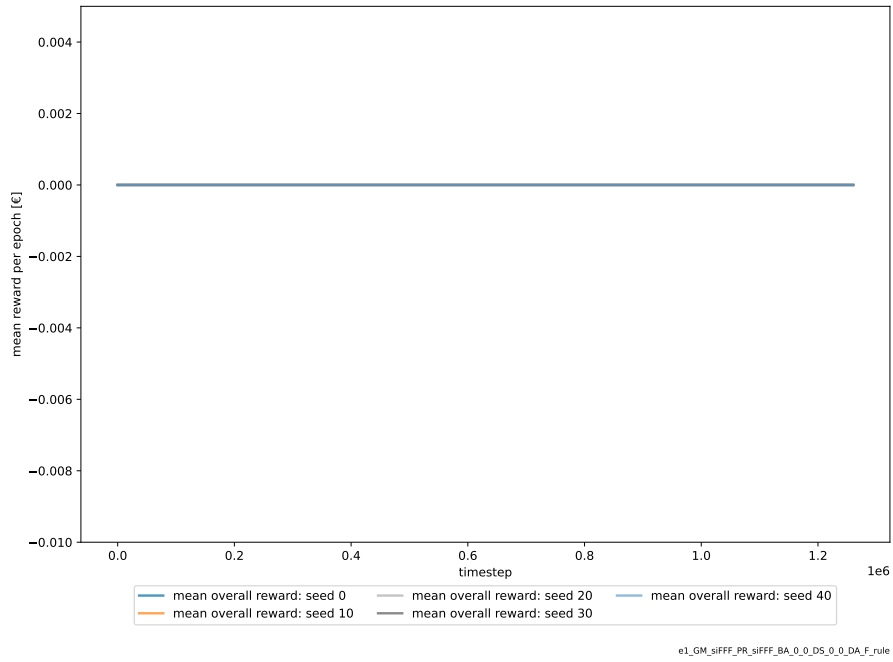


Figure B.1: Overview of the mean rewards per epoch for all agents combined for experiment B.2.1 with rule-based agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

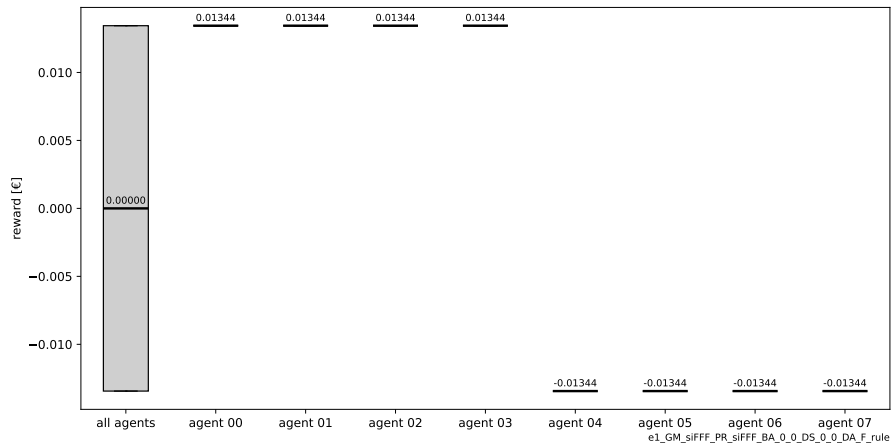
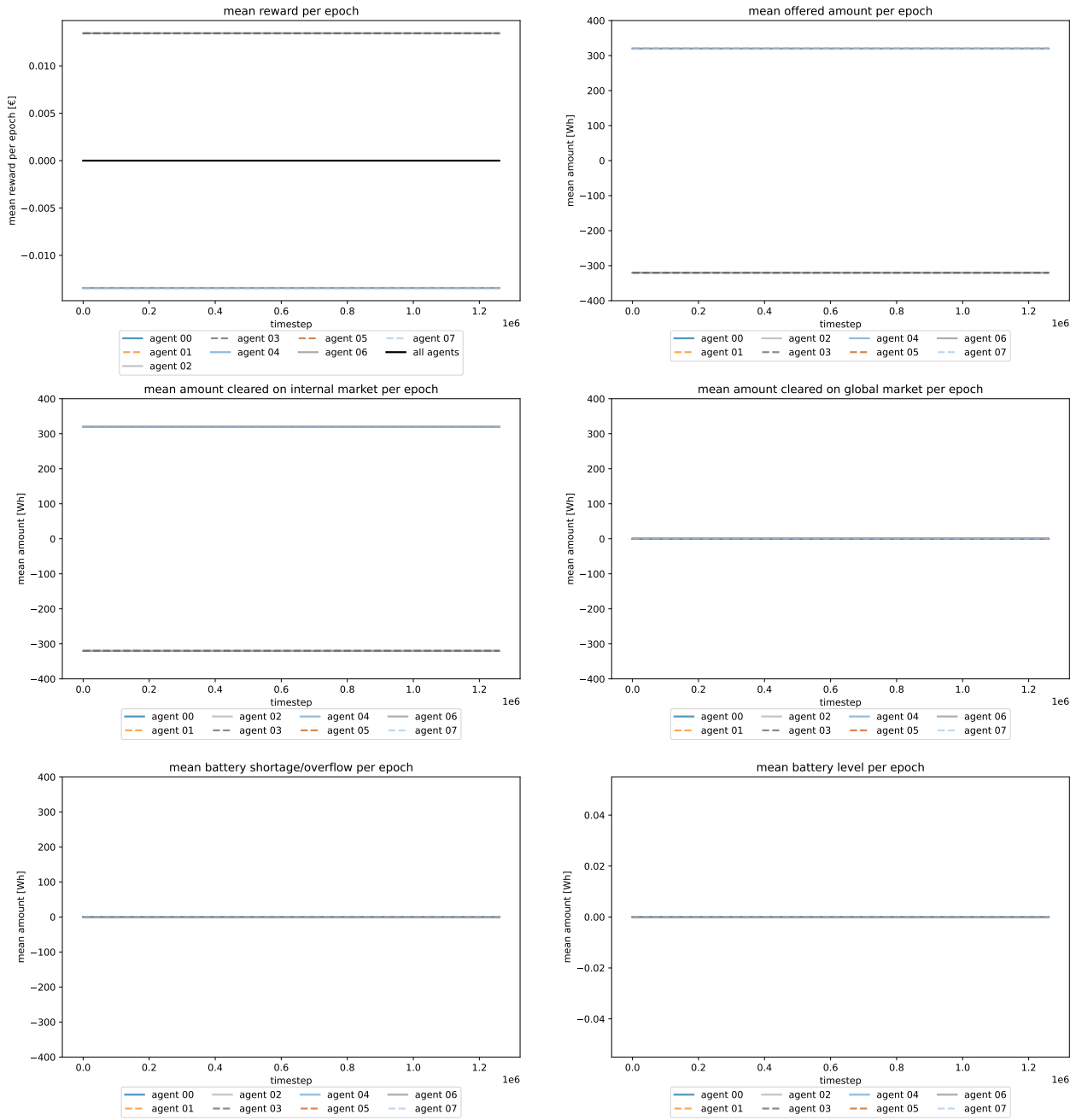


Figure B.2: Reward distribution in the last epoch for the run with seed 0 of experiment B.2.1 with rule-based agents.



e1_GM_sifff_PR_sifff_BA_0_0_DS_0_0_DA_F_rule

Figure B.3: Detailed view of key metrics for the run with seed 0 of experiment B.2.1 with rule-based agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

PPO Agents

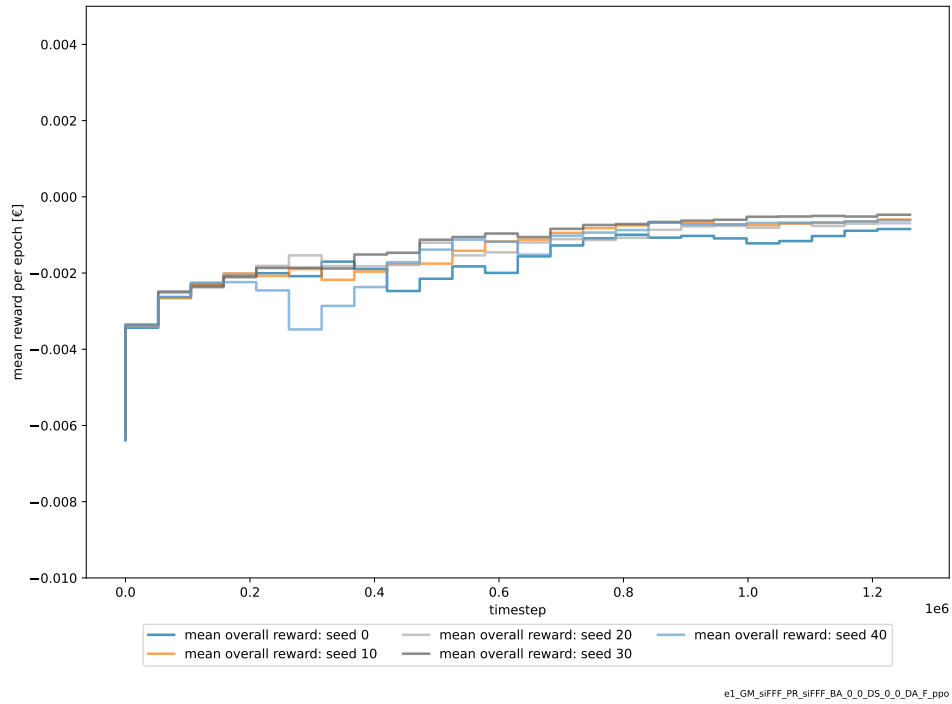


Figure B.4: Overview of the mean rewards per epoch for all agents combined for experiment B.2.1 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

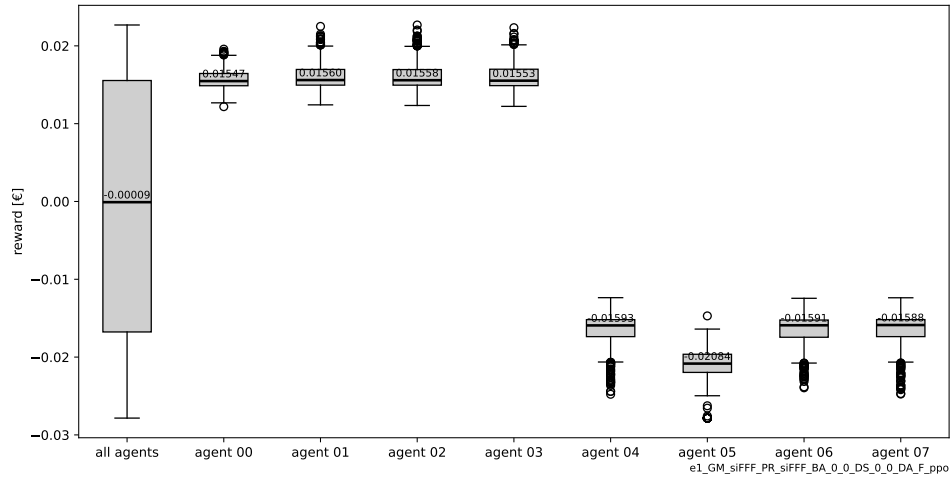
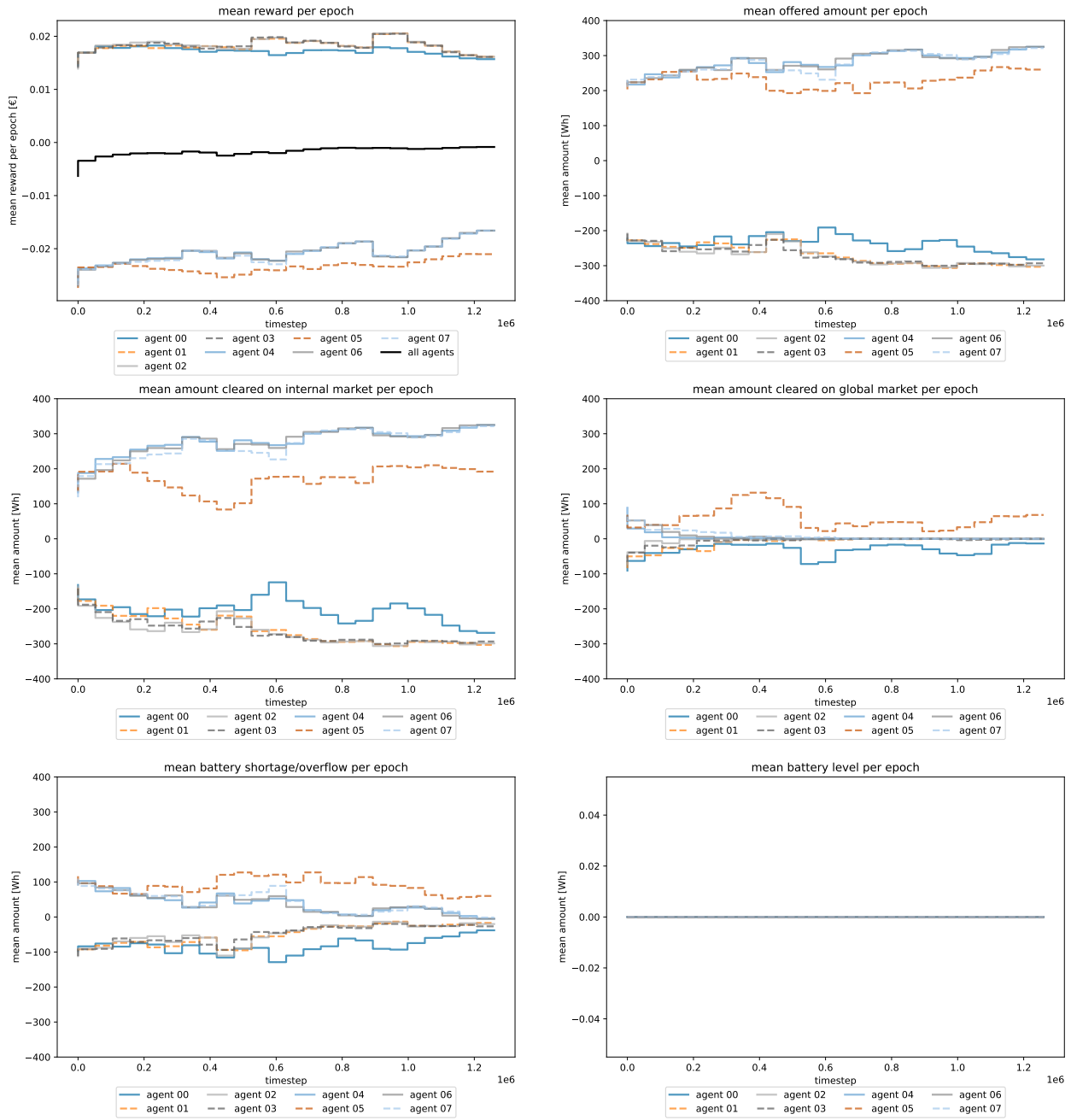


Figure B.5: Reward distribution in the last epoch for the run with seed 0 for experiment B.2.1 with PPO agents.



e1_GM_siFFF_PR_siFFF_BA_0_0_DS_0_0_DA_F_ppo

Figure B.6: Detailed view of key metrics for the run with seed 0 of experiment B.2.1 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents

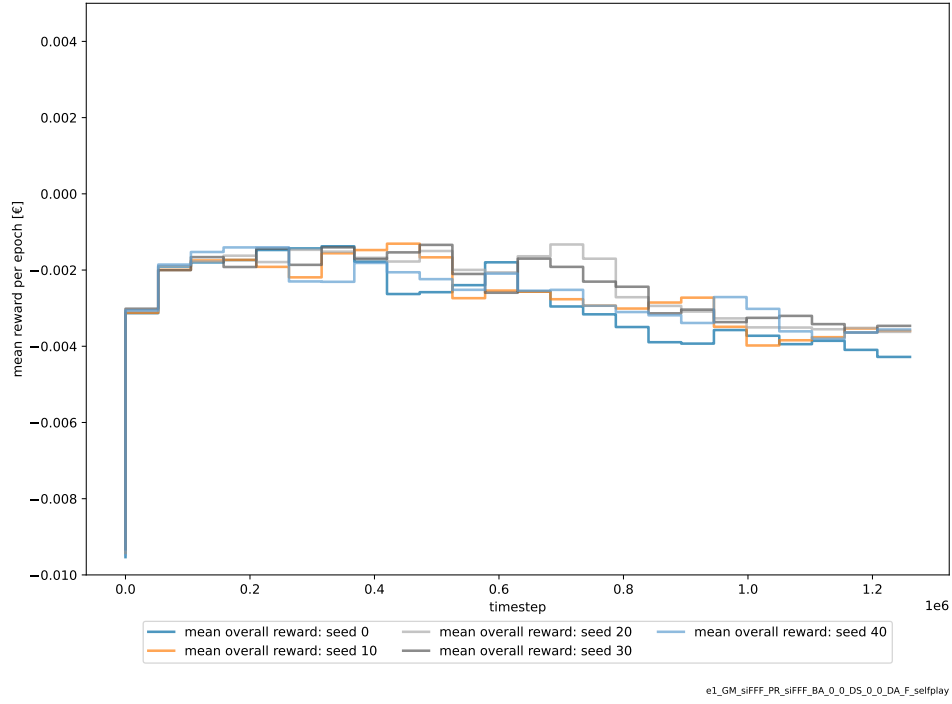


Figure B.7: Overview of the mean rewards per epoch for all agents combined for experiment B.2.1 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

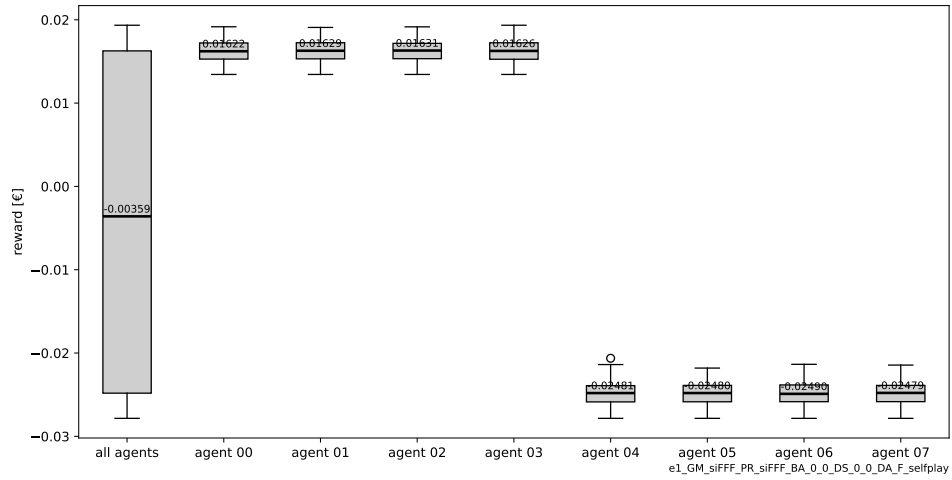


Figure B.8: Reward distribution in the last epoch for the run with seed 0 for experiment B.2.1 with self-play PPO agents.

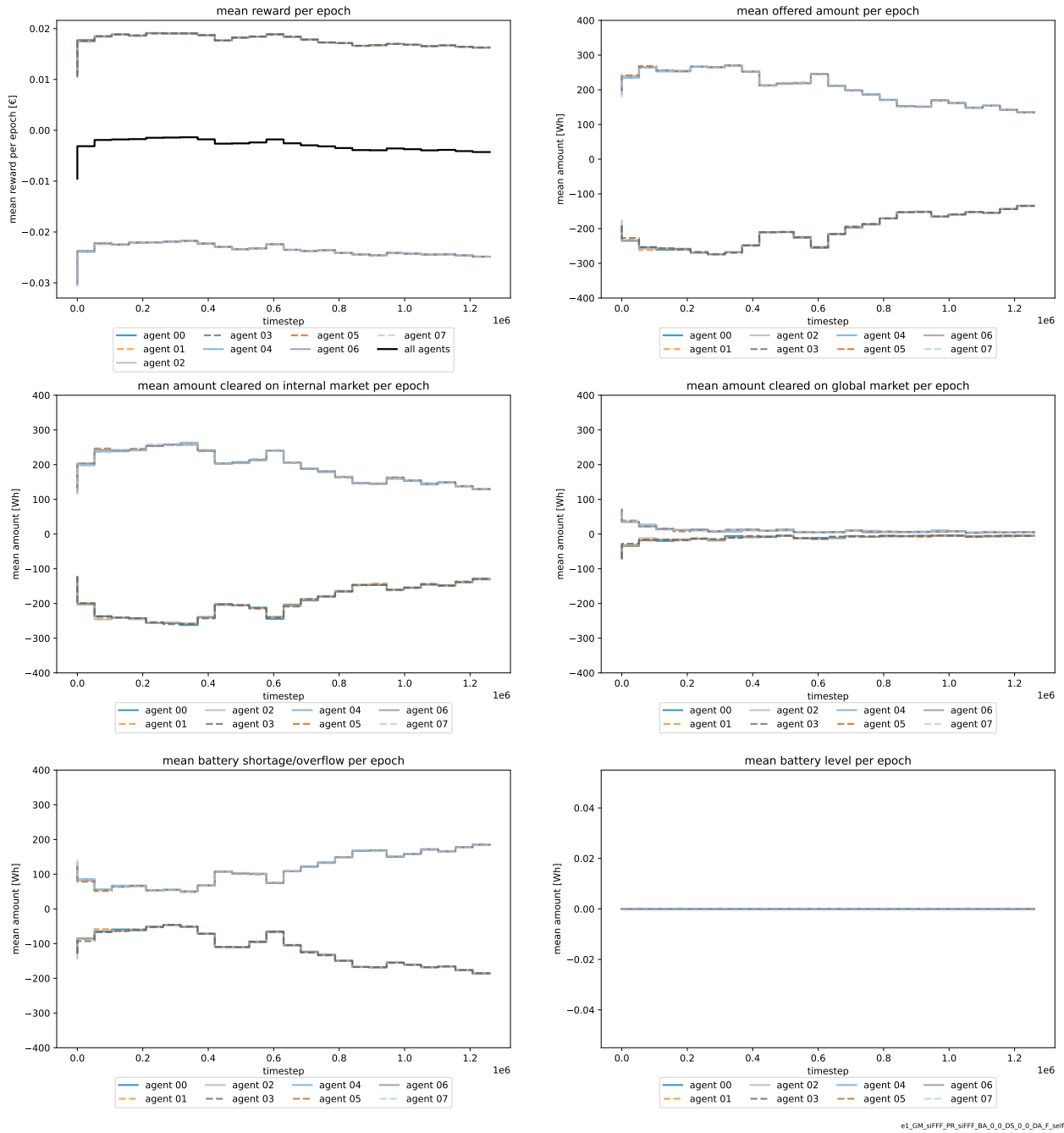


Figure B.9: Detailed view of key metrics for the run with seed 0 of experiment B.2.1 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.2.2 Cte. Prosumers and a Yearly Oscillating Global Market

Parameter	Setting	Parameter	Setting
global market data generation	formula	global market Time-of-use cost	0.000045 €/Wh
prosumers data generation	formula	global market mean day-ahead value	0.000042 €/Wh
Day-ahead window	deactivated	global market yearly day-ahead amplitude	0.000021 €/Wh
Data Sharing	deactivated	global market daily day-ahead amplitude	0 €/Wh
Producer agents	00,01,02,03	prosumer mean energy demand value	+/-320 Wh
Consumer Agents	04,05,06,07	prosumer yearly energy demand amplitude	0 Wh

Table B.3: Overview of settings for experiment B.2.2

Rule-Based Agents

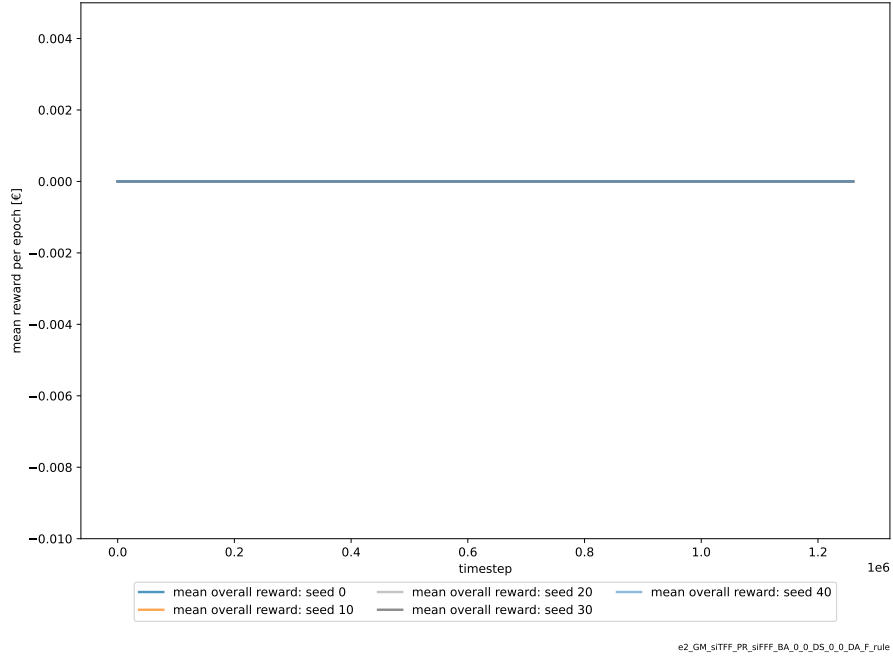


Figure B.10: Overview of the mean rewards per epoch for all agents combined for experiment B.2.2 with rule-based agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

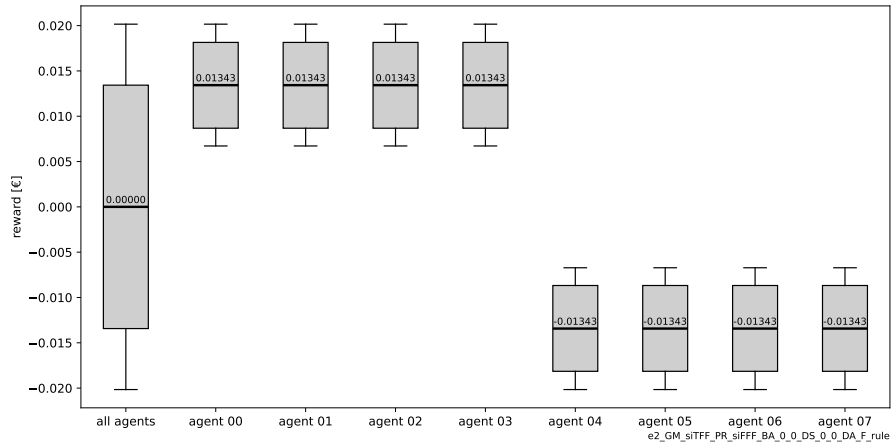
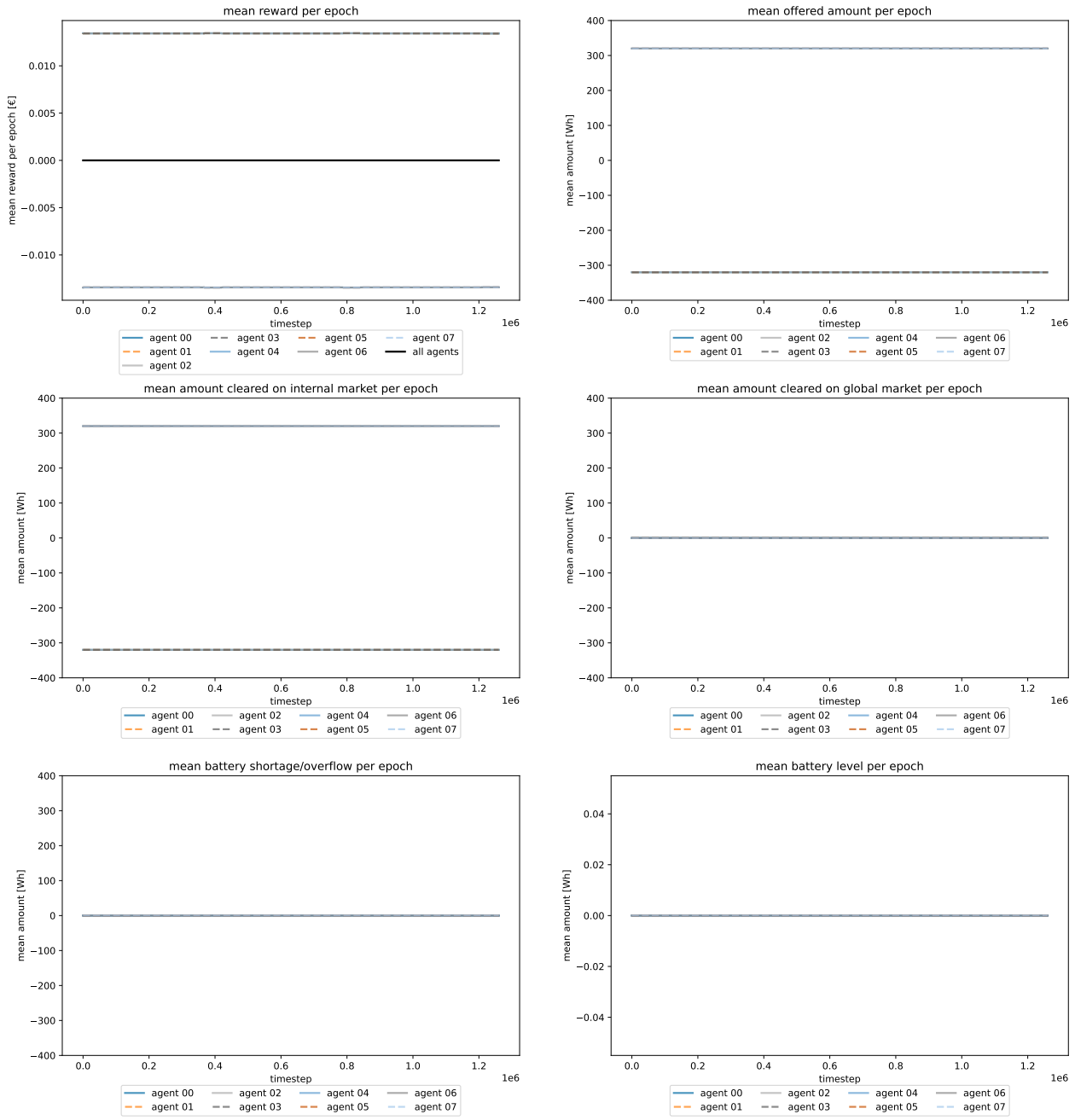


Figure B.11: Reward distribution in the last epoch for the run with seed 0 of experiment B.2.2 with rule-based agents.



e2_GM_siTFF_PR_siFFF_BA_0_0_DS_0_0_DA_F_rule

Figure B.12: Detailed view of key metrics for the run with seed 0 of experiment B.2.2 with rule-based agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

PPO Agents

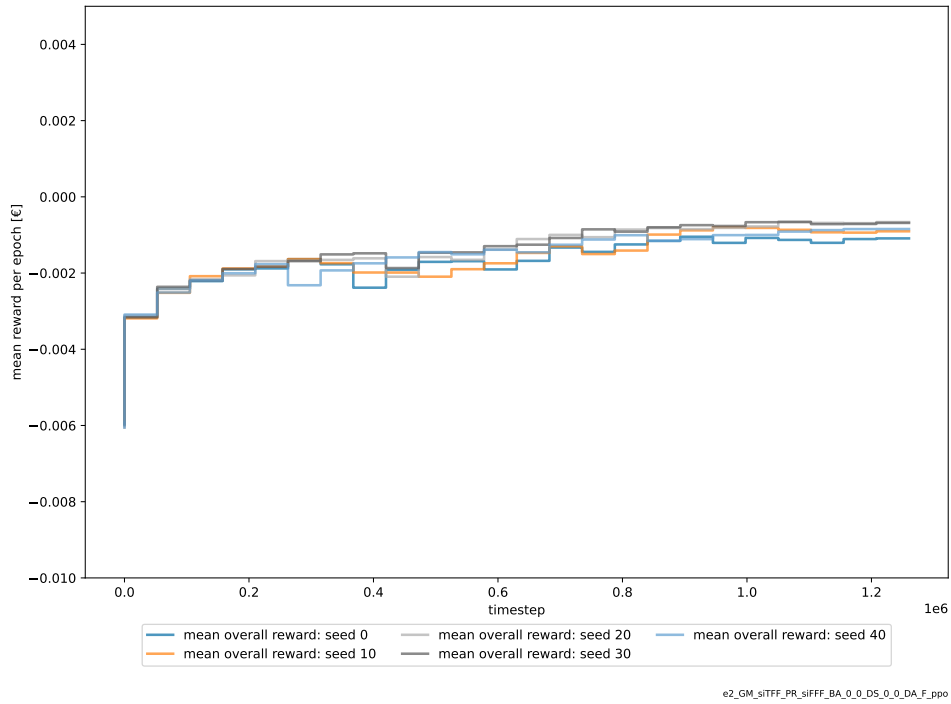


Figure B.13: Overview of the mean rewards per epoch for all agents combined for experiment B.2.2 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

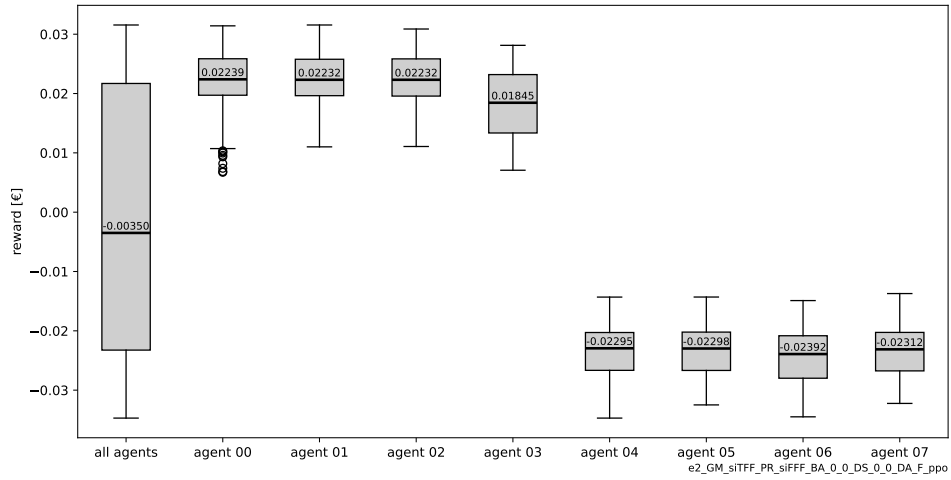
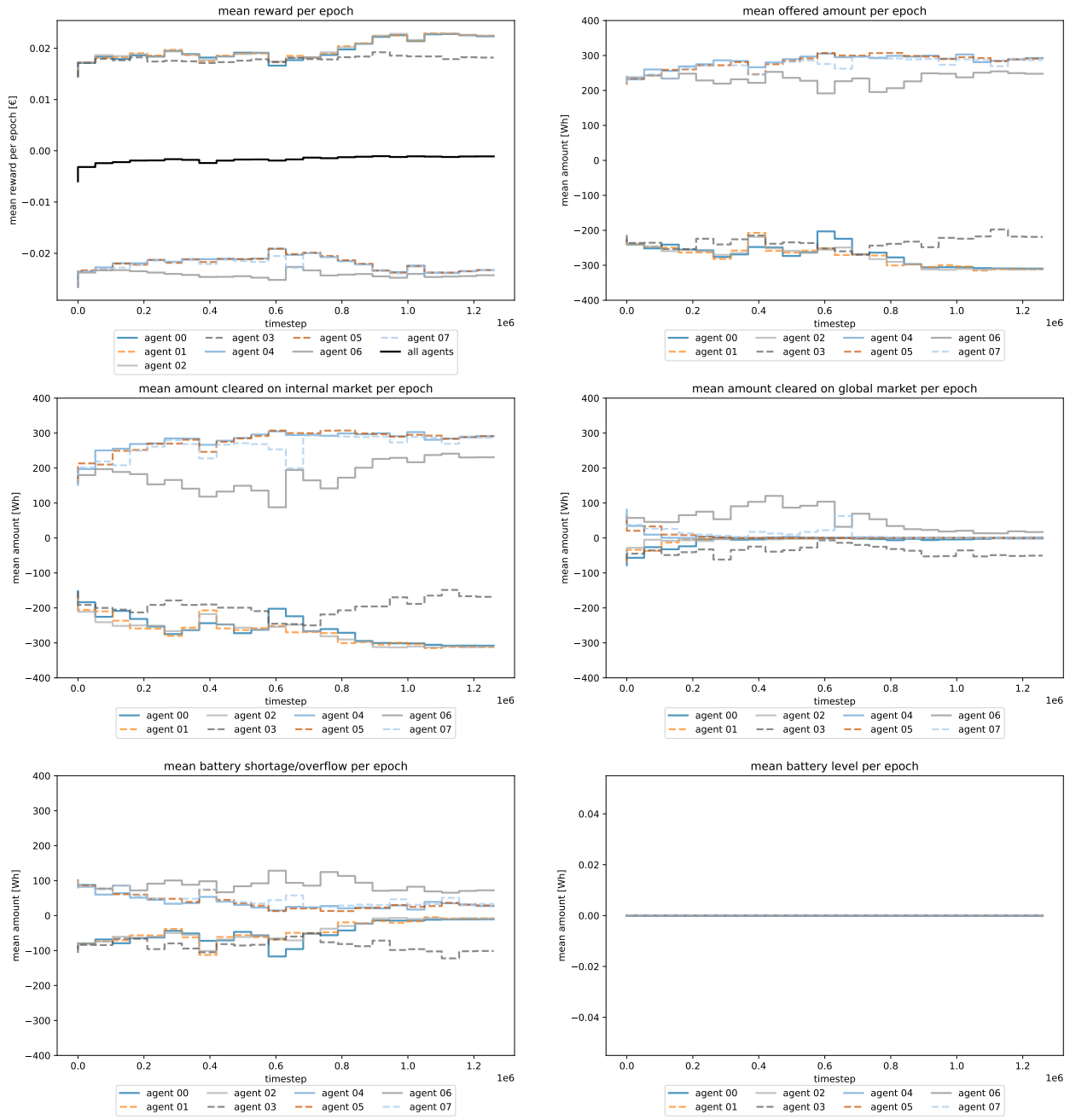


Figure B.14: Reward distribution in the last epoch for the run with seed 0 for experiment B.2.2 with PPO agents.



e2_GM_stTFF_PR_stFFF_BA_0_0_DS_0_0_DA_F_ppo

Figure B.15: Detailed view of key metrics for the run with seed 0 of experiment B.2.2 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

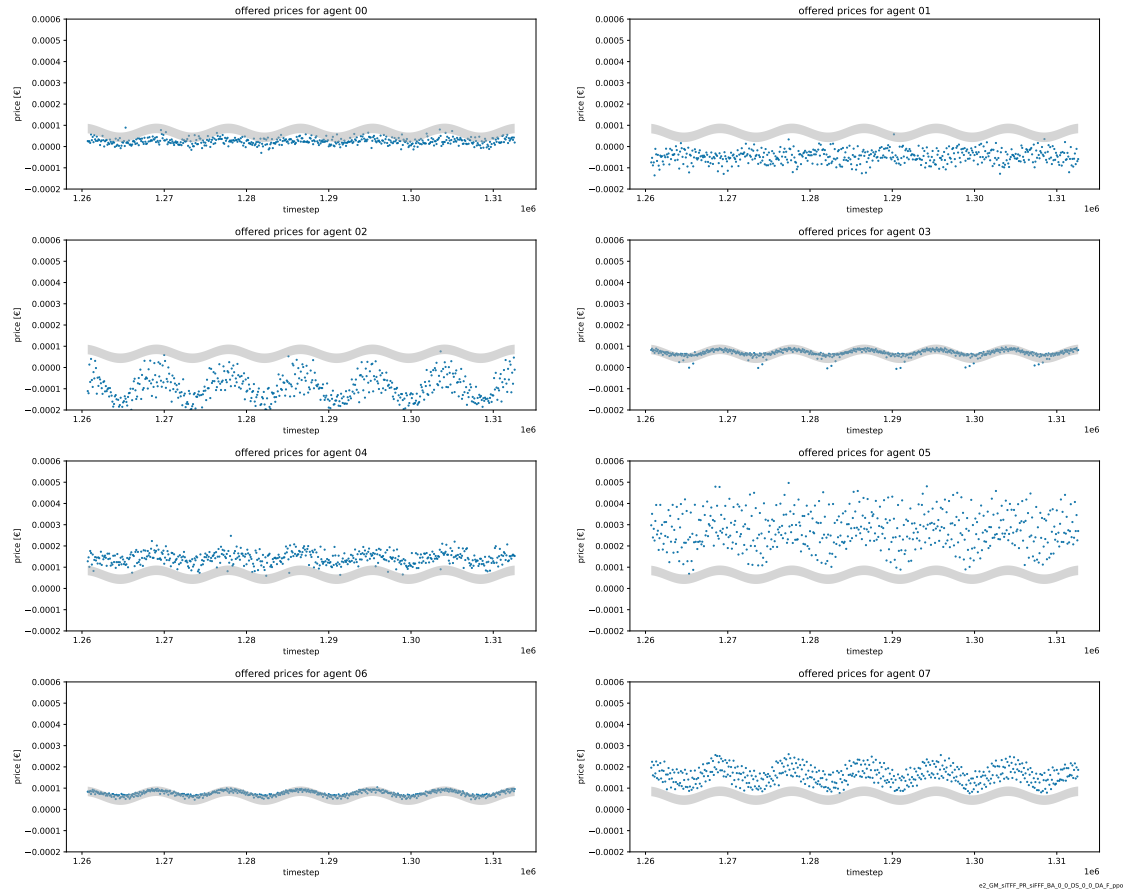
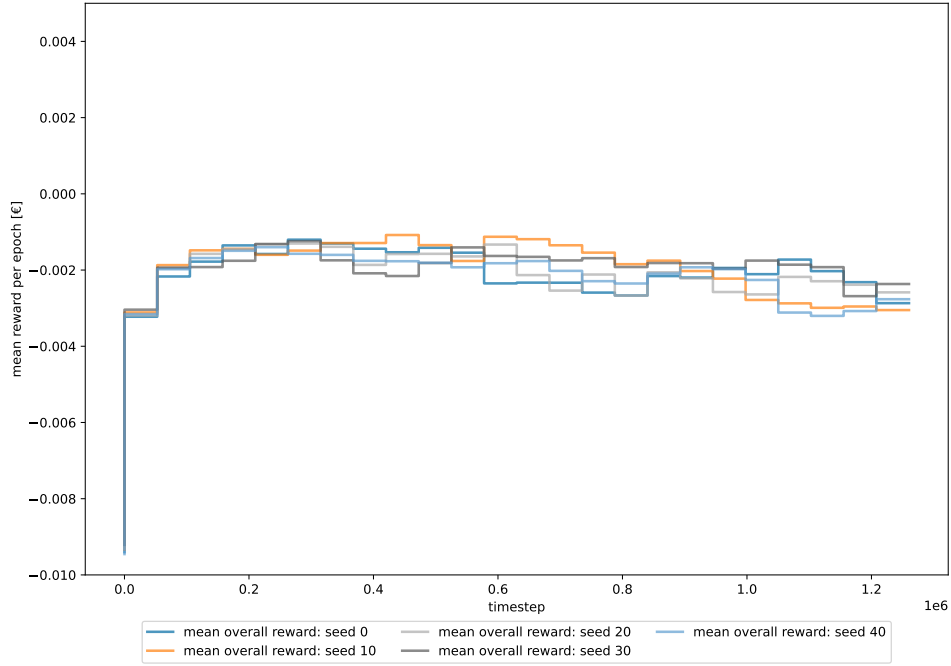


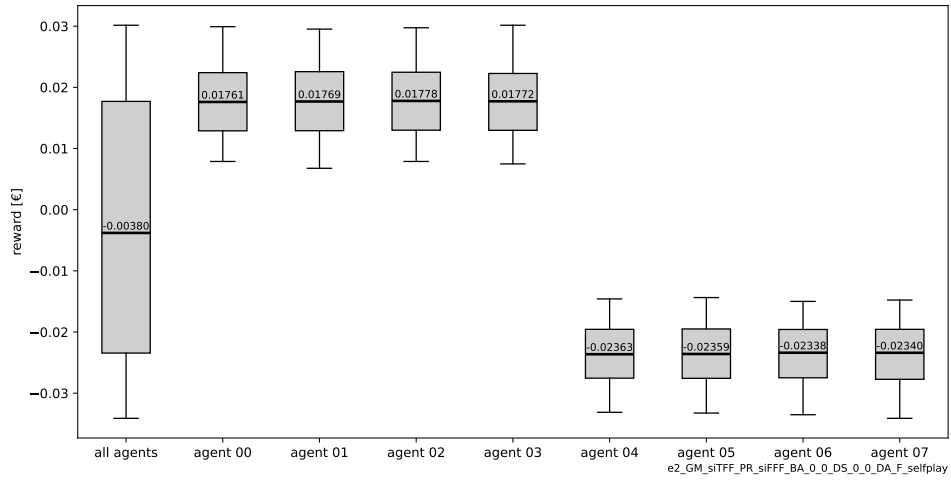
Figure B.16: Scatter plot containing all individual PPO agent offer prices during the last epoch for experiment B.2.2 with a yearly oscillating global market.

Self-play PPO Agents



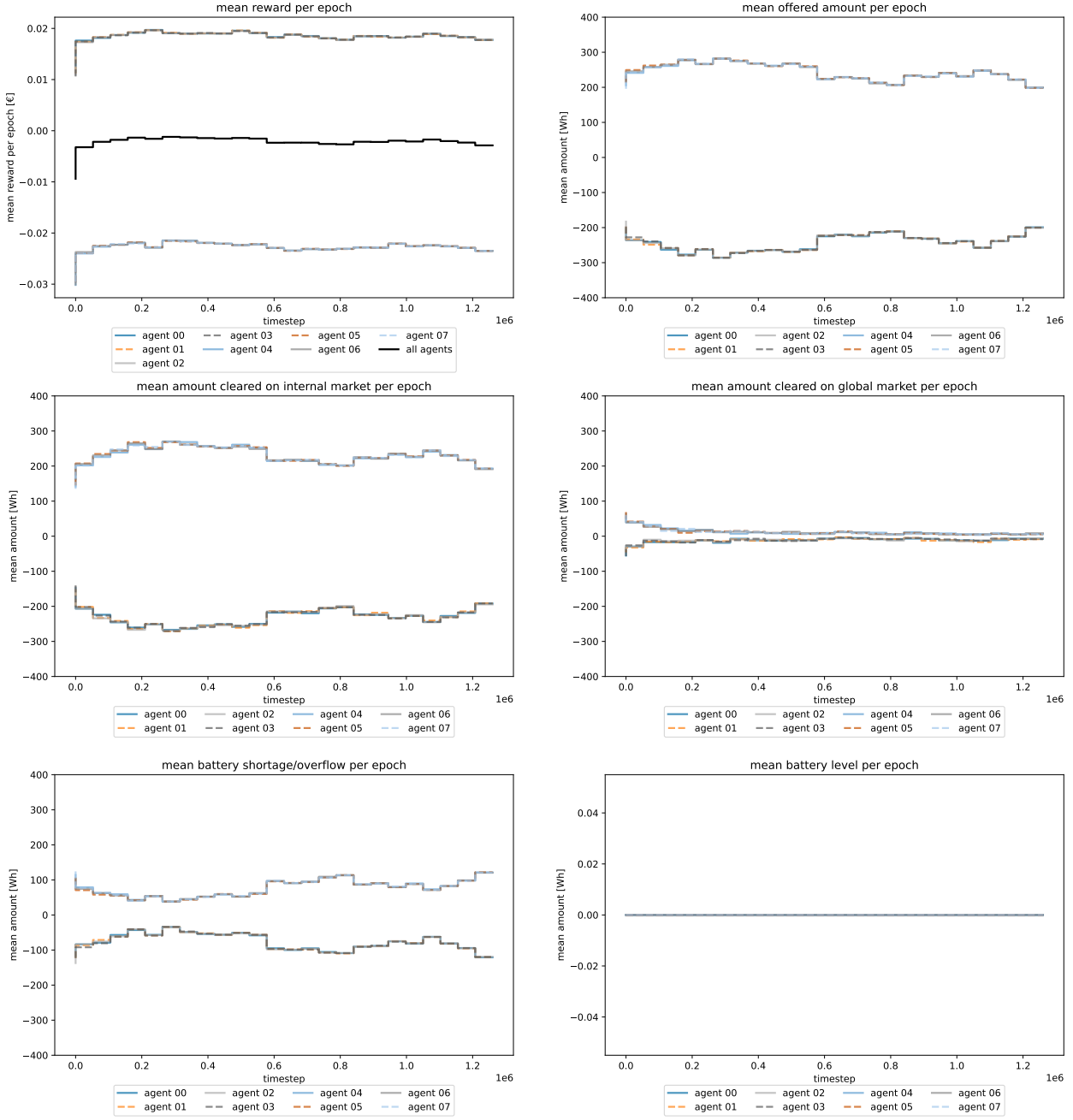
e2_GM_siTFF_PR_siFFF_BA_0_0_DS_0_0_DA_F_selfplay

Figure B.17: Overview of the mean rewards per epoch for all agents combined for experiment B.2.2 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.



e2_GM_siTFF_PR_siFFF_BA_0_0_DS_0_0_DA_F_selfplay

Figure B.18: Reward distribution in the last epoch for the run with seed 0 for experiment B.2.2 with self-play PPO agents.



e2_GM_sITFF_PR_sITFF_BA_0_0_DS_0_0_DA_F_selfplay

Figure B.19: Detailed view of key metrics for the run with seed 0 of experiment B.2.2 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.2.3 Cte. Prosumers and a Yearly and Daily oscillating Global Market

Parameter	Setting	Parameter	Setting
global market data generation	formula	global market Time-of-use cost	0.000045 €/Wh
prosumers data generation	formula	global market mean day-ahead value	0.000042 €/Wh
Day-ahead window	deactivated	global market yearly day-ahead amplitude	0.000021 €/Wh
Data Sharing	deactivated	global market daily day-ahead amplitude	0.000005 €/Wh
Producer agents	00,01,02,03	prosumer mean energy demand value	+/-320 Wh
Consumer Agents	04,05,06,07	prosumer yearly energy demand amplitude	0 Wh

Table B.4: Overview of settings for experiment B.2.3

Rule-Based Agents

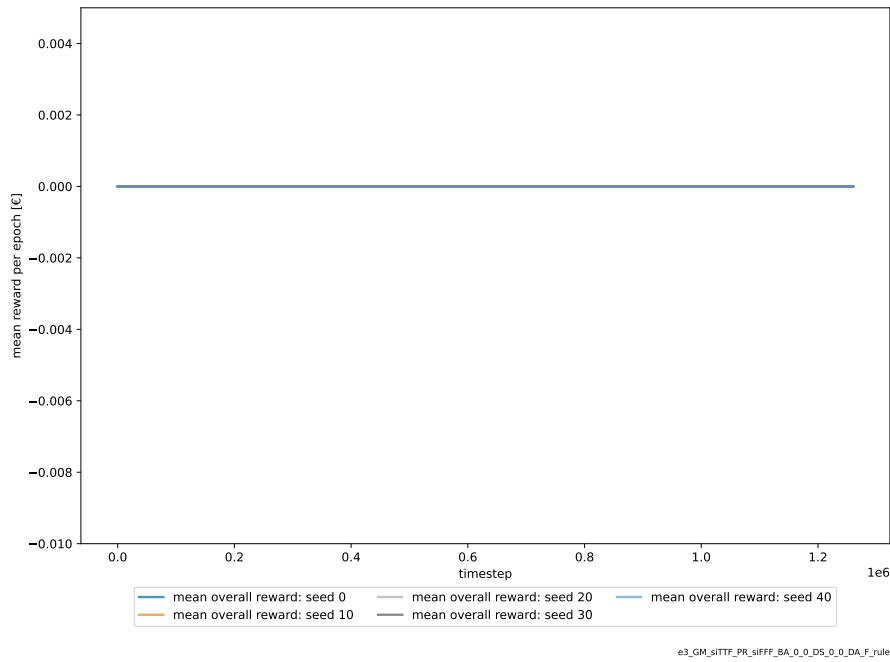


Figure B.20: Overview of the mean rewards per epoch for all agents combined for experiment B.2.3 with rule-based agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

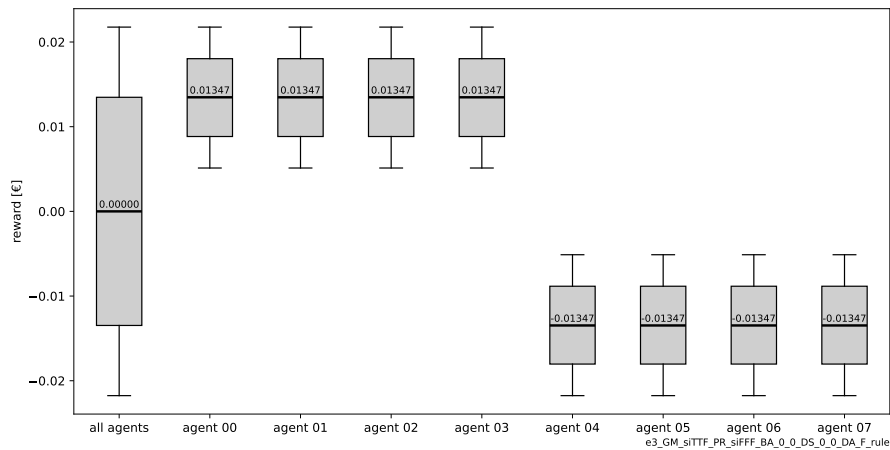
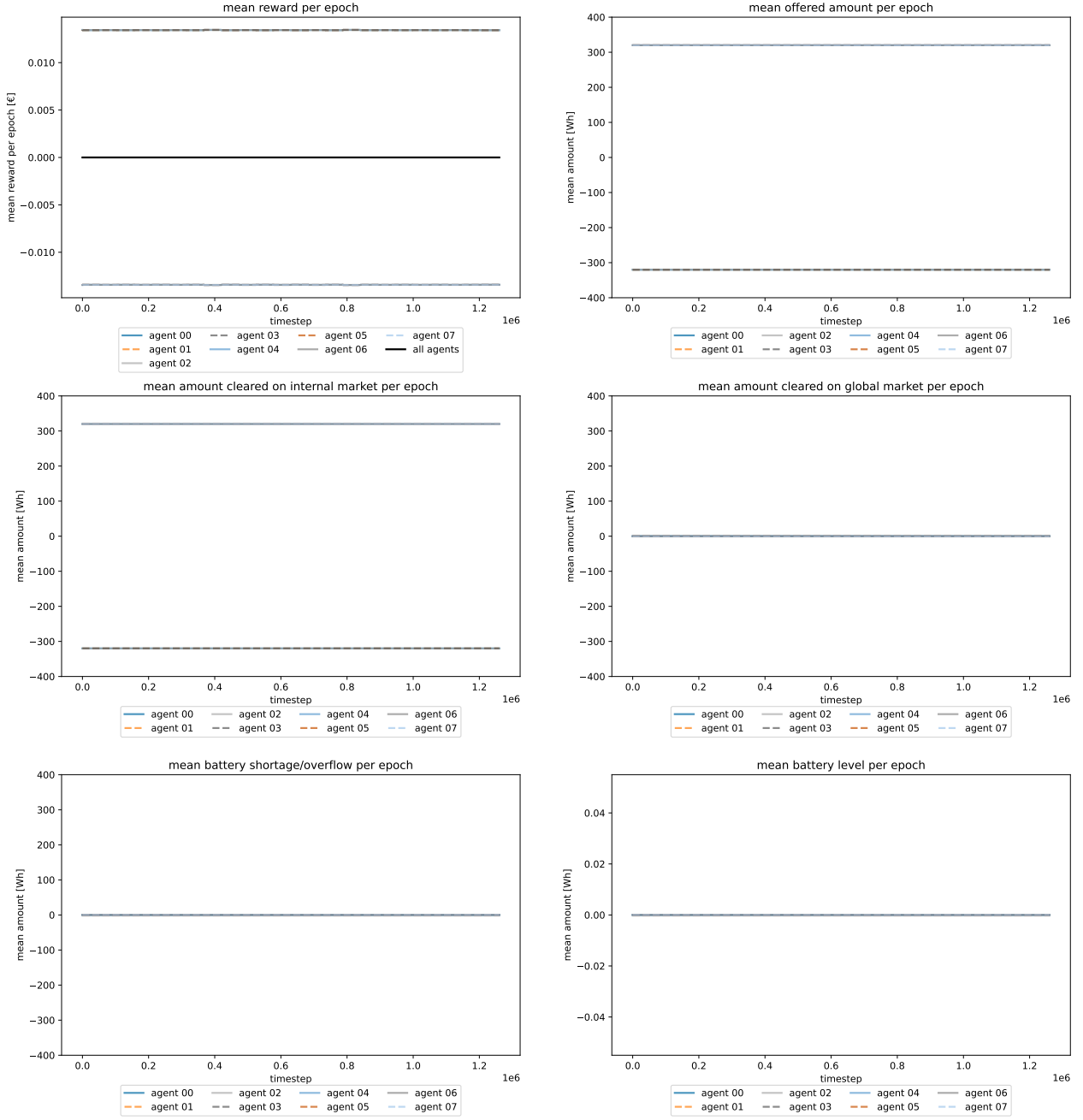


Figure B.21: Reward distribution in the last epoch for the run with seed 0 of experiment B.2.3 with rule-based agents.



e3_GM_siTTF_PR_siFFF_BA_0_0_DS_0_0_DA_F_rule

Figure B.22: Detailed view of key metrics for the run with seed 0 of experiment B.2.3 with rule-based agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

PPO Agents

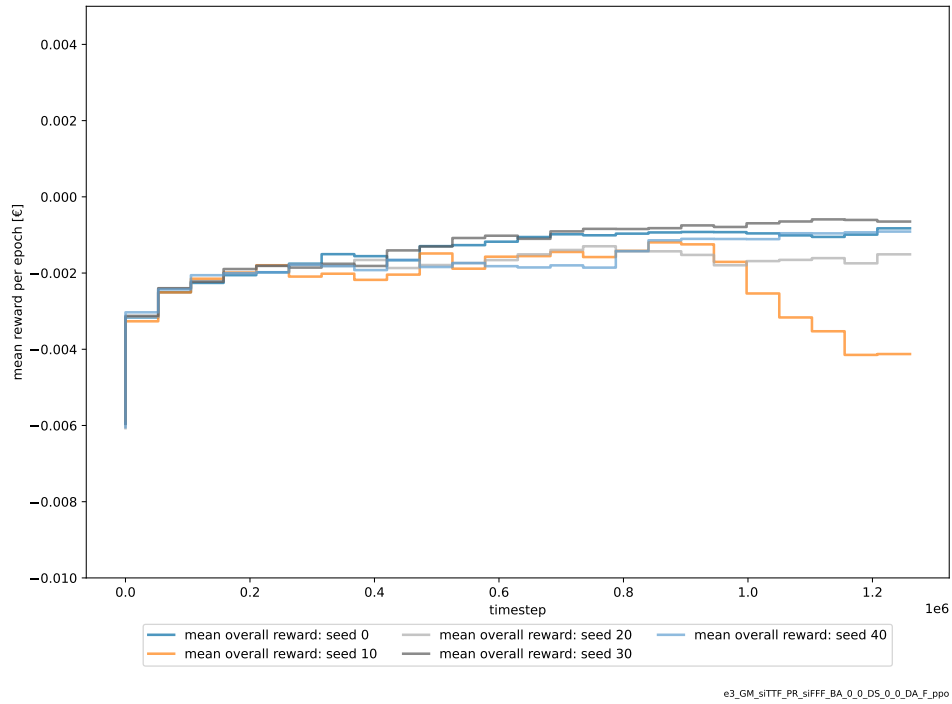


Figure B.23: Overview of the mean rewards per epoch for all agents combined for experiment B.2.3 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

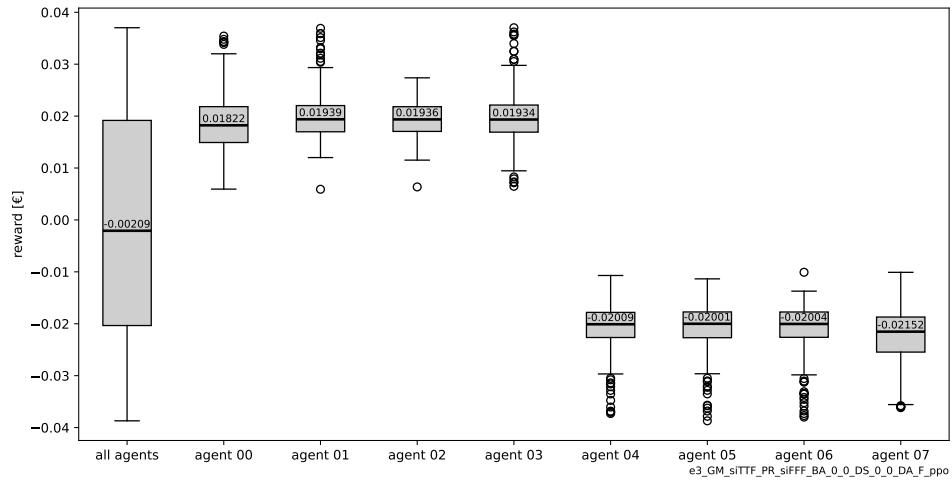
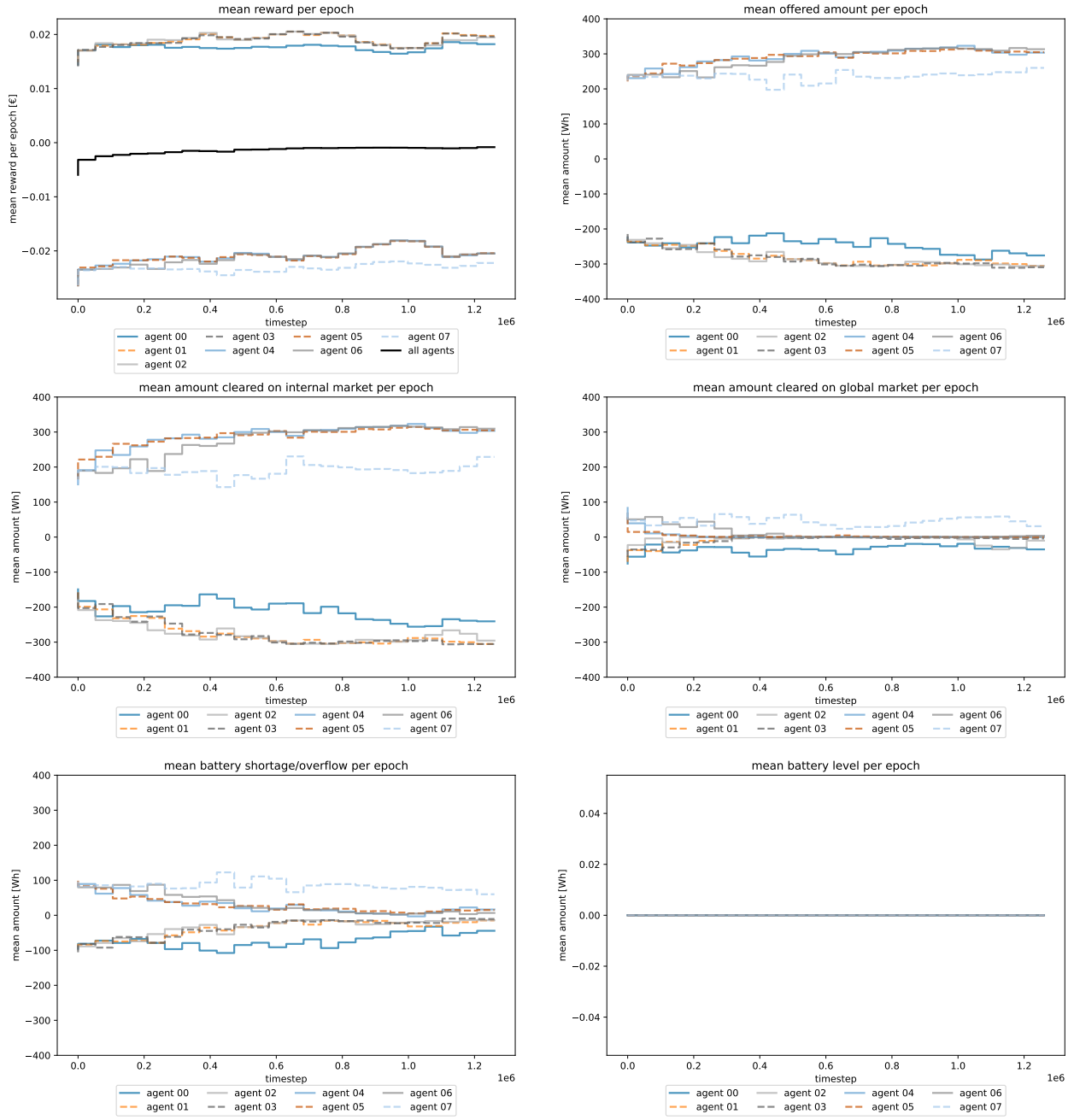


Figure B.24: Reward distribution in the last epoch for the run with seed 0 for experiment B.2.3 with PPO agents.



e3_GM_sITTF_PR_sIFFF_BA_0_0_DS_0_0_DA_F_ppo

Figure B.25: Detailed view of key metrics for the run with seed 0 of experiment B.2.3 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

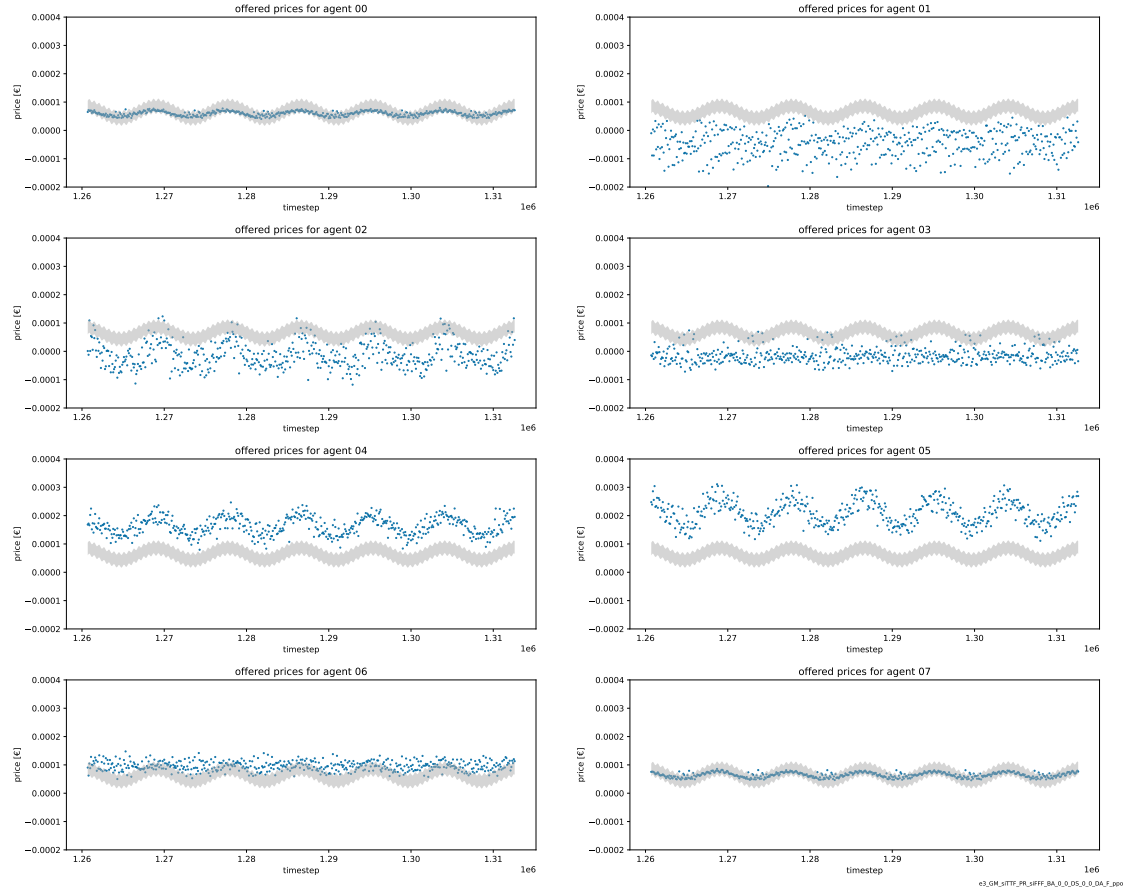


Figure B.26: Scatter plot containing all individual PPO agent offer prices during the last epoch for experiment B.2.3 with a yearly and daily oscillating global market. Note that the daily fluctuations are not fully observable due to the logging sample interval of 100 timesteps.

Self-play PPO Agents

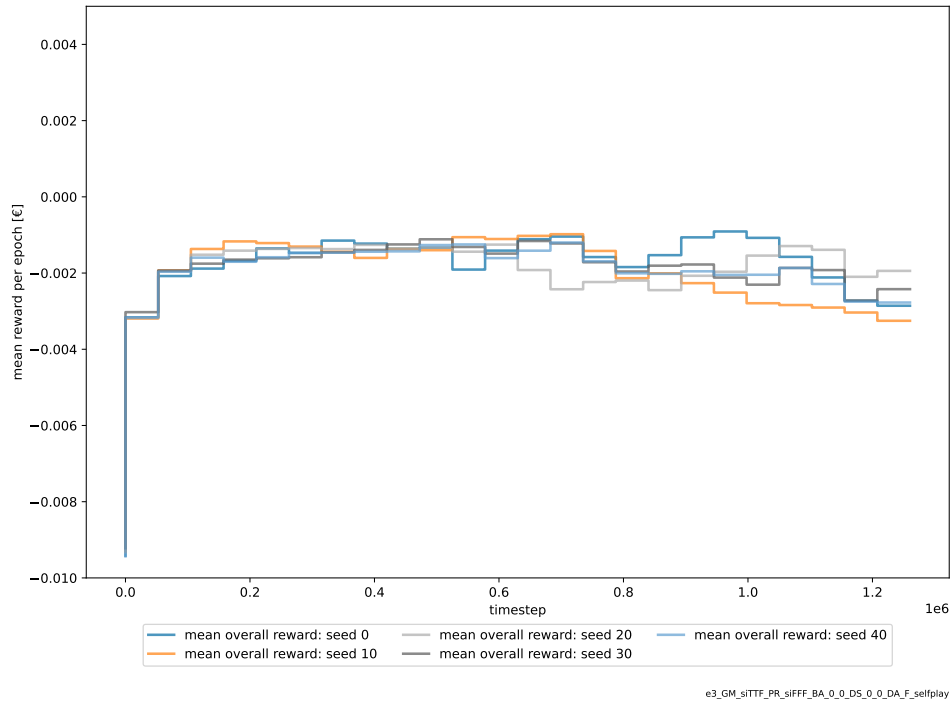


Figure B.27: Overview of the mean rewards per epoch for all agents combined for experiment B.2.3 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

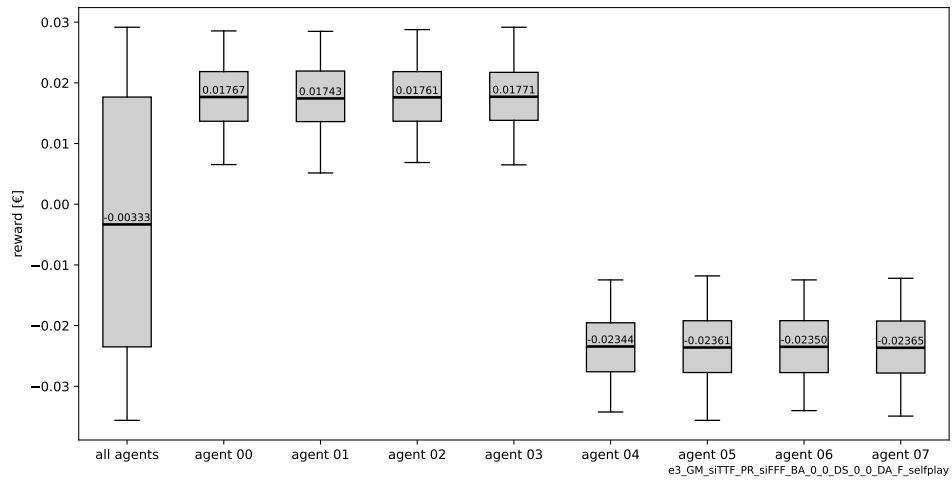
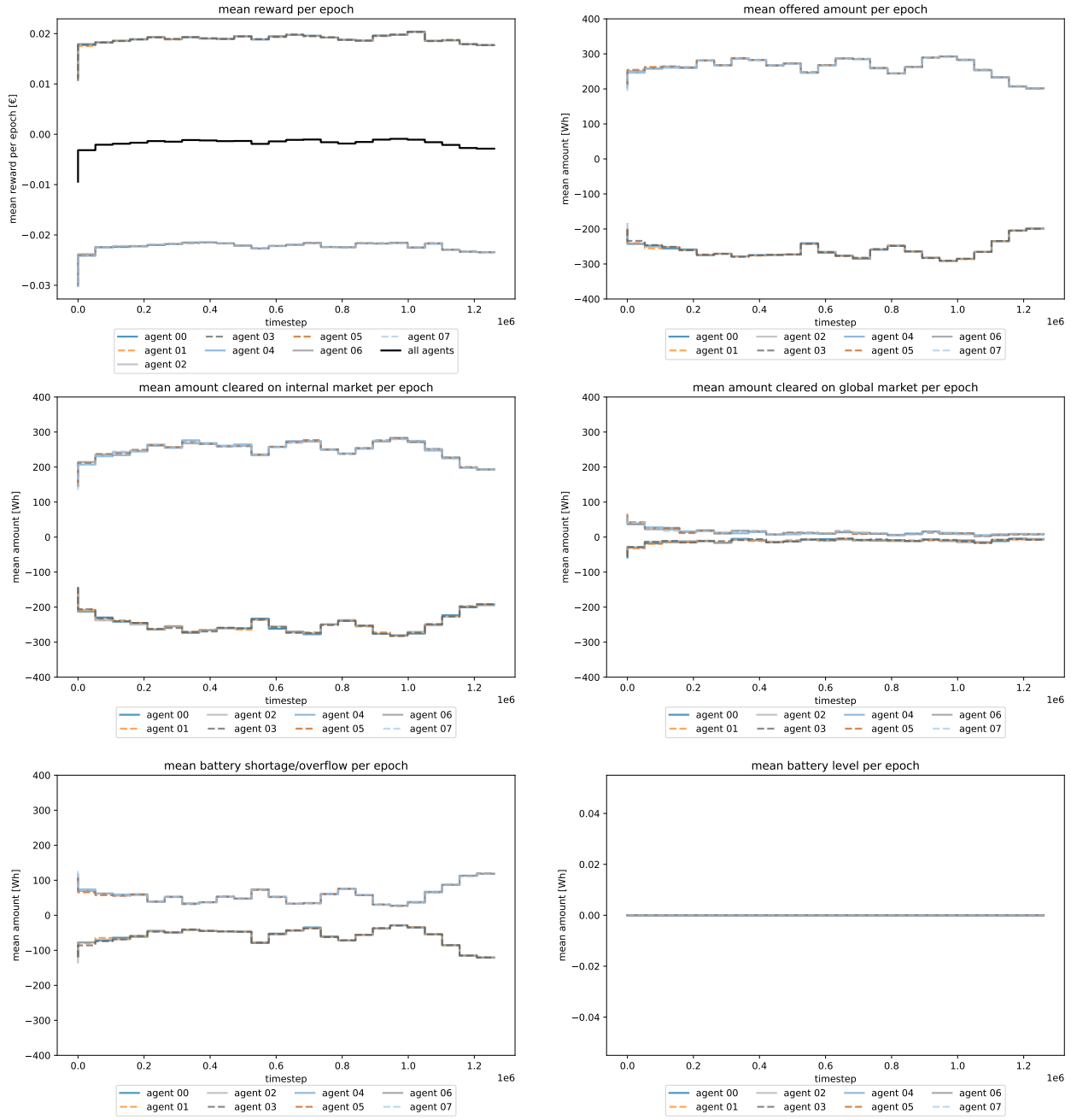


Figure B.28: Reward distribution in the last epoch for the run with seed 0 for experiment B.2.3 with self-play PPO agents.



e3_GM_sITTF_PR_sIFFF_BA_0_0_DS_0_0_DA_F_selfplay

Figure B.29: Detailed view of key metrics for the run with seed 0 of experiment B.2.3 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.2.4 Yearly Oscillating Prosumers and a Cte. Global Market

Parameter	Setting	Parameter	Setting
global market data generation	formula	global market Time-of-use cost	0.000045 €/Wh
prosumers data generation	formula	global market mean day-ahead value	0.000042 €/Wh
Day-ahead window	deactivated	global market yearly day-ahead amplitude	0 €/Wh
Data Sharing	deactivated	global market daily day-ahead amplitude	0 €/Wh
Producer agents	00,01,02,03	prosumer mean energy demand value	+/-320 Wh
Consumer Agents	04,05,06,07	prosumer yearly energy demand amplitude	350 Wh

Table B.5: Overview of settings for experiment B.2.4

Rule-Based Agents

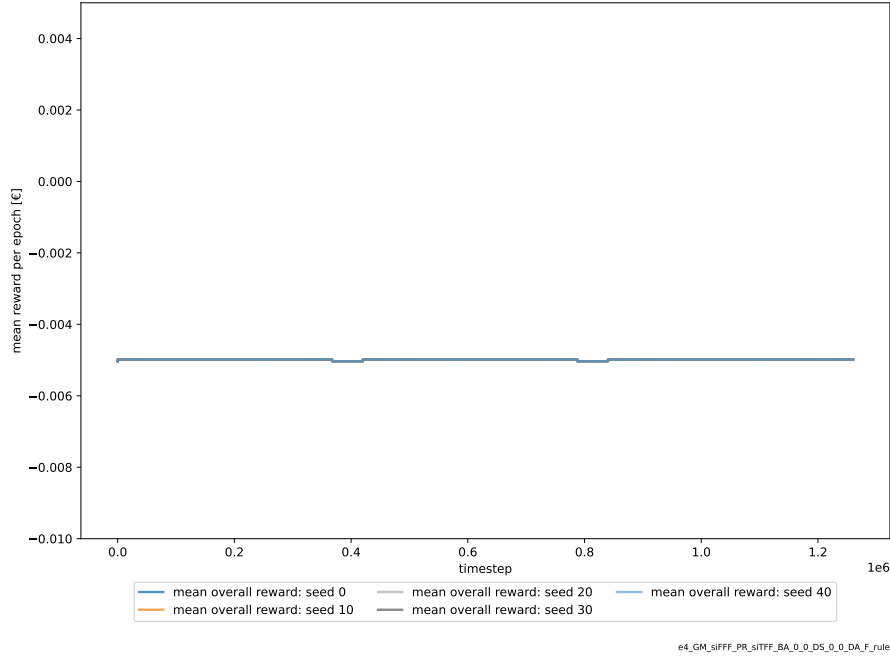


Figure B.30: Overview of the mean rewards per epoch for all agents combined for experiment B.2.4 with rule-based agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

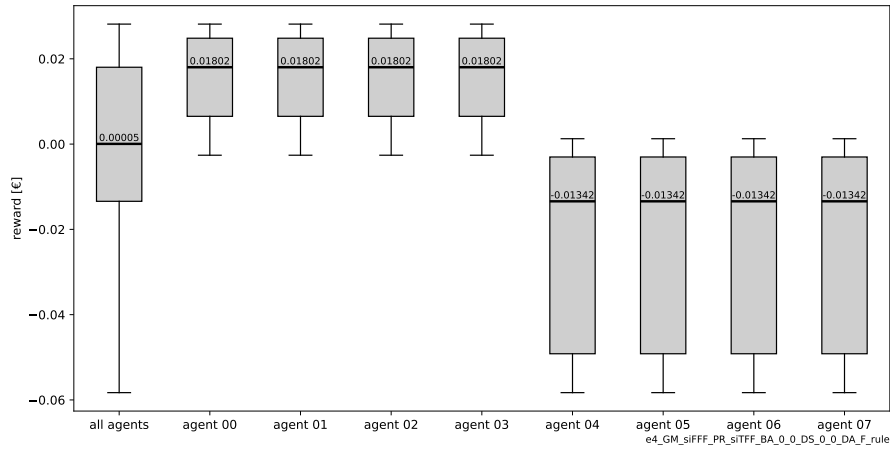
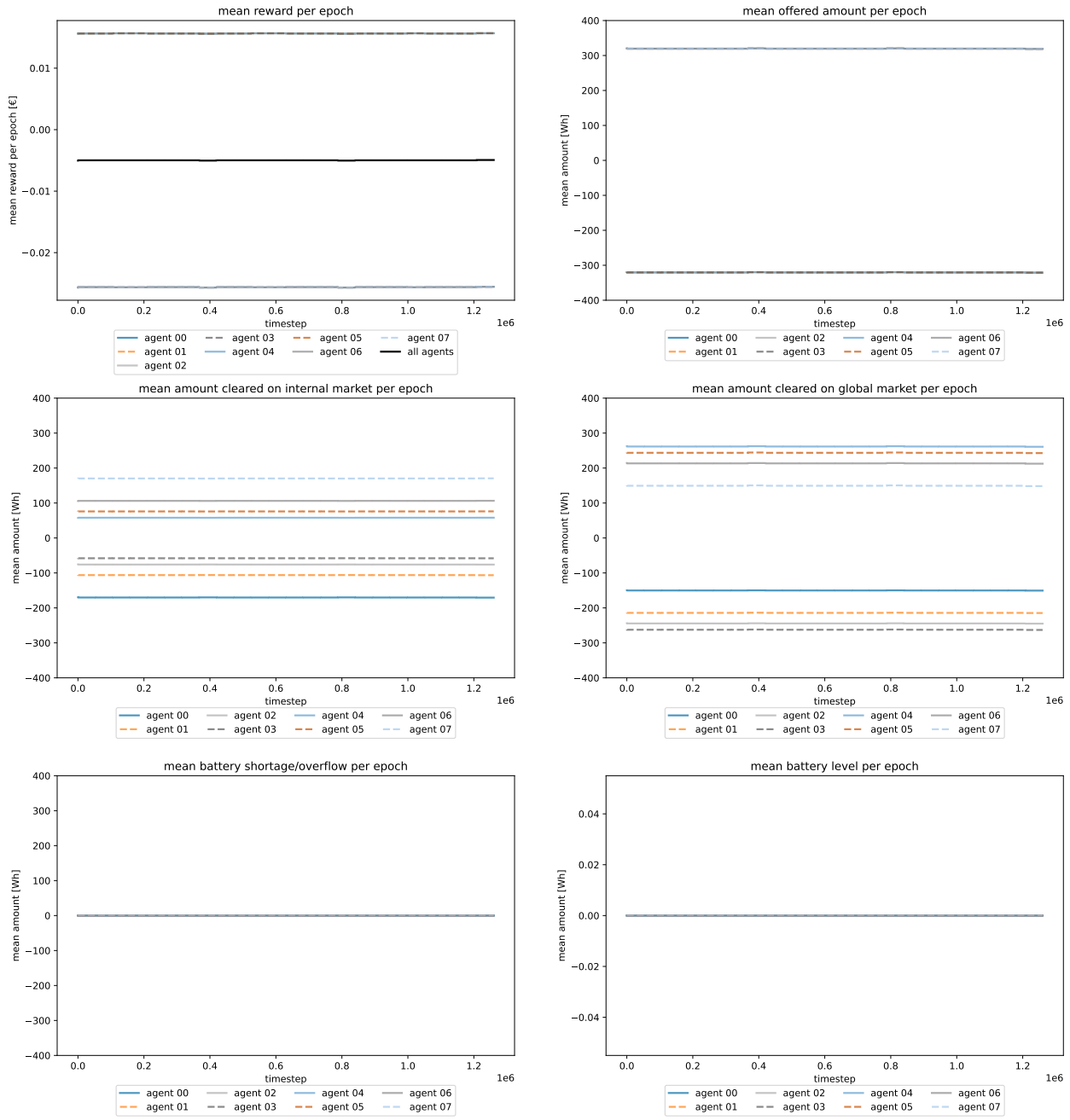


Figure B.31: Reward distribution in the last epoch for the run with seed 0 of experiment B.2.4 with rule-based agents



e4_GM_sifff_PR_sifff_BA_0_0_DS_0_0_DA_F_rule

Figure B.32: Detailed view of key metrics for the run with seed 0 of experiment B.2.4 with rule-based agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

PPO Agents

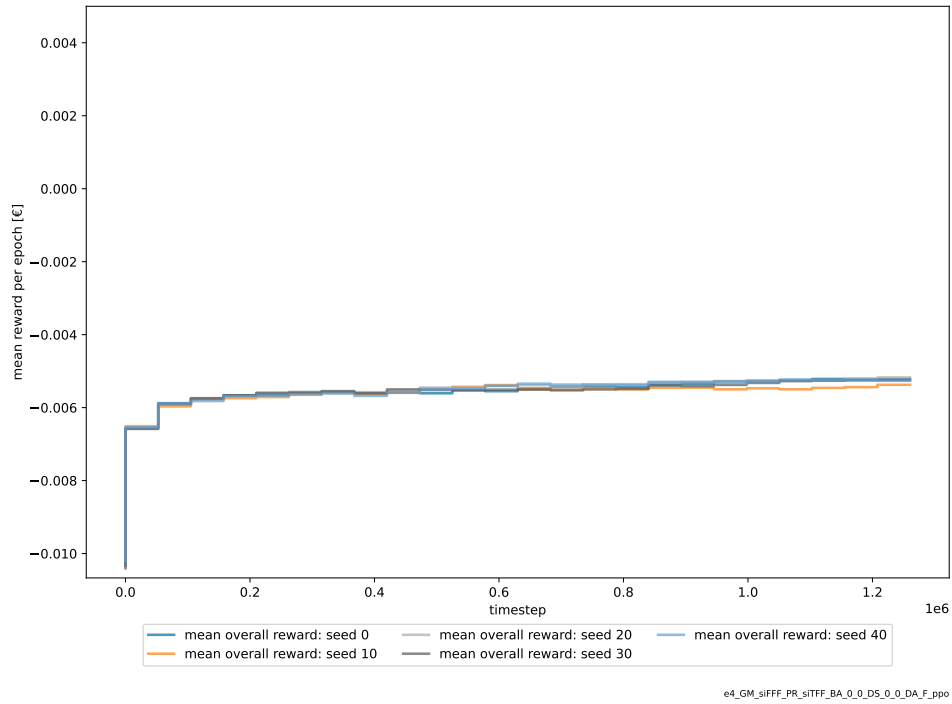


Figure B.33: Overview of the mean rewards per epoch for all agents combined for experiment B.2.4 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

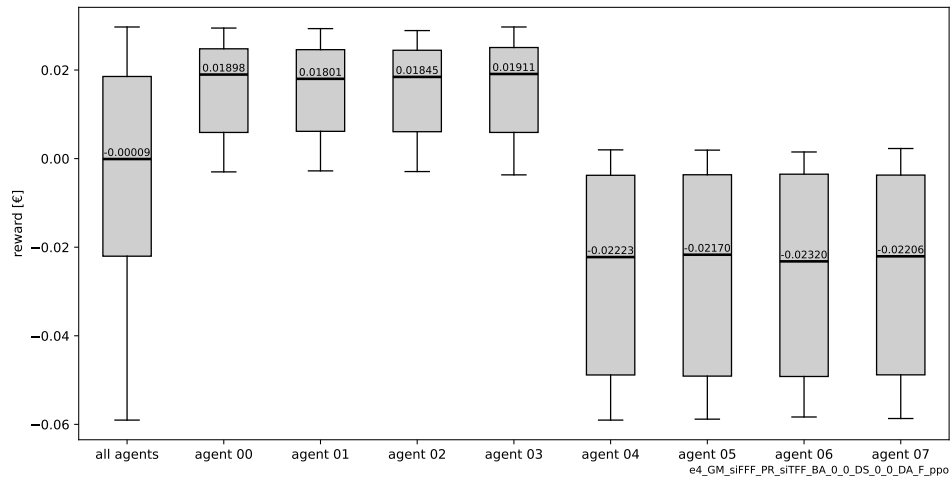
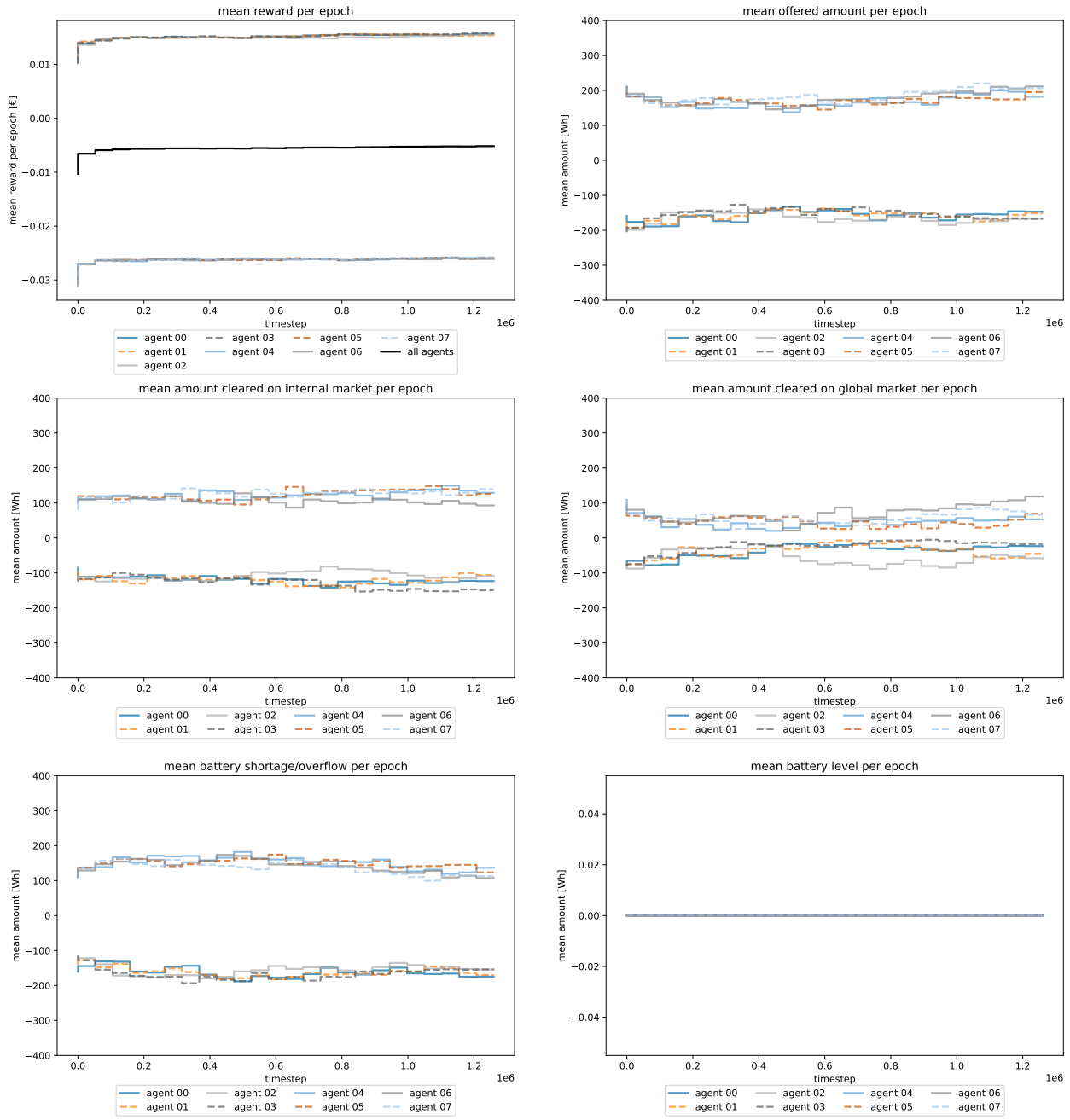


Figure B.34: Reward distribution in the last epoch for the run with seed 0 for experiment B.2.4 with PPO agents



e4_GM_sifff_PR_sifff_BA_0_0_DS_0_0_DA_F_ppo

Figure B.35: Detailed view of key metrics for the run with seed 0 of experiment B.2.4 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents

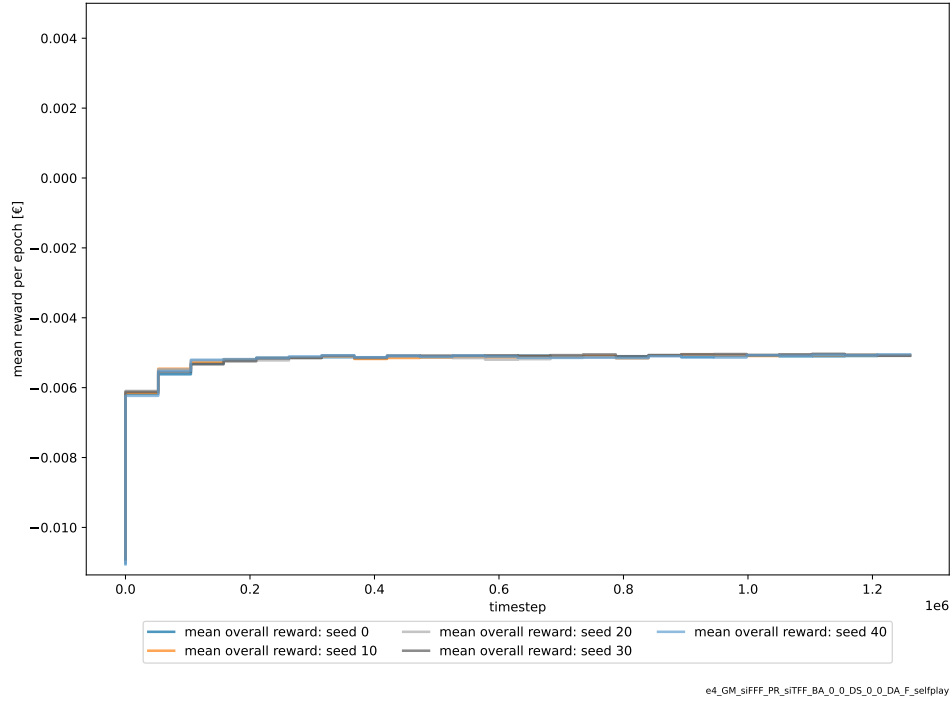


Figure B.36: Overview of the mean rewards per epoch for all agents combined for experiment B.2.4 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

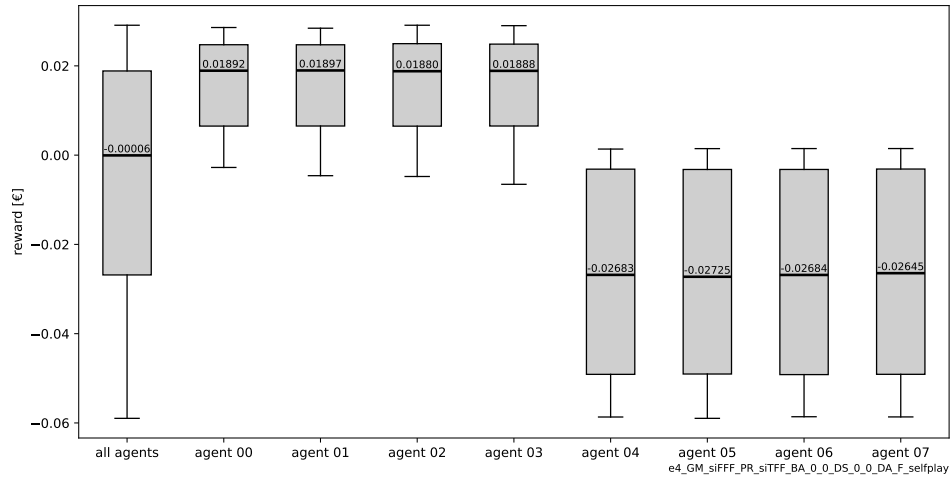
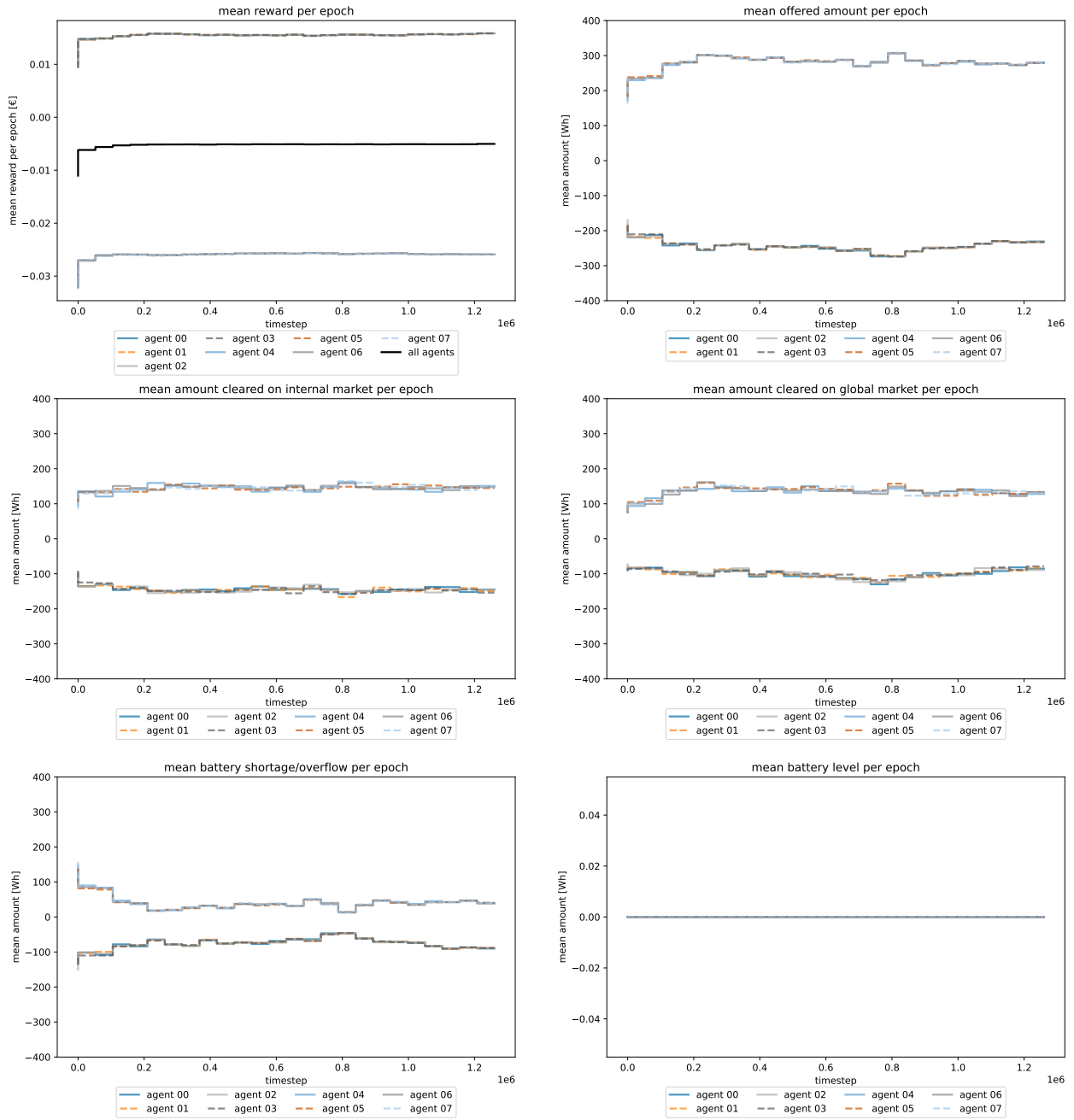


Figure B.37: Reward distribution in the last epoch for the run with seed 0 for experiment B.2.4 with self-play PPO agents.



e4_GM_sifff_PR_sTFF_BA_0_0_DS_0_0_DA_F_selfplay

Figure B.38: Detailed view of key metrics for the run with seed 0 of experiment B.2.4 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.2.5 Using the Synthetic Datasets

Parameter	Setting
global market data generation	Synthetic
prosumers data generation	Synthetic
Day-ahead window	deactivated
Data Sharing	deactivated
global market Time-of-use cost	0.000045 €/Wh

Table B.6: Overview of settings for experiment B.2.5

Rule-Based Agents

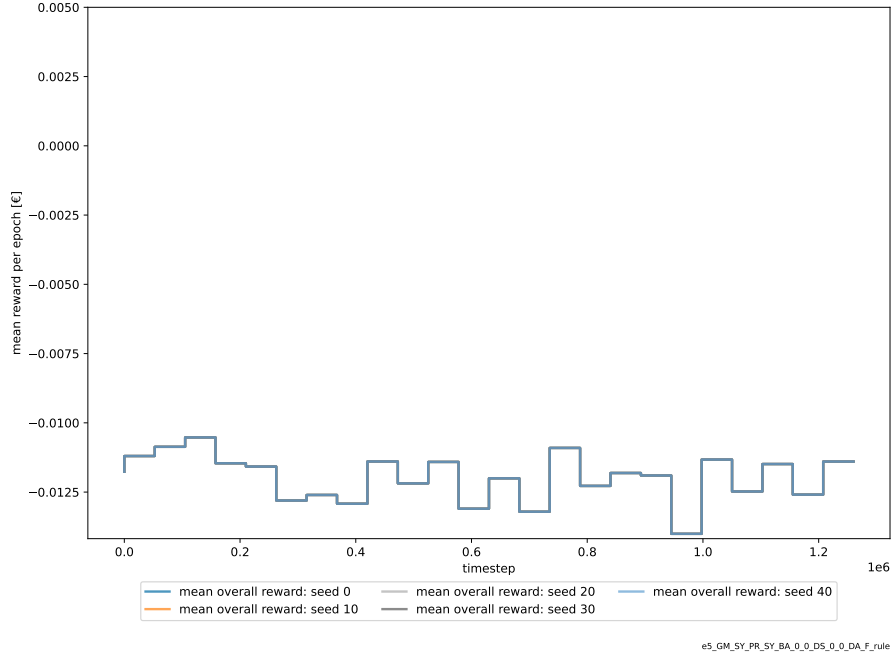


Figure B.39: Overview of the mean rewards per epoch for all agents combined for experiment B.2.5 with rule-based agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

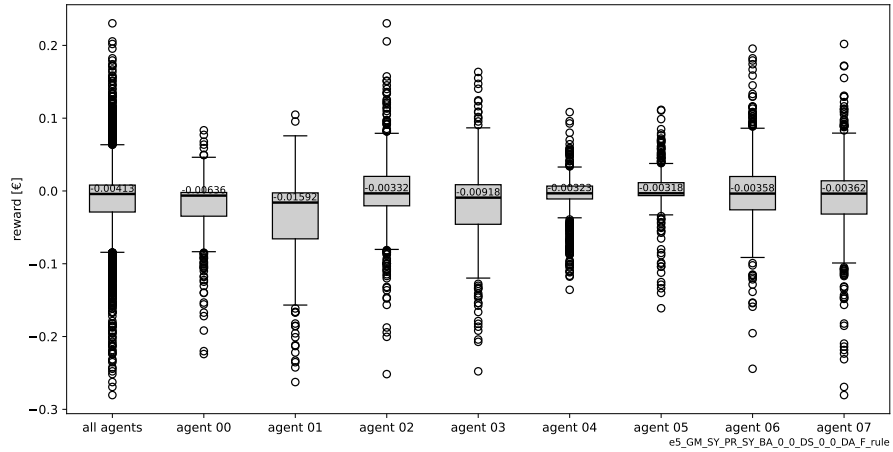
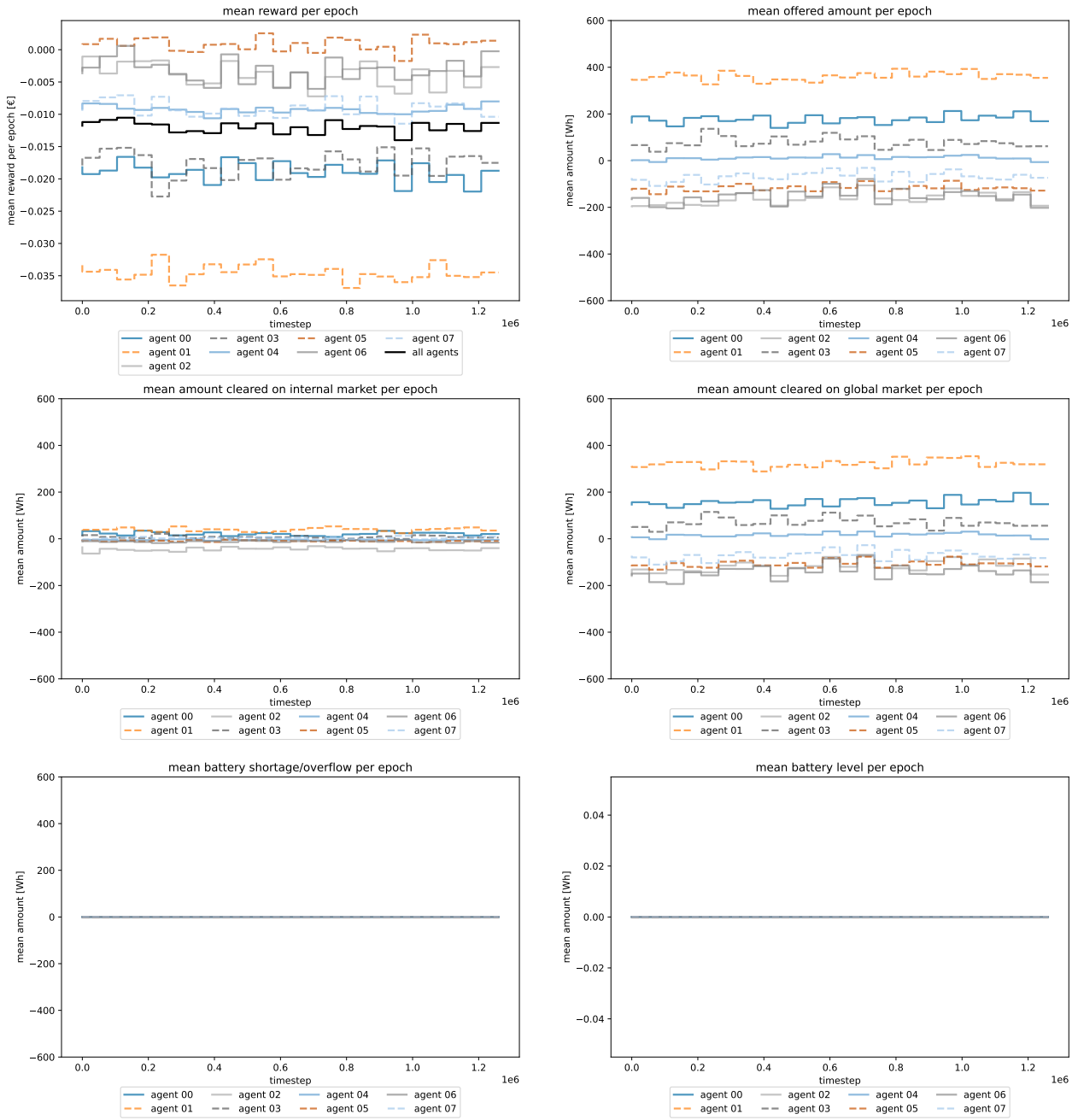


Figure B.40: Reward distribution in the last epoch for the run with seed 0 of experiment B.2.5 with rule-based agents



e5_GM_SY_FR_SY_BA_0_0_DS_0_0_DA_F_rule

Figure B.41: Detailed view of key metrics for the run with seed 0 of experiment B.2.5 with rule-based agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

PPO Agents

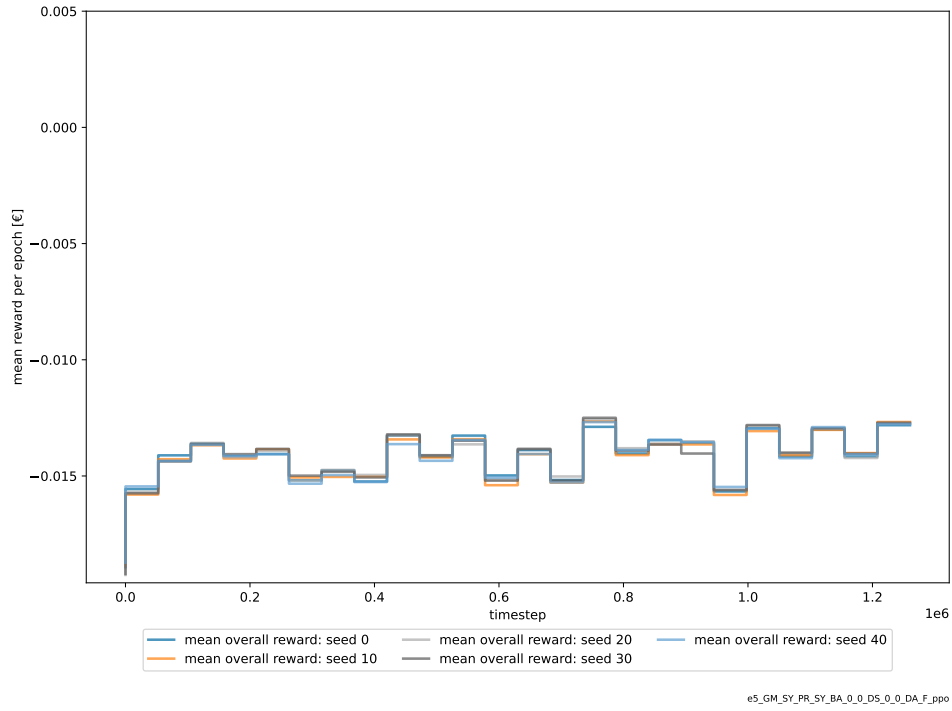


Figure B.42: Overview of the mean rewards per epoch for all agents combined for experiment B.2.5 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

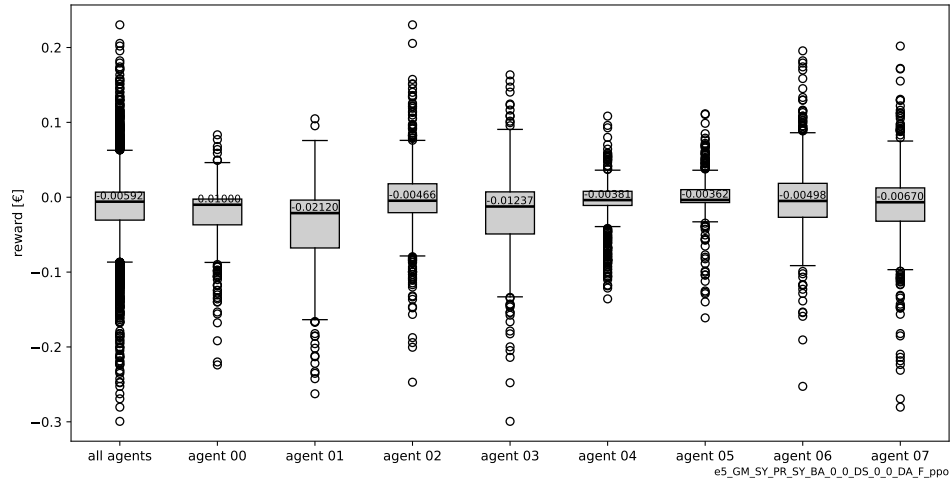
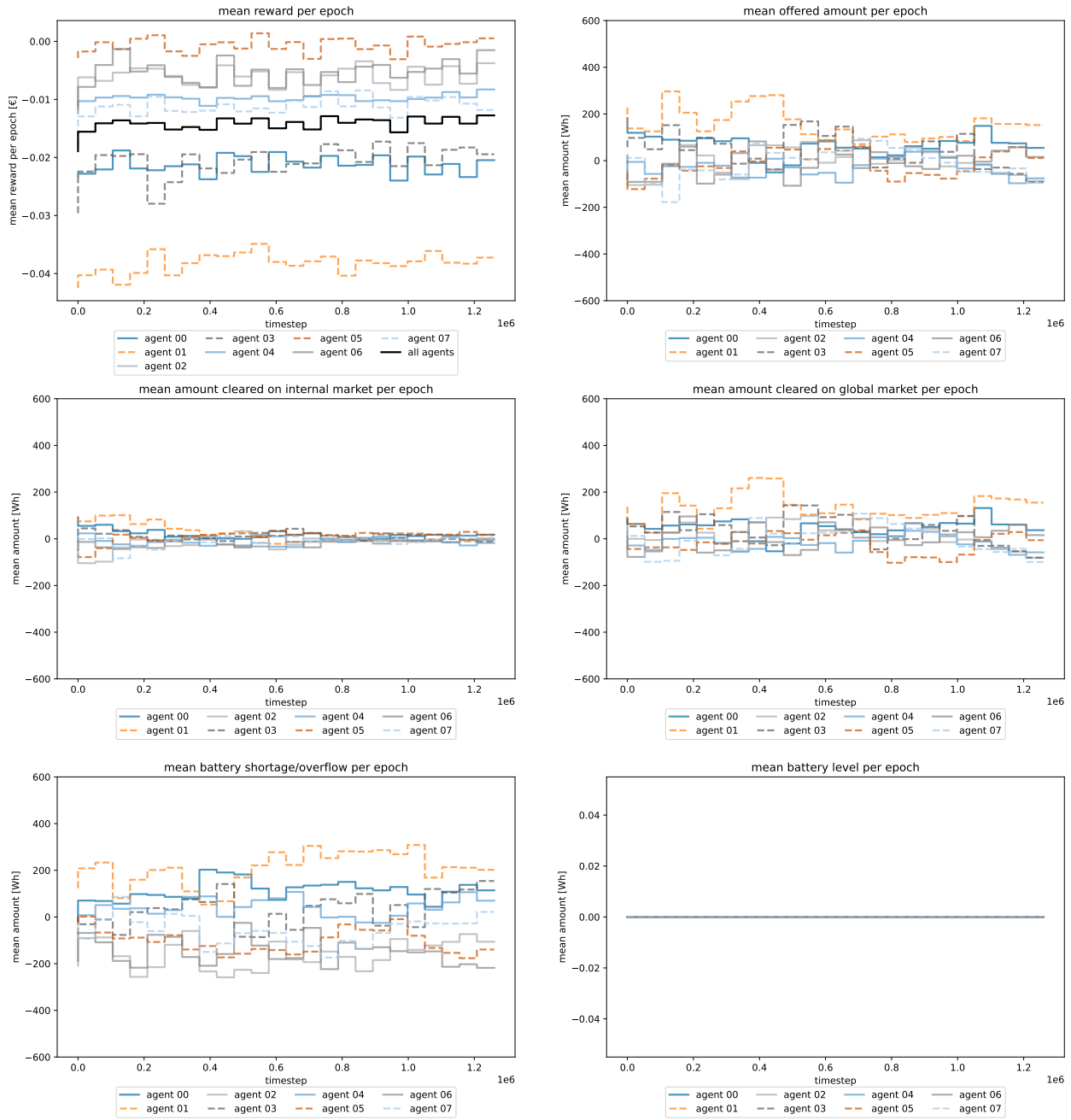


Figure B.43: Reward distribution in the last epoch for the run with seed 0 for experiment B.2.5 with PPO agents



e5_GM_SY_PR_SY_BA_0_0_DS_0_0_DA_F_ppo

Figure B.44: Detailed view of key metrics for the run with seed 0 of experiment B.2.5 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents

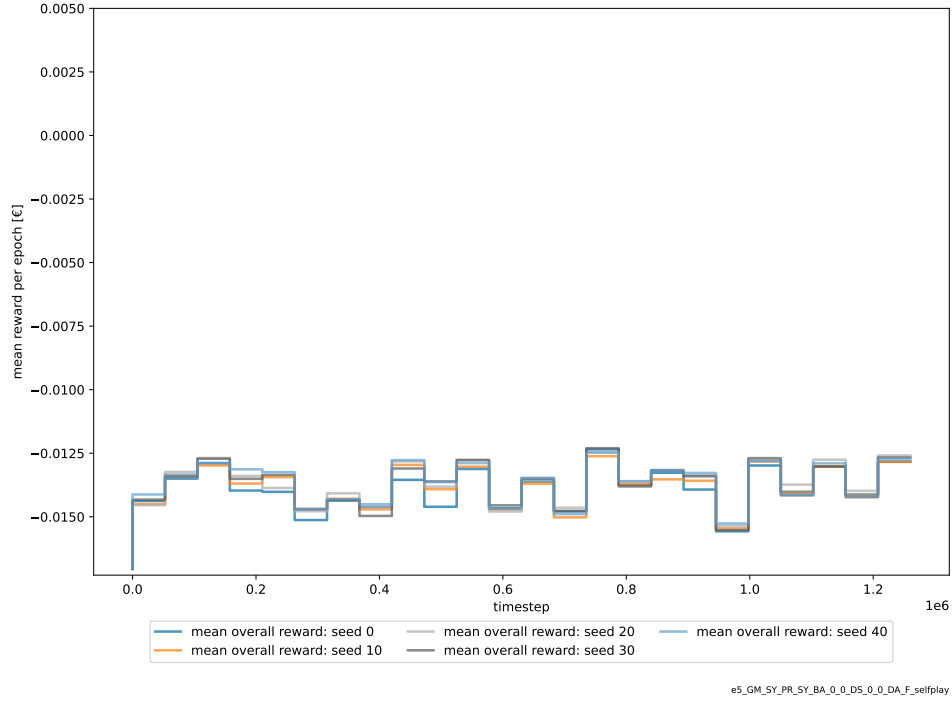


Figure B.45: Overview of the mean rewards per epoch for all agents combined for experiment B.2.5 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

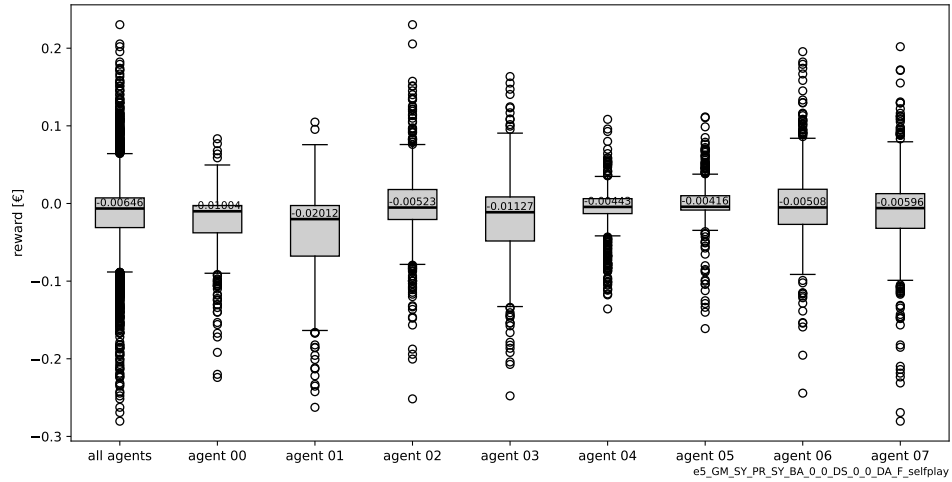


Figure B.46: Reward distribution in the last epoch for the run with seed 0 for experiment B.2.5 with self-play PPO agents.

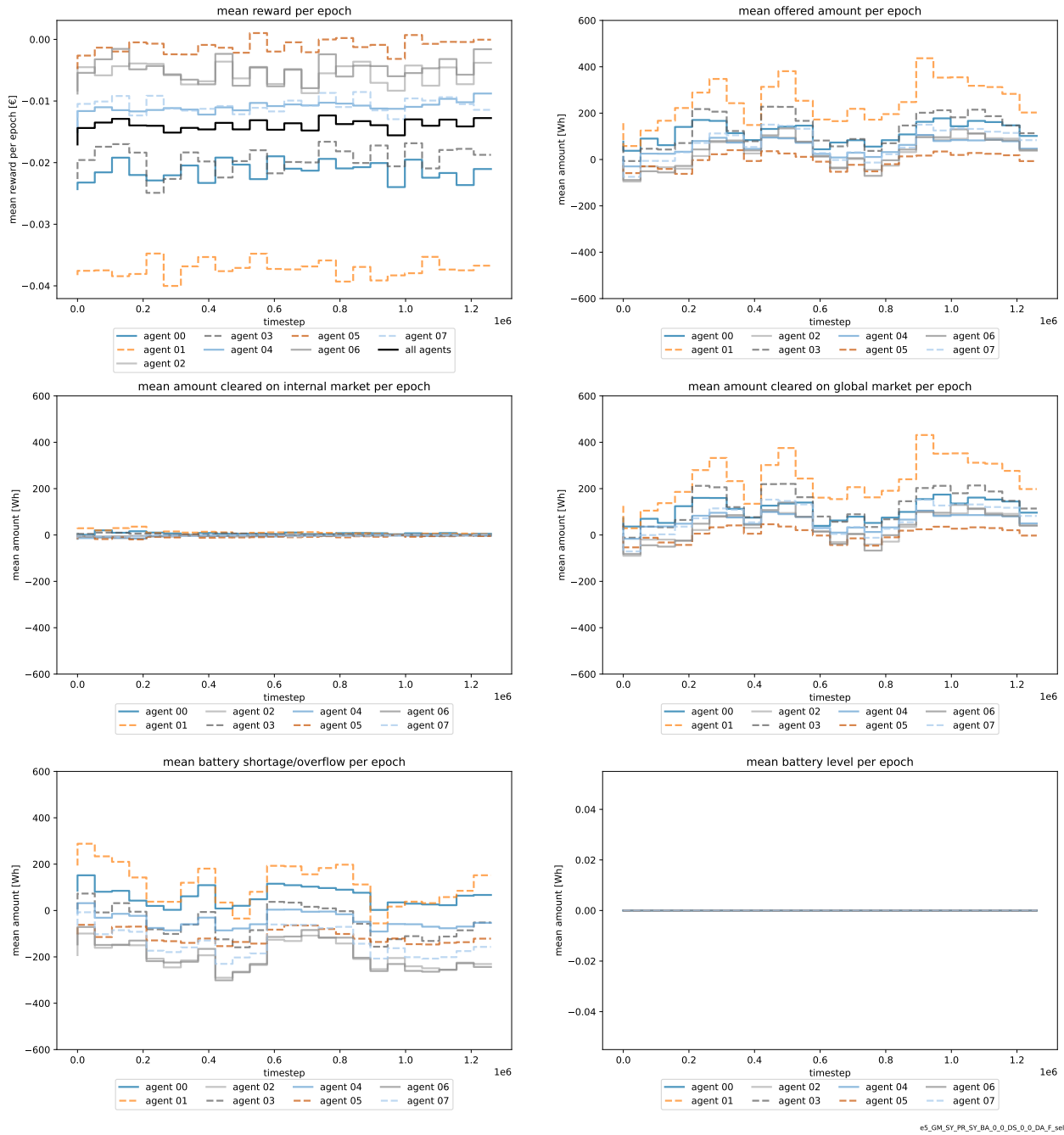


Figure B.47: Detailed view of key metrics for the run with seed 0 of experiment B.2.5 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.3 With Batteries and no Day-Ahead Window or Data Sharing

B.3.1 Cte. Prosumers and a Yearly Oscillating Global Market

Parameter	Setting	Parameter	Setting
global market data generation	formula	global market Time-of-use cost	0.000045 €/Wh
prosumers data generation	formula	global market mean day-ahead value	0.000042 €/Wh
Day-ahead window	deactivated	global market yearly day-ahead amplitude	0.000021 €/Wh
Data Sharing	deactivated	global market daily day-ahead amplitude	0 €/Wh
Producer agents	00,01,02,03	prosumer mean energy demand value	+/-320 Wh
Consumer Agents	04,05,06,07	prosumer yearly energy demand amplitude	0 Wh

Table B.7: Overview of settings for experiment B.3.1

PPO Agents

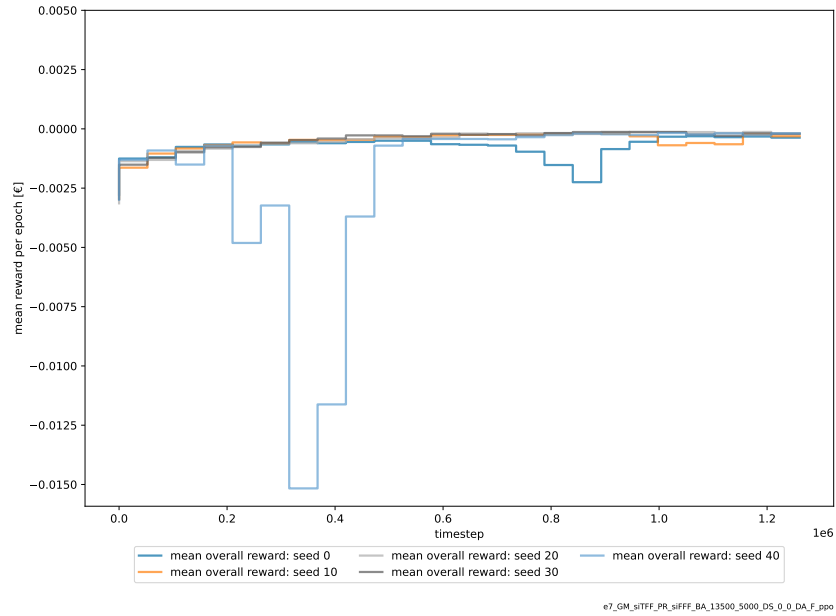


Figure B.48: Overview of the mean rewards per epoch for all agents combined for experiment B.3.1 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

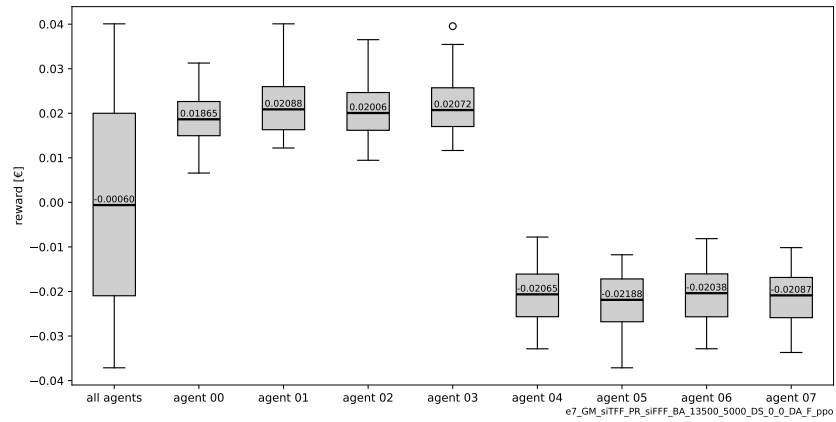
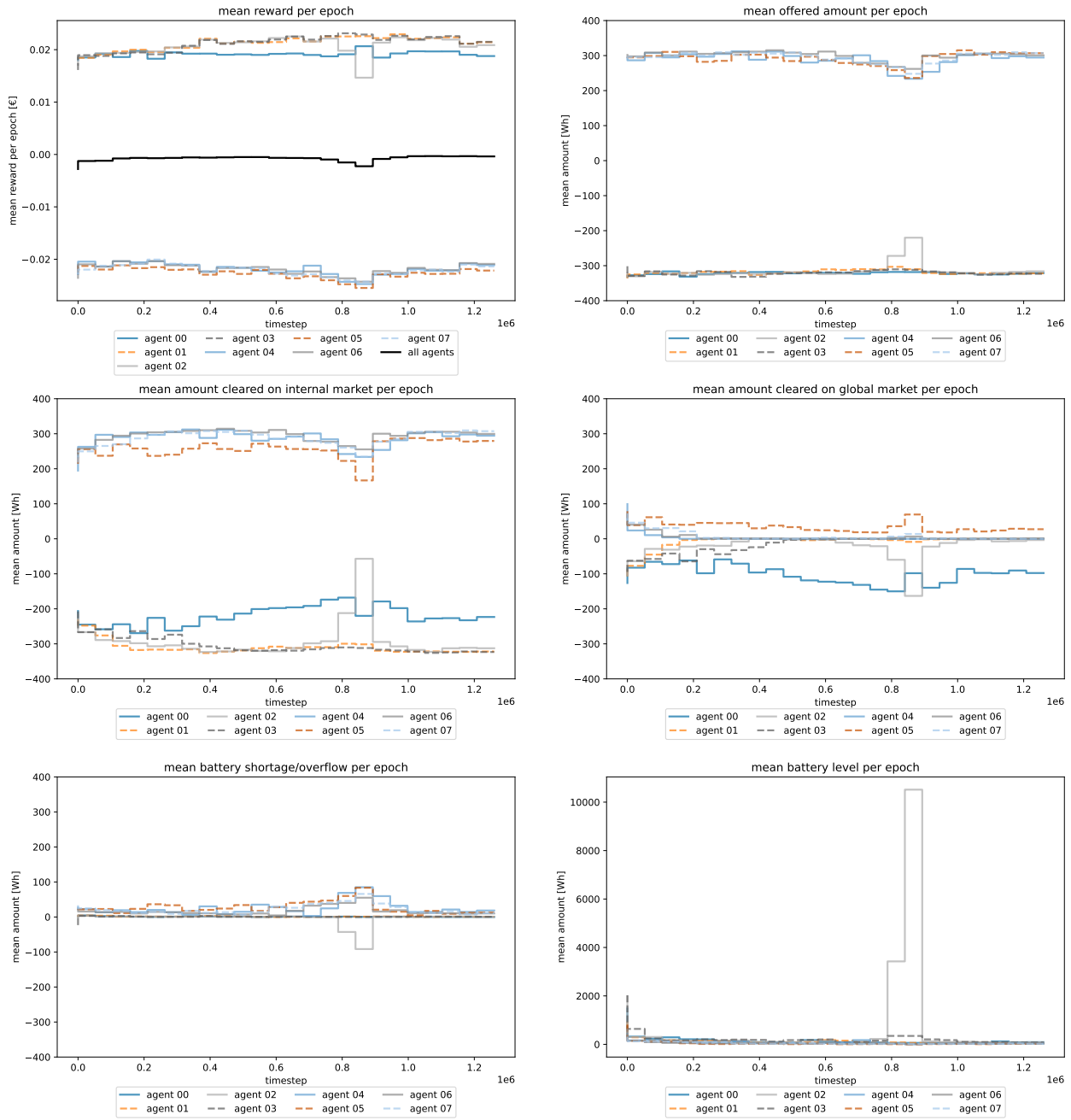


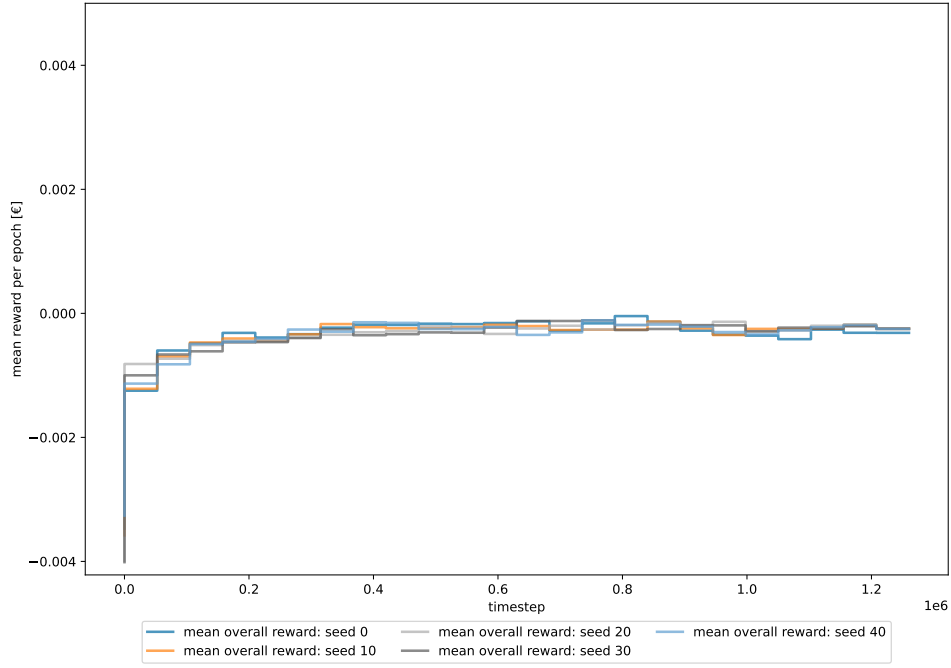
Figure B.49: Reward distribution in the last epoch for the run with seed 0 for experiment B.3.1 with PPO agents.



e7_GM_sITFF_PR_sITFF_BA_13500_5000_DS_0_0_DA_F_ppo

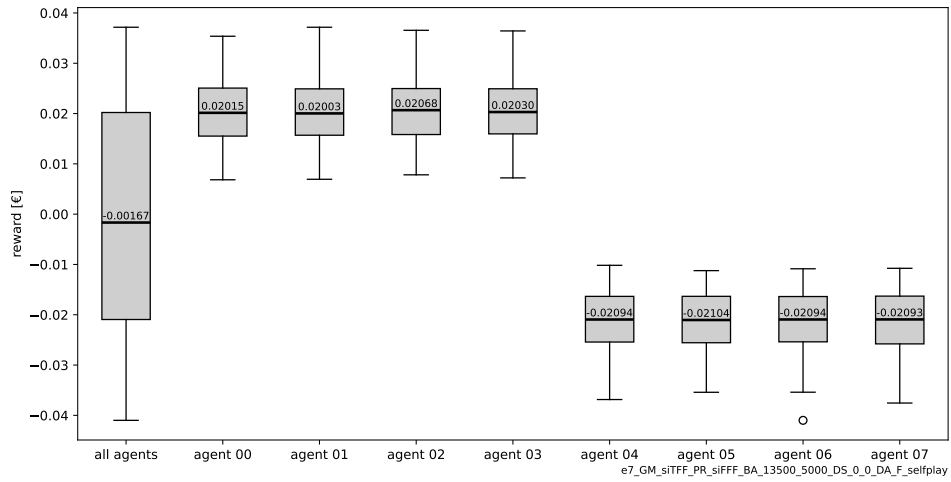
Figure B.50: Detailed view of key metrics for the run with seed 0 of experiment B.3.1 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents



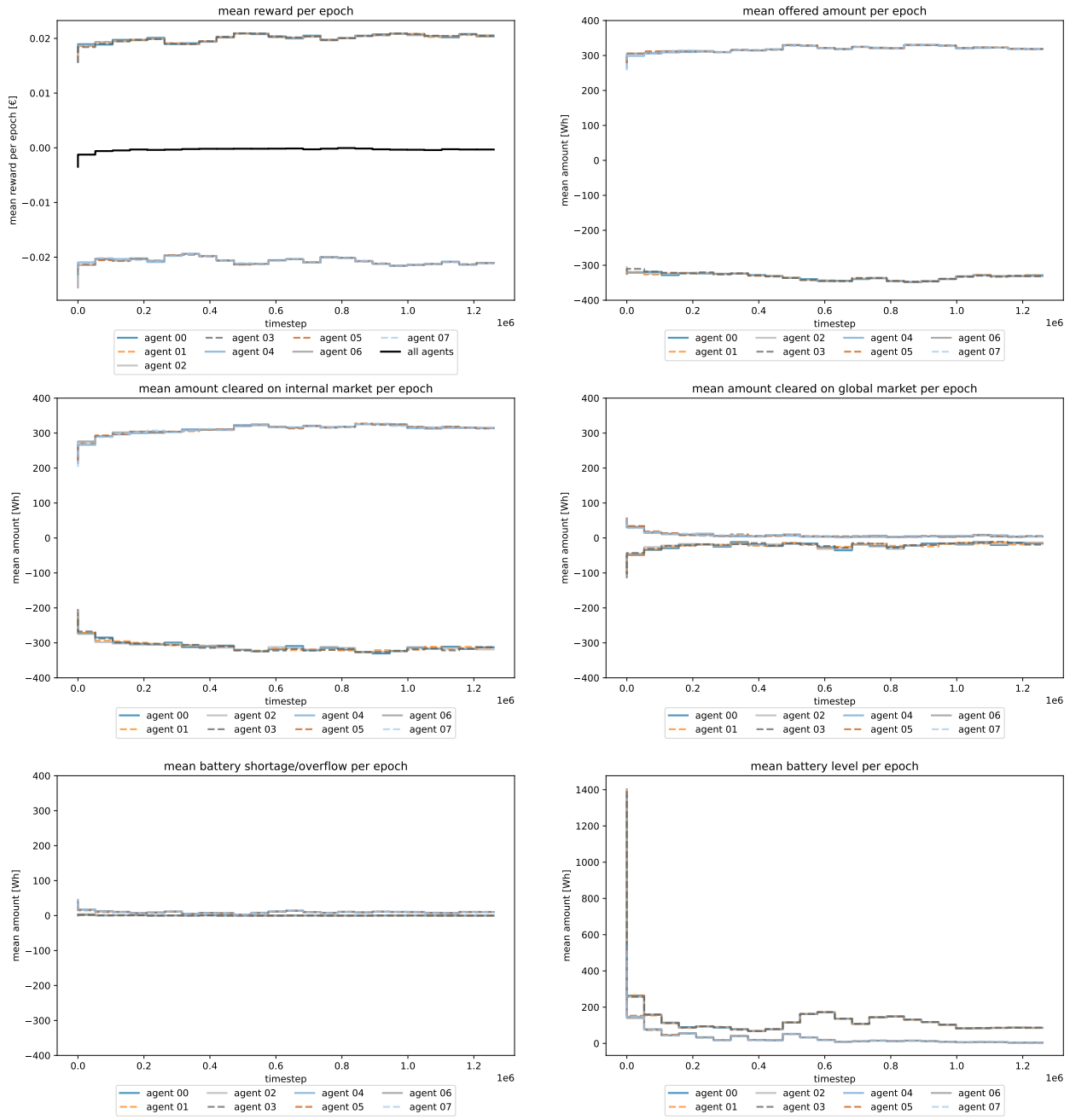
e7_GM_siTFF_PR_siFFF_BA_13500_5000_DS_0_0_DA_F_selfplay

Figure B.51: Overview of the mean rewards per epoch for all agents combined for experiment B.3.1 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.



e7_GM_siTFF_PR_siFFF_BA_13500_5000_DS_0_0_DA_F_selfplay

Figure B.52: Reward distribution in the last epoch for the run with seed 0 for experiment B.3.1 with self-play PPO agents.



e7_GM_siTFF_PR_siFFF_BA_13500_5000_DS_0_0_DA_F_selfplay

Figure B.53: Detailed view of key metrics for the run with seed 0 of experiment B.3.1 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.3.2 Cte. Prosumers and a Yearly and Daily oscillating Global Market

Parameter	Setting	Parameter	Setting
global market data generation	formula	global market Time-of-use cost	0.000045 €/Wh
prosumers data generation	formula	global market mean day-ahead value	0.000042 €/Wh
Day-ahead window	deactivated	global market yearly day-ahead amplitude	0.000021 €/Wh
Data Sharing	deactivated	global market daily day-ahead amplitude	0.000005 €/Wh
Producer agents	00,01,02,03	prosumer mean energy demand value	+/-320 Wh
Consumer Agents	04,05,06,07	prosumer yearly energy demand amplitude	0 Wh

Table B.8: Overview of settings for experiment B.3.2

PPO Agents

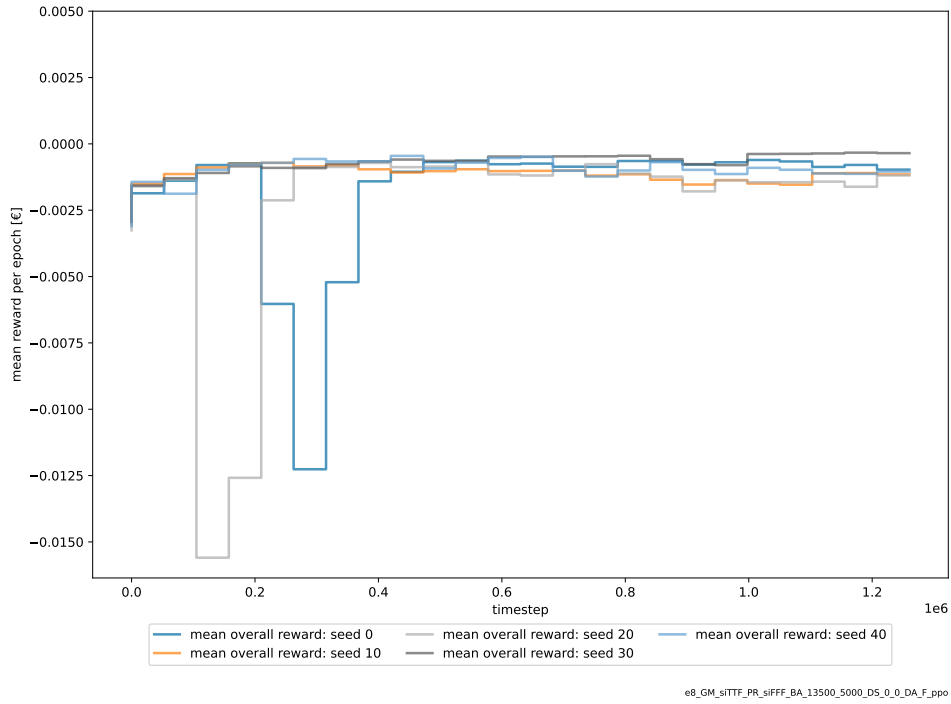


Figure B.54: Overview of the mean rewards per epoch for all agents combined for experiment B.3.2 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

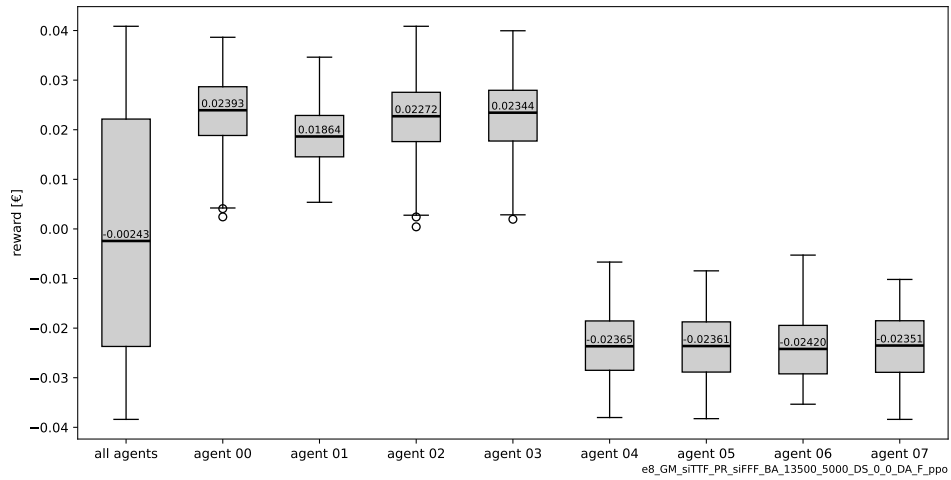
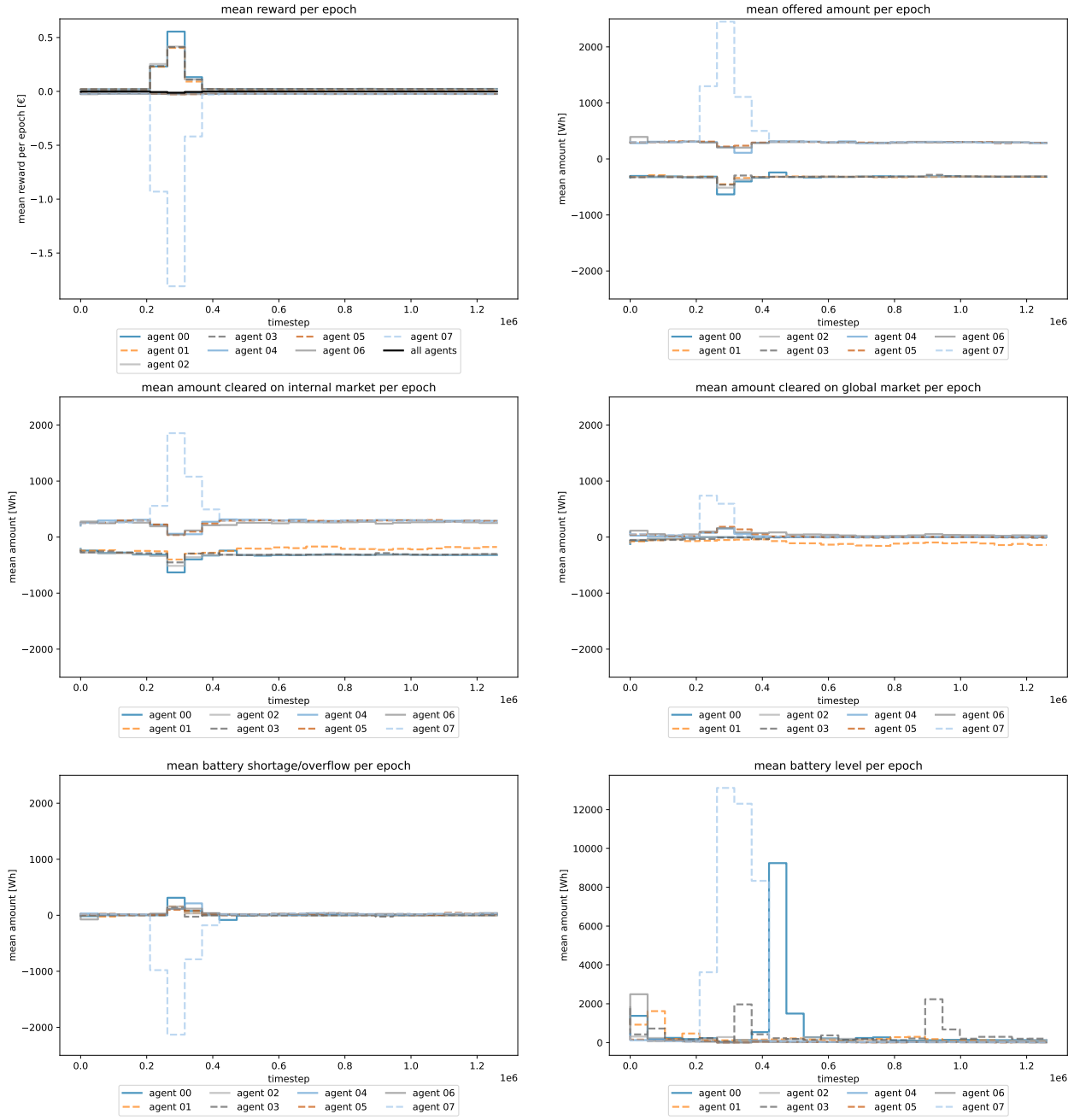


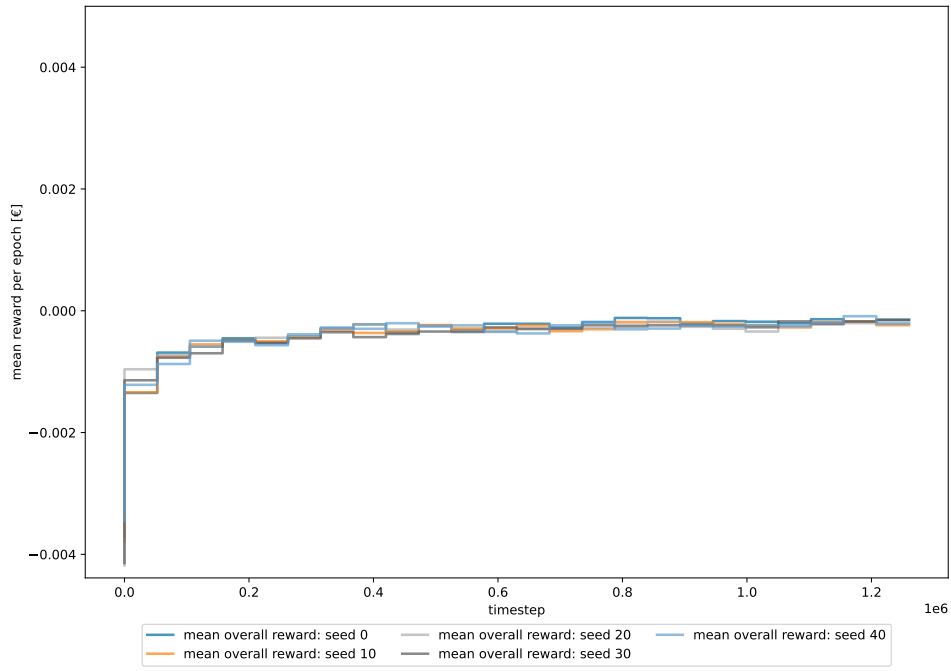
Figure B.55: Reward distribution in the last epoch for the run with seed 0 for experiment B.3.2 with PPO agents.



e8_GM_siTTF_PR_siFFF_BA_13500_5000_DS_0_0_DA_F_ppo

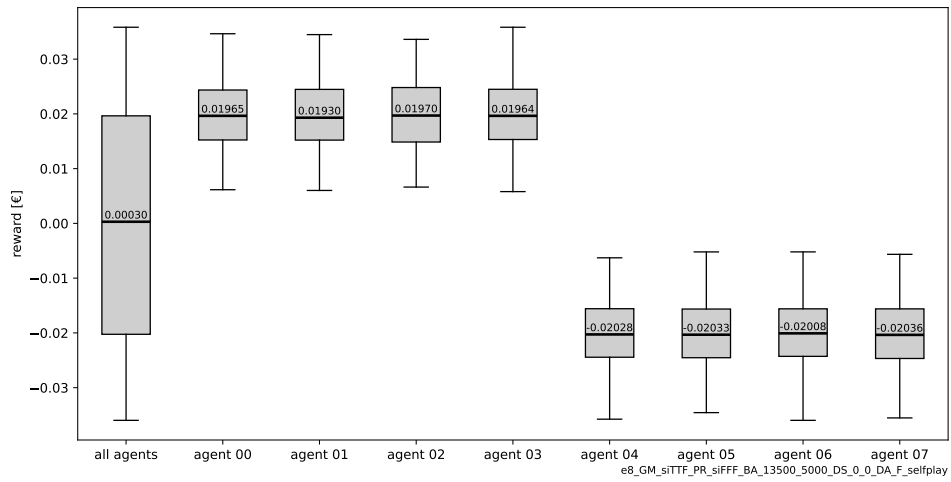
Figure B.56: Detailed view of key metrics for the run with seed 0 of experiment B.3.2 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents



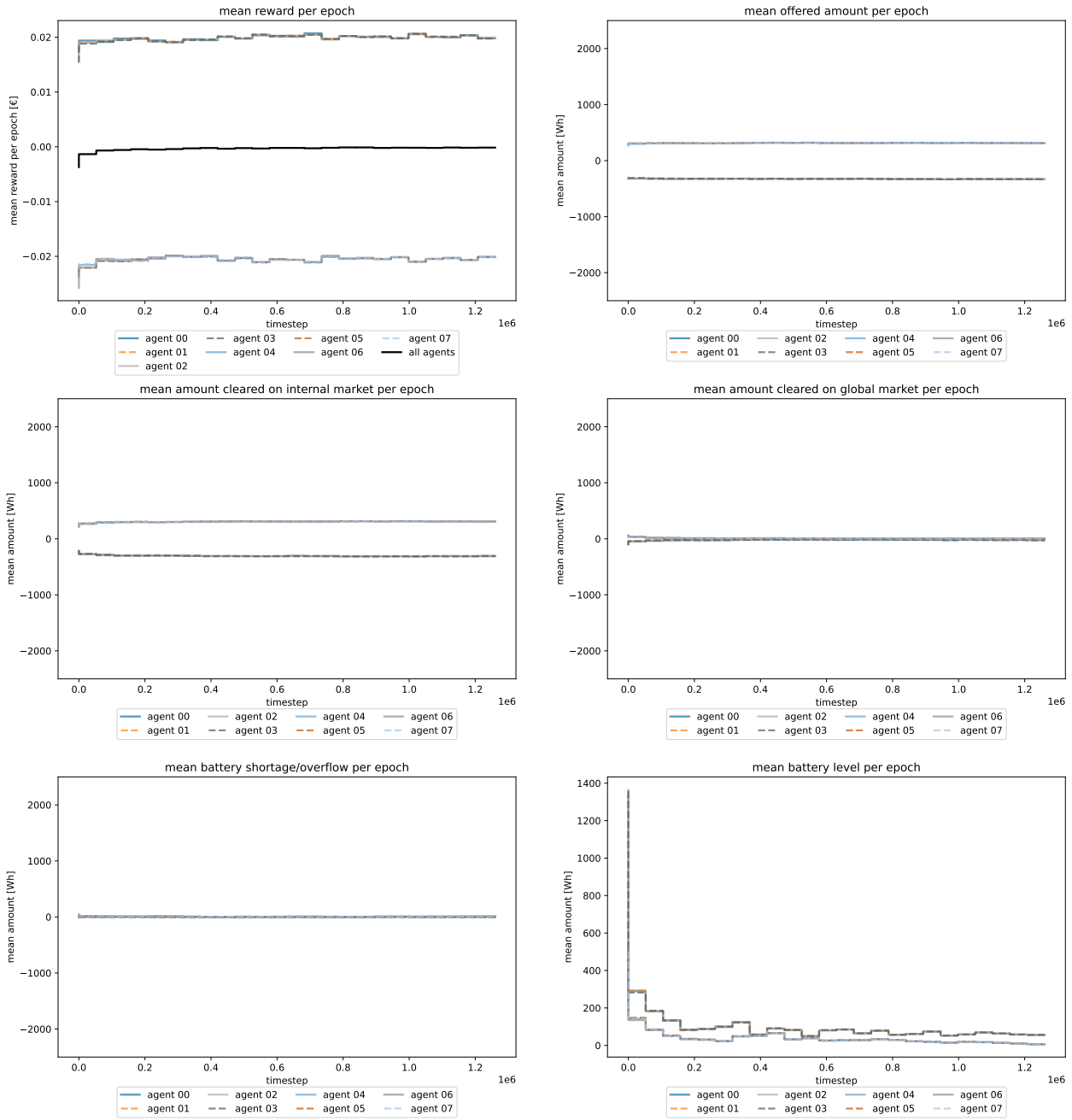
e8_GM_siTTF_PR_siFFF_BA_13500_5000_DS_0_0_DA_F_selfplay

Figure B.57: Overview of the mean rewards per epoch for all agents combined for experiment B.3.2 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.



e8_GM_siTTF_PR_siFFF_BA_13500_5000_DS_0_0_DA_F_selfplay

Figure B.58: Reward distribution in the last epoch for the run with seed 0 for experiment B.3.2 with self-play PPO agents.



e8_GM_siTTF_PR_siFFF_BA_13500_5000_DS_0_0_DA_F_selfplay

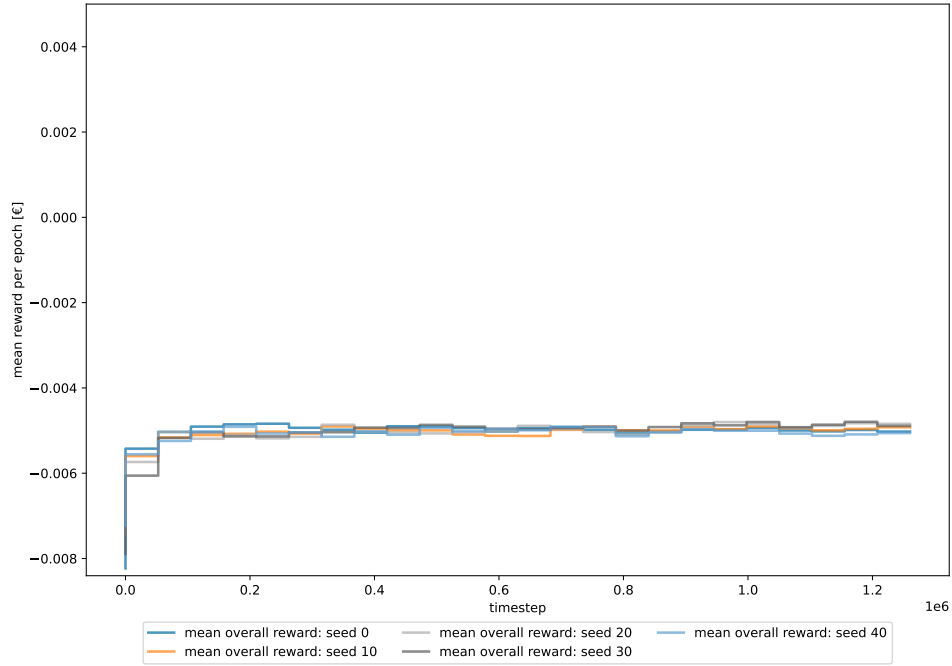
Figure B.59: Detailed view of key metrics for the run with seed 0 of experiment B.3.2 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.3.3 Yearly Oscillating Prosumers and a Cte. Global Market

Parameter	Setting	Parameter	Setting
global market data generation	formula	global market Time-of-use cost	0.000045 €/Wh
prosumers data generation	formula	global market mean day-ahead value	0.000042 €/Wh
Day-ahead window	deactivated	global market yearly day-ahead amplitude	0 €/Wh
Data Sharing	deactivated	global market daily day-ahead amplitude	0 €/Wh
Producer agents	00,01,02,03	prosumer mean energy demand value	+/-320 Wh
Consumer Agents	04,05,06,07	prosumer yearly energy demand amplitude	350 Wh

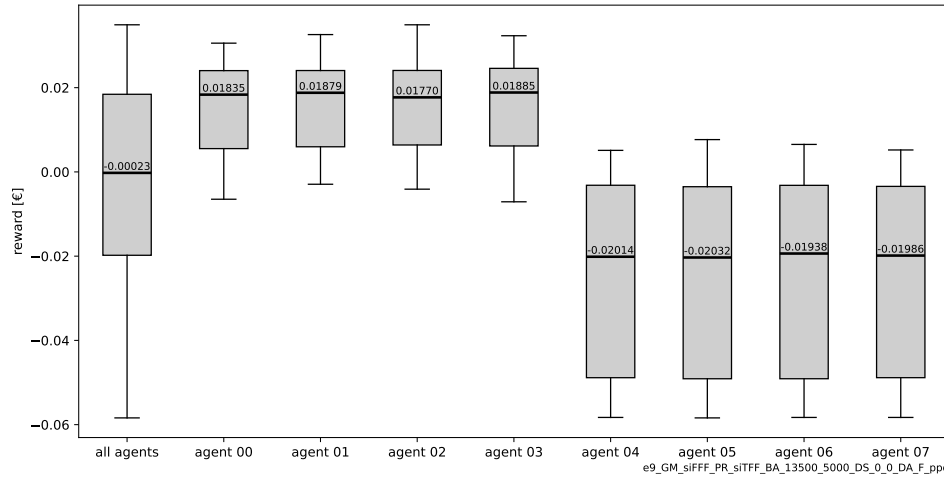
Table B.9: Overview of settings for experiment B.3.3

PPO Agents



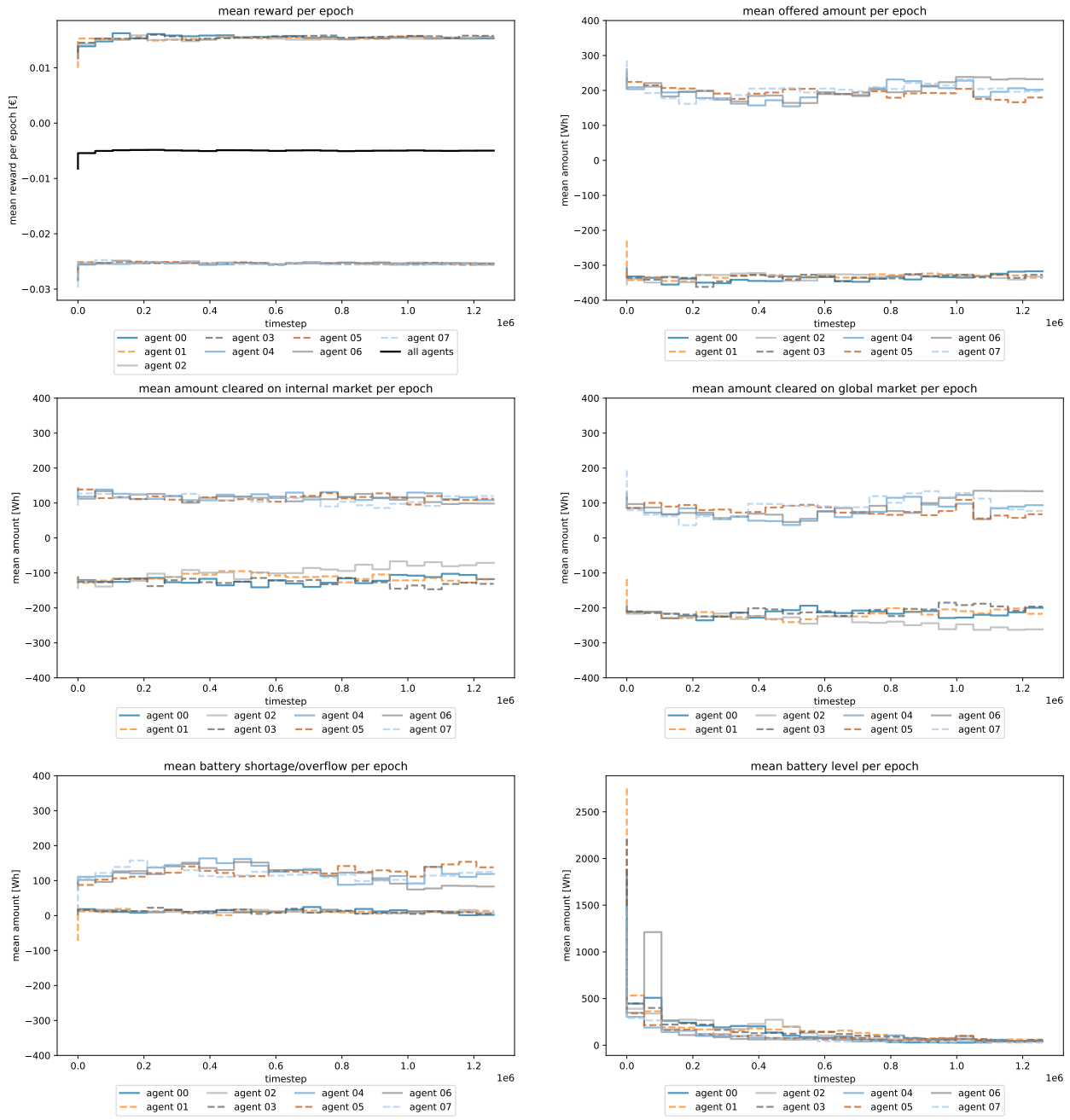
e9_GM_sifff_PR_sitff_BA_13500_5000_DS_0_0_DA_F_ppo

Figure B.60: Overview of the mean rewards per epoch for all agents combined for experiment B.3.3 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.



e9_GM_sifff_PR_sitff_BA_13500_5000_DS_0_0_DA_F_ppo

Figure B.61: Reward distribution in the last epoch for the run with seed 0 for experiment B.3.3 with PPO agents



e9_GM_sIFFF_PR_sITFF_BA_13500_5000_DS_0_0_DA_F_ppo

Figure B.62: Detailed view of key metrics for the run with seed 0 of experiment B.3.3 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents

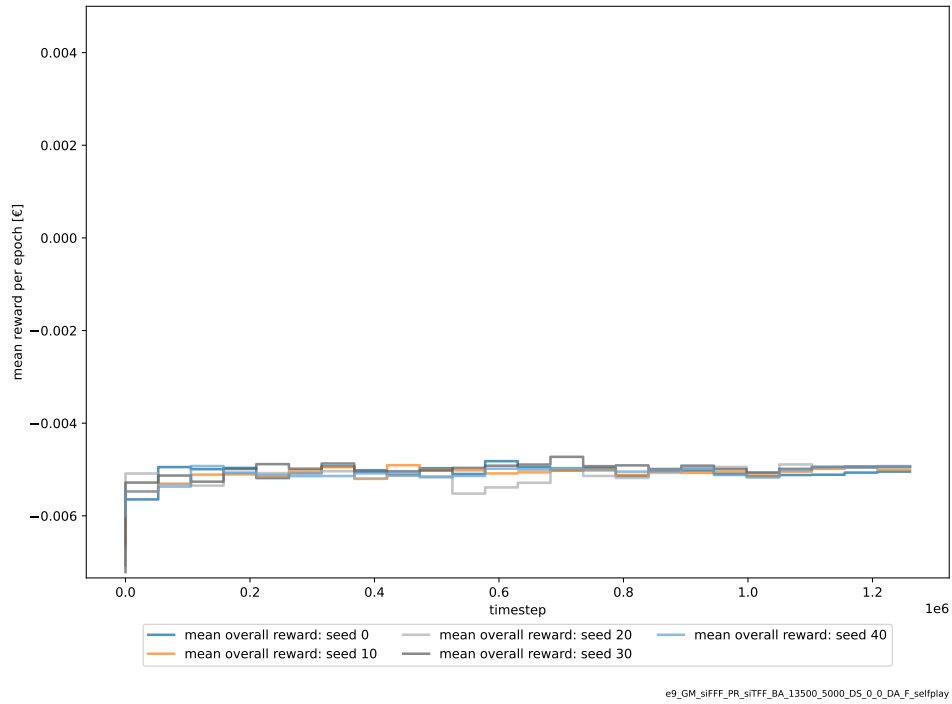


Figure B.63: Overview of the mean rewards per epoch for all agents combined for experiment B.3.3 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

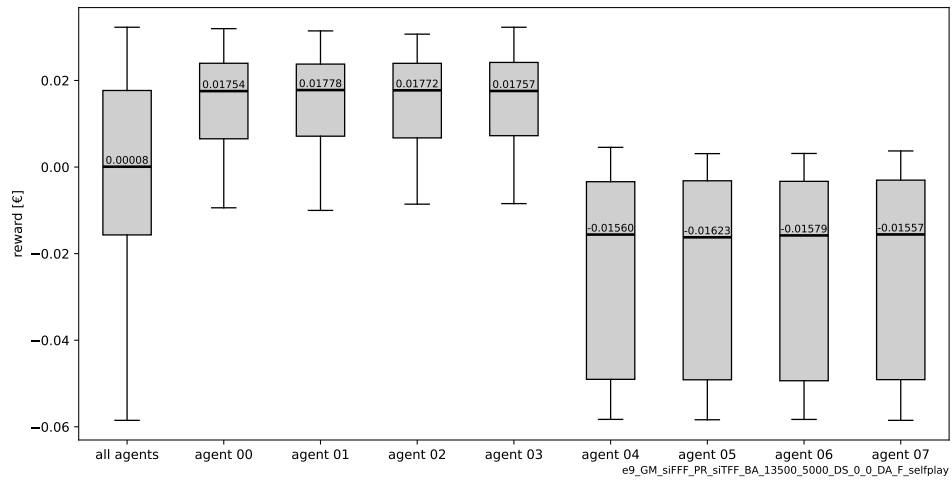
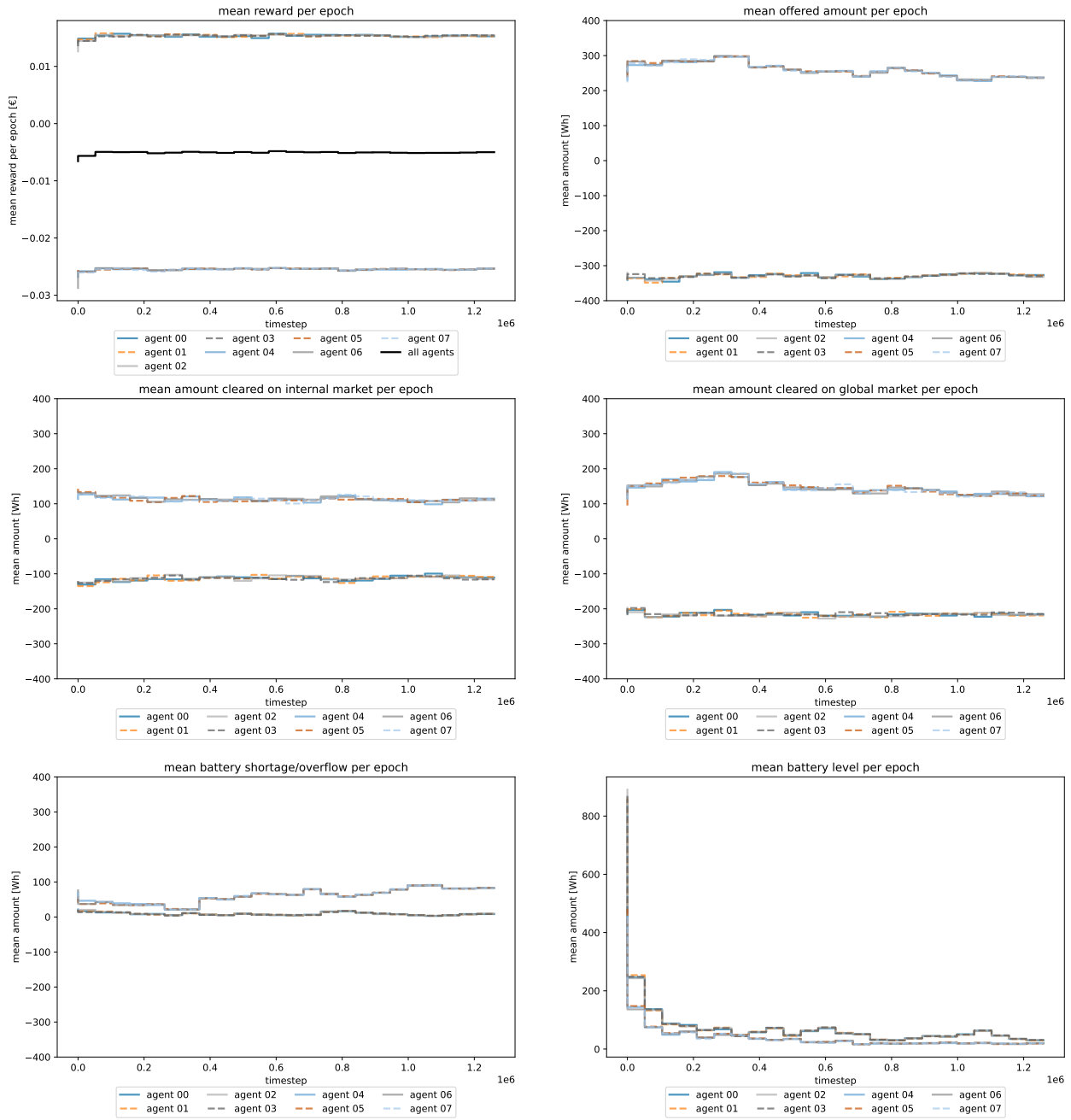


Figure B.64: Reward distribution in the last epoch for the run with seed 0 for experiment B.3.3 with self-play PPO agents.



e9_gM_sIFFF_PR_sITFF_BA_13500_5000_DS_0_0_DA_F_selfplay

Figure B.65: Detailed view of key metrics for the run with seed 0 of experiment B.3.3 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.3.4 Using the Synthetic Datasets

Parameter	Setting
global market data generation	Synthetic
prosumers data generation	Synthetic
Day-ahead window	deactivated
Data Sharing	deactivated
global market Time-of-use cost	0.000045 €/Wh

Table B.10: Overview of settings for experiment B.3.4

PPO Agents

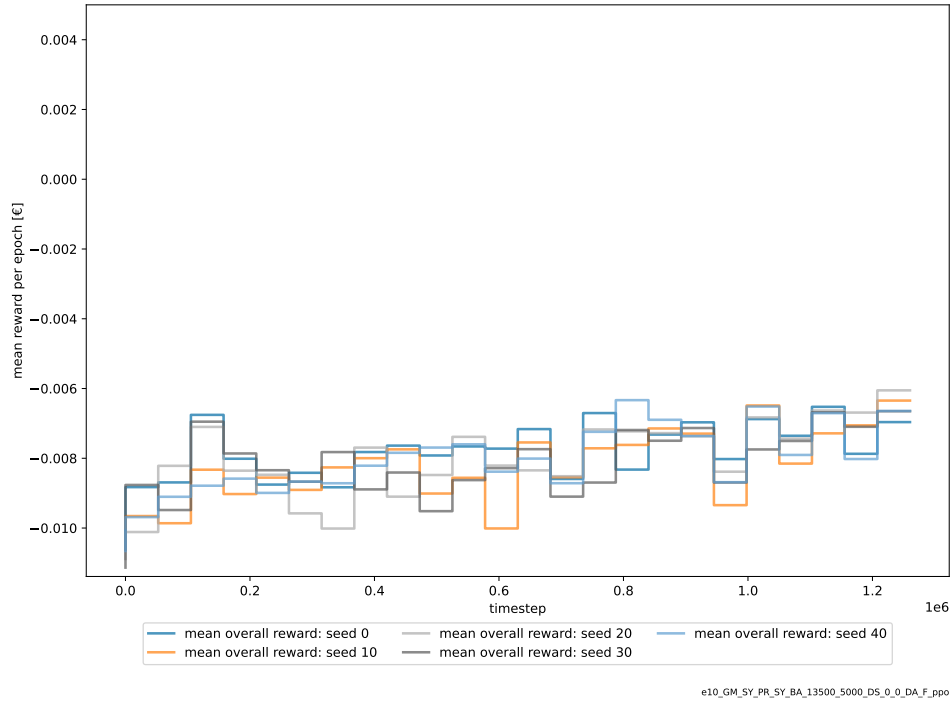


Figure B.66: Overview of the mean rewards per epoch for all agents combined for experiment B.3.4 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

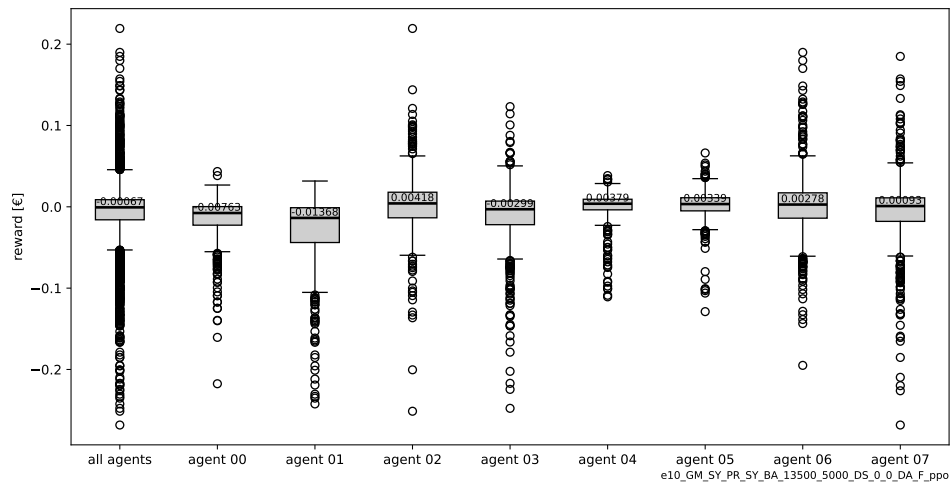
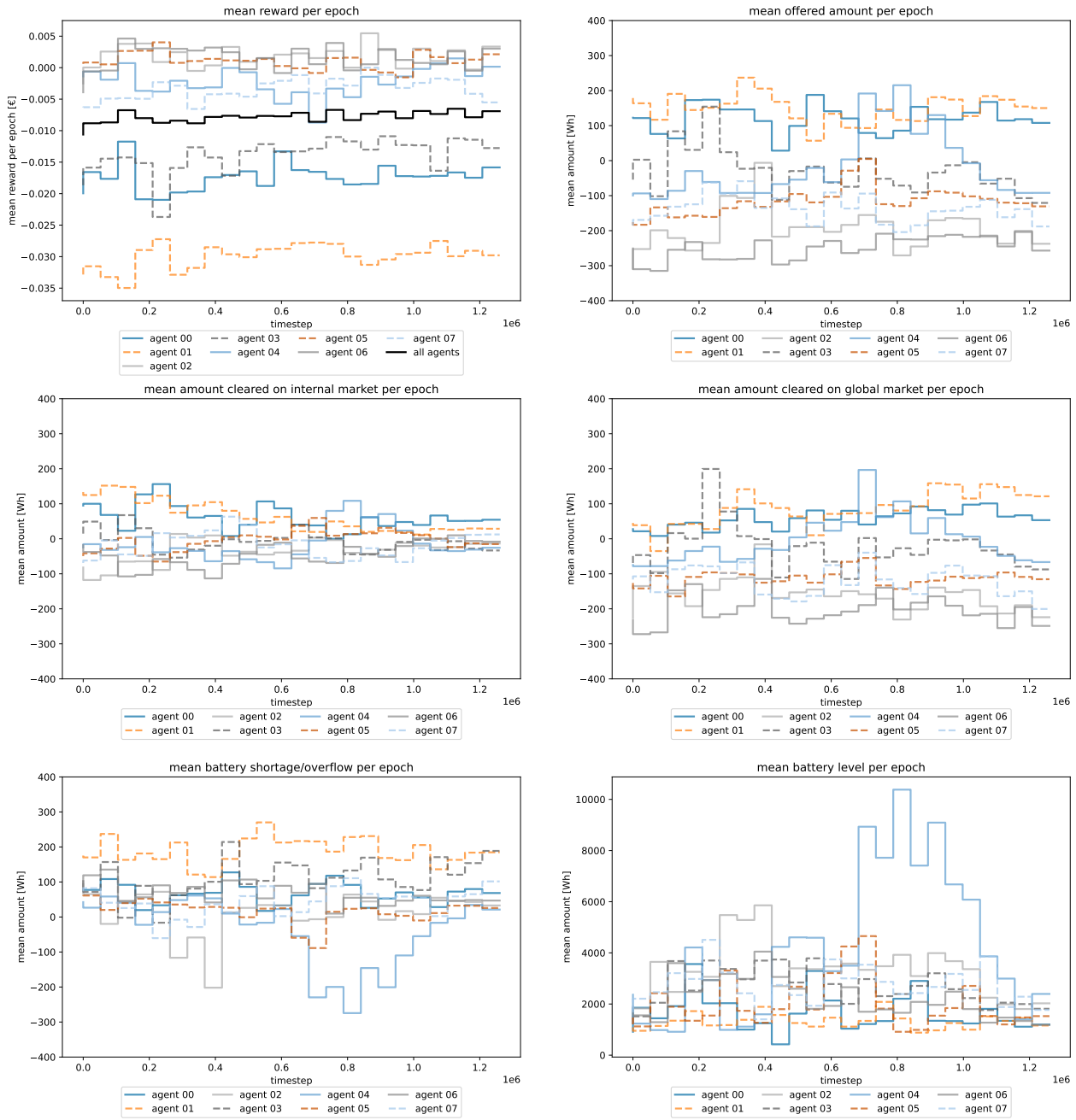


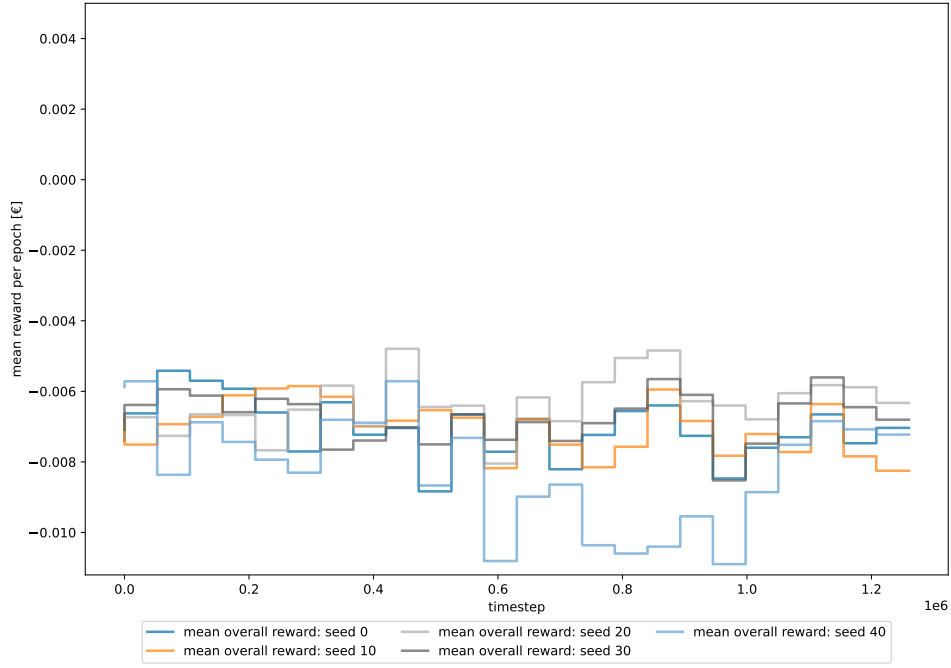
Figure B.67: Reward distribution in the last epoch for the run with seed 0 for experiment B.3.4 with PPO agents



e10_GM_SY_PR_SY_BA_13500_5000_DS_0_0_DA_F_ppo

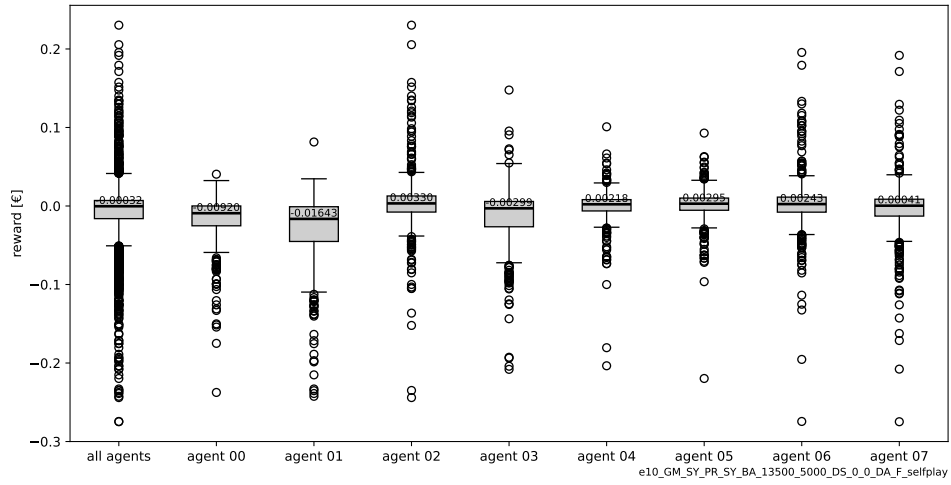
Figure B.68: Detailed view of key metrics for the run with seed 0 of experiment B.3.4 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents



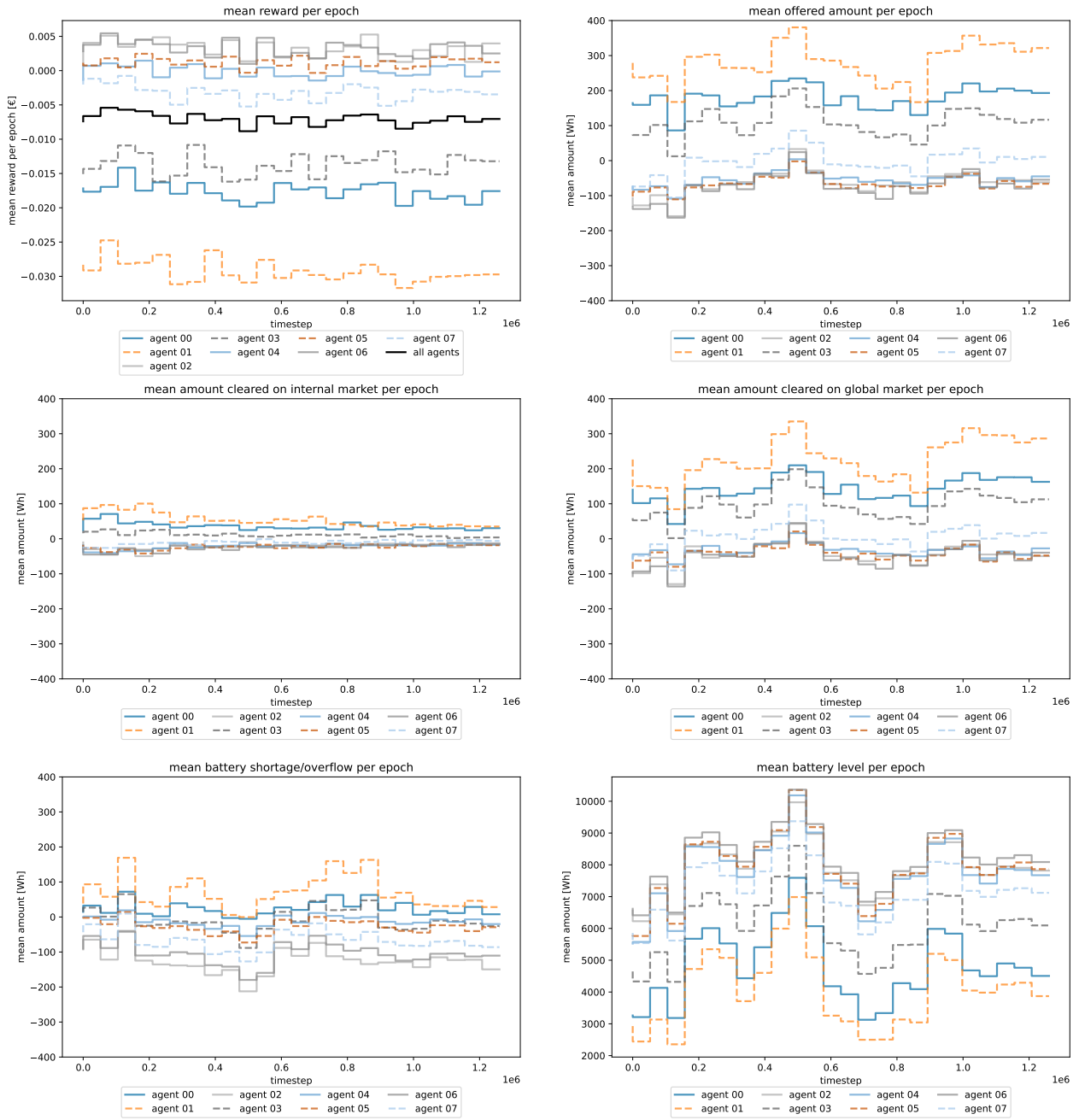
e10_GM_SY_PR_SY_BA_13500_5000_DS_0_0_DA_F_selfplay

Figure B.69: Overview of the mean rewards per epoch for all agents combined for experiment B.3.4 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.



e10_GM_SY_PR_SY_BA_13500_5000_DS_0_0_DA_F_selfplay

Figure B.70: Reward distribution in the last epoch for the run with seed 0 for experiment B.3.4 with self-play PPO agents.



e10_GM_SY_PR_SY_BA_13500_5000_DS_0_0_DA_F_selfplay

Figure B.71: Detailed view of key metrics for the run with seed 0 of experiment B.3.4 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.4 With Batteries and Day-Ahead Window and no Data Sharing

B.4.1 Cte. Prosumers and a Yearly Oscillating Global Market

Parameter	Setting	Parameter	Setting
global market data generation	formula	global market Time-of-use cost	0.000045 €/Wh
prosumers data generation	formula	global market mean day-ahead value	0.000042 €/Wh
Day-ahead window	activated	global market yearly day-ahead amplitude	0.000021 €/Wh
Data Sharing	deactivated	global market daily day-ahead amplitude	0 €/Wh
Producer agents	00,01,02,03	prosumer mean energy demand value	+/-320 Wh
Consumer Agents	04,05,06,07	prosumer yearly energy demand amplitude	0 Wh

Table B.11: Overview of settings for experiment B.4.1

PPO Agents

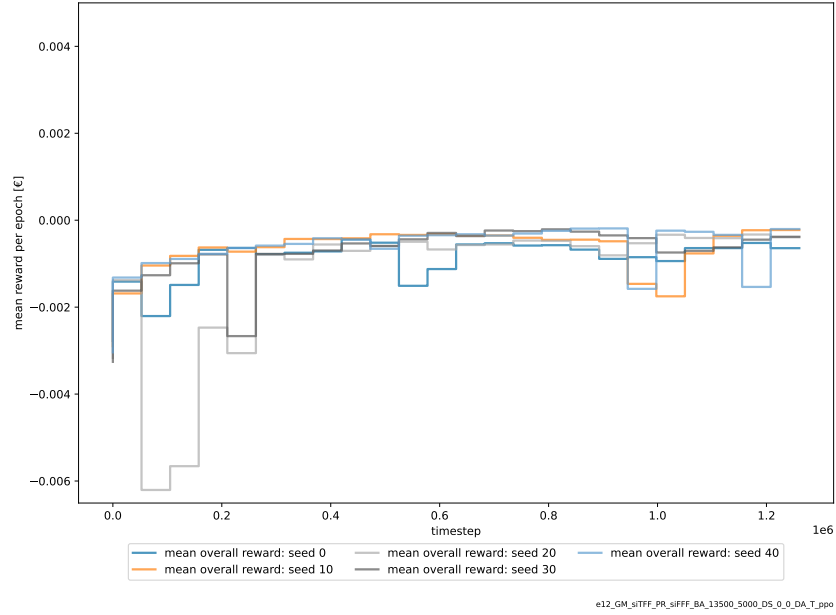


Figure B.72: Overview of the mean rewards per epoch for all agents combined for experiment B.4.1 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

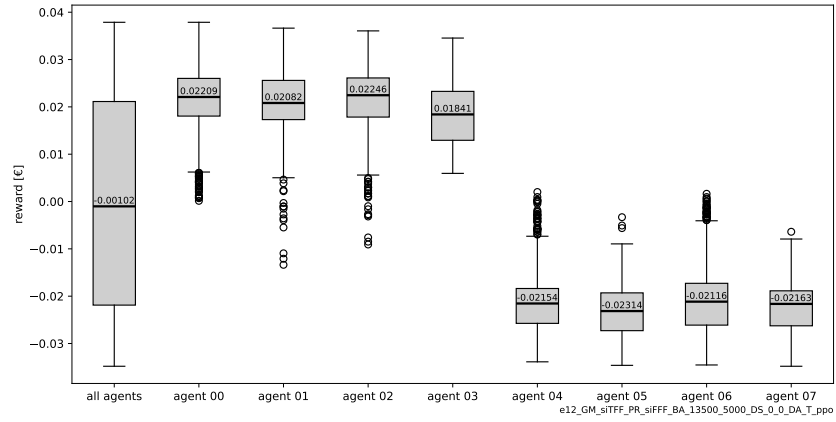
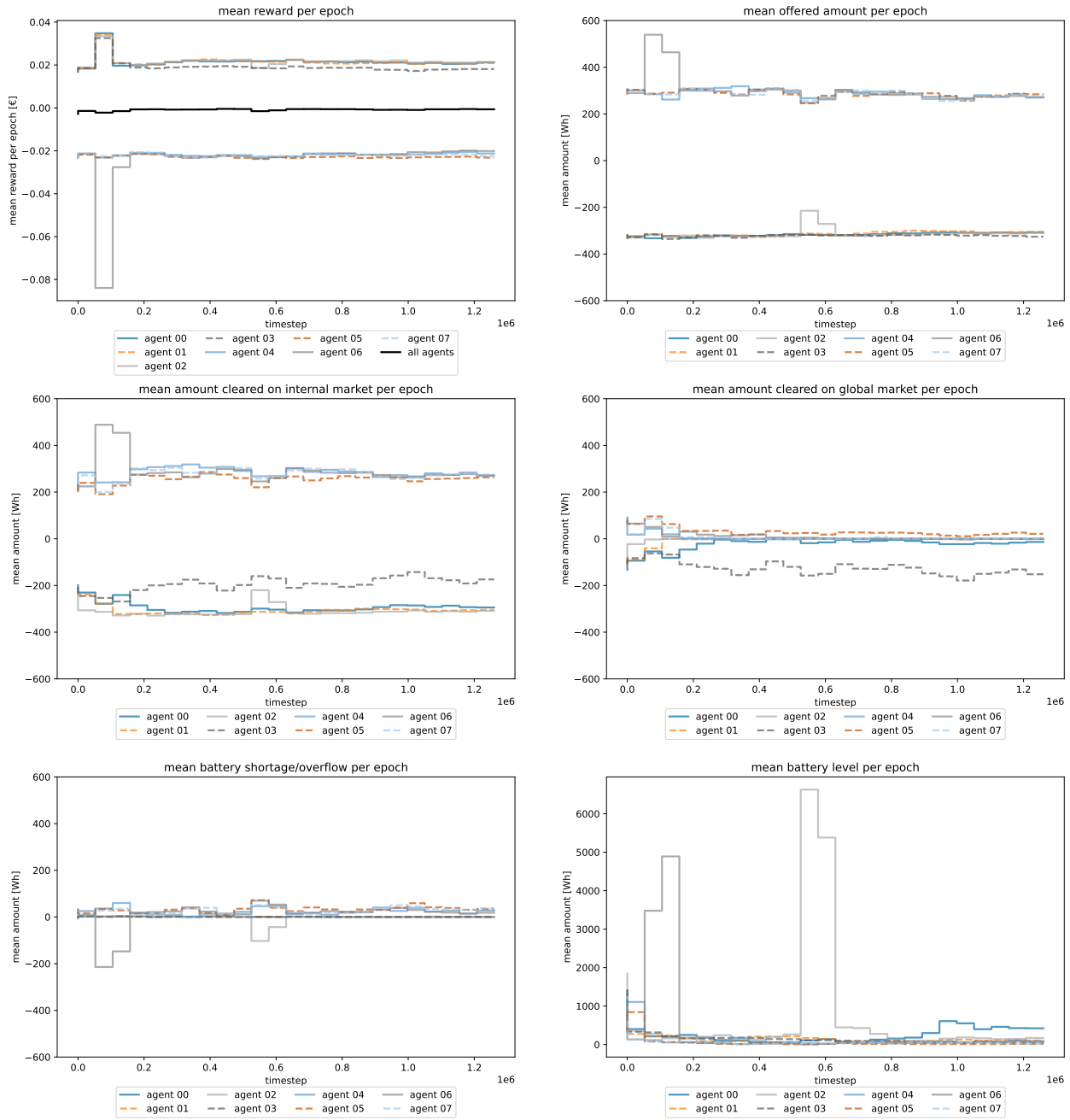


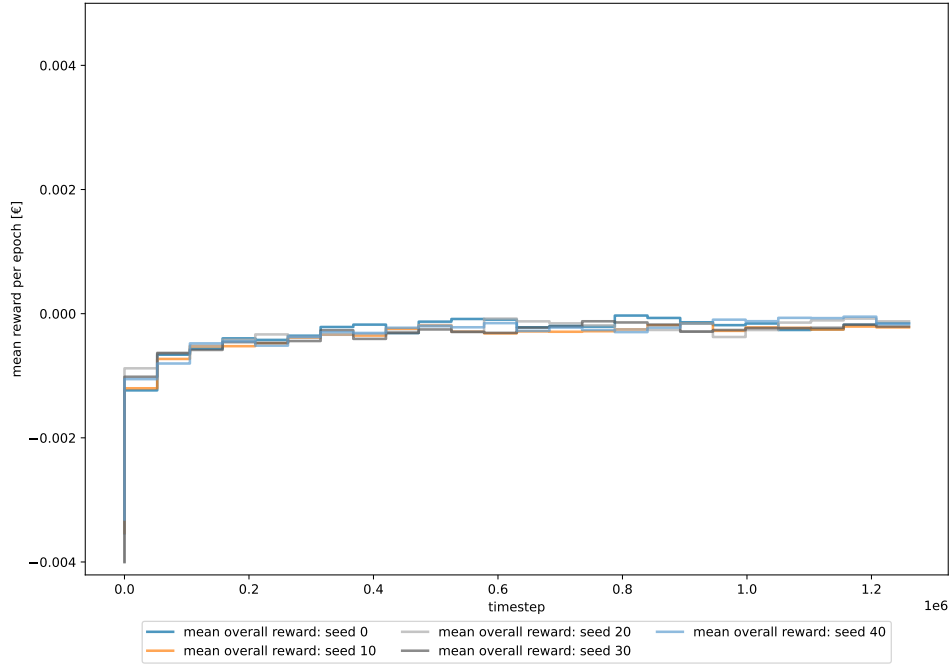
Figure B.73: Reward distribution in the last epoch for the run with seed 0 for experiment B.4.1 with PPO agents.



e12_GM_siTFF_PR_siFFF_BA_13500_5000_DS_0_0_DA_T_ppo

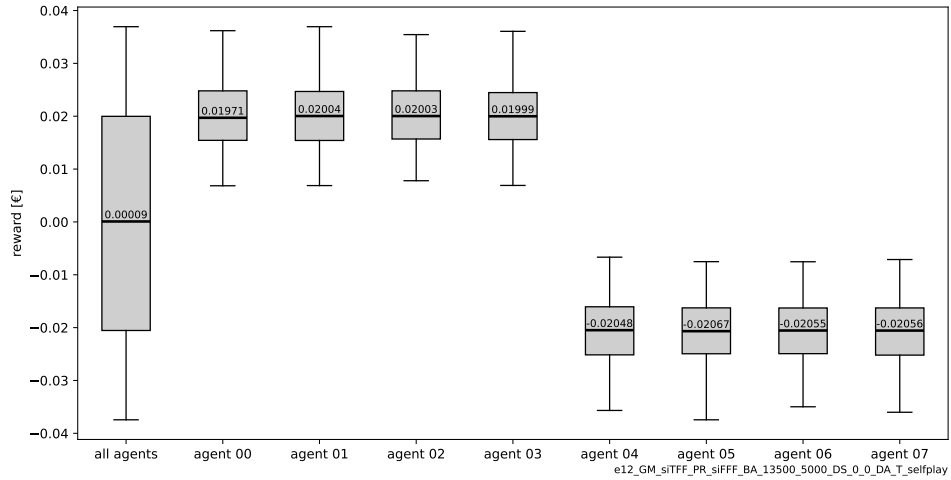
Figure B.74: Detailed view of key metrics for the run with seed 0 of experiment B.4.1 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents



e12_GM_siTFF_PR_siFFF_BA_13500_5000_DS_0_0_DA_T_selfplay

Figure B.75: Overview of the mean rewards per epoch for all agents combined for experiment B.4.1 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.



e12_GM_siTFF_PR_siFFF_BA_13500_5000_DS_0_0_DA_T_selfplay

Figure B.76: Reward distribution in the last epoch for the run with seed 0 for experiment B.4.1 with self-play PPO agents.

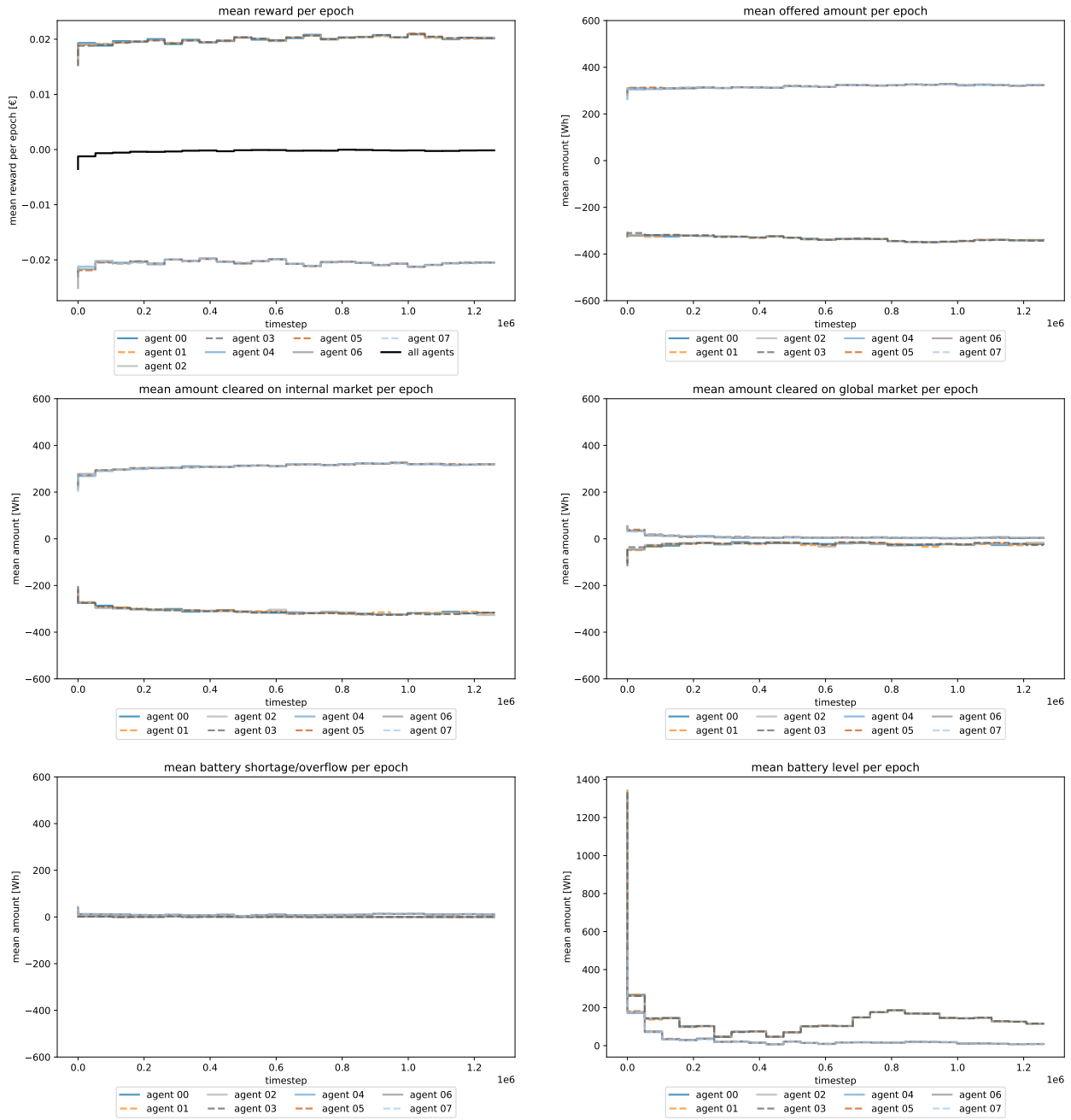


Figure B.77: Detailed view of key metrics for the run with seed 0 of experiment B.4.1 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.4.2 Cte. Prosumers and a Yearly and Daily oscillating Global Market

Parameter	Setting	Parameter	Setting
global market data generation	formula	global market Time-of-use cost	0.000045 €/Wh
prosumers data generation	formula	global market mean day-ahead value	0.000042 €/Wh
Day-ahead window	activated	global market yearly day-ahead amplitude	0.000021 €/Wh
Data Sharing	deactivated	global market daily day-ahead amplitude	0.000005 €/Wh
Producer agents	00,01,02,03	prosumer mean energy demand value	+/-320 Wh
Consumer Agents	04,05,06,07	prosumer yearly energy demand amplitude	0 Wh

Table B.12: Overview of settings for experiment B.4.2

PPO Agents

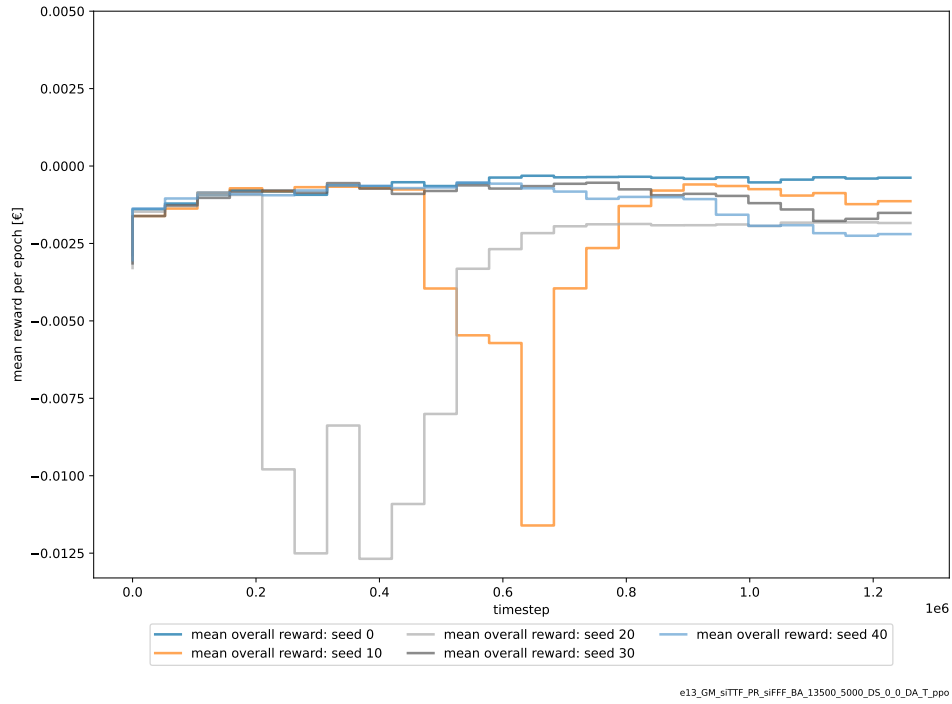


Figure B.78: Overview of the mean rewards per epoch for all agents combined for experiment B.4.2 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

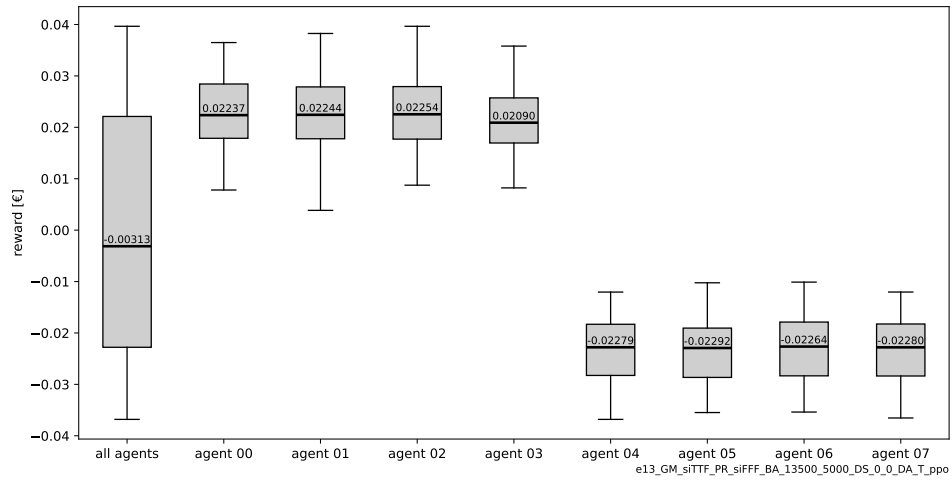
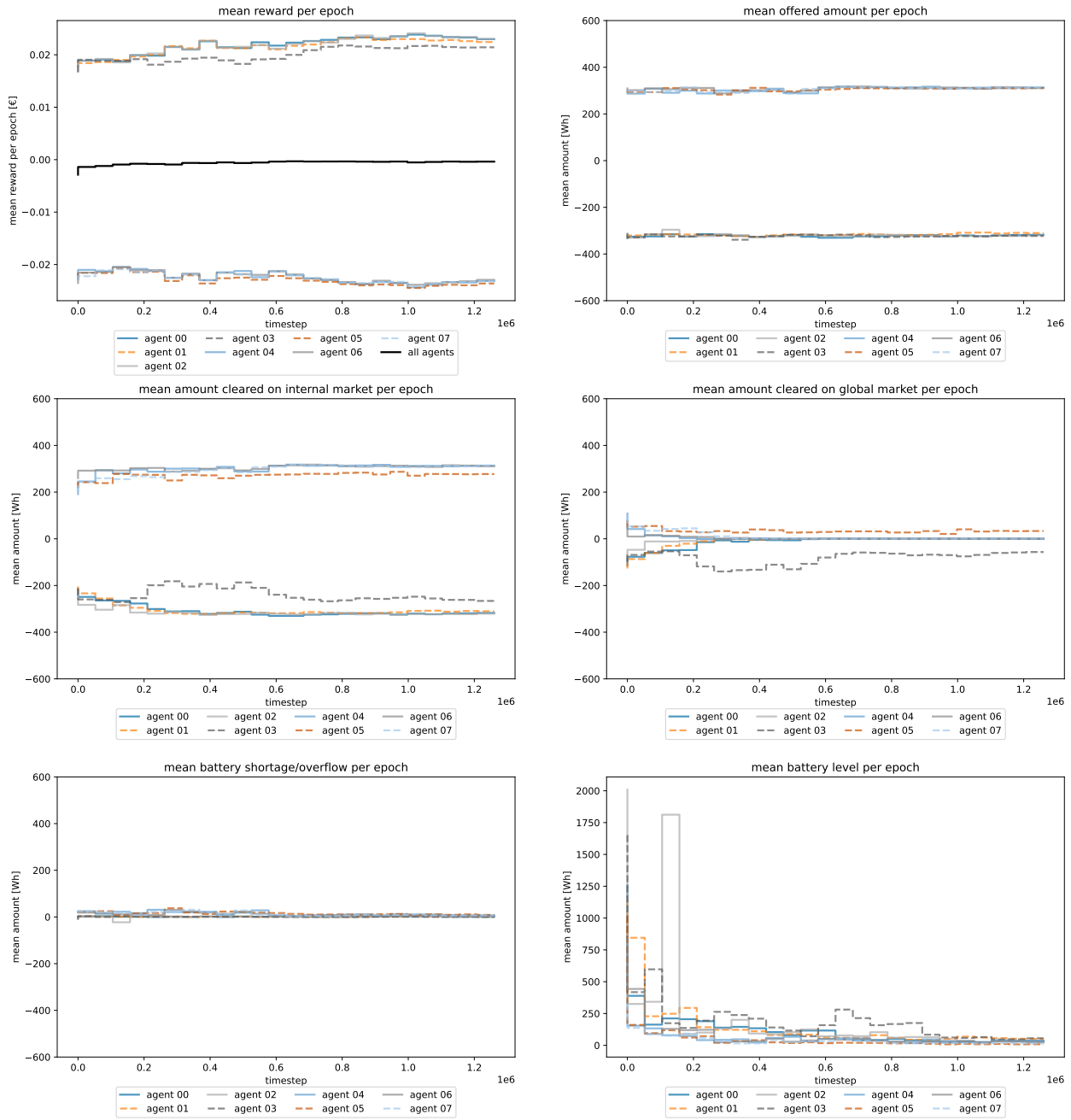


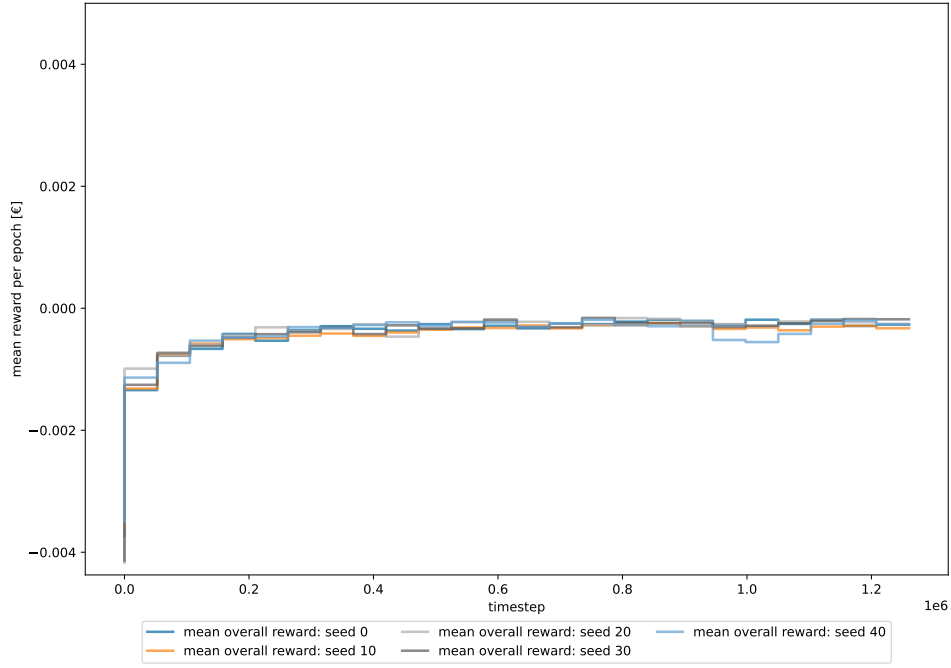
Figure B.79: Reward distribution in the last epoch for the run with seed 0 for experiment B.4.2 with PPO agents.



e13_GM_sITTF_PR_sIFFF_BA_13500_5000_DS_0_0_DA_T_ppo

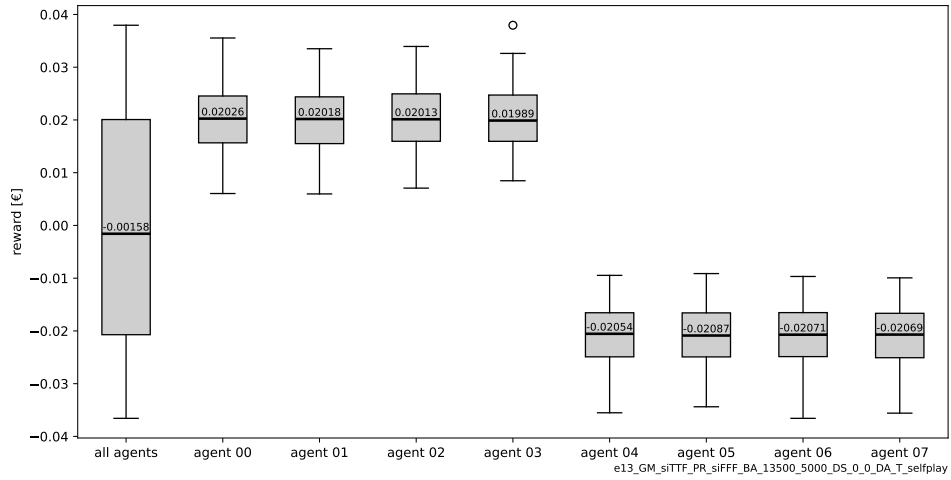
Figure B.80: Detailed view of key metrics for the run with seed 0 of experiment B.4.2 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents



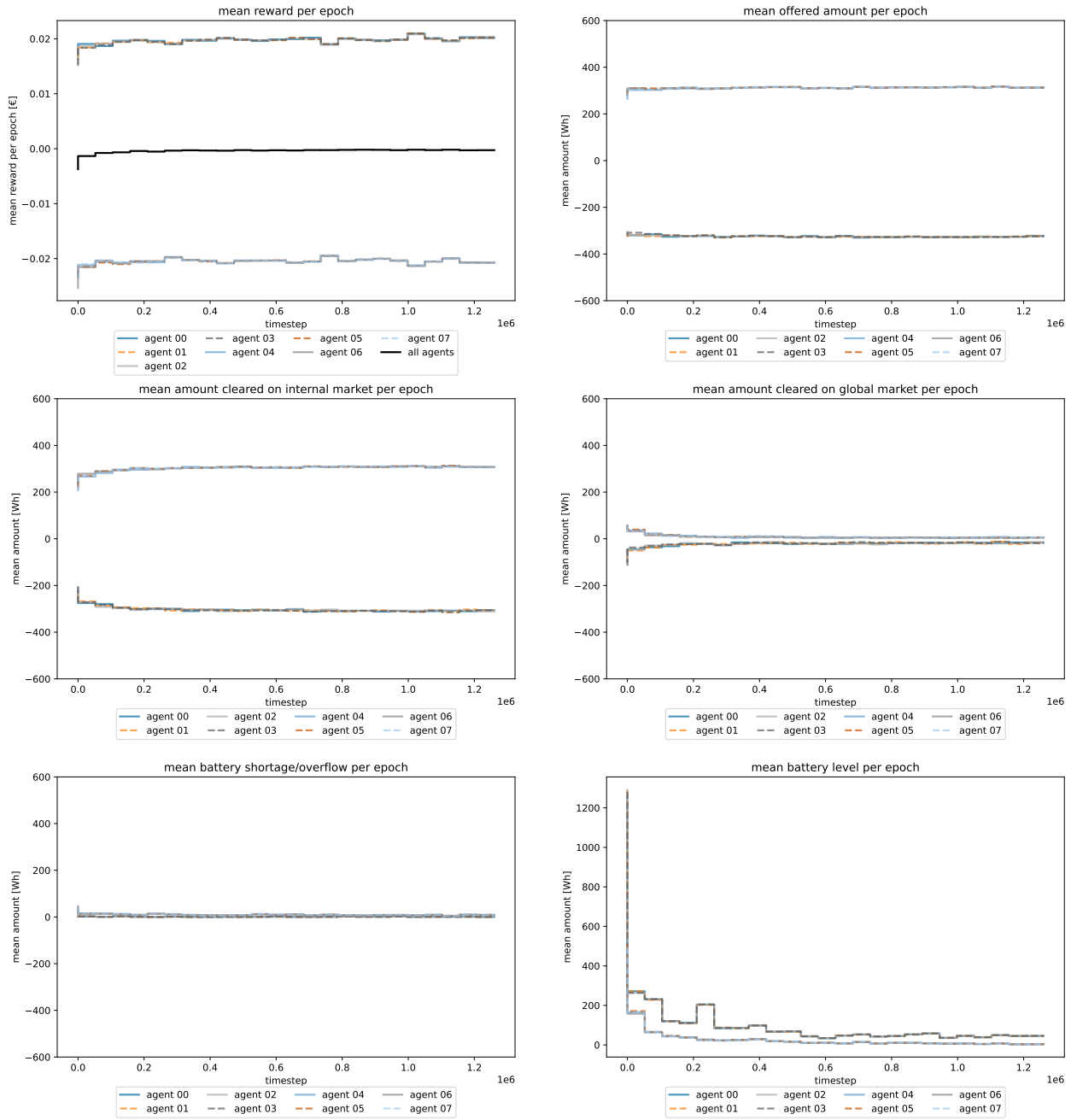
e13_GM_siTTF_PR_siFFF_BA_13500_5000_DS_0_0_DA_T_selfplay

Figure B.81: Overview of the mean rewards per epoch for all agents combined for experiment B.4.2 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.



e13_GM_siTTF_PR_siFFF_BA_13500_5000_DS_0_0_DA_T_selfplay

Figure B.82: Reward distribution in the last epoch for the run with seed 0 for experiment B.4.2 with self-play PPO agents.



e13_GM_siTTF_PR_siFFF_BA_13500_5000_DS_0_0_DA_T_selfplay

Figure B.83: Detailed view of key metrics for the run with seed 0 of experiment B.4.2 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.4.3 Using the Synthetic Datasets

Parameter	Setting
global market data generation	Synthetic
prosumers data generation	Synthetic
Day-ahead window	activated
Data Sharing	deactivated
global market Time-of-use cost	0.000045 €/Wh

Table B.13: Overview of settings for experiment B.4.3

PPO Agents

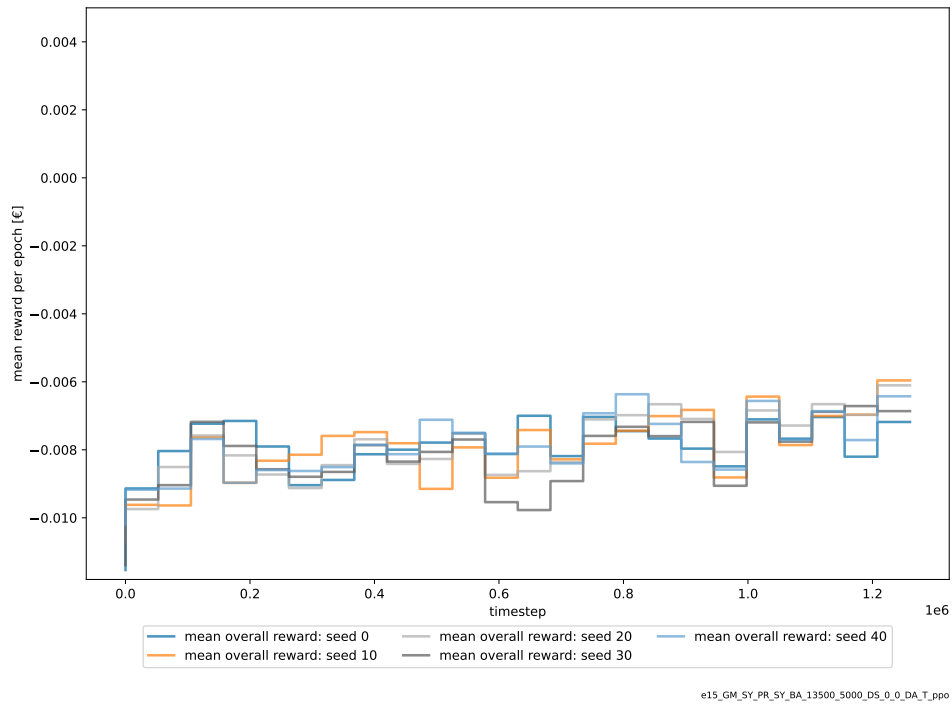


Figure B.84: Overview of the mean rewards per epoch for all agents combined for experiment B.4.3 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

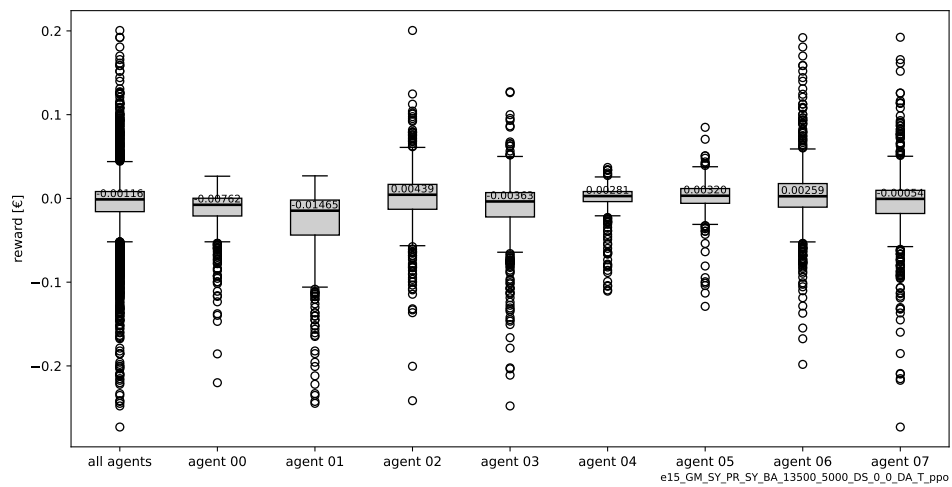
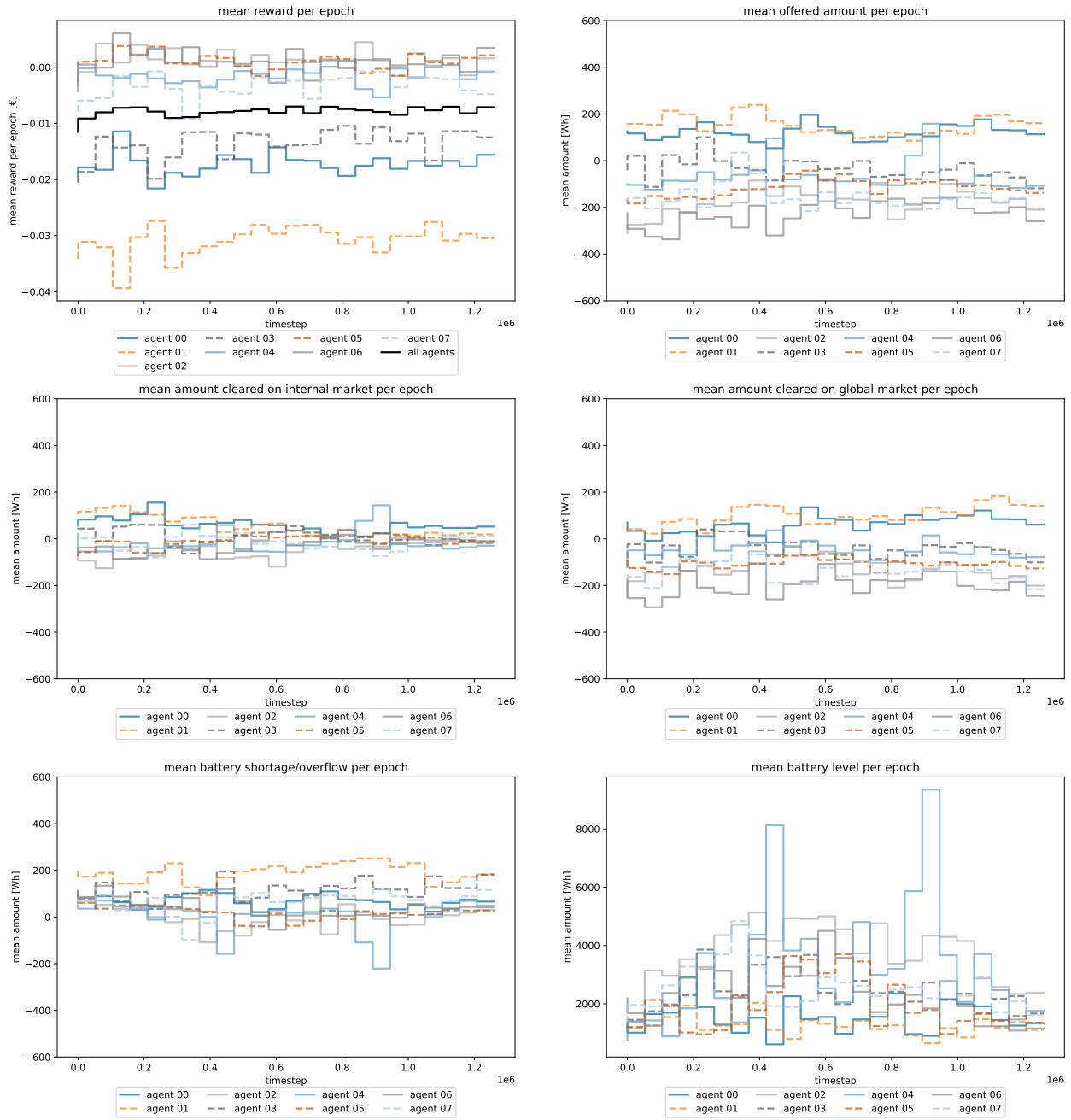


Figure B.85: Reward distribution in the last epoch for the run with seed 0 for experiment B.4.3 with PPO agents



e15_GM_SY_PR_SY_BA_13500_5000_DS_0_0_DA_T_ppo

Figure B.86: Detailed view of key metrics for the run with seed 0 of experiment B.4.3 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents

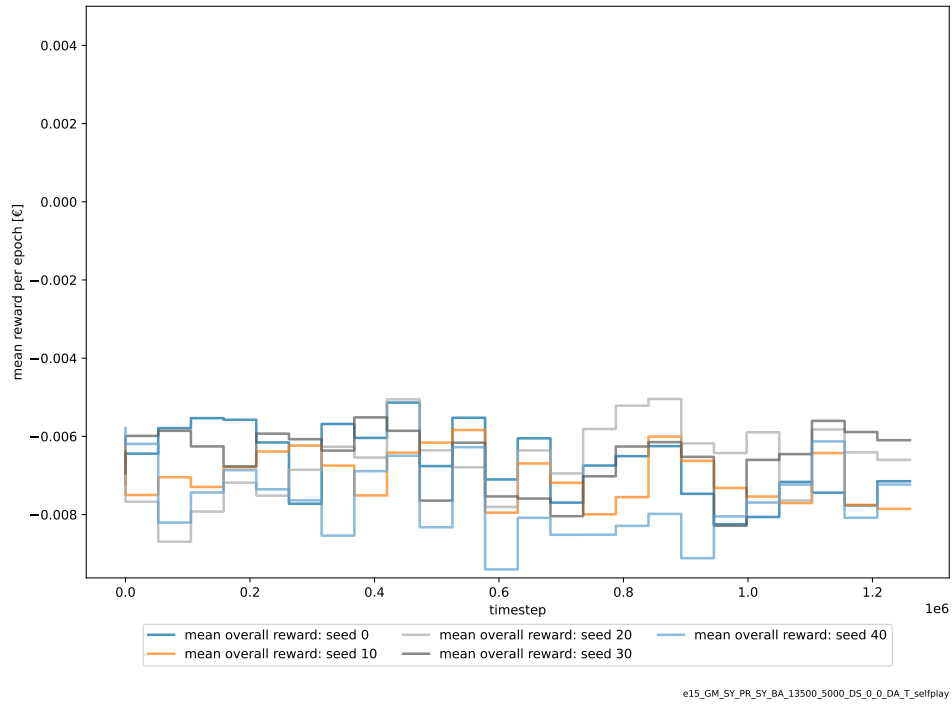


Figure B.87: Overview of the mean rewards per epoch for all agents combined for experiment B.4.3 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

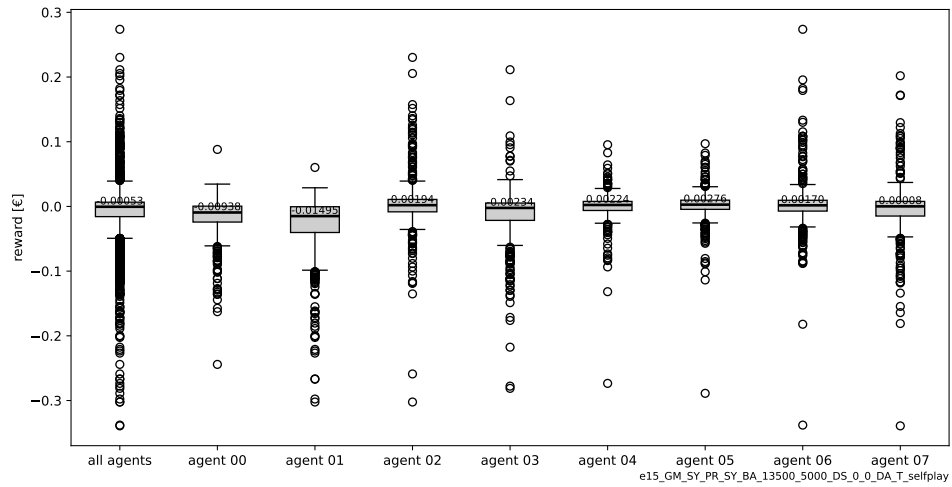
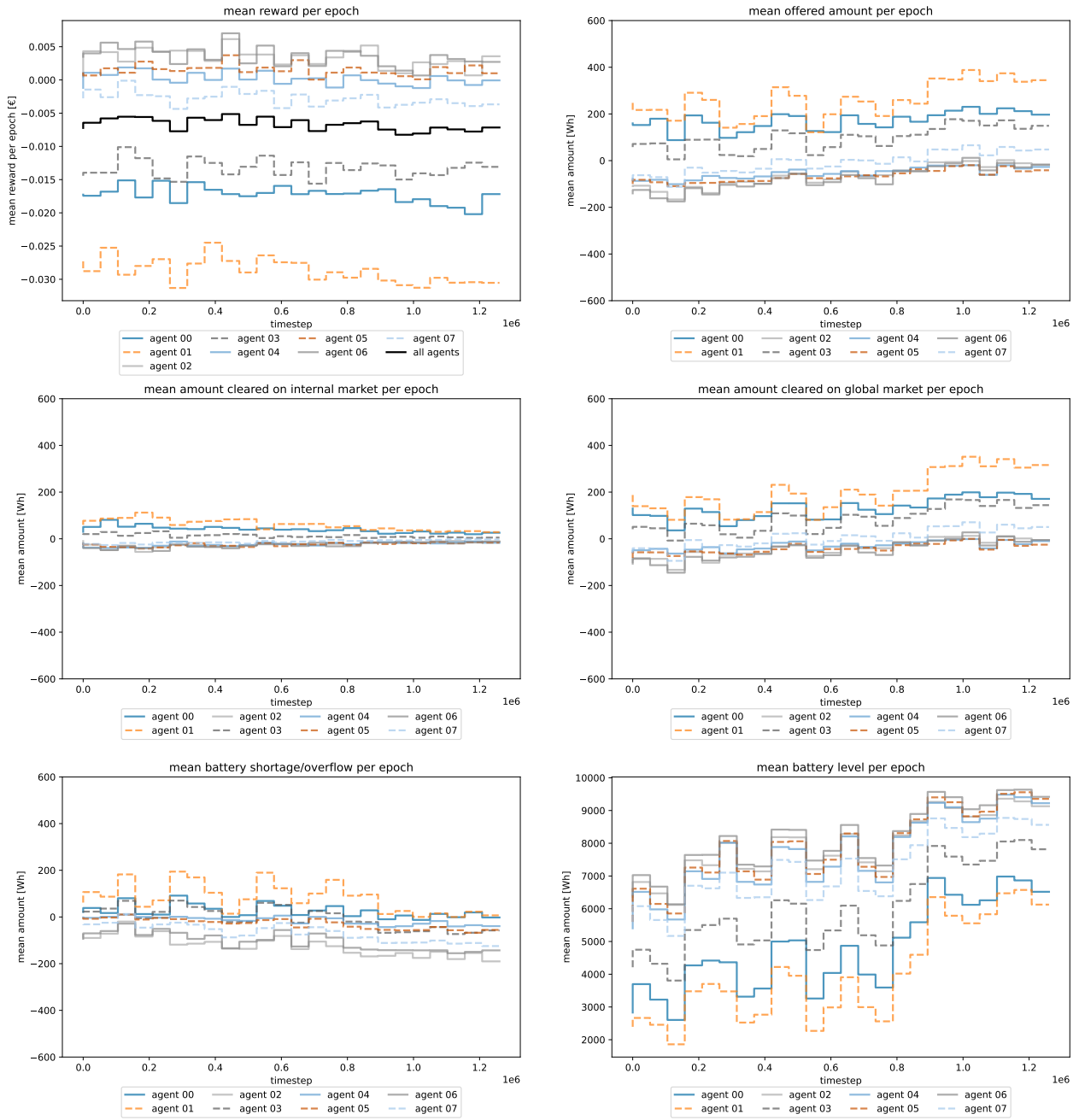


Figure B.88: Reward distribution in the last epoch for the run with seed 0 for experiment B.4.3 with self-play PPO agents.



e15_GM_SY_PR_SY_BA_13500_5000_DS_0_0_DA_T_selfplay

Figure B.89: Detailed view of key metrics for the run with seed 0 of experiment B.4.3 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.5 With Batteries, Day-Ahead Window and Data Sharing

B.5.1 Yearly Oscillating Prosumers and a Cte. Global Market

Parameter	Setting	Parameter	Setting
global market data generation	formula	global market Time-of-use cost	0.000045 €/Wh
prosumers data generation	formula	global market mean day-ahead value	0.000042 €/Wh
Day-ahead window	activated	global market yearly day-ahead amplitude	0 €/Wh
Data Sharing	activated	global market daily day-ahead amplitude	0 €/Wh
Producer agents	00,01,02,03	prosumer mean energy demand value	+/-320 Wh
Consumer Agents	04,05,06,07	prosumer yearly energy demand amplitude	350 Wh

Table B.14: Overview of settings for experiment B.5.1

PPO Agents

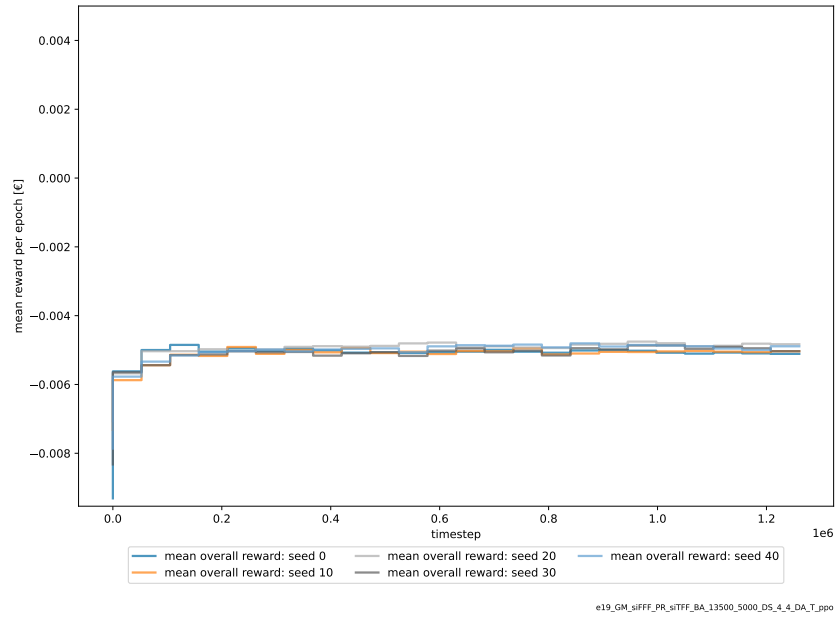


Figure B.90: Overview of the mean rewards per epoch for all agents combined for experiment B.5.1 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

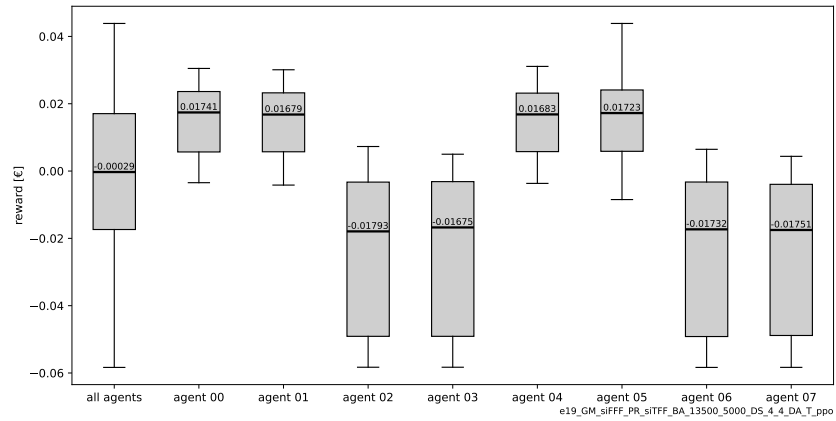
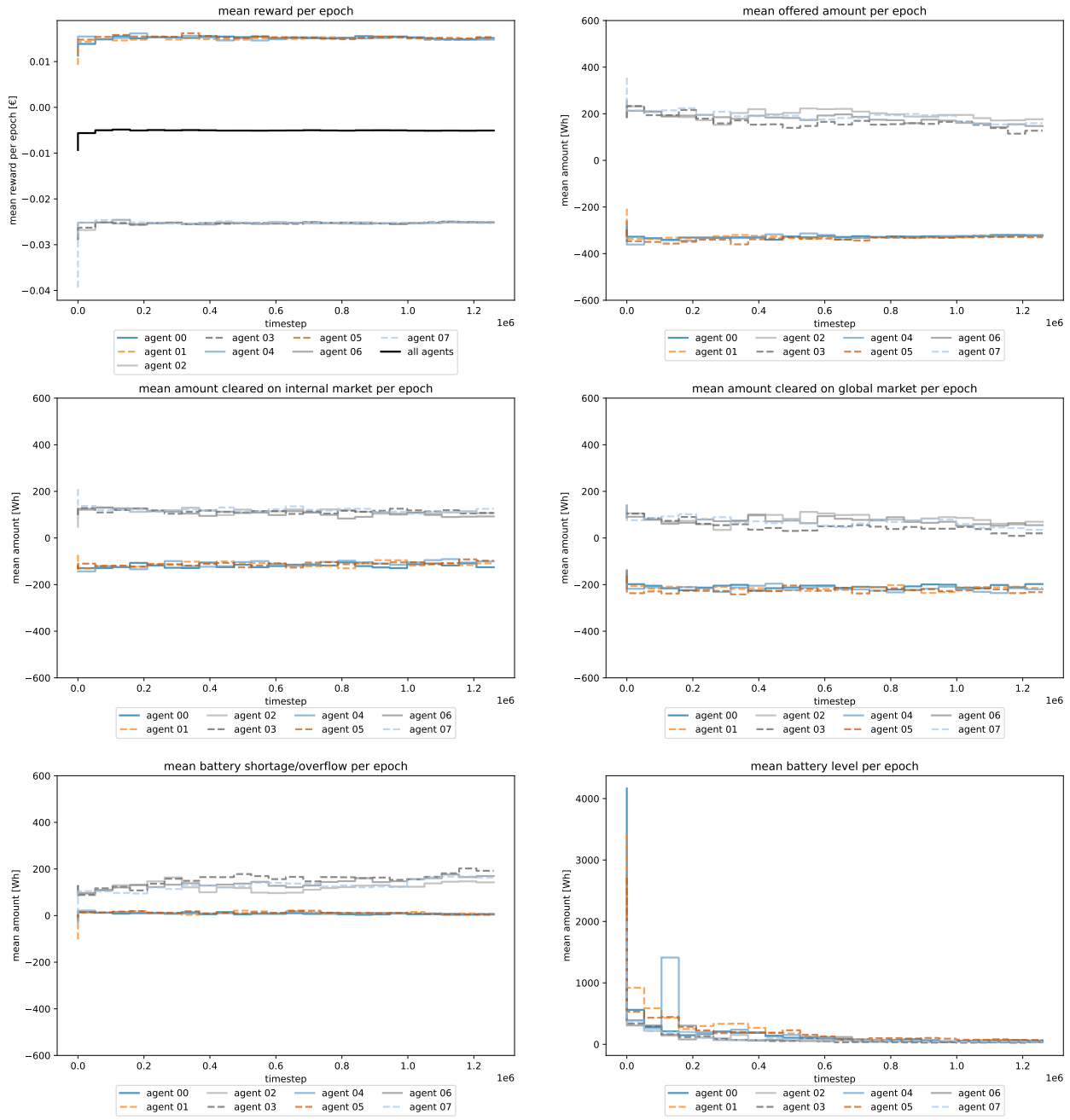


Figure B.91: Reward distribution in the last epoch for the run with seed 0 for experiment B.5.1 with PPO agents



e19_GM_sIFFF_PR_siTFF_BA_13500_5000_DS_4_4_DA_T_ppo

Figure B.92: Detailed view of key metrics for the run with seed 0 of experiment B.5.1 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents

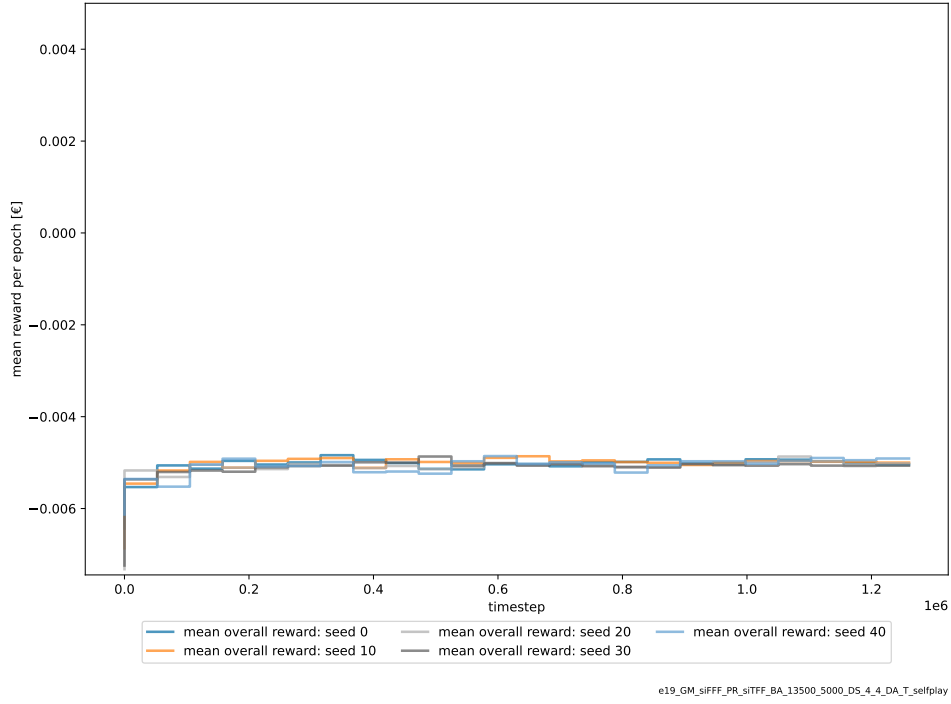


Figure B.93: Overview of the mean rewards per epoch for all agents combined for experiment B.5.1 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

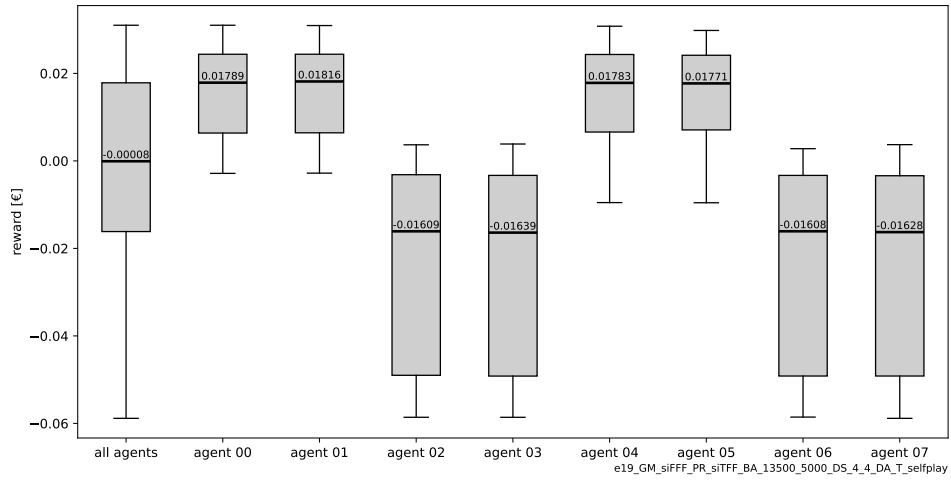
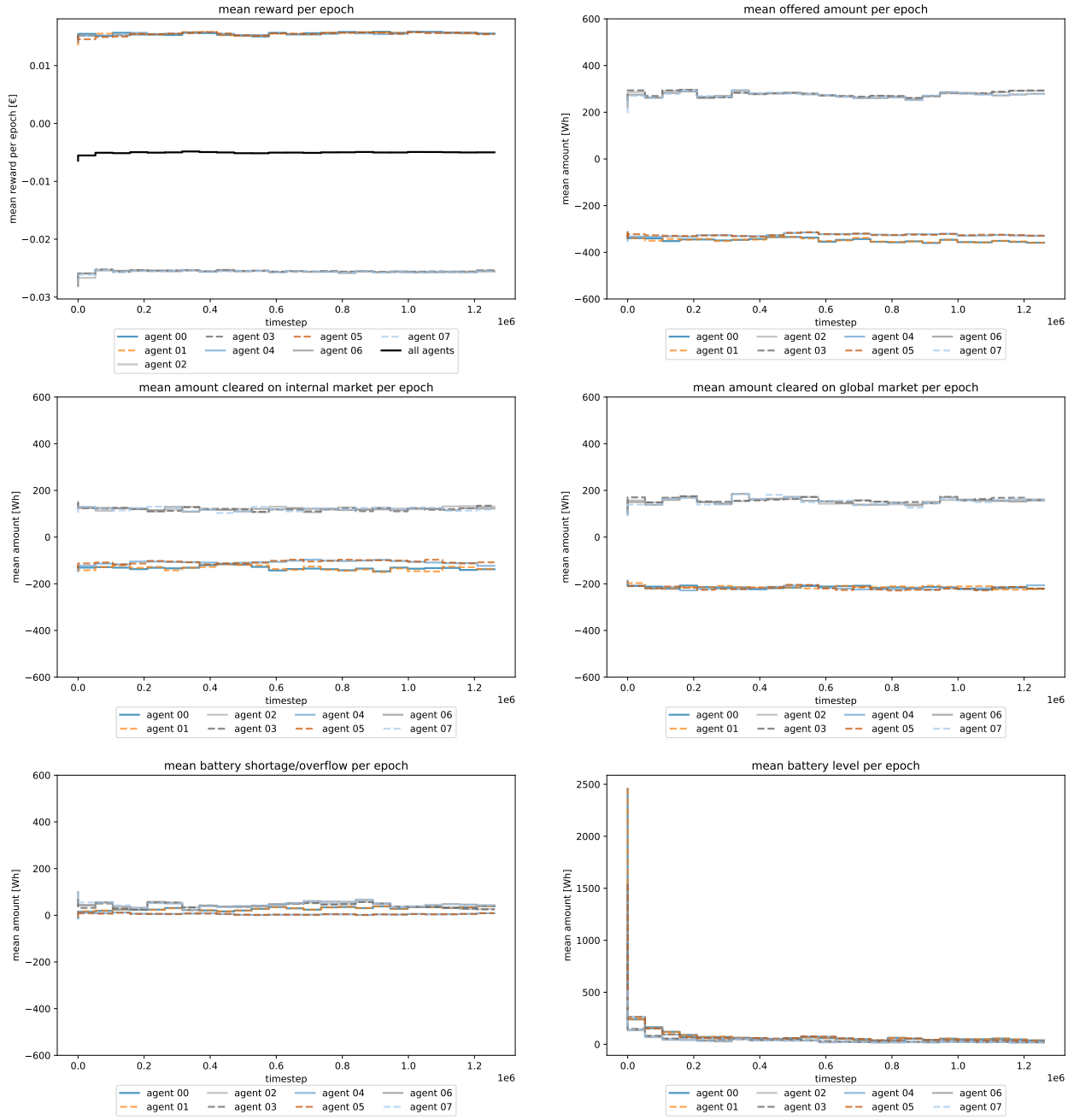


Figure B.94: Reward distribution in the last epoch for the run with seed 0 for experiment B.5.1 with self-play PPO agents.



e19_GM_sIFFF_PR_sITFF_BA_13500_5000_DS_4_4_DA_T_selfplay

Figure B.95: Detailed view of key metrics for the run with seed 0 of experiment B.5.1 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

B.5.2 Using the Synthetic Datasets

Parameter	Setting
global market data generation	Synthetic
prosumers data generation	Synthetic
Day-ahead window	activated
Data Sharing	activated
global market Time-of-use cost	0.000045 €/Wh

Table B.15: Overview of settings for experiment B.5.2

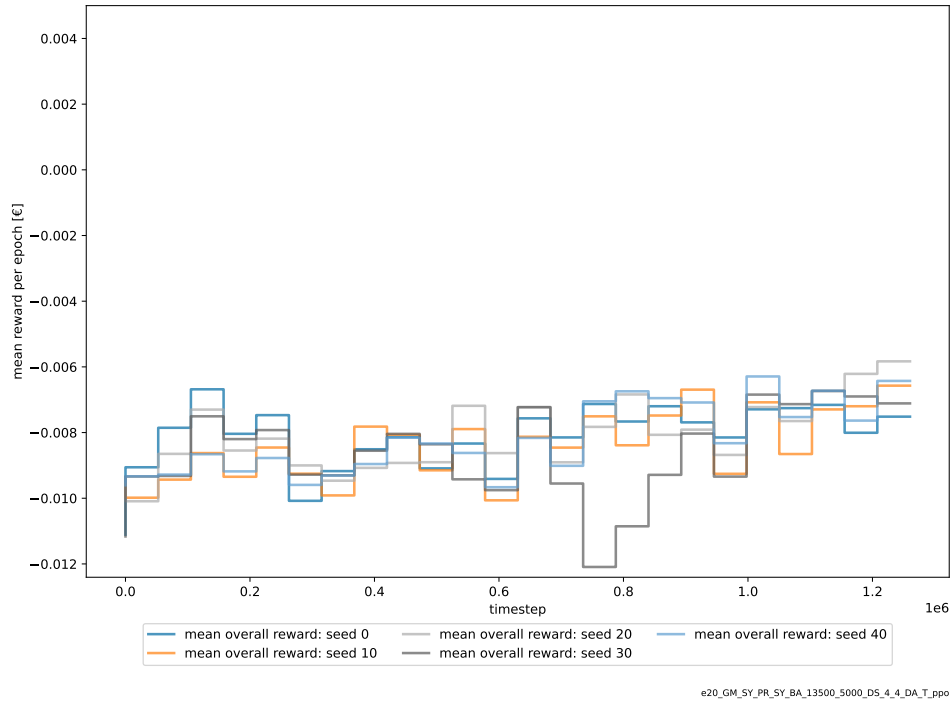


Figure B.96: Overview of the mean rewards per epoch for all agents combined for experiment B.5.2 with PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

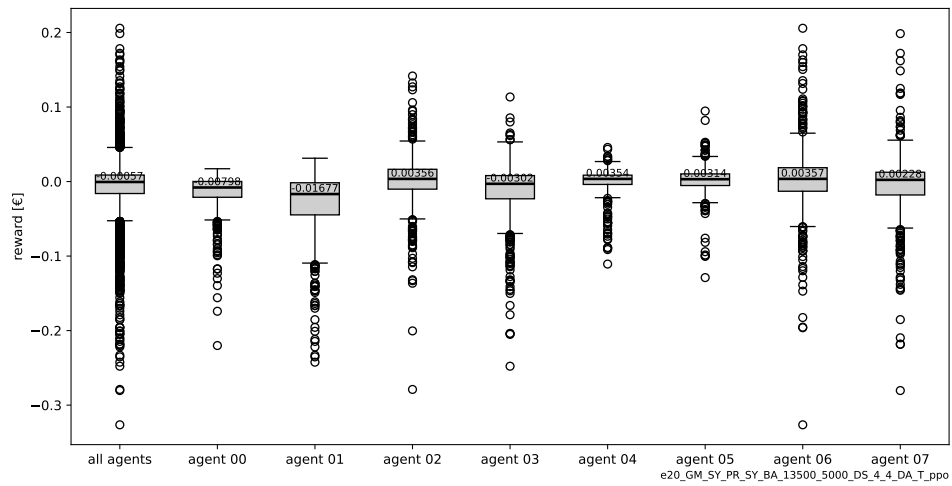
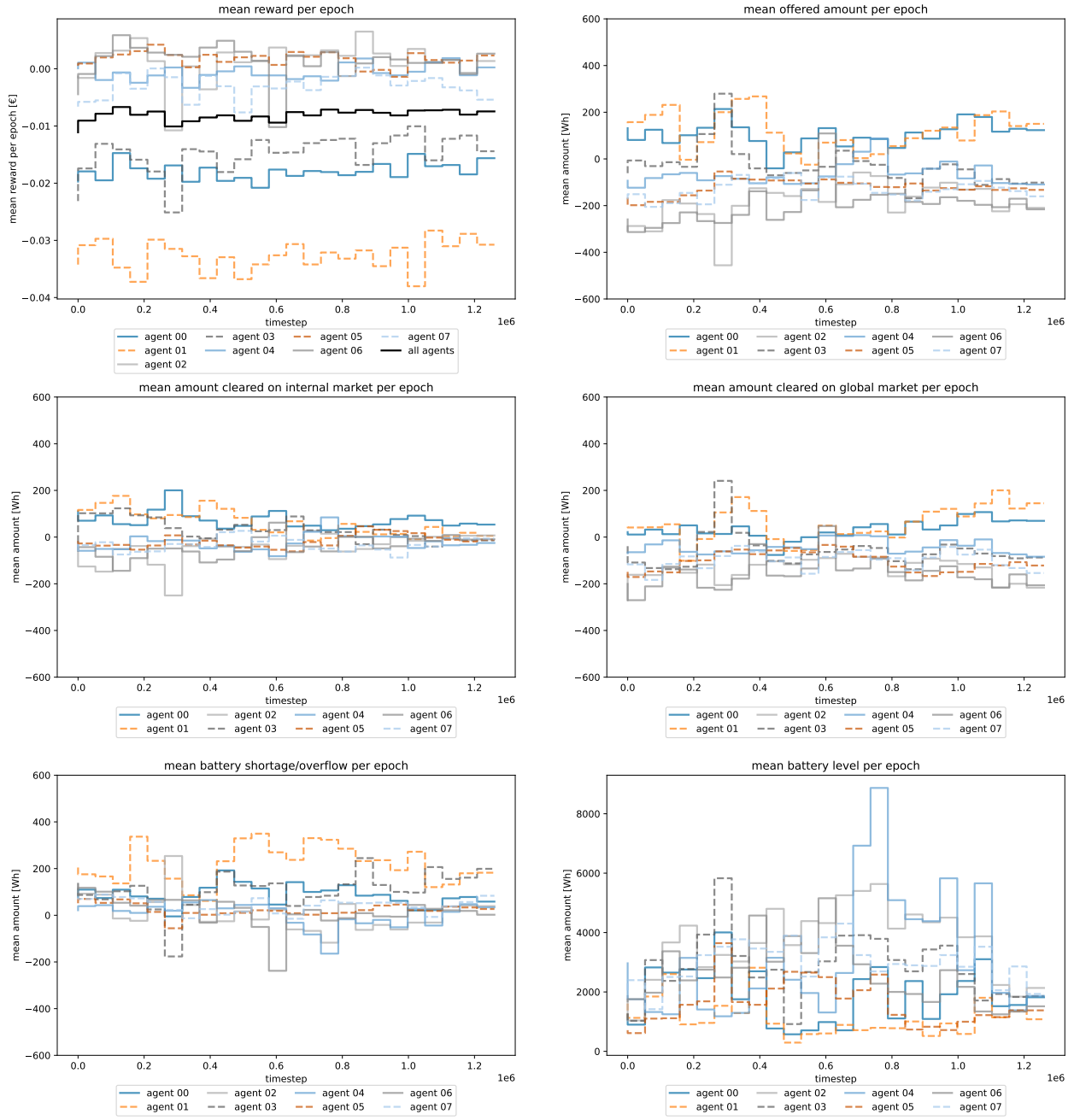


Figure B.97: Reward distribution in the last epoch for the run with seed 0 for experiment B.5.2 with PPO agents



e20_GM_SY_PR_SY_BA_13500_5000_DS_4_4_DA_T_ppo

Figure B.98: Detailed view of key metrics for the run with seed 0 of experiment B.5.2 with PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.

Self-play PPO Agents

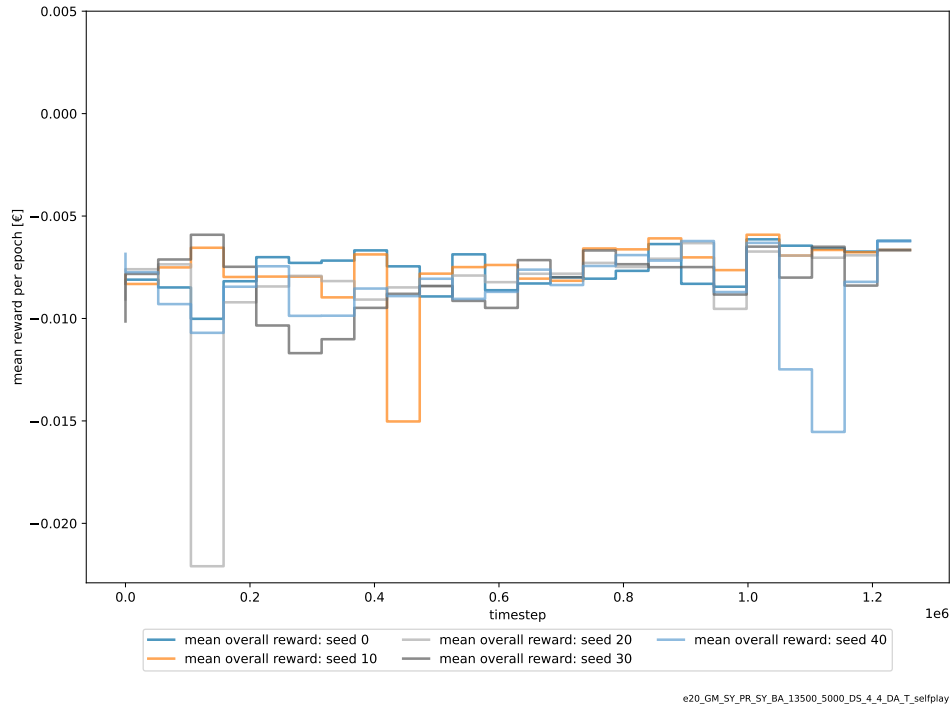


Figure B.99: Overview of the mean rewards per epoch for all agents combined for experiment B.5.2 with self-play PPO agents. The experiment was repeated five times using seeds 0,10,20,30,40, each comprising 25 epochs with a length of 52512 timesteps.

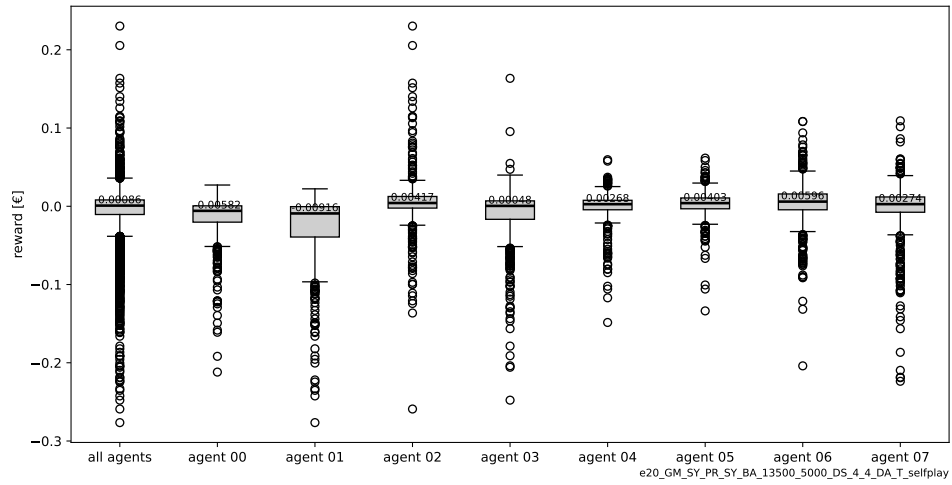


Figure B.100: Reward distribution in the last epoch for the run with seed 0 for experiment B.5.2 with self-play PPO agents.

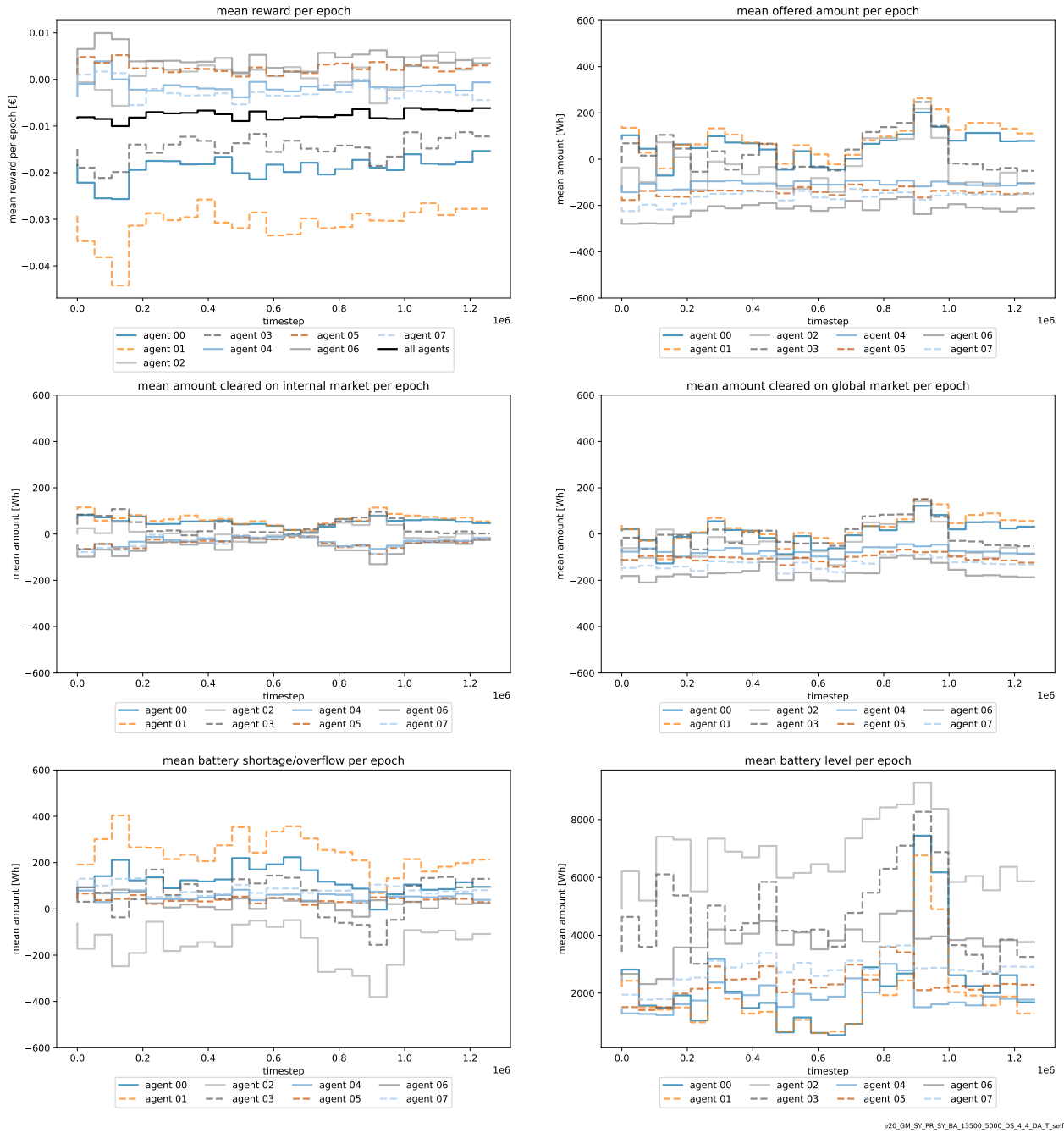


Figure B.101: Detailed view of key metrics for the run with seed 0 of experiment B.5.2 with self-play PPO agents. The experiment was run for 25 epochs, each comprising 52512 timesteps.